

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



Kiến trúc Phần mềm (CO3017)

BÁO CÁO BÀI TẬP LỚN

Intelligent Restaurant Management System (IRMS)

Giảng viên: Trần Trương Tuấn Phát

Lớp học phần: L01

Nhóm: 8

Khoa: Khoa học và Kỹ thuật Máy tính

MSSV	Họ và tên	Email
2311896	Đỗ Quang Long	long.doquang1896@hcmut.edu.vn
2311982	Lê Hoàng Luân	luan.le2311982@hcmut.edu.vn
2310990	Lê Thúy Hiền	hien.lethuy@hcmut.edu.vn
2213698	Nguyễn Hải Trung	trung.nguyen2004@hcmut.edu.vn
2211384	Tô Thế Hưng	hung.totothehung@hcmut.edu.vn
2310737	Trần Nhật Đăng	dang.trannh388@hcmut.edu.vn

Mục lục

1	Tổng quan hệ thống	3
1.1	Bối cảnh nghiệp vụ	3
1.2	Mục tiêu của hệ thống	3
1.3	Phạm vi hệ thống (IRMS)	4
1.4	Các bên liên quan	6
1.4.1	Cơ sở đo tải và ràng buộc dự án	7
1.5	Yêu cầu chức năng	7
1.6	Yêu cầu phi chức năng ở góc nhìn toàn dự án	9
2	Mô hình hóa hệ thống	10
2.1	Danh mục use cases	10
2.2	Góc nhìn sử dụng	15
2.2.1	Kịch bản 1: Từ đặt bàn đến tiếp nhận và xếp bàn	15
2.2.2	Kịch bản 2: Từ gọi món đến điều phối bếp và phục vụ	16
2.2.3	Kịch bản 3: Từ hóa đơn đến thanh toán và biên lai	16
2.2.4	Kịch bản 4: Từ cảnh báo tồn kho thấp đến quyết định của quản lý	16
2.3	Đặc tả use cases	21
3	Kiến trúc phần mềm	50
3.1	Tổng quan kiến trúc	51
3.2	Tổng quan cho các góc nhìn chính	52
3.2.1	Tổng quan cho góc nhìn logic	53
3.2.2	Tổng quan cho góc nhìn mô-đun	54
3.2.3	Góc nhìn phân rã dịch vụ cho tuyến order-to-cash	57
3.2.4	Tổng quan cho góc nhìn phân bổ	58
3.3	Đặc trưng kiến trúc	59
3.4	Góc nhìn logic	60
3.4.1	Sơ đồ phân rã năng lực logic ban đầu	60
3.4.2	Sơ đồ phân rã năng lực logic hoàn chỉnh	61
3.4.3	Sơ đồ chuỗi phục vụ lỗi	62
3.4.4	Ý nghĩa kiến trúc của góc nhìn logic	63
3.5	Góc nhìn phân rã	63
3.5.1	Các ứng dụng phía người dùng	63
3.5.2	Các dịch vụ nghiệp vụ phía sau	63
3.5.3	Sơ đồ tổng quan	65
3.6	Góc nhìn thành phần và kết nối	69
3.6.1	Các thành phần chính	69
3.6.2	Các kiểu kết nối chính	70
3.6.3	Biện minh cho mô hình kết nối hỗn hợp	70
3.6.4	Sơ đồ thành phần cho tuyến lệnh vận hành	74
3.6.5	Sơ đồ thành phần cho xử lý nền, quản trị và mô hình đọc	75
3.7	Góc nhìn triển khai	76

3.7.1	Tổng quan phân bổ hệ thống	76
3.7.2	Giải thích thiết kế triển khai	78
3.7.3	Sơ đồ triển khai cho runtime Docker local và cô lập lỗi	83
3.7.4	Install View cho gói triển khai Docker local	85
3.8	Nguyên tắc thiết kế	88
3.9	Áp dụng nguyên lý SOLID vào thiết kế	88
3.9.1	Các ví dụ áp dụng rõ nhất trong code	90
3.10	Khả năng mở rộng trong tương lai	91
4	Thiết kế chi tiết	92
4.1	Góc nhìn lớp	92
4.1.1	Phạm vi bao phủ của sơ đồ lớp	92
4.2	Giải thích theo từng cụm lớp	97
4.3	Thiết kế hành vi	107
4.3.1	Nhóm đặt bàn và bàn ăn	108
4.3.2	Nhóm gọi món và điều phối bếp	113
4.3.3	Nhóm hóa đơn, thanh toán và biên lai	120
4.3.4	Nhóm tồn kho, cảnh báo và phân tích	125
4.3.5	Nhóm quản trị cấu hình và vận hành	129
4.4	Phản tư về toàn bộ báo cáo và định hướng hiện thực	133
A	Phụ lục	135
A.1	Lý do lựa chọn kiến trúc và hồ sơ quyết định kiến trúc	135
A.1.1	So sánh các kiểu kiến trúc ứng viên	136
A.2	Liên kết giữa phụ lục và các phần chính	139
A.3	Lý do chọn các góc nhìn và loại sơ đồ	140
A.4	Lý do chọn các thành phần và loại hình kiến trúc nền	141
A.5	Phân công công việc	143

1 Tổng quan hệ thống

1.1 Bối cảnh nghiệp vụ

Hệ thống **Intelligent Restaurant Management System (IRMS)** được thiết kế nhằm tối ưu hóa hoạt động vận hành của nhà hàng, bao gồm phục vụ tại bàn, khu vực bếp, quầy thanh toán và các hoạt động điều phối theo ca. Trong thực tế, nhiều sai sót thường xảy ra vào các giờ cao điểm, chẳng hạn như đơn gọi món bị ghi nhầm, ghi chú dị ứng không được lưu, trạng thái bàn không cập nhật kịp thời, hóa đơn bị lệch khi tách hóa đơn hoặc áp dụng khuyến mãi, cũng như tồn kho cuối ngày không phản ánh đúng mức tiêu hao thực tế. Những vấn đề này ảnh hưởng trực tiếp đến chất lượng phục vụ, trải nghiệm khách hàng và hiệu quả vận hành của nhà hàng.

Vì vậy, IRMS không chỉ đơn thuần là một ứng dụng ghi nhận đơn gọi món trên màn hình mà phải đóng vai trò như một **nền tảng số trung tâm**, liên kết chặt chẽ các khu vực phục vụ phía trước, khu bếp và bộ phận quản trị. Hệ thống cần đảm bảo các chức năng nghiệp vụ toàn diện, bao gồm quản lý đặt bàn và xếp bàn, tiếp nhận và điều phối đơn gọi món, quản lý tiến trình chế biến trong bếp, lập và thanh toán hóa đơn, theo dõi tồn kho, cung cấp báo cáo và phân tích hiệu suất vận hành, kiểm soát truy cập dựa trên vai trò, ghi nhận nhật ký kiểm toán, đồng thời có khả năng tích hợp với các hệ thống bên ngoài khi cần thiết.

IRMS được xây dựng như một hệ thống nghiệp vụ hoàn chỉnh với luồng xử lý lỗi rõ ràng, hỗ trợ các quy trình bất đồng bộ, đồng thời đảm bảo khả năng mở rộng linh hoạt để đáp ứng nhu cầu vận hành phức tạp và đa dạng trong môi trường nhà hàng hiện đại.

1.2 Mục tiêu của hệ thống

Hệ thống *Intelligent Restaurant Management System (IRMS)* được xây dựng nhằm hỗ trợ số hóa và tối ưu hóa các hoạt động vận hành trong nhà hàng. Thông qua việc tích hợp các chức năng quản lý đơn hàng, điều phối bếp, quản lý bàn, thanh toán và theo dõi tồn kho, hệ thống hướng tới việc nâng cao hiệu quả làm việc của nhân viên cũng như cải thiện trải nghiệm phục vụ khách hàng. Các mục tiêu chính của hệ thống bao gồm:

1. Giảm sai sót trong quá trình gọi món và xử lý đơn hàng

Hệ thống sử dụng thực đơn số hóa và giao diện nhập liệu có cấu trúc để giúp nhân viên phục vụ tạo đơn nhanh chóng và chính xác. Các tùy chọn như ghi chú món ăn, yêu cầu đặc biệt hoặc cảnh báo dị ứng giúp hạn chế nhầm lẫn trong quá trình phục vụ và chế biến.

2. Tăng hiệu quả phối hợp giữa bộ phận phục vụ và bếp

Khi đơn hàng được xác nhận, hệ thống sẽ tự động chuyển thông tin đến màn hình hiển thị bếp (*Kitchen Display System – KDS*). Điều này giúp đầu bếp nắm bắt yêu cầu món ăn kịp thời, giảm thời gian chờ và hạn chế việc truyền đạt thông tin thủ công.

3. Đảm bảo cập nhật và đồng bộ trạng thái hoạt động theo thời gian gần thực

Các thông tin quan trọng như trạng thái bàn, tiến độ chế biến món ăn, trạng thái đơn hàng và thanh toán cần được cập nhật liên tục trên các giao diện liên quan. Điều này giúp các bộ phận trong nhà hàng có cùng nguồn dữ liệu và phối hợp làm việc hiệu quả hơn.

4. Hỗ trợ quản lý bàn và tối ưu hóa quy trình phục vụ khách hàng

Hệ thống giúp nhân viên theo dõi tình trạng bàn, quản lý đặt bàn và ước lượng thời gian chờ của khách. Nhờ đó, nhà hàng có thể sắp xếp khách hợp lý và nâng cao chất lượng dịch vụ.

5. Tăng hiệu quả quản lý tài chính và thanh toán

IRMS hỗ trợ tạo hóa đơn tự động, tính toán tổng chi phí bao gồm món ăn, dịch vụ và khuyến mãi. Hệ thống

cho phép nhiều phương thức thanh toán khác nhau, đồng thời hỗ trợ tách hóa đơn khi cần thiết.

6. Hỗ trợ theo dõi tồn kho và cung cấp dữ liệu phân tích cho quản lý

Hệ thống ghi nhận việc sử dụng nguyên liệu và cung cấp cảnh báo khi tồn kho xuống thấp. Ngoài ra, các báo cáo bán hàng và phân tích hoạt động giúp nhà quản lý đánh giá hiệu suất kinh doanh và đưa ra quyết định phù hợp.

7. Tạo nền tảng hệ thống linh hoạt và có khả năng mở rộng

Kiến trúc của IRMS được thiết kế theo hướng dịch vụ kết hợp điều phối bằng sự kiện. Các năng lực nghiệp vụ được đặt trong những service có trách nhiệm rõ ràng, giao tiếp đồng bộ qua API khi thao tác cần phản hồi trực tiếp và phát sinh event cho các hệ quả nghiệp vụ có thể xử lý bất đồng bộ. Cách tổ chức này cho phép mở rộng từng vùng chức năng, cô lập các adapter biên và giảm phụ thuộc chéo giữa đặt bàn, gọi món, bếp, thanh toán, tồn kho, báo cáo và kiểm toán.

1.3 Phạm vi hệ thống (IRMS)

Hệ thống được thiết kế để số hóa và tối ưu hóa toàn bộ quy trình vận hành của một nhà hàng, từ khâu tiếp đón khách hàng, chế biến tại bếp đến quản lý kho và phân tích kinh doanh. Xét theo kiến trúc hướng dịch vụ, các nhóm chức năng này là cơ sở để phân rã IRMS thành các service theo năng lực nghiệp vụ như Reservation & Seating Service, Ordering & Menu Service, Kitchen Coordination Service, Billing & Settlement Service, Inventory Monitoring Service, Reporting Service, IAM & Audit Service, Notification Service và các adapter biên nội bộ. Cụ thể, hệ thống bao gồm các nhóm năng lực chính sau:

• Phân hệ Phục vụ và Gọi món (Front-of-House)

- **Quản lý gọi món:** Hỗ trợ nhân viên thực hiện gọi món nhanh chóng trên máy tính bảng/terminal. Hệ thống xử lý các yêu cầu phức tạp như tùy chỉnh món, ghi chú dị ứng, các gói combo và thay đổi nguyên liệu.
- **Cập nhật thực đơn thực tế:** Hiển thị danh mục món ăn và trạng thái còn/hết hàng theo thời gian thực để nhân viên tư vấn chính xác cho khách.
- **Quản lý bàn và đặt chỗ:** Cung cấp sơ đồ bàn trực tiếp, quản lý trạng thái bàn (trống/đang dùng), danh sách chờ và lịch hẹn đặt trước cho bộ phận Host.

• Phân hệ Điều hành Bếp (Kitchen Display System – KDS)

- **Điều phối tự động:** Tự động chuyển đơn hàng từ bàn đến các trạm bếp tương ứng.
- **Tối ưu hóa quy trình:** Sắp xếp và ưu tiên đơn hàng dựa trên vị trí trạm, loại món và thời gian chế biến để đảm bảo tốc độ phục vụ.
- **Tương tác thời gian thực:** Cho phép đầu bếp cập nhật tiến độ món ăn để nhân viên phục vụ có thể theo dõi và trả món kịp thời.

• Phân hệ Thanh toán và Tài chính

- **Xử lý hóa đơn linh hoạt:** Hỗ trợ tính phí dịch vụ, áp dụng chương trình khuyến mãi, tách hóa đơn và quản lý tiền tip.
- **Đa dạng phương thức thanh toán:** Hỗ trợ tiền mặt, thẻ và ví điện tử thông qua PaymentGateway trong code, đồng thời phát hành biên lai qua ReceiptDeliveryAdapter.

• Phân hệ Quản trị và Kho vận

- **Quản lý kho tự động:** Tự động khấu trừ nguyên liệu khi có đơn hàng và phát thông báo cảnh báo khi lượng tồn kho xuống thấp.
- **Báo cáo và Phân tích (Analytics):** Cung cấp hệ thống dashboard về doanh thu, các món bán chạy, hiệu suất nhân viên và nhận diện các điểm nghẽn trong vận hành.
- **Kiểm soát và Bảo mật:** Quản lý quyền truy cập dựa trên vai trò (RBAC) và lưu vết lịch sử (audit logs) cho các tác động quan trọng vào hệ thống.

1.4 Các bên liên quan

Bảng 1: Các bên liên quan chính của IRMS

Bên liên quan	Vai trò	Mối quan tâm chính
Khách hàng	Người dùng/đơn vị vận hành	- Đặt bàn trước - Nhận thông báo xác nhận và nhắc nhở - Thanh toán đa phương thức, nhận hóa đơn - Gọi món, yêu cầu tùy chỉnh và ghi chú dị ứng khi gọi món
Lễ tân	Người dùng/đơn vị vận hành	- Quản lý sơ đồ bàn, theo dõi trạng thái trống/bận - Xử lý đặt bàn trước và danh sách chờ - Gửi thông báo xác nhận/nhắc nhở cho khách hàng - Ước tính thời gian chờ, phân bổ bàn hợp lý
Phục vụ	Người dùng/đơn vị vận hành	- Tạo đơn gọi món, tiếp nhận yêu cầu gọi món từ khách - Theo dõi đơn, thời gian phục vụ và tình trạng thanh toán theo từng bàn
Nhân viên bếp	Người dùng/đơn vị vận hành	- Cập nhật trạng thái món: Đã bắt đầu, Đang nấu, Sẵn sàng phục vụ - Phản hồi cảnh báo khi món sắp trễ - Cập nhật nguyên liệu đã sử dụng sau khi chế biến
Thu ngân	Người dùng/đơn vị vận hành	- Lập hóa đơn tổng hợp (đồ ăn, đồ uống, dịch vụ) - Xử lý thanh toán: tiền mặt, ví điện tử - Tách hóa đơn và quản lý tiền bo - Thực hiện hoàn tiền khi cần và ghi vào nhật ký kiểm toán
Quản lý	Người dùng/đơn vị vận hành	- Xem báo cáo: doanh thu, giờ cao điểm, món bán chạy - Giám sát tồn kho, nhận cảnh báo ngưỡng tối thiểu - Cấu hình thực đơn, giá và chương trình khuyến mãi - Xuất báo cáo cho kế toán, đánh giá hiệu suất - Theo dõi hiệu quả nhân viên và điểm nghẽn bếp
Quản trị viên	Người dùng/đơn vị vận hành	- Phân quyền truy cập theo vai trò - Xem và quản lý nhật ký kiểm toán toàn hệ thống - Xử lý sự cố kỹ thuật, bảo đảm hệ thống luôn sẵn sàng hoạt động - Cấu hình và bảo trì hệ thống bán hàng tại quầy

1.4.1 Cơ sở đo tải và ràng buộc dự án

Đây là giả định thiết kế cho một nhà hàng tầm trung, được dùng thống nhất ở toàn bộ báo cáo khi nói về hiệu năng, độ tin cậy và khả năng mở rộng.

Bảng 2: Hai điều kiện vận hành dùng làm cơ sở thiết kế của IRMS

Điều kiện	Tải giả định	Ý nghĩa với kiến trúc
Điều kiện 1 — Ca thông thường	20–25 bàn hoạt động, 8–12 nhân viên đăng nhập đồng thời, tối đa 35 đơn gọi món mỗi giờ, khoảng 120 dòng món mỗi giờ.	Hệ thống phải phản hồi mượt cho các tác vụ tạo đơn gọi món, cập nhật màn hình bếp, đổi trạng thái bàn và chốt hóa đơn mà chưa cần mở rộng hay hy sinh trải nghiệm thao tác.
Điều kiện 2 — Giờ cao điểm	35–40 bàn hoạt động, 15–18 nhân viên đăng nhập đồng thời, tối đa 60 đơn gọi món mỗi giờ, khoảng 220 dòng món mỗi giờ và 20 hóa đơn mỗi giờ.	Kiến trúc phải giữ ổn định tuyến giao dịch nóng; các luồng nền như báo cáo, cảnh báo mức tồn thấp và đồng bộ thông báo không được làm nghẽn gọi món, điều phối bếp hay thanh toán.

Trong hai điều kiện trên, dự án chịu các ràng buộc thực tế sau:

- **Ràng buộc phạm vi:** phiên bản hiện tại phục vụ một nhà hàng đơn lẻ, chưa tối ưu cho vận hành nhiều chi nhánh ở quy mô đầy đủ, nhưng kiến trúc vẫn chưa điểm mở cho mở rộng về sau.
- **Ràng buộc thiết bị:** nhân viên thao tác qua máy tính bảng gọi món, màn hình bếp, thiết bị thu ngân hoặc giao diện web nội bộ; trải nghiệm phải ưu tiên thao tác nhanh, ít bước và dễ học theo ca.
- **Ràng buộc mạng:** mạng nội bộ có thể chấp chừa ngắn hạn trong phạm vi dưới 60 giây; hệ thống được phép thử lại nhưng không được làm mất đơn gọi món đã xác nhận hoặc tạo phiếu trùng.
- **Ràng buộc thanh toán:** thanh toán điện tử luôn đi qua cổng đối tác ngoài; IRMS không lưu dữ liệu thẻ thô mà chỉ lưu kết quả giao dịch và mã tham chiếu.
- **Ràng buộc kiểm soát:** dữ liệu nghiệp vụ phải được lưu tập trung và có nhật ký kiểm toán cho hoàn tiền, hủy bỏ, ghi đề hoặc thay đổi nhạy cảm.
- **Ràng buộc học phần:** phạm vi hiện thực có thể tập trung vào một phần phân hệ lõi, nhưng kiến trúc vẫn phải phản ánh đầy đủ bức tranh hệ thống để chứng minh tư duy thiết kế tổng thể.

1.5 Yêu cầu chức năng

Để đáp ứng các mục tiêu vận hành và giải quyết các bài toán trong bối cảnh nghiệp vụ, IRMS được phân rã thành các service theo năng lực nghiệp vụ. Mỗi service giữ một ranh giới trách nhiệm riêng, có dữ liệu và luật nghiệp vụ thuộc phạm vi của mình, đồng thời phối hợp với service khác bằng API đồng bộ hoặc event bất đồng bộ tùy theo tính chất của thao tác.

1. Reservation & Seating Service

Service này hỗ trợ trực tiếp hoạt động tiếp đón khách, quản lý không gian nhà hàng và mở phiên phục vụ tại bàn. Thành phần này chịu trách nhiệm quản lý trạng thái bàn, đặt bàn trước, danh sách chờ, ghép bàn, chuyển bàn và giải phóng bàn sau khi khách rời đi. Những thao tác cần kết quả tức thời như xác nhận đặt bàn, nhận khách, gán

bàn hoặc mở TableSession được xử lý qua API đồng bộ để giao diện lễ tân và phục vụ nhận được phản hồi rõ ràng.

Quản lý trạng thái bàn cho phép nhân viên xem sơ đồ nhà hàng theo thời gian thực, cập nhật trạng thái từng bàn và điều phối khách theo sức chứa thực tế. Khi một phiên bàn được mở, service có thể phát event TableSessionOpened để các service khác như gọi món, báo cáo hoặc kiểm toán nhận biết ngữ cảnh phục vụ mới mà không cần gọi trực tiếp vào service đặt bàn cho mọi hệ quả phụ.

2. Ordering & Menu Service

Service này chịu trách nhiệm quản lý thực đơn số, tùy chọn món, trạng thái còn hoặc tạm hết, đơn gọi món và ảnh chụp giao dịch tại thời điểm xác nhận. Thành phần này giải quyết mâu thuẫn giữa thực đơn có thể thay đổi liên tục và đơn gọi món đã xác nhận cần ổn định để phục vụ bếp, hóa đơn và kiểm toán. Thao tác tạo hoặc xác nhận đơn gọi món cần phản hồi ngay nên được thực hiện qua API đồng bộ.

Duyệt thực đơn và gọi món cung cấp thông tin món, giá, tình trạng khả dụng, ghi chú đặc biệt, dị ứng thực phẩm và tùy chọn từng món. Khi đơn gọi món được xác nhận, service phát event OrderConfirmed. Event này là nguồn dữ kiện cho Kitchen Coordination Service, Inventory Monitoring Service, Reporting Service, IAM & Audit Service và Notification Service. Nhờ đó, các hệ quả sau giao dịch chính được xử lý bất đồng bộ thay vì làm dài tuyến xác nhận đơn.

3. Kitchen Coordination Service

Service này chịu trách nhiệm biến đơn gọi món đã xác nhận thành công việc chế biến có thể điều phối tại bếp. Thành phần này quản lý phiếu bếp, trạm bếp, hàng đợi, mức ưu tiên, tiến độ món và bàn giao món đã hoàn tất cho tuyến phục vụ. Ranh giới này giúp logic chế biến không bị trộn vào Order hoặc OrderItem.

Tiếp nhận và hiển thị phiếu bếp được kích hoạt từ event OrderConfirmed hoặc KitchenTicketCreated. Khi đầu bếp cập nhật tiến độ món, các event như OrderItemPrepared được phát ra để giao diện phục vụ, báo cáo và thông báo cùng tiêu thụ. Các cập nhật trạng thái cần hiển thị nhanh cho nhân viên có thể đi qua API đồng bộ, trong khi thông báo và tổng hợp dữ liệu vận hành đi qua event bất đồng bộ.

4. Billing & Settlement Service

Service này chịu trách nhiệm toàn bộ vùng quyết toán, bao gồm lập hóa đơn, tách hóa đơn, tính thuế, phí phục vụ, tiền boa, áp mã khuyến mãi, ghi nhận thanh toán, phát hành biên lai và xử lý hoàn tiền. Các thao tác như lập hóa đơn, ghi nhận thanh toán hoặc hoàn tiền cần phản hồi rõ ràng nên được xử lý qua API đồng bộ.

Lập hóa đơn và thanh toán sử dụng ảnh chụp đáng tin cậy của các món đã gọi và đã phục vụ để tạo nghĩa vụ tài chính. Sau khi hóa đơn được phát hành hoặc thanh toán được ghi nhận, service phát event BillIssued, PaymentCaptured hoặc RefundRequested. Các event này phục vụ in biên lai, ghi audit, cập nhật báo cáo và đồng bộ trạng thái bàn mà không buộc service quyết toán phải gọi trực tiếp mọi service phụ trợ.

5. Inventory Monitoring Service

Service này chịu trách nhiệm theo dõi tồn kho, công thức nguyên liệu, giao dịch kho, quy tắc cảnh báo và trạng thái mức tồn thấp. Thành phần này không nằm trên tuyến phản hồi trực tiếp của thao tác gọi món hoặc thanh toán, mà chủ yếu tiêu thụ event nghiệp vụ để cập nhật biến động kho theo nhịp bất đồng bộ có kiểm soát.

Cập nhật tồn kho và cảnh báo được kích hoạt từ các event như OrderConfirmed, OrderItemPrepared hoặc PaymentCaptured. Khi mức tồn giảm dưới ngưỡng an toàn, service phát InventoryLow để Notification Service, Reporting Service hoặc giao diện quản lý xử lý tiếp. Cách tổ chức này giúp tồn kho phản ánh tiêu hao thực tế

nhưng không làm nghề thao tác gọi món ở tuyến đầu.

6. Reporting Service, IAM & Audit Service và Notification Service

Reporting Service tổng hợp dữ liệu vận hành thành mô hình đọc phục vụ báo cáo doanh thu, món bán chạy, khung giờ cao điểm, điểm nghẽn bếp, tỷ lệ hoàn tiền và hiệu suất nhân viên. Dữ liệu báo cáo được làm mới thông qua event và Background Worker, nhờ đó truy vấn quản trị không gây áp lực trực tiếp lên dữ liệu giao dịch nóng.

IAM & Audit Service quản lý định danh người dùng, phân quyền theo vai trò, phiên làm việc và nhật ký kiểm toán bất biến. Mọi thao tác nhạy cảm như hoàn tiền, hủy món đã gửi bếp, ghi đề giá, mở lại hóa đơn, thay đổi quyền truy cập hoặc điều chỉnh tồn kho thủ công đều cần được ghi nhận đầy đủ người thực hiện, thời điểm, giá trị trước, giá trị sau và lý do.

Notification Service xử lý việc ghi nhận và điều phối xác nhận đặt bàn, nhắc lịch, cảnh báo tồn kho, thông báo món sẵn sàng và thông báo thanh toán. Service này tiêu thụ event thay vì được gọi trực tiếp bởi mọi service nghiệp vụ, nhờ đó adapter kênh thông báo có thể thay đổi hoặc lỗi tạm thời mà không làm hỏng giao dịch chính.

1.6 Yêu cầu phi chức năng ở góc nhìn toàn dự án

Khác với ca sử dụng vốn mô tả hành vi theo từng kịch bản, yêu cầu phi chức năng của IRMS phải được nhìn ở mức **toàn dự án**. Chúng quyết định kiến trúc hướng dịch vụ được tổ chức ra sao, service nào cần ranh giới trách nhiệm riêng, dữ liệu nào cần đồng bộ mạnh và hệ quả nào có thể xử lý bất đồng bộ qua event. Vì vậy, mỗi yêu cầu dưới đây đều được gắn với **số liệu mục tiêu, điều kiện áp dụng và cách đáp ứng trong kiến trúc**.

Bảng 3: Yêu cầu phi chức năng của IRMS ở góc nhìn toàn dự án

Thuộc tính	Mục tiêu đo được	Điều kiện áp dụng	Cách đáp ứng trong kiến trúc
Hiệu năng & độ phản hồi	Tra cứu thực đơn và trạng thái bàn có thời gian phản hồi dưới 1 giây; thao tác xác nhận đơn gọi món và gửi bếp có thời gian phản hồi dưới 2 giây ở Điều kiện 1 và dưới 3 giây ở Điều kiện 2; phiếu phải hiển thị trên màn hình bếp không quá 2 giây sau khi đơn được xác nhận.	Áp dụng cho toàn bộ giờ mở cửa, đặc biệt tại màn hình gọi món, màn hình bếp và quầy thu ngân là ba điểm không được gây nghẽn thao tác người dùng.	Tách riêng các miền Ordering, Kitchen Coordination và Billing để giảm cạnh tranh tài nguyên; dùng mô hình đọc hoặc bộ nhớ đệm cho thực đơn và trạng thái thường đọc; giữ các tác vụ trên tuyến giao dịch nóng ở dạng gọi đồng bộ ngắn, còn báo cáo và cảnh báo chuyển sang xử lý nền.
Độ tin cậy & toàn vẹn giao dịch	Không làm mất đơn gọi món đã xác nhận; tỷ lệ tạo phiếu trùng hoặc ghi nhận thanh toán trùng phải thấp hơn 0.1%; khôi phục sau lỗi một nút ứng dụng trong vòng tối đa 10 phút.	Áp dụng ở cả Điều kiện 1 và Điều kiện 2; vẫn phải giữ đúng khi mạng nội bộ gián đoạn ngắn hạn dưới 60 giây hoặc khi đối tác thanh toán phản hồi chậm.	Dùng giao dịch cục bộ theo dịch vụ, khóa chống xử lý lặp cho gọi món, thanh toán và hoàn tiền, khóa lạc quan khi cập nhật trạng thái, cùng hộp thư ra và bộ truyền sự kiện cho sự kiện miền để tránh vừa mất dữ liệu vừa xử lý lặp.

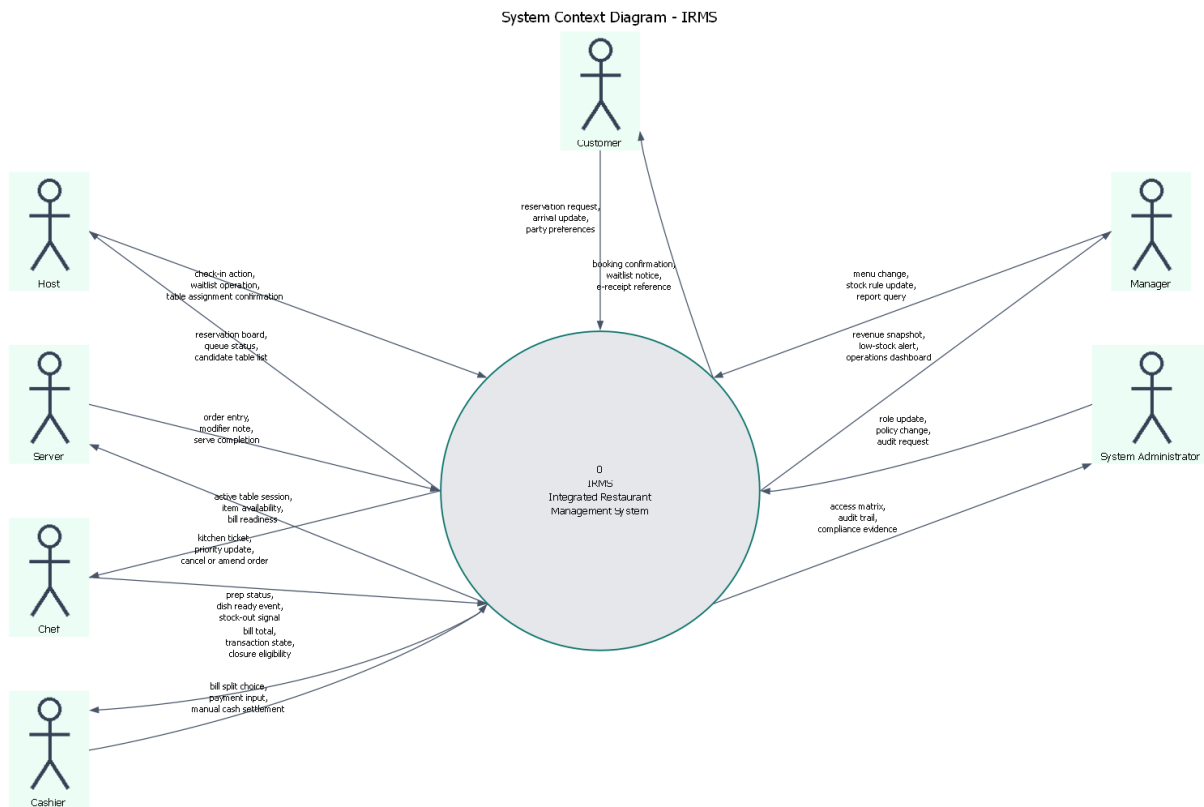
Thuộc tính	Mục tiêu đo được	Điều kiện áp dụng	Cách đáp ứng trong kiến trúc
Khả năng mở rộng	Hệ thống phải chịu được tối thiểu 1.5 lần tải của Điều kiện 2 trong các đợt cao điểm ngắn 10–15 phút mà không cần thiết kế lại; riêng các miền Ordering, Kitchen Coordination và Billing phải có thể tăng số phiên bản chạy độc lập khi tải tăng, báo cáo và phân tích không được kéo chậm tuyến giao dịch nóng.	Áp dụng cho bữa trưa, bữa tối, ngày cuối tuần hoặc các đơn đặt nhóm lớn	Chọn kiến trúc tách miền nghiệp vụ rõ ràng để phân bổ tải theo nhu cầu thực tế; mở rộng theo chiều ngang các miền có nhu cầu cao; cô lập báo cáo bằng mô hình đọc và ảnh chụp dữ liệu để truy vấn phân tích không tranh chấp với cơ sở dữ liệu giao dịch.
Bảo mật & khả năng kiểm toán	100% thao tác hoàn tiền, hủy món đã gửi bếp, ghi đề giá, mở lại hóa đơn và thay đổi quyền phải có nhật ký người thực hiện, thời điểm, giá trị trước, giá trị sau và lý do; phiên thu ngân hoặc quản lý tự khóa sau 15 phút không hoạt động; hệ thống không lưu dữ liệu thẻ gốc.	Áp dụng liên tục trong toàn bộ vòng đời vận hành và đặc biệt quan trọng ở các ca đổi nhân sự, bàn giao ca hoặc xử lý khiếu nại sau thanh toán.	Dùng IAM tập trung, phân quyền theo vai trò, chính sách kiểm tra quyền trong từng dịch vụ, nhật ký kiểm toán bất biến cho hành động nhạy cảm, và tích hợp thanh toán qua đối tác ngoài để giới hạn phạm vi dữ liệu nhạy cảm nằm trong IRMS.
Khả năng quan sát & vận hành	100% yêu cầu ghi dữ liệu phải mang mã liên kết xuyên suốt; cảnh báo mức tồn thấp phải xuất hiện trong vòng 60 giây sau khi vượt ngưỡng; bảng điều khiển vận hành có độ trễ dữ liệu không quá 5 phút; lỗi ở cổng vào hoặc PaymentGateway phải sinh cảnh báo vận hành trong vòng 1 phút.	Áp dụng cho cả thời gian phục vụ và giai đoạn chốt ca, khi quản lý cần đối chiếu chênh lệch đơn gọi món, hóa đơn, tồn kho và thao tác hoàn tiền.	Ghi nhật ký tập trung theo dịch vụ, truy vết xuyên suốt bằng mã liên kết, thu thập số đo cho độ trễ phiếu và thanh toán, tổng hợp bảng điều khiển từ mô hình đọc, cùng với dấu thời gian sự kiện để có thể truy vết một đơn từ lúc tạo đến lúc đóng hóa đơn.
Khả năng bảo trì & mở rộng	Khi thêm một phương thức thanh toán hoặc một kênh thông báo mới, thay đổi chủ yếu chỉ nằm ở bộ kết nối, cấu hình và kiểm thử tích hợp; thay đổi thực đơn, giá, khuyến mãi hoặc ngưỡng mức tồn thấp không được buộc sửa chéo bộ điều khiển, thanh toán và tồn kho cùng lúc.	Áp dụng trong suốt vòng đời dự án, đặc biệt khi nhà hàng đổi chính sách giá, khuyến mãi theo mùa hoặc cần thí điểm thêm kênh thanh toán mới.	Phân rã theo miền nghiệp vụ, áp dụng nguyên lý SOLID và đảo ngược phụ thuộc cho cổng ngoài, tách lớp điều phối ứng dụng khỏi quy tắc miền, chuẩn hóa hợp đồng giữa các dịch vụ để mỗi thay đổi được khu trú trong biên chức năng tương ứng.

2 Mô hình hóa hệ thống

2.1 Danh mục use cases

Phần mô hình hóa này đóng vai trò cầu nối giữa Chương 1 và chương kiến trúc. Mục tiêu không chỉ là liệt kê chức năng, mà là khóa ba vấn đề tiền kiến trúc của IRMS: **biên hệ thống nằm ở đâu, những năng lực nghiệp vụ nào sẽ trở thành các miền trách nhiệm chính, và những luồng đầu-cuối nào phải được bảo vệ khi lựa chọn**

cấu trúc phần mềm. Với hướng đọc service-based, ba mô hình mở đầu được dùng để chốt ranh giới hệ thống, chốt các nhóm use case sẽ ánh xạ thành domain service, và chốt các điểm cần phối hợp đồng bộ hay bất đồng bộ. Việc đọc ba hình này cũng giúp phần đặc tả use cases phía sau bám sát toàn hệ thống thay vì bị nhìn như các kịch bản rời rạc; phần định vị và biện minh tương ứng được nối trực tiếp về Bảng 36 và Bảng 37 trong Phụ lục A.2 và Phụ lục A.3.



Hình 1: Sơ đồ ngữ cảnh của IRMS

Hình 1 khóa rõ biên hệ thống theo đúng tinh thần *context/process diagram*: IRMS được đặt ở trung tâm như một hệ thống điều phối vận hành, còn các vai trò con người nằm ngoài ranh giới này. Giá trị của hình không nằm ở việc liệt kê actor, mà ở việc chỉ rõ các hợp đồng trao đổi dữ liệu giữa IRMS với người dùng bên ngoài trước khi bàn đến cấu trúc bên trong. Từ ranh giới này, phần bên trong mới có thể được phân rã thành các dịch vụ cốt lõi:

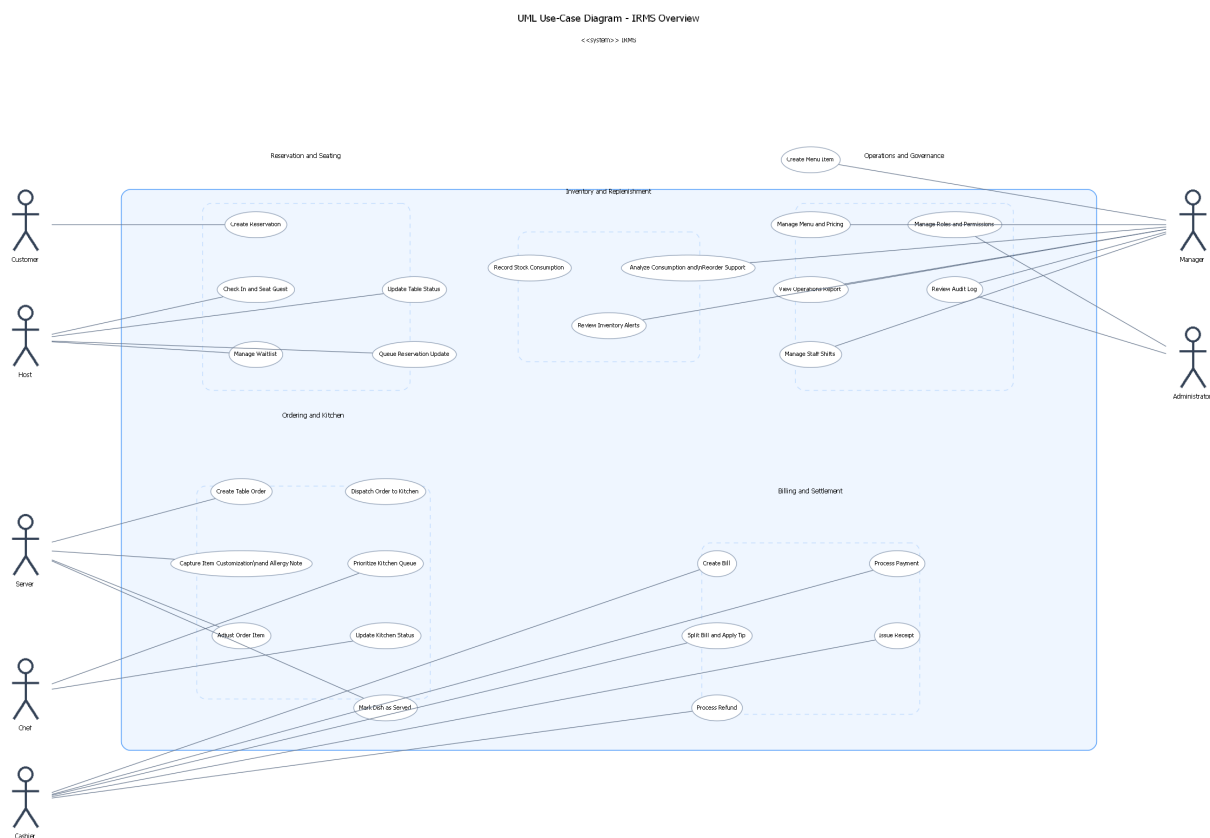
- Reservation & Seating Service
- Ordering & Menu Service
- Kitchen Coordination Service
- Billing & Settlement Service
- Inventory Monitoring Service
- Reporting Service
- IAM & Audit Service
- Notification Service

cùng các adapter biên đã hiện thực trong code như `PaymentGateway`, `NotificationChannelAdapter` và `ReceiptDeliveryAdapter`.

Các điểm cần đọc từ sơ đồ ngữ cảnh.

- **Các vai trò vận hành trực tiếp** gồm Customer, Host / Reception, Server / Floor Staff, Kitchen Operations, Cashier, Manager và System Administrator. Đây là những điểm chạm sinh ra tải tương tác và quyết định yêu cầu phản hồi của hệ thống.
- **Các vai trò không phải hệ thống ngoài:** repo hiện tại chưa có bằng chứng cấu hình cho công thanh toán, nhà cung cấp thông báo hay máy in biên lai bên ngoài, nên sơ đồ chỉ thể hiện các actor con người và giữ các adapter đó ở bên trong IRMS.
- **Các luồng đều có tên gọi nghiệp vụ rõ ràng** như “reservation request”, “kitchen ticket”, “transaction state” hay “audit trail”. Điều này giúp khóa trách nhiệm của IRMS ở mức trao đổi dữ liệu, không chỉ ở mức vai trò sử dụng.
- **Ranh giới trách nhiệm được làm rõ từ sớm:** IRMS chịu trách nhiệm điều phối đặt bàn, gọi món, bếp, hóa đơn, tồn kho và quản trị; các chi tiết như thanh toán, thông báo và biên lai được xử lý qua service hoặc adapter nội bộ đã có trong code.

Vì vậy, sơ đồ ngữ cảnh là nền cho mọi quyết định kiến trúc phía sau: chỉ khi biên hệ thống được khóa rõ thì việc phân rã thành dịch vụ, thành phần hay kênh tích hợp ở Mục ?? mới có cơ sở vững chắc.



Hình 2: Sơ đồ tổng quan use cases của IRMS

Hình 2 trình bày tập use cases ở mức bao quát. Để giữ sơ đồ đủ khả năng đọc nhưng vẫn bám sát đặc tả use case, các use cases được gom theo các cụm lớn bám theo dòng giá trị nghiệp vụ thay vì triển khai toàn bộ chi tiết mức đặc tả. Ở mức overview, hình nhấn mạnh các khối Reservation and Seating, Ordering and Kitchen, Billing and Settlement, Inventory and Replenishment

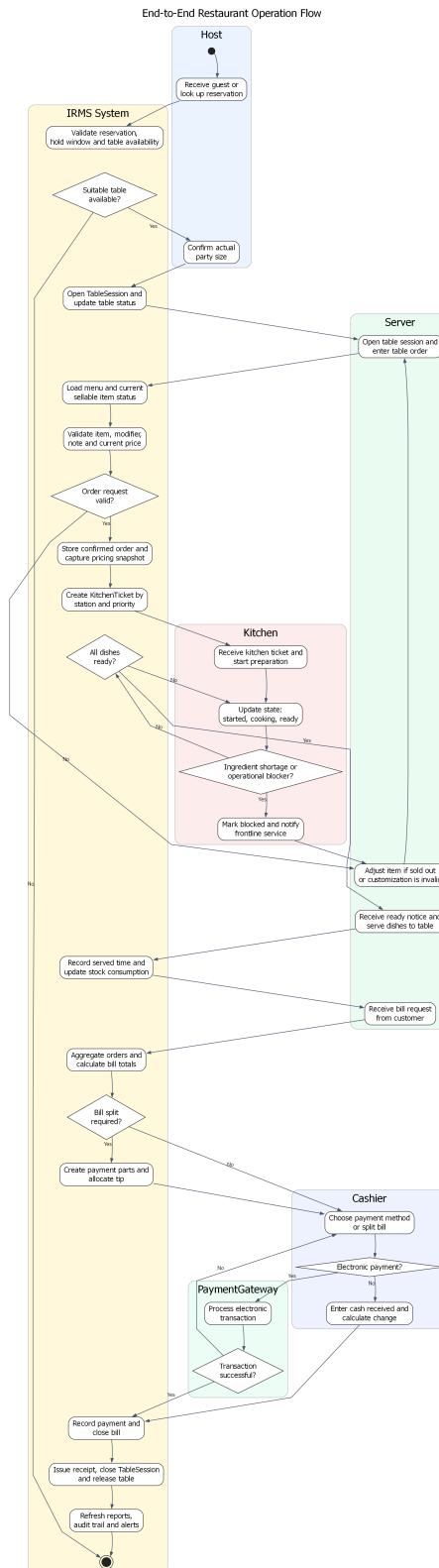
Support và Operations and Governance. Cách nhóm này cũng là cơ sở trực tiếp để ánh xạ sang các domain service ở chương kiến trúc.

Nhìn dưới góc độ kiến trúc, sơ đồ tổng quan use cases không chỉ trả lời “hệ thống có những chức năng gì”, mà còn chỉ ra “chức năng nào thuộc một miền dịch vụ riêng”, “chức năng nào nằm trên đường giao dịch nóng”, và “chức năng nào phù hợp hơn với luồng phản ứng theo sự kiện”. Đây là bước trung gian quan trọng trước khi ánh xạ các năng lực này sang logic view, module view và component-and-connector view ở chương sau.

Các cụm năng lực cần đọc từ sơ đồ.

- **Reservation and Seating:** bao phủ đặt bàn, tiếp nhận khách, danh sách chờ, giữ chỗ và mở phiên phục vụ tại bàn; đây là nền cho Reservation & Seating Service.
- **Ordering and Kitchen:** bao phủ tạo đơn, điều phối phiếu bếp, cập nhật tiến độ chế biến, xử lý ưu tiên và xác nhận món đã phục vụ; đây là phần use case trung tâm của Ordering & Menu Service và Kitchen Coordination Service.
- **Billing and Settlement:** bao phủ tạo hóa đơn, tách hóa đơn, xử lý thanh toán, hoàn tiền và phát hành biên lai; đây là phạm vi của Billing & Settlement Service.
- **Inventory and Replenishment Support:** bao phủ ghi nhận tiêu hao, cảnh báo mức tồn thấp và hỗ trợ quyết định nhập thêm nguyên liệu từ dữ liệu lịch sử; đây là phạm vi của Inventory Monitoring Service.
- **Governance and Analytics:** bao phủ quản trị thực đơn, phân quyền, kiểm toán, báo cáo và điều hành vận hành; đây là phạm vi của Reporting Service và IAM & Audit Service.

Việc gom theo cụm ở mức tổng quan giúp hình giữ được khả năng đọc, còn phần chi tiết của từng mã usecases được triển khai ngay sau đó. Cách trình bày này cũng tạo nhịp chuyển trực tiếp sang các góc nhìn logic và mô-đun, nơi các cụm nghiệp vụ này được dùng như ranh giới trách nhiệm của từng service.



Hình 3: Sơ đồ hoạt động của luồng vận hành đầu-cuối nhà hàng

Hình 3 mô tả một vòng vận hành điển hình của nhà hàng, từ tiếp nhận khách đến giải phóng bàn sau thanh toán. Giá trị quan trọng nhất của hình là nó chỉ ra những trạng thái nào phải được quản lý như các mốc nghiệp vụ có hậu quả rõ ràng: một bàn chỉ được có một TableSession đang mở, một đơn đã xác nhận phải sinh phiếu cho bếp, và một hóa đơn chỉ được đóng sau khi thanh toán được ghi nhận hợp lệ.

Ở góc nhìn kiến trúc, sơ đồ này cũng tách rõ **đường giao dịch nóng** khỏi **đường hỗ trợ**. Các thao tác như xác nhận đơn, cập nhật trạng thái món sẵn sàng, ghi nhận thanh toán và đóng phiên bản là các điểm phải phản hồi dứt khoát qua gọi đồng bộ giữa các biên phù hợp. Ngược lại, các tác vụ như làm mới báo cáo, phát cảnh báo mức tồn thấp, đồng bộ trạng thái phụ trợ hay gửi thông báo có thể đi trên tuyến bất đồng bộ theo hướng event-driven nếu điều đó không làm sai trạng thái phục vụ trước mặt khách hàng.

Những mắt xích kiến trúc được khóa bởi sơ đồ đầu-cuối.

- **Điểm bàn giao giữa các khu vực:** lễ tân mở phiên bản, phục vụ xác nhận đơn, bếp cập nhật trạng thái, thu ngân quyết toán; mỗi điểm bàn giao đều gắn với một thay đổi trạng thái có hậu quả nghiệp vụ.
- **Điểm đồng bộ bắt buộc:** mở TableSession, xác nhận đơn, đánh dấu món sẵn sàng, ghi nhận thanh toán và giải phóng bàn không được nhập nhằng về trạng thái.
- **Điểm bất đồng bộ chấp nhận được:** thông báo, báo cáo và cảnh báo mức tồn thấp có thể được đẩy ra sau với điều kiện không làm sai các trạng thái phục vụ cốt lõi.
- **Điểm nối sang chương kiến trúc:** hình này là cơ sở để kiểm tra xem các sơ đồ logic, phân rã dịch vụ, thành phần và triển khai ở chương sau có thật sự bảo vệ được luồng vận hành đầu-cuối hay không.

Bảng 4: Danh mục 26 use cases chính được đặc tả chi tiết

Nhóm	Mã use case
Nhóm đặt bàn và bàn ăn	UC-RES-01, UC-RES-02, UC-RES-03, UC-RES-04, UC-RES-05
Nhóm gọi món và điều phối bếp	UC-ORD-01, UC-ORD-02, UC-ORD-03, UC-KIT-01, UC-KIT-02, UC-KIT-03, UC-ORD-04
Nhóm thanh toán	UC-BIL-01, UC-BIL-02, UC-PAY-01, UC-PAY-02, UC-PAY-03
Nhóm tồn kho, cảnh báo và hỗ trợ nhập hàng	UC-INV-01, UC-INV-02, UC-INV-03
Nhóm quản trị, kiểm toán và phân tích	UC-REP-01, UC-ADM-01, UC-ADM-02, UC-ADM-03, UC-ADM-04, UC-OPS-01

Năm use cases giữ vai trò khóa kín phạm vi ở phần này là UC-RES-04, UC-RES-05, UC-KIT-03, UC-INV-03 và UC-ADM-03. Chúng làm rõ các mắt xích dễ bị xem nhẹ khi mô tả hệ thống nhà hàng ở mức khái quát: kết thúc vòng đời bàn sau thanh toán, giao tiếp chủ động với khách hàng, điều phối ưu tiên trong bếp, hỗ trợ quyết định nhập hàng từ dữ liệu tiêu hao lịch sử và truy vết thao tác nhạy cảm ở tầng quản trị.

2.2 Góc nhìn sử dụng

Góc nhìn sử dụng được dùng như bộ kịch bản kiểm chứng kiến trúc. Trong các tình huống chuỗi tương tác logic quá dài sẽ gây ra tình huống khó quản lý, vận hành. Bốn kịch bản dưới đây được chọn vì chạm vào bốn mối quan tâm kiến trúc lớn của IRMS: tính đúng đắn khi gán bàn, độ đáp ứng của luồng gọi món, độ tin cậy của thanh toán, và khả năng cô lập các luồng hỗ trợ như tồn kho và cảnh báo. Đồng thời, đây cũng là bốn kịch bản thể hiện rõ nhất điểm nào nên sử dụng API đồng bộ và điểm nào nên phối hợp theo sự kiện nội bộ.

2.2.1 Kịch bản 1: Từ đặt bàn đến tiếp nhận và xếp bàn

1. Khách hàng hoặc lễ tân tạo đặt bàn.

2. Reservation & Seating Service kiểm tra khả dụng, gán bàn hoặc giữ chỗ và ghi nhận ReservationCreated nếu cần phát thông báo.
3. Khi khách đến, Host UI xác nhận tiếp nhận và mở TableSession.
4. Trạng thái bàn được cập nhật qua TableStatusChanged để tuyến phục vụ nhìn thấy bàn sẵn sàng nhận order.

2.2.2 Kịch bản 2: Từ gọi món đến điều phối bếp và phục vụ

1. Phục vụ tạo đơn gọi món và cấu hình tùy chọn món hoặc gợi ý đi ứng trên Server POS UI.
2. Ordering & Menu Service xác nhận đơn gọi món và phát OrderPlaced.
3. Kitchen Coordination Service nhận order, tách phiếu theo trạm bếp và hiển thị lên Kitchen Display UI.
4. Bếp cập nhật trạng thái từng dòng phiếu qua các mốc như OrderItemStarted và OrderReady; màn hình gọi món nhận lại trạng thái sẵn sàng.
5. Phục vụ giao món và đánh dấu đã phục vụ; Inventory Monitoring Service nhận sự kiện liên quan để ghi nhận tiêu hao và cập nhật kho.

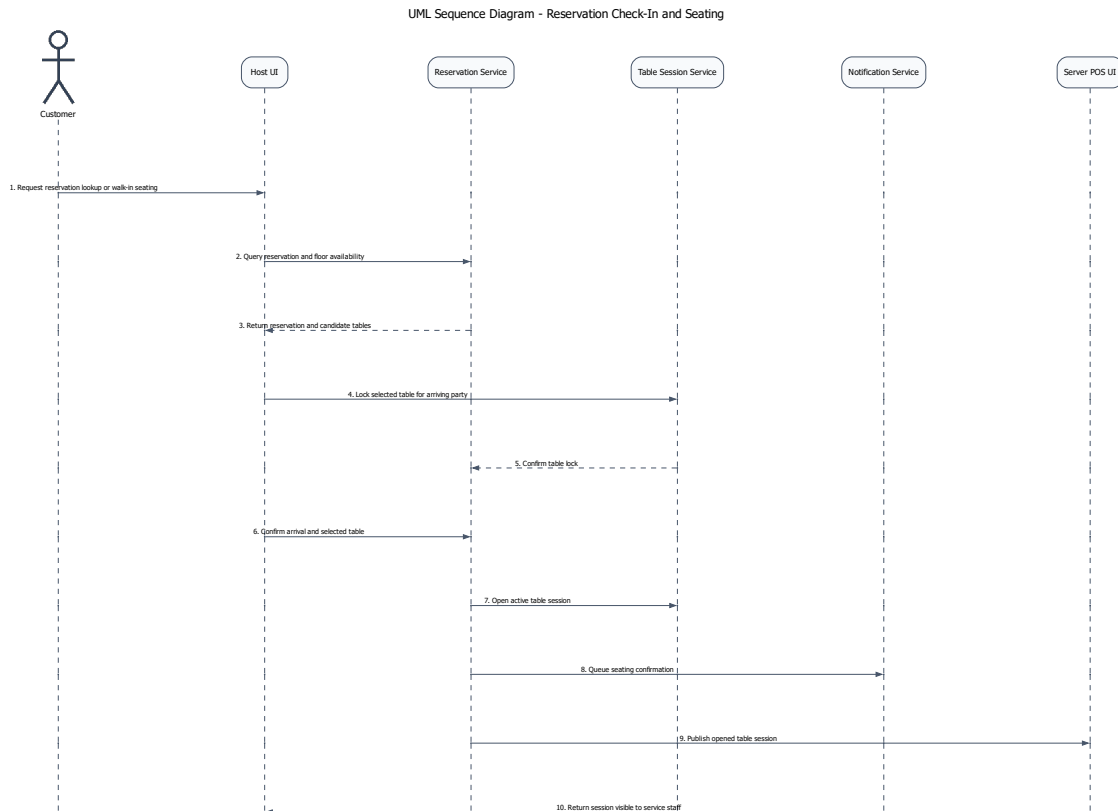
2.2.3 Kịch bản 3: Từ hóa đơn đến thanh toán và biên lai

1. Thu ngân hoặc phục vụ gom các đơn gọi món của bàn thành hóa đơn.
2. Billing & Settlement Service áp dụng khuyến mãi, tiền boa và tách hóa đơn nếu cần.
3. Thanh toán được thực hiện qua tiền mặt hoặc qua PaymentGateway đã hiện thực trong code.
4. Khi thanh toán đủ, hệ thống ghi nhận PaymentCompleted, phát hành biên lai và chuyển TableSession sang chuẩn bị đóng.

2.2.4 Kịch bản 4: Từ cảnh báo tồn kho thấp đến quyết định của quản lý

1. Inventory Monitoring Service cập nhật lượng tồn kho thực tế sau mỗi món đã phục vụ hoặc sau mỗi lần điều chỉnh thủ công.
2. Nếu xuống dưới ngưỡng, hệ thống phát StockLow.
3. Manager Console nhận cảnh báo, hiển thị nguyên liệu bị ảnh hưởng và món có nguy cơ hết hàng.
4. Quản lý quyết định nhập thêm hàng hoặc tạm ẩn món trong menu.

Giải thích chi tiết cho góc nhìn sử dụng. Mỗi kịch bản trong phần này được chọn để ép kiến trúc phải trả lời một câu hỏi cụ thể: trạng thái nào cần đồng bộ mạnh, nơi nào cần idempotency, nơi nào phải chặn lỗi từ adapter biên, và nơi nào có thể chấp nhận nhất quán trễ để bảo vệ tuyến giao dịch nóng.



Hình 4: Sơ đồ trình tự cho kịch bản từ đặt bàn đến tiếp nhận và xếp bàn

Hình 4 được chọn để kiểm tra một quyết định mô hình hóa quan trọng: Reservation và TableSession phải được tách thành hai khái niệm khác nhau. Reservation đại diện cho cam kết trước khi khách ngồi vào bàn; TableSession đại diện cho phiên phục vụ đã thực sự được mở trong vận hành. Việc tách này giúp hệ thống xử lý đúng các tình huống thường gặp như no-show, walk-in, đổi bàn do thay đổi sức chứa hoặc check-in muộn so với khung giữ chỗ.

Đánh đổi của cách mô hình hóa này là trạng thái phải được kiểm soát chặt hơn: đặt bàn có thể còn hiệu lực nhưng bàn chưa sẵn sàng; bàn có thể vừa được giải phóng nhưng vẫn cần dọn dẹp; cùng một yêu cầu tiếp nhận không được mở trùng hai phiên phục vụ. Vì vậy, đây là kịch bản dùng để kiểm tra nơi nào kiến trúc cần đồng bộ mạnh và nơi nào có thể chỉ cập nhật hiển thị.

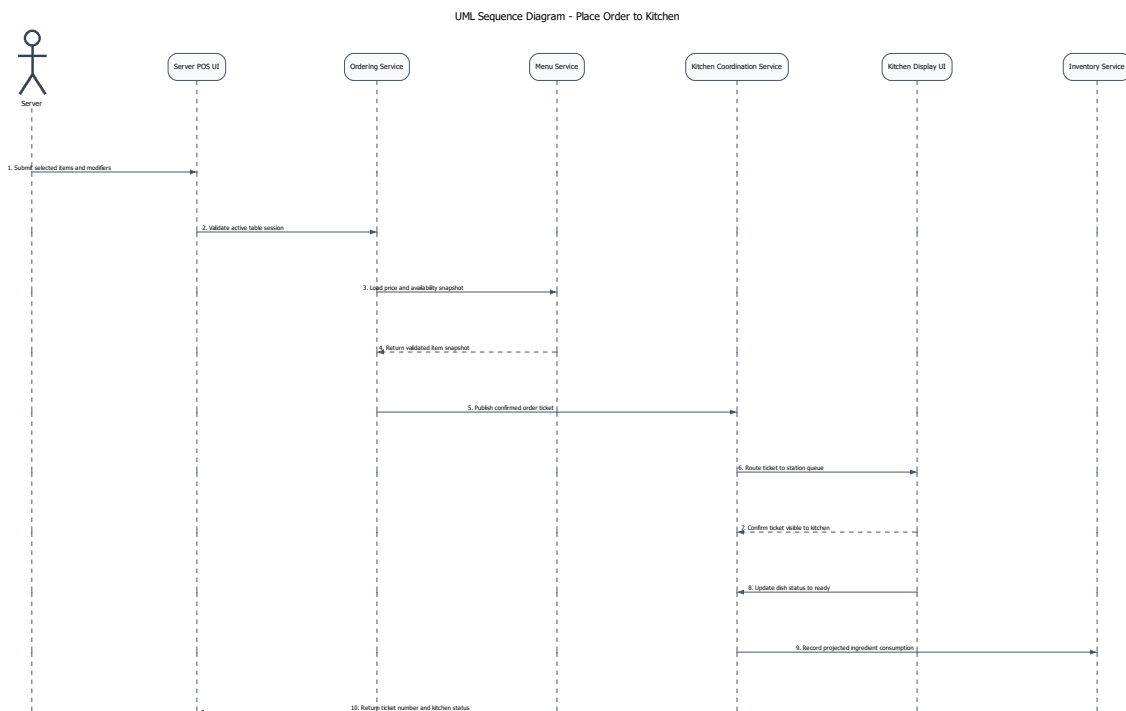
Các điểm kiến trúc đọc được từ sơ đồ.

- **Host UI và Reservation & Seating Service** chịu trách nhiệm kiểm tra khả dụng, giữ chỗ và xác nhận tiếp nhận khách.
- **Reservation & Seating Service** là ranh giới nơi bàn được chuyển từ trạng thái chờ sang trạng thái đang phục vụ thông qua TableSession.
- **Notification Service và Server POS UI** chỉ được kích hoạt sau khi phiên bàn đã được mở thành công, tránh để tuyến phục vụ nhìn thấy trạng thái nửa chừng.

Hàm ý đối với kiến trúc.

- Phải có cơ chế chống mở trùng TableSession cho cùng một bàn hoặc cùng một lượt tiếp nhận.

- Trạng thái bàn và trạng thái đặt bàn không nên bị ép gộp thành một thực thể duy nhất, nếu không các chuyển tiếp quan trọng của vận hành sẽ trở nên mơ hồ.



Hình 5: Sơ đồ trình tự cho kịch bản từ gọi món đến điều phối bếp và phục vụ

Hình 5 là kịch bản quan trọng nhất để kiểm chứng **khả năng đáp ứng** của IRMS. Nguyên tắc chủ đạo ở đây là *xác nhận đơn thành công trước, rồi mới lan truyền sang bếp và các luồng hỗ trợ*. Cách tổ chức này rút ngắn thời gian phản hồi tại màn hình gọi món và bảo vệ đơn đã chốt khi một thành phần phía sau như KDS hoặc đồng bộ trạng thái gặp trục trặc tạm thời.

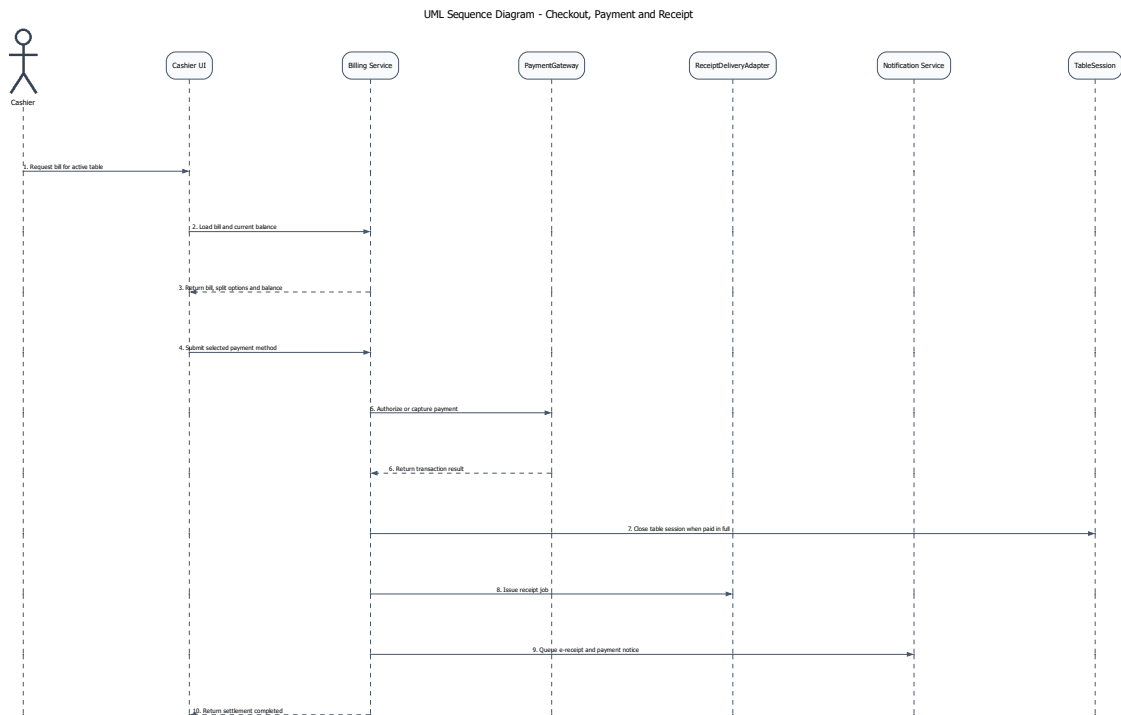
Đổi lại, hệ thống phải chấp nhận và quản lý các trạng thái trung gian có kiểm soát: đơn đã xác nhận nhưng phiếu chưa hiện ngay trên màn hình bếp; bếp đã cập nhật trạng thái nhưng POS chưa nhận bản sao mới nhất; tiêu hao nguyên liệu được ghi nhận sau sự kiện món đã phục vụ. Kịch bản này giúp chứng minh cho việc dùng nhất quán trễ ở những đoạn không trực tiếp quyết định trải nghiệm đặt món trước mặt khách hàng.

Các điểm kiến trúc đọc được từ sơ đồ.

- **Ordering & Menu Service** là ranh giới chốt giao dịch gọi món; đây là nơi không được làm mất đơn đã xác nhận.
- **Kitchen Coordination Service** và **Kitchen Display UI** xử lý phần lan truyền vận hành sau giao dịch, bao gồm định tuyến phiếu và phản hồi trạng thái chế biến.
- **Menu Service** lưu giữ trạng thái khả dụng và giá tại thời điểm xác nhận đơn, giúp tránh nhập nhằng khi menu thay đổi sau đó.
- **Inventory Monitoring Service** chỉ tham gia ở nhánh hỗ trợ, không nên chặn bước xác nhận đơn ở tuyến đầu.

Hàm ý đối với kiến trúc.

- Đơn gọi món, phiếu bếp và cập nhật trạng thái không nên bị ràng buộc trong cùng một bước đồng bộ dài.
- Cơ chế chống lặp, correlation ID và khả năng thử lại là bắt buộc nếu kiến trúc muốn vừa nhanh vừa không làm mất dữ liệu đơn.



Hình 6: Sơ đồ trình tự cho kịch bản từ hóa đơn đến thanh toán và biên lai

Hình 6 là kịch bản mà kiến trúc phải ưu tiên **độ tin cậy và tính toàn vẹn giao dịch**. Ở đây, Billing & Settlement Service không chỉ tính tổng cuối cùng mà còn điều phối khuyến mãi, tách hóa đơn, chọn phương thức thanh toán, gọi PaymentGateway, phát hành biên lai và đóng phiên bản. Sai sót ở luồng này kéo theo các hậu quả: ghi nhận thanh toán trùng, đóng sai hóa đơn hoặc thay đổi trạng thái bàn khi giao dịch chưa hoàn tất.

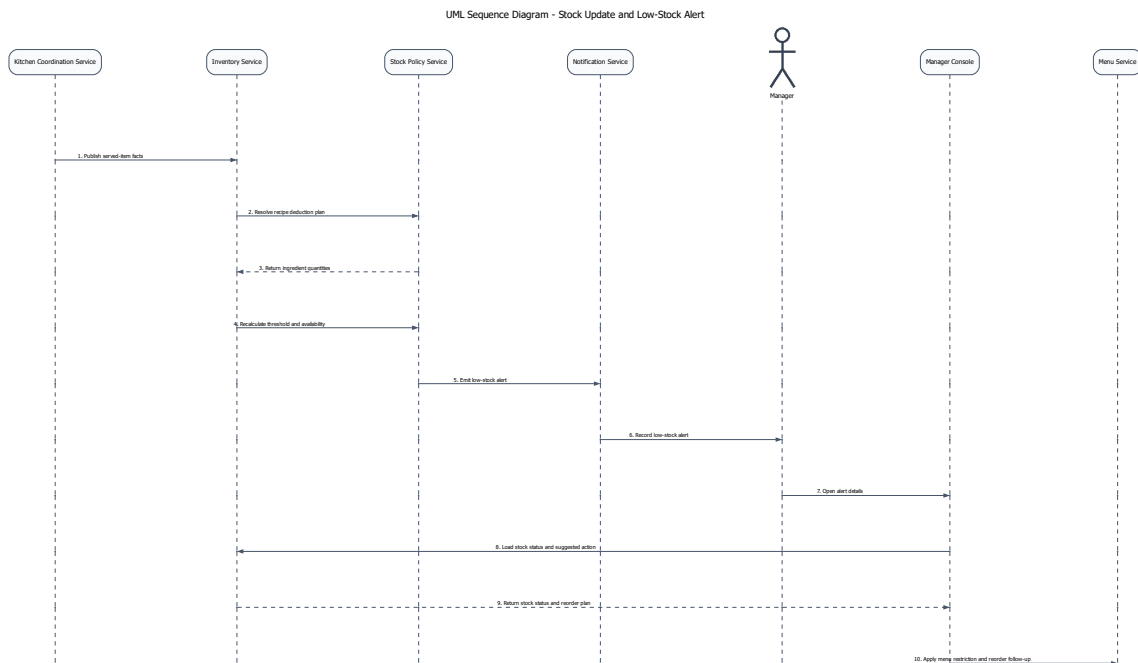
Vì vậy, thanh toán cần được giữ trong một tuyến đồng bộ đủ ngắn để hệ thống có thể trả lời rõ ràng giao dịch đã thành công hay chưa. Đánh đổi là luồng này trở nên nhạy cảm với độ trễ và lỗi của adapter biên; do đó, nó cần timeout rõ ràng, chống xử lý lặp, ghi vết kiểm toán và tách riêng adapter tích hợp để cô lập rủi ro.

Các điểm kiến trúc đọc được từ sơ đồ.

- **Billing & Settlement Service** giữ vai trò bộ điều phối của tuyến quyết toán và là nơi phải đảm bảo idempotency cho các thao tác nhạy cảm.
- **PaymentGateway** đại diện cho port thanh toán cần được cô lập khỏi logic quyết toán lỗi.
- **ReceiptDeliveryAdapter**, **Notification Service** và **TableSession** chỉ được kích hoạt sau khi kết quả thanh toán đã rõ ràng.

Hàm ý đối với kiến trúc.

- PaymentGateway không nên được phép chi phối trực tiếp trạng thái hóa đơn mà không đi qua lớp điều phối của hệ thống.
- Việc đóng TableSession, phát hành biên lai và gửi e-receipt phải phụ thuộc vào cùng một kết quả quyết toán đã được xác nhận.



Hình 7: Sơ đồ trình tự cho kịch bản từ cảnh báo mức tồn thấp đến quyết định của quản lý

Hình 7 cho thấy vì sao cập nhật tồn kho và phát cảnh báo phải được đặt trên một tuyến hỗ trợ thay vì đặt trực tiếp vào đường giao dịch nóng. Trong kịch bản này, Kitchen Coordination Service phát sinh dữ kiện món đã phục vụ, Inventory Monitoring Service ghi nhận tiêu hao, Stock Policy Service đánh giá ngưỡng và Notification Service chuyển cảnh báo đến quản lý. Cách tách này giúp thay đổi công thức món, ngưỡng cảnh báo hay chính sách ăn món mà không làm nặng thêm tuyến gọi món và thanh toán.

Đánh đổi là trạng thái tồn kho và cảnh báo quản trị có thể đến chậm hơn giao dịch mới nhất nếu luồng nền đang thử lại hoặc xử lý dồn hàng. Độ trễ này chấp nhận được, miễn là nó không đi cùng với các quyết định cần phản hồi tức thời như xác nhận đơn hay xác nhận thanh toán.

Các điểm kiến trúc đọc được từ sơ đồ.

- **Kitchen Coordination Service** là nguồn phát sinh dữ kiện phục vụ dùng để đưa ra tiêu hao.
- **Inventory Monitoring Service** và **Stock Policy Service** chịu trách nhiệm cập nhật tồn kho và đánh giá ngưỡng cảnh báo.
- **Notification Service, Manager Console** và **Menu Service** tạo thành tuyến phản ứng quản trị sau khi cảnh báo đã được phát ra.

Hàm ý đối với kiến trúc.

- Tồn kho và cảnh báo là đúng đắn về vận hành, nhưng không nên được ép vào cùng một bước phản hồi thời gian thực với gọi món hoặc thanh toán.

- Việc tách riêng tầng chính sách và tầng thông báo làm tăng khả năng thay đổi.

2.3 Đặc tả use cases

Use Case ID	UC-RES-01
Use Case Name	Tạo đặt bàn
Primary Actors	Khách hàng, Lễ tân
Description	Tạo một đặt bàn hợp lệ cho ngày, giờ và số lượng khách cụ thể.
Trigger	Khách hàng gửi yêu cầu đặt bàn qua quầy lễ tân, điện thoại hoặc web/app.
Preconditions	• Nhà hàng có mở nhận đặt bàn trong khung giờ yêu cầu. • Có tối thiểu tên và số điện thoại khách hàng. • Sơ đồ bàn và năng lực phục vụ đã được cấu hình.
Postconditions	• Tạo đặt bàn với trạng thái Pending hoặc Confirmed. • Hệ thống giữ chỗ hoặc gợi ý bàn phù hợp. • Khách nhận mã đặt bàn và thông báo xác nhận.
Normal Flow	1. Lễ tân nhập thời gian, số khách, tên và số điện thoại. 2. Hệ thống kiểm tra và validate dữ liệu đầu vào. 3. Hệ thống kiểm tra sức chứa, bàn trống và xung đột đặt bàn. 4. Hệ thống gợi ý bàn hoặc nhóm bàn phù hợp. 5. Lễ tân chọn bàn và nhập ghi chú nếu có. 6. Hệ thống tạo đặt bàn và sinh mã đặt bàn. 7. Hệ thống gửi thông báo xác nhận cho khách.
Alternative Flows	A1. Không đủ sức chứa (từ bước 3) 3.1 Hệ thống đề xuất khung giờ khác hoặc đưa khách vào danh sách chờ. A2. Yêu cầu đặc biệt (từ bước 4) 4.1 Lễ tân lọc danh sách bàn theo khu vực hoặc thuộc tính yêu cầu. A3. Không gán được bàn (từ bước 5) 5.1 Hệ thống tạo đặt bàn trạng thái Pending và giữ chỗ theo số lượng khách.
Exception Flows	E1. Dữ liệu không hợp lệ (từ bước 2) 2.1 Hệ thống yêu cầu nhập lại thông tin khách. E2. Lỗi đồng bộ sơ đồ bàn (từ bước 3) 3.2 Hệ thống khóa thao tác và yêu cầu tải lại dữ liệu.

Business Rules	• Không vượt quá sức chứa tối đa của bàn hoặc nhóm bàn. • Nhóm khách lớn phải được xác nhận bởi lễ tân hoặc quản lý.
-----------------------	---

Use Case ID	UC-RES-02
Use Case Name	Nhận khách và sắp bàn
Primary Actors	Lễ tân, Phục vụ
Description	Tiếp nhận khách đến nhà hàng và gán khách vào bàn hoặc nhóm bàn phù hợp.
Trigger	Khách đến quầy lễ tân hoặc được gọi từ danh sách chờ.
Preconditions	• Khách có đặt bàn hoặc có thể được xếp chỗ tại chỗ. • Sơ đồ bàn và trạng thái bàn được đồng bộ theo thời gian thực.
Postconditions	• Một phiên phục vụ bàn (TableSession) được mở. • Bàn được chuyển sang trạng thái Occupied hoặc Reserved-Arrived. • Phục vụ nhận được thông báo tiếp nhận bàn mới.
Normal Flow	1. Lễ tân tìm đặt bàn theo mã, số điện thoại hoặc tên khách. 2. Hệ thống hiển thị thông tin đặt bàn, trạng thái, thời điểm đến và bàn gợi ý. 3. Lễ tân xác nhận khách đến, số khách thực tế và thời gian check-in. 4. Hệ thống kiểm tra hiệu lực đặt bàn và trạng thái bàn. 5. Lễ tân điều chỉnh hoặc ghép bàn nếu số lượng khách thay đổi. 6. Hệ thống mở phiên phục vụ (TableSession). 7. Hệ thống cập nhật trạng thái bàn thành Occupied hoặc Reserved-Arrived. 8. Hệ thống phân công bàn cho phục vụ theo khu vực. 9. Phục vụ nhận thông báo và chuẩn bị phục vụ.
Alternative Flows	A1. Khách vắng lại (từ bước 1) 1.1 Lễ tân chọn chức năng xếp bàn trực tiếp. 1.2 Hệ thống gợi ý bàn trống phù hợp. 1.3 Lễ tân chọn bàn và tiếp tục từ bước 5. A2. Thay đổi số lượng khách (từ bước 3) 3.1 Lễ tân cập nhật số khách thực tế. 3.2 Hệ thống gợi ý lại bàn phù hợp hoặc yêu cầu ghép bàn.

Exception Flows	E1. Bàn chưa sẵn sàng (từ bước 4) 4.1 Hệ thống đề xuất bàn thay thế hoặc đưa khách vào danh sách chờ. E2. Đặt bàn hết hiệu lực (từ bước 4) 4.2 Hệ thống yêu cầu xác nhận phục hồi từ lễ tân hoặc quản lý. E3. Lỗi đồng bộ sơ đồ bàn (từ bước 4) 4.3 Hệ thống tạm dừng thao tác và yêu cầu tải lại dữ liệu.
Business Rules	- Mỗi bàn chỉ có một TableSession hoạt động tại một thời điểm. - Không được gán bàn nếu bàn đang được sử dụng. - Thời gian giữ chỗ tối đa được cấu hình theo ca phục vụ.

Use Case ID	UC-RES-03
Use Case Name	Quản lý danh sách chờ
Primary Actors	Lễ tân
Description	Ghi nhận khách chờ bàn và điều phối khách khi có bàn phù hợp.
Trigger	Nhà hàng hết bàn phù hợp hoặc khách bị trì hoãn.
Preconditions	- Lễ tân có quyền truy cập danh sách chờ. - Hệ thống có khả năng ước tính thời gian chờ.
Postconditions	- Khách được thêm vào danh sách chờ với trạng thái Waiting. - Khách được gọi theo thứ tự ưu tiên khi có bàn phù hợp.
Normal Flow	- Lễ tân nhập thông tin khách, số lượng khách và kênh liên hệ. - Hệ thống ước tính thời gian chờ dựa trên tình trạng bàn. - Lễ tân xác nhận thêm khách vào danh sách chờ. - Khi có bàn trống, hệ thống tìm khách phù hợp theo sức chứa và ưu tiên. - Lễ tân chọn khách để gọi. - Hệ thống gửi thông báo và cập nhật trạng thái Called. - Lễ tân xác nhận khách đến và chuyển sang use case Nhận khách và sắp bàn.
Alternative Flows	A1. Ưu tiên khách (từ bước 4) 4.1 Lễ tân chọn khách có ưu tiên cao hơn (VIP, đặt trước) thay vì theo FIFO. A2. Cập nhật thông tin (từ bước 3) 3.1 Lễ tân cập nhật số điện thoại, số lượng khách hoặc hủy yêu cầu.

Exception Flows	E1. Khách không phản hồi (từ bước 6) 6.1 Nếu khách không phản hồi trong thời gian quy định, hệ thống chuyển trạng thái sang Skipped. E2. Thời gian chờ thay đổi (từ bước 2) 2.1 Nếu thời gian chờ tăng đột biến, hệ thống yêu cầu lễ tân xác nhận lại trước khi hiển thị.
Business Rules	- Thứ tự mặc định là FIFO trong cùng nhóm sức chứa. - Có thể override thứ tự theo ưu tiên. - Chỉ gọi khách khi có bàn phù hợp với số lượng và yêu cầu. - Thời gian giữ lượt gọi được cấu hình (timeout).

Use Case ID	UC-RES-04
Use Case Name	Cập nhật trạng thái bàn và giải phóng bàn
Primary Actors	Lễ tân, Phục vụ, Thu ngân
Description	Cập nhật trạng thái bàn, đóng phiên phục vụ và giải phóng bàn cho lượt khách tiếp theo.
Trigger	Khách rời bàn, hoàn tất phục vụ hoặc cần thay đổi trạng thái bàn.
Preconditions	- Bàn tồn tại trong hệ thống. - Người thao tác có quyền cập nhật trạng thái. - Nếu có TableSession, hệ thống truy xuất được trạng thái hóa đơn.
Postconditions	- Trạng thái bàn được cập nhật nhất quán. - TableSession được đóng nếu đủ điều kiện. - Lịch sử thay đổi được ghi nhận.
Normal Flow	- Người dùng chọn bàn trên sơ đồ bàn. - Hệ thống hiển thị trạng thái bàn, phiên phục vụ và hóa đơn liên quan. - Người dùng chọn trạng thái đích (Cleaning, Available, OutOfService). - Nếu chuyển sang trạng thái giải phóng, hệ thống kiểm tra hóa đơn hoặc split còn mở. - Nếu tất cả hóa đơn đã hoàn tất, hệ thống đóng TableSession và chuyển bàn sang Cleaning. - Phục vụ xác nhận đã dọn bàn xong. - Hệ thống chuyển bàn sang trạng thái Available. - Hệ thống cập nhật danh sách chờ và gợi ý khách tiếp theo nếu có.

Alternative Flows	A1. Chuyển sang OutOfService (từ bước 3) 3.1 Người dùng chuyển bàn sang trạng thái OutOfService để bảo trì hoặc khóa khu vực. A2. Chuyển bàn trong khi phục vụ (từ bước 3) 3.2 Nếu khách đổi bàn, hệ thống cập nhật bàn mới nhưng giữ nguyên TableSession.
Exception Flows	E1. Chưa thanh toán (từ bước 4) 4.1 Hệ thống từ chối giải phóng bàn và yêu cầu hoàn tất thanh toán. E2. Xung đột cập nhật (từ bước 2) 2.1 Hệ thống phát hiện cập nhật đồng thời và yêu cầu tải lại dữ liệu. E3. Không đủ quyền (từ bước 3) 3.3 Hệ thống chặn thao tác nếu người dùng không có quyền.
Business Rules	- Chỉ chuyển sang Available khi không còn TableSession và hóa đơn mở. - Mọi thay đổi trạng thái phải được ghi log (thời gian, người thực hiện). - Không được giải phóng bàn khi còn công nợ.

Use Case ID	UC-RES-05
Use Case Name	Gửi thông báo cho khách hàng
Primary Actors	Hệ thống, Lễ tân
Description	Gửi thông báo cho khách hàng khi có sự kiện như đặt bàn, thay đổi hoặc đến lượt chờ.
Trigger	Sự kiện nghiệp vụ phát sinh hoặc lễ tân kích hoạt gửi thủ công.
Preconditions	- Có thông tin liên hệ hợp lệ. - Mẫu thông báo và kênh gửi đã được cấu hình. - Hệ thống có kênh thông báo nội bộ qua NotificationChannelAdapter.
Postconditions	- Tạo bản ghi NotificationMessage. - Thông báo được gửi hoặc đưa vào hàng chờ. - Lịch sử gửi được lưu lại.

Normal Flow	1. Hệ thống nhận sự kiện cần gửi thông báo. 2. Hệ thống xác định loại thông báo và chọn mẫu nội dung phù hợp. 3. Hệ thống kiểm tra thông tin liên hệ và kênh gửi ưu tiên. 4. Hệ thống cá nhân hóa nội dung thông báo. 5. Hệ thống tạo NotificationMessage và gửi qua kênh đã chọn. 6. Hệ thống ghi nhận kết quả và cập nhật trạng thái gửi.
Alternative Flows	A1. Gửi thử công (từ bước 1) 1.1 Lễ tân kích hoạt gửi lại thông báo cho khách. A2. Chuyển kênh gửi (từ bước 3) 3.1 Nếu kênh chính không khả dụng, hệ thống chuyển sang kênh dự phòng.
Exception Flows	E1. Thiếu thông tin liên hệ (từ bước 3) 3.2 Hệ thống đánh dấu cần xử lý thủ công. E2. Lỗi dịch vụ gửi (từ bước 5) 5.1 Hệ thống đưa thông báo vào hàng chờ retry. E3. Bị chặn spam (từ bước 5) 5.2 Hệ thống bỏ qua gửi trùng và ghi nhận lý do.
Business Rules	• Mỗi thông báo phải liên kết với Reservation hoặc WaitlistEntry. • Thông báo có thời hạn phải kèm thời gian phản hồi rõ ràng. • Không gửi trùng thông báo trong khoảng thời gian cấu hình.

Use Case ID	UC-ORD-01
Use Case Name	Tạo đơn gọi món tại bàn
Primary Actors	Phục vụ
Description	Tạo đơn gọi món cho một bàn đang phục vụ, bao gồm món, số lượng và ghi chú.
Trigger	Phục vụ chọn chức năng tạo đơn gọi món trên bàn đang hoạt động.
Preconditions	• TableSession đang hoạt động. • Phục vụ có quyền thao tác trên khu vực.
Postconditions	• Đơn gọi món được lưu với trạng thái Draft hoặc Confirmed. • Các dòng món được liên kết với phiên phục vụ.

Normal Flow	1. Phục vụ mở bàn và chọn chức năng tạo đơn gọi món. 2. Hệ thống kiểm tra trạng thái TableSession. 3. Hệ thống hiển thị thực đơn và danh sách món. 4. Phục vụ chọn món, số lượng và ghi chú cho từng dòng món. 5. Hệ thống tính tạm tính theo giá hiện hành. 6. Phục vụ lưu đơn ở trạng thái Draft hoặc xác nhận gửi bếp. 7. Hệ thống lưu đơn gọi món và cập nhật trạng thái.
Alternative Flows	A1. Tạo nhiều đơn (từ bước 1) 1.1 Phục vụ tạo nhiều đơn gọi món liên tiếp trong cùng một phiên phục vụ. A2. Dừng mẫu đơn (từ bước 3) 3.1 Phục vụ chọn mẫu đơn có sẵn cho bàn đoàn hoặc combo.
Exception Flows	E1. Món hết hàng (từ bước 4) 4.1 Hệ thống từ chối thêm món và yêu cầu chọn món thay thế. E2. Phiên bàn không hợp lệ (từ bước 2) 2.1 Hệ thống từ chối tạo đơn nếu TableSession đã đóng.
Business Rules	• Mỗi đơn phải thuộc một TableSession. • Giá món được cố định tại thời điểm xác nhận đơn. • Một bàn có thể có nhiều đơn trong cùng phiên phục vụ.

Use Case ID	UC-ORD-02
Use Case Name	Tùy biến món và ghi chú dị ứng
Primary Actors	Phục vụ, Bếp
Description	Ghi nhận tùy chọn món, mức chế biến và ghi chú dị ứng cho từng dòng món.
Trigger	Phục vụ chọn chỉnh sửa chi tiết một dòng món trong đơn gọi món.
Preconditions	• Món hỗ trợ tùy chọn hoặc ghi chú. • Đơn gọi món ở trạng thái Draft hoặc chưa gửi bếp. • Cấu hình modifier đã được thiết lập.
Postconditions	• Tùy chọn được lưu dưới dạng OrderItemModifier. • Ghi chú dị ứng được gắn vào dòng món.

Normal Flow	1. Phục vụ mở chi tiết dòng món. 2. Hệ thống hiển thị các nhóm tùy chọn và giới hạn chọn. 3. Phục vụ chọn các tùy biến cho món. 4. Phục vụ nhập ghi chú đi ứng hoặc yêu cầu đặc biệt. 5. Hệ thống kiểm tra quy tắc chọn (min/max). 6. Hệ thống lưu cấu hình tùy chọn vào dòng món. 7. Khi đơn được gửi bếp, thông tin tùy biến và ghi chú được hiển thị.
Alternative Flows	A1. Dừng cấu hình mẫu (từ bước 2) 2.1 Phục vụ chọn nhanh cấu hình có sẵn cho món. A2. Gắn cờ đi ứng (từ bước 4) 4.1 Hệ thống tự động đánh dấu nổi bật các ghi chú đi ứng quan trọng.
Exception Flows	E1. Vi phạm quy tắc chọn (từ bước 5) 5.1 Hệ thống yêu cầu điều chỉnh số lượng lựa chọn. E2. Ghi chú quá dài (từ bước 4) 4.2 Hệ thống yêu cầu rút gọn hoặc chuyển sang ghi chú nội bộ. E3. Đơn đã gửi bếp (từ bước 1) 1.1 Hệ thống từ chối chỉnh sửa nếu đơn đã được xác nhận.
Business Rules	• Ghi chú đi ứng phải hiển thị rõ ràng trên phiếu bếp. • Modifier có thể thay đổi giá món. • Không cho phép chỉnh sửa sau khi gửi bếp.

Use Case ID	UC-ORD-03
Use Case Name	Gửi đơn gọi món sang bếp
Primary Actors	Phục vụ, Bếp
Description	Chuyển đơn gọi món đã xác nhận đến các trạm bếp để chế biến.
Trigger	Phục vụ nhấn gửi bếp hoặc hệ thống tự động gửi.
Preconditions	• Đơn có ít nhất một dòng món hợp lệ. • Đơn ở trạng thái Draft. • Kết nối tới hệ thống bếp khả dụng.
Postconditions	• Tạo một hoặc nhiều KitchenTicket. • Đơn chuyển sang trạng thái Confirmed/Queued.

Normal Flow	1. Phục vụ xác nhận các dòng món cần gửi. 2. Hệ thống kiểm tra trạng thái món, modifier và tồn kho logic. 3. Hệ thống xác định trạm bếp cho từng dòng món. 4. Hệ thống tách đơn thành các KitchenTicket theo trạm bếp. 5. Hệ thống xếp hàng phiếu bếp theo ưu tiên và thời gian. 6. Hệ thống cập nhật trạng thái đơn thành Confirmed/Queued. 7. Màn hình bếp hiển thị phiếu và phát tín hiệu thông báo.
Alternative Flows	A1. Gửi món sau (từ bước 1) 1.1 Phục vụ đánh dấu một số món để gửi sau. A2. Gộp phiếu bếp (từ bước 4) 4.1 Hệ thống gộp các phiếu bếp gần nhau theo cấu hình thời gian.
Exception Flows	E1. Không có trạm bếp (từ bước 3) 3.1 Hệ thống chặn gửi và cảnh báo lỗi cấu hình menu. E2. Mất kết nối bếp (từ bước 5) 5.1 Hệ thống đưa phiếu vào hàng đợi và retry sau.
Business Rules	• Mỗi dòng món phải được định tuyến đến ít nhất một trạm bếp. • Chỉ dòng món hợp lệ mới được gửi. • Không cho phép gửi lại các dòng đã được gửi trước đó.

Use Case ID	UC-KIT-01
Use Case Name	Cập nhật tiến độ chế biến
Primary Actors	Bếp
Description	Theo dõi và cập nhật trạng thái món từ khi nhận phiếu bếp đến khi sẵn sàng phục vụ.
Trigger	Nhân viên bếp chọn một phiếu bếp trên màn hình KDS.
Preconditions	• KitchenTicket đã được tạo và gán trạm bếp. • Nhân viên đã đăng nhập vào hệ thống bếp.
Postconditions	• Trạng thái món và phiếu bếp được cập nhật. • POS nhận được tiến độ mới nhất.

Normal Flow	1. Nhân viên bếp mở phiếu bếp ở trạng thái Queued. 2. Nhân viên cập nhật trạng thái từng món (Started, Cooking, Ready). 3. Hệ thống ghi nhận thời gian cho từng trạng thái. 4. Hệ thống so sánh thời gian chế biến với SLA. 5. Khi tất cả món hoàn tất, hệ thống chuyển phiếu sang Ready. 6. Hệ thống gửi thông báo đến POS để phục vụ ra món.
Alternative Flows	A1. Ưu tiên phiếu (từ bước 1) 1.1 Nhân viên bếp ưu tiên xử lý phiếu VIP hoặc món cần ra trước. A2. Giữ món (từ bước 2) 2.1 Nhân viên đánh dấu Hold for service để chờ đồng bộ với món khác.
Exception Flows	E1. Thiếu nguyên liệu (từ bước 2) 2.2 Nhân viên đánh dấu Blocked và gửi yêu cầu thay món. E2. Mất kết nối KDS (từ bước 3) 3.1 Hệ thống chuyển sang hàng đợi cục bộ và đồng bộ lại sau.
Business Rules	• Chỉ trạm bếp được phân công mới được cập nhật trạng thái. • Mọi thay đổi trạng thái phải được ghi log. • Trạng thái phiếu phụ thuộc vào trạng thái tất cả món.

Use Case ID	UC-KIT-02
Use Case Name	Ra món cho bàn
Primary Actors	Phục vụ
Description	Nhận món từ bếp và phục vụ đúng bàn, đúng thứ tự.
Trigger	Hệ thống thông báo món hoặc phiếu bếp ở trạng thái Ready.
Preconditions	• Món ở trạng thái Ready. • TableSession còn hoạt động.
Postconditions	• Dòng món được cập nhật trạng thái Served. • Thời gian phục vụ được ghi nhận.

Normal Flow	1. Phục vụ mở danh sách món sẵn sàng. 2. Phục vụ xác nhận nhận món từ bếp. 3. Hệ thống hiển thị thông tin bàn, vị trí ngồi và ghi chú. 4. Phục vụ giao món cho khách. 5. Phục vụ xác nhận “Đã phục vụ”. 6. Hệ thống cập nhật trạng thái từng dòng món thành Served. 7. Nếu tất cả món đã phục vụ, hệ thống cập nhật trạng thái phục vụ của bàn.
Alternative Flows	A1. Xác nhận theo phiếu (từ bước 1) 1.1 Phục vụ xác nhận toàn bộ phiếu thay vì từng món. A2. Giữ món theo course (từ bước 3) 3.1 Phục vụ trì hoãn ra món để đồng bộ theo course.
Exception Flows	E1. Sai món hoặc sai tùy biến (từ bước 3) 3.2 Phục vụ trả món về bếp và ghi lý do. E2. Bàn tạm vắng (từ bước 4) 4.1 Phục vụ đánh dấu trì hoãn phục vụ. E3. Trạng thái không hợp lệ (từ bước 5) 5.1 Hệ thống từ chối nếu món chưa ở trạng thái Ready.
Business Rules	• Chỉ món ở trạng thái Ready mới được phục vụ. • Mọi thao tác trả món phải có lý do. • Trạng thái phục vụ phải được ghi log để phân tích SLA.

Use Case ID	UC-KIT-03
Use Case Name	Ưu tiên hàng chờ bếp và xử lý món gấp
Primary Actors	Bếp, Quản lý
Description	Điều chỉnh mức ưu tiên của ticket hoặc món để xử lý các trường hợp gấp và giảm trễ hạn.
Trigger	Ticket gần vượt SLA, khách VIP hoặc yêu cầu can thiệp từ quản lý.
Preconditions	• KitchenTicket hoặc item tồn tại trong hàng chờ. • Có dữ liệu SLA và thứ tự hàng chờ. • Người thao tác có quyền thay đổi ưu tiên.
Postconditions	• Mức ưu tiên được cập nhật. • Hàng chờ được sắp xếp lại. • Lý do thay đổi được ghi log.

Normal Flow	1. Nhân viên mở hàng chờ của trạm bếp. 2. Hệ thống hiển thị độ trễ và các ticket có nguy cơ trễ. 3. Người thao tác chọn ticket hoặc món cần ưu tiên. 4. Hệ thống áp dụng quy tắc ưu tiên (SLA, VIP, course). 5. Người thao tác xác nhận hoặc nhập lý do ưu tiên. 6. Hệ thống cập nhật mức ưu tiên. 7. Hệ thống sắp xếp lại hàng chờ. 8. Hệ thống làm nổi bật ticket ưu tiên trên KDS.
Alternative Flows	A1. Tự động ưu tiên (từ bước 2) 2.1 Hệ thống tự động tăng ưu tiên nếu ticket gần vượt SLA. A2. Ưu tiên theo món (từ bước 3) 3.1 Người thao tác chỉ ưu tiên một phần của ticket (ví dụ món khai vị).
Exception Flows	E1. Không đủ quyền (từ bước 3) 3.2 Hệ thống yêu cầu xác nhận từ quản lý. E2. Quá tải trạm bếp (từ bước 2) 2.2 Hệ thống cảnh báo và gợi ý điều phối sang trạm khác. E3. Thiếu dữ liệu SLA (từ bước 4) 4.1 Hệ thống chỉ cho phép ưu tiên thủ công.
Business Rules	• Mọi hành động ưu tiên phải có lý do. • Không được gây trì hoãn vô hạn cho ticket khác (anti-starvation). • Ưu tiên có thể áp dụng ở mức ticket hoặc item.

Use Case ID	UC-ORD-04
Use Case Name	Sửa hoặc hủy dòng món
Primary Actors	Phục vụ, Quản lý
Description	Điều chỉnh hoặc hủy dòng món trong đơn gọi món theo trạng thái và quyền hạn.
Trigger	Phục vụ chọn dòng món để sửa hoặc hủy.
Preconditions	• Đơn chưa hoàn tất thanh toán. • Người dùng có quyền chỉnh sửa theo trạng thái món.
Postconditions	• Dòng món được cập nhật hoặc chuyển trạng thái Voided. • Lịch sử thay đổi được ghi nhận. • Nếu đã gửi bếp, hệ thống phát thông báo đến KDS.

Normal Flow	1. Phục vụ chọn dòng món cần chỉnh sửa. 2. Hệ thống hiển thị trạng thái món và quyền chỉnh sửa. 3. Phục vụ chọn sửa thông tin hoặc hủy dòng món. 4. Nếu món đã gửi bếp, hệ thống yêu cầu nhập lý do. 5. Nếu cần, hệ thống yêu cầu phê duyệt từ quản lý. 6. Hệ thống cập nhật dữ liệu dòng món. 7. Hệ thống phát sự kiện đến KDS và cập nhật tạm tính.
Alternative Flows	A1. Chỉnh sửa trước khi gửi bếp (từ bước 3) 3.1 Phục vụ chỉnh sửa tự do khi món ở trạng thái Draft. A2. Hủy sau khi chế biến (từ bước 4) 4.1 Hệ thống yêu cầu phê duyệt bắt buộc khi món đã ở trạng thái Cooking hoặc Ready.
Exception Flows	E1. Không đủ quyền (từ bước 5) 5.1 Hệ thống từ chối thao tác nếu không có quyền. E2. Xung đột cập nhật (từ bước 2) 2.1 Hệ thống phát hiện chỉnh sửa đồng thời và yêu cầu tải lại.
Business Rules	• Hủy món sau khi gửi bếp phải có lý do. • Có thể yêu cầu phê duyệt nếu vượt ngưỡng cấu hình. • Không được mất lịch sử trạng thái và giá ban đầu.

Use Case ID	UC-BIL-01
Use Case Name	Lập hóa đơn
Primary Actor	Thu ngân
Description	Use case này cho phép thu ngân tạo hóa đơn thanh toán cho bàn.
Preconditions	• Bàn đã có đơn gọi món. • Các món đã được ghi nhận trong hệ thống. • Thu ngân đã đăng nhập vào hệ thống.
Postconditions	• Hóa đơn được tạo và lưu trong hệ thống. • Tổng số tiền cần thanh toán được tính toán và hiển thị. • Hóa đơn ở trạng thái chờ thanh toán.

Main Flow	1. Thu ngân chọn bàn cần thanh toán trong hệ thống. 2. Hệ thống hiển thị danh sách các món ăn, đồ uống và dịch vụ đã được gọi cho bàn đó. 3. Thu ngân kiểm tra thông tin hóa đơn và nhấn tiếp theo. 4. Hệ thống tính tổng tiền bao gồm các khoản phí và thuế (nếu có). 5. Thu ngân xác nhận lập hóa đơn. 6. Hệ thống tạo hóa đơn và hiển thị thông tin chi tiết. 7. Thu ngân tiến hành bước thanh toán.
Alternative Flows	A1. Điều chỉnh món trong hóa đơn: Tại bước 3, nếu thu ngân phát hiện sai sót trong danh sách món. • 3a. Thu ngân yêu cầu điều chỉnh món (thay đổi số lượng hoặc hủy món). • 3b. Hệ thống cập nhật lại danh sách món. • 3c. Hệ thống quay lại bước 3 của luồng chính. A2. Hủy thao tác lập hóa đơn: Tại bước 5, nếu thu ngân quyết định không tiếp tục lập hóa đơn. • 5a. Thu ngân chọn hủy thao tác. • 5b. Hệ thống hủy quá trình lập hóa đơn và quay lại màn hình quản lý bàn.
Exception Flows	E1. Lỗi hệ thống khi tạo hóa đơn: • 5a. Hệ thống không thể tạo hóa đơn do lỗi kỹ thuật. • 5b. Hệ thống hiển thị thông báo: “Không thể tạo hóa đơn. Vui lòng thử lại sau.” • 5c. Use case kết thúc. E2. Không tìm thấy dữ liệu đơn món: • 2a. Hệ thống không tìm thấy dữ liệu đơn món của bàn đã chọn. • 2b. Hệ thống hiển thị thông báo: “Không có dữ liệu để lập hóa đơn.” • 2c. Hệ thống quay lại màn hình quản lý bàn.

Use Case ID	UC-BIL-02
Use Case Name	Tách hóa đơn và thêm tiền boa

Primary Actor	Thu ngân, Phục vụ
Description	Use case này cho phép nhân viên tách hóa đơn thành nhiều phần thanh toán và ghi nhận tiền boia theo yêu cầu của khách.
Trigger	Khách yêu cầu thanh toán riêng theo đầu người, theo món, hoặc theo số tiền.
Preconditions	- Hóa đơn đã được tạo trong hệ thống. - Hóa đơn chưa hoàn tất thanh toán.
Postconditions	- Các phần thanh toán (Bill Split) được tạo trong hệ thống. - Mỗi phần hóa đơn sẵn sàng để thực hiện thanh toán. - Tiền boia được ghi nhận theo cấu hình của nhân viên.
Main Flow	- Thu ngân mở hóa đơn cần tách. - Hệ thống hiển thị danh sách món, số ghế và tổng tiền của hóa đơn. - Thu ngân chọn phương thức chia hóa đơn (chia đều, theo món, theo ghế, theo số tiền hoặc kết hợp). - Thu ngân chọn mức tiền boia gợi ý. - Hệ thống tính lại số tiền của từng phần thanh toán. - Hệ thống hiển thị tổng tiền của từng phần và chênh lệch (nếu có). - Thu ngân xác nhận cấu hình tách hóa đơn. - Hệ thống tạo các phần hóa đơn tương ứng.
Alternative Flows	A1. Tách một phần món ra thanh toán trước: Tại bước 3. 3a. Thu ngân chọn các món cần tách ra thanh toán. 3b. Hệ thống tạo một phần hóa đơn mới cho các món đã chọn. 3c. Phần còn lại vẫn giữ trong hóa đơn gốc. A2. Áp dụng tiền boia cho từng phần hóa đơn: Tại bước 4. 4a. Thu ngân nhập tiền boia cho từng phần thanh toán. 4b. Hệ thống cập nhật tổng tiền của từng phần hóa đơn.

Exception Flows	E1. Tổng tiền các phần không khớp • 7a. Hệ thống phát hiện tổng các phần thanh toán không khớp với tổng hóa đơn. • 7b. Hệ thống hiển thị thông báo yêu cầu điều chỉnh lại cấu hình tách hóa đơn. • 7c. Hệ thống quay lại bước 3. E2. Phần hóa đơn đã thanh toán • 3a. Hệ thống phát hiện một phần hóa đơn đã được thanh toán. • 3b. Hệ thống không cho phép thay đổi phạm vi món trong phần đó. • 3c. Hệ thống yêu cầu hoàn tiền hoặc có phê duyệt trước khi chỉnh sửa.
------------------------	---

Use Case ID	UC-PAY-01
Use Case Name	Xử lý thanh toán
Primary Actor	Thu ngân
Description	Use case này cho phép thu ngân thực hiện thanh toán cho hóa đơn hoặc từng phần hóa đơn bằng các phương thức như tiền mặt, thẻ hoặc ví điện tử.
Trigger	Thu ngân chọn chức năng thanh toán trên một hóa đơn hoặc một phần hóa đơn trong hệ thống.
Preconditions	• Hóa đơn hoặc phần hóa đơn đã được tạo trong hệ thống. • Hóa đơn chưa hoàn tất thanh toán. • Thu ngân đã đăng nhập vào hệ thống.
Postconditions	• Một bản ghi thanh toán được lưu trong hệ thống. • Số tiền đã thanh toán của hóa đơn được cập nhật. • Nếu tổng tiền đã thanh toán đủ, hóa đơn được đánh dấu hoàn tất.

Main Flow	1. Thu ngân chọn hóa đơn hoặc phân hóa đơn cần thanh toán. 2. Hệ thống hiển thị tổng số tiền cần thanh toán. 3. Thu ngân chọn phương thức thanh toán. 4. Nếu thanh toán điện tử, hệ thống gửi yêu cầu xử lý giao dịch. 5. Nếu thanh toán tiền mặt, thu ngân nhập số tiền khách đưa. 6. Hệ thống tính tiền thối (nếu có). 7. Hệ thống ghi nhận giao dịch thanh toán và cập nhật số tiền còn lại của hóa đơn. 8. Nếu hóa đơn chưa thanh toán đủ, hệ thống cho phép tiếp tục thực hiện thanh toán. 9. Nếu hóa đơn đã thanh toán đủ, hệ thống đóng hóa đơn và hiển thị biên lai.
Alternative Flows	A1. Sử dụng tiền đặt cọc Tại bước 2 của Main Flow. • 2a. Hệ thống phát hiện hóa đơn có tiền đặt cọc từ trước. • 2b. Hệ thống hiển thị số tiền đặt cọc và số tiền còn lại cần thanh toán. • 2c. Hệ thống tự động trừ tiền đặt cọc vào tổng hóa đơn. • 2d. Thu ngân tiếp tục thanh toán phần tiền còn lại.
Exception Flows	E1. Thanh toán điện tử thất bại • 4a. Hệ thống nhận phản hồi giao dịch thất bại. • 4b. Hệ thống hiển thị thông báo lỗi thanh toán. • 4c. Thu ngân chọn phương thức thanh toán khác hoặc thử lại. E2. Thiết bị thanh toán không phản hồi • 4a. Hệ thống không nhận được phản hồi từ thiết bị thanh toán. • 4b. Hệ thống hiển thị thông báo lỗi. • 4c. Thu ngân chuyển sang phương thức thanh toán khác.

Use Case ID	UC-PAY-02
Use Case Name	Phát hành biên lai
Primary Actor	Thu ngân, Khách hàng

Description	Use case này cho phép hệ thống tạo và phát hành biên lai cho khách hàng sau khi thanh toán được ghi nhận thành công.
Trigger	Thanh toán của hóa đơn hoặc split bill được xác nhận thành công.
Preconditions	• Thanh toán đã được ghi nhận hợp lệ trong hệ thống. • Hệ thống đã cấu hình kênh phát hành biên lai qua ReceiptDeliveryAdapter (print, email hoặc sms ở mức adapter nội bộ).
Postconditions	• Biên lai (Receipt) được tạo và lưu trong hệ thống. • Biên lai được liên kết với hóa đơn và giao dịch thanh toán. • Yêu cầu phát hành biên lai được ghi nhận theo kênh đã chọn.
Main Flow	1. Hệ thống tạo nội dung biên lai gồm danh sách món, giảm giá, phí, phương thức thanh toán và thời gian. 2. Thu ngân chọn phương thức phát hành biên lai qua print adapter hoặc kênh điện tử. 3. Hệ thống ghi nhận yêu cầu phát hành biên lai theo lựa chọn. 4. Hệ thống lưu biên lai và đánh dấu trạng thái yêu cầu phát hành. 5. Nếu có thông tin khách hàng, hệ thống tạo yêu cầu phát hành bản sao biên lai qua email hoặc sms ở mức adapter nội bộ.
Alternative Flows	A1. Xuất biên lai rút gọn hoặc chi tiết • 2a. Thu ngân chọn loại biên lai (rút gọn hoặc chi tiết). • 2b. Hệ thống tạo biên lai theo định dạng tương ứng. A2. Biên lai theo split bill • 2a. Hệ thống phát hiện giao dịch thuộc split bill. • 2b. Mỗi split được tạo một biên lai riêng tương ứng.

Exception Flows	E1. Kênh phát hành biên lai không hợp lệ 3a. Hệ thống không tìm thấy ReceiptDeliveryAdapter phù hợp hoặc thiếu địa chỉ nhận đối với kênh điện tử. 3b. Hệ thống hiển thị thông báo lỗi và cho phép thu ngân chọn lại kênh phát hành. E2. Không tạo được yêu cầu gửi biên lai điện tử 5a. Hệ thống không tạo được yêu cầu gửi biên lai điện tử qua NotificationChannelAdapter. 5b. Hệ thống vẫn lưu biên lai và đánh dấu “chưa gửi”. 5c. Thu ngân có thể gửi lại thủ công sau.
------------------------	---

Use Case ID	UC-PAY-03
Use Case Name	Hoàn tiền
Primary Actor	Thu ngân, Quản lý
Description	Use case này cho phép hệ thống thực hiện hoàn tiền một phần hoặc toàn phần cho giao dịch thanh toán đã được ghi nhận trước đó.
Trigger	Khách yêu cầu hoàn tiền do hủy món, khiếu nại hoặc điều chỉnh hóa đơn sau thanh toán.
Preconditions	- Tồn tại giao dịch thanh toán thành công cần hoàn tiền. - Người thực hiện có quyền hoàn tiền hoặc được quản lý phê duyệt.
Postconditions	- Một bản ghi hoàn tiền (Refund) được tạo và liên kết với giao dịch gốc. - Hệ thống cập nhật lại số dư hóa đơn và trạng thái thanh toán. - Nhật ký kiểm toán ghi nhận người thực hiện, lý do và số tiền hoàn.
Main Flow	- Thu ngân chọn giao dịch thanh toán cần hoàn tiền. - Hệ thống hiển thị số tiền đã thanh toán và số tiền còn có thể hoàn. - Thu ngân nhập số tiền hoàn và lý do hoàn tiền. - Nếu vượt ngưỡng cho phép, hệ thống yêu cầu quản lý xác nhận. - Hệ thống xử lý hoàn tiền qua PaymentGateway hoặc ghi nhận hoàn tiền mặt. - Hệ thống cập nhật trạng thái hoàn tiền và số dư hóa đơn.

Alternative Flows	A1. Hoàn tiền một phần theo món • 3a. Thu ngân chọn các món cần hoàn tiền. • 3b. Hệ thống tính lại số tiền tương ứng. • 3c. Tiếp tục luồng hoàn tiền như bình thường. A2. Chuyển hoàn tiền thành tín dụng khách hàng • 4a. Quản lý chọn phương án không hoàn tiền trực tiếp. • 4b. Hệ thống ghi nhận số tiền vào ví nội bộ của khách hàng.
Exception Flows	E1. PaymentGateway từ chối hoàn tiền • 5a. Hệ thống gửi yêu cầu hoàn tiền thất bại. • 5b. Hệ thống ghi nhận trạng thái “Pending Review”. • 5c. Thông báo cho kế toán hoặc quản lý xử lý. E2. Số tiền hoàn vượt giới hạn • 3a. Hệ thống phát hiện số tiền hoàn lớn hơn mức còn lại. • 3b. Hệ thống chặn thao tác và yêu cầu nhập lại. E3. Giao dịch không còn đủ điều kiện hoàn tiền • 2a. Hệ thống kiểm tra giao dịch đã vượt quá thời hạn hoàn tiền được cấu hình (refund_window_hours, mặc định 24 giờ). • 2b. Hệ thống hiển thị thông báo và yêu cầu xử lý ngoại lệ.

Use Case ID	UC-INV-01
Use Case Name	Cập nhật tiêu hao tồn kho sau phục vụ
Primary Actor	Hệ thống, Quản lý kho
Description	Use case này mô tả việc hệ thống tự động khấu trừ tồn kho dựa trên món đã phục vụ, đồng thời cho phép quản lý kho thực hiện điều chỉnh thủ công khi cần thiết.
Trigger	Một món ăn được xác nhận đã phục vụ hoặc quản lý kho tạo giao dịch điều chỉnh tồn kho.
Preconditions	• Món ăn đã được ánh xạ công thức nguyên liệu (RecipeLine). • Hệ thống tồn kho đang hoạt động bình thường.
Postconditions	• Giao dịch kho (StockTransaction) được ghi nhận. • Số lượng tồn kho của nguyên liệu được cập nhật chính xác.

Main Flow	1. Hệ thống ghi nhận món ăn được phục vụ thành công. 2. Hệ thống tra cứu công thức nguyên liệu (RecipeLine) của món. 3. Hệ thống tính toán lượng nguyên liệu cần trừ theo số lượng món. 4. Hệ thống tạo giao dịch kho để khấu trừ tồn kho. 5. Quản lý kho có thể tạo giao dịch điều chỉnh thủ công nếu cần. 6. Hệ thống cập nhật số lượng tồn kho và lưu lịch sử giao dịch.
Alternative Flows	A1. Khấu trừ tồn kho khi gửi bếp • 2a. Hệ thống được cấu hình khấu trừ tại thời điểm gửi bếp thay vì phục vụ. • 2b. Tồn kho được trừ ngay khi món được chuyển sang bếp chế biến. A2. Ghi nhận hao hụt nguyên liệu đầu ca • 5a. Quản lý kho tạo giao dịch prep/waste đầu ca. • 5b. Hệ thống cập nhật tồn kho tương ứng.
Exception Flows	E1. Thiếu công thức nguyên liệu • 2a. Hệ thống không tìm thấy RecipeLine của món. • 2b. Hệ thống không thực hiện khấu trừ và tạo cảnh báo cho quản lý. E2. Xung đột cập nhật tồn kho • 4a. Hệ thống phát hiện đồng thời nhiều giao dịch gây sai lệch tồn kho. • 4b. Hệ thống tạm khóa cập nhật hoặc đưa vào hàng chờ xử lý.

Use Case ID	UC-INV-02
Use Case Name	Nhận cảnh báo sắp hết hàng
Primary Actor	Hệ thống, Quản lý, Bếp
Description	Use case này mô tả việc hệ thống tự động phát hiện nguyên liệu sắp hết khi tồn kho giảm xuống dưới ngưỡng cấu hình và gửi cảnh báo đến các bộ phận liên quan.
Trigger	Số lượng tồn kho của nguyên liệu giảm xuống thấp hơn ngưỡng cảnh báo (LowStockRule).
Preconditions	• Ngưỡng cảnh báo tồn kho đã được cấu hình cho từng nguyên liệu. • Hệ thống thông báo nội bộ đang hoạt động.

Postconditions	• Cảnh báo sắp hết hàng được tạo và gửi đến các vai trò liên quan. • Danh sách nguyên liệu cần bổ sung được cập nhật trong hệ thống.
Main Flow	1. Hệ thống kiểm tra tồn kho sau mỗi lần cập nhật. 2. Nếu tồn kho của nguyên liệu thấp hơn ngưỡng cấu hình, hệ thống tạo cảnh báo. 3. Hệ thống gửi thông báo đến Quản lý và Bếp. 4. Quản lý xem danh sách nguyên liệu sắp hết và các món liên quan. 5. Quản lý quyết định đặt hàng bổ sung hoặc điều chỉnh kế hoạch sử dụng.
Alternative Flows	A1. Gọi ý đặt hàng tự động • 3a. Hệ thống phân tích lịch sử tiêu thụ nguyên liệu. • 3b. Hệ thống đề xuất số lượng đặt hàng phù hợp. A2. Gom cảnh báo theo ca • 2a. Hệ thống gom nhiều cảnh báo trong một khoảng thời gian. • 2b. Hệ thống gửi một thông báo tổng hợp thay vì gửi riêng lẻ.
Exception Flows	E1. Cấu hình ngưỡng không hợp lệ • 1a. Hệ thống phát hiện ngưỡng cảnh báo thiếu hoặc sai định dạng. • 1b. Hệ thống bỏ qua việc tạo cảnh báo và ghi nhận lỗi cấu hình. • 1c. Quản lý được thông báo để cập nhật lại cấu hình. E2. Lỗi kênh thông báo • 3a. Hệ thống không gửi được thông báo đến người dùng. • 3b. Hệ thống lưu cảnh báo vào hàng đợi để gửi lại sau.

Use Case ID	UC-INV-03
Use Case Name	Phân tích tiêu hao và đề xuất nhập hàng
Primary Actor	Quản lý, Quản lý kho
Description	Use case này mô tả việc hệ thống phân tích dữ liệu tiêu hao, tồn kho và cảnh báo để đề xuất số lượng nhập hàng phù hợp cho kỳ vận hành tiếp theo.
Trigger	Quản lý mở chức năng phân tích nhập hàng theo ngày, tuần hoặc kỳ vận hành.

Preconditions	- Hệ thống đã có dữ liệu tồn kho và lịch sử tiêu hao (StockTransaction). - Có cấu hình mức tồn an toàn hoặc min-max. - Người dùng có quyền xem dữ liệu kho và báo cáo.
Postconditions	- Hệ thống tạo danh sách đề xuất nhập hàng cho từng nguyên liệu. - Quản lý có thể lưu hoặc xuất báo cáo đề xuất. - Các nguyên liệu có rủi ro được đánh dấu cảnh báo.
Main Flow	- Quản lý chọn kỳ phân tích và nhóm nguyên liệu. - Hệ thống tải dữ liệu tiêu hao và tồn kho liên quan. - Hệ thống tính mức tiêu hao trung bình theo kỳ. - Hệ thống xác định mức tồn mục tiêu và điểm đặt hàng lại. - Hệ thống tạo đề xuất số lượng nhập cho từng nguyên liệu. - Quản lý xem và điều chỉnh đề xuất nếu cần. - Quản lý lưu hoặc xuất báo cáo.
Alternative Flows	A1. Thiếu dữ liệu lịch sử 3a. Hệ thống phát hiện dữ liệu không đủ để phân tích. 3b. Hệ thống chuyển sang phương pháp min-max hoặc safety stock. 3c. Đề xuất vẫn được tạo nhưng gắn nhãn độ tin cậy thấp. A2. So sánh nhiều kỳ 1a. Quản lý chọn nhiều kỳ (cuối tuần, lễ, cao điểm). 1b. Hệ thống tính và so sánh mức tiêu hao giữa các kỳ. 1c. Hiện thị đề xuất theo từng kịch bản.
Exception Flows	E1. Dữ liệu không đồng nhất 2a. Hệ thống phát hiện sai đơn vị hoặc thiếu ánh xạ nguyên liệu. 2b. Hệ thống không tạo đề xuất cho nguyên liệu đó. 2c. Ghi nhận cần xử lý thủ công. E2. Lỗi hệ thống 2a. Hệ thống không truy xuất được dữ liệu kho. 2b. Hiện thị thông báo lỗi và yêu cầu thử lại.

Use Case ID	UC-REP-01
--------------------	-----------

Use Case Name	Xem báo cáo doanh thu và vận hành
Primary Actor	Quản lý
Description	Use case này mô tả việc hệ thống cung cấp báo cáo tổng hợp về doanh thu, hiệu suất phục vụ, hoạt động bếp và các chỉ số vận hành của nhà hàng.
Trigger	Quản lý mở dashboard báo cáo hoặc yêu cầu xuất báo cáo theo khoảng thời gian.
Preconditions	- Dữ liệu giao dịch, gọi món, bếp và tồn kho đã được tổng hợp vào hệ thống báo cáo. - Người dùng có quyền truy cập chức năng báo cáo.
Postconditions	- Hệ thống hiển thị dashboard báo cáo hoặc file xuất (PDF/CSV). - Các chỉ số vận hành được tổng hợp theo bộ lọc đã chọn.
Main Flow	- Quản lý chọn khoảng thời gian và bộ lọc báo cáo. - Hệ thống truy xuất dữ liệu báo cáo từ read model. - Hệ thống hiển thị doanh thu, món bán chạy, giờ cao điểm và hiệu suất phục vụ. - Quản lý xem chi tiết theo ca, khu vực hoặc bếp. - Quản lý chọn xuất báo cáo nếu cần.
Alternative Flows	A1. Lọc và cập nhật báo cáo 1a. Quản lý thay đổi bộ lọc (thời gian, khu vực, ca làm). 1b. Hệ thống truy vấn lại dữ liệu và cập nhật dashboard. A2. Xuất báo cáo 5a. Quản lý chọn định dạng xuất (PDF/CSV). 5b. Hệ thống tạo file và cung cấp cho người dùng tải về. A3. Hiển thị cảnh báo bất thường 3a. Hệ thống phát hiện chỉ số bất thường (ví dụ: hoàn tiền tăng đột biến). 3b. Hệ thống hiển thị cảnh báo trên dashboard.

Exception Flows	E1. Dữ liệu báo cáo chưa cập nhật • 2a. Hệ thống hiển thị thời điểm đồng bộ gần nhất. • 2b. Báo cáo có thể bị giới hạn độ mới của dữ liệu. E2. Lỗi truy xuất dữ liệu • 2a. Hệ thống không truy xuất được dữ liệu báo cáo. • 2b. Hiển thị thông báo lỗi và yêu cầu thử lại.
------------------------	--

Use Case ID	UC-ADM-01
Use Case Name	Quản lý menu và giá bán
Primary Actor	Quản lý
Description	Use case này mô tả việc quản lý cập nhật món ăn, điều chỉnh giá bán, khuyến mãi và trạng thái hiển thị của thực đơn trong hệ thống.
Trigger	Quản lý mở màn hình cấu hình thực đơn trong hệ thống.
Preconditions	• Người dùng có quyền quản trị thực đơn. • Hệ thống đã có danh mục món cơ sở.
Postconditions	• Thực đơn hoặc thay đổi giá được lưu trong hệ thống. • Các thay đổi được áp dụng theo thời điểm công bố. • Các đơn hàng đang mở giữ nguyên giá đã ghi nhận trước đó.
Main Flow	1. Quản lý chọn món hoặc danh mục món cần chỉnh sửa. 2. Hệ thống hiển thị thông tin chi tiết (giá, trạng thái, khuyến mãi). 3. Quản lý chỉnh sửa thông tin món (tên, giá, trạng thái, khuyến mãi). 4. Quản lý lưu thay đổi và chọn công bố ngay hoặc lên lịch. 5. Hệ thống kiểm tra dữ liệu và cập nhật phiên bản thực đơn mới.
Alternative Flows	A1. Công bố theo lịch • 4a. Quản lý chọn thời gian áp dụng (ca sáng, ca tối, ngày lễ). • 4b. Hệ thống lưu phiên bản và kích hoạt theo thời gian đã đặt. A2. Áp dụng khuyến mãi • 3a. Quản lý áp dụng khuyến mãi theo món, danh mục hoặc hóa đơn. • 3b. Hệ thống cập nhật chính sách giá tương ứng.

Exception Flows	E1. Thiếu dữ liệu cấu hình • 5a. Hệ thống phát hiện thiếu thông tin bắt buộc (ví dụ: trạm bếp hoặc thuế). • 5b. Hệ thống không cho phép công bố thực đơn. • 5c. Yêu cầu người dùng bổ sung thông tin. E2. Xung đột lịch hiệu lực • 4a. Hệ thống phát hiện trùng lịch áp dụng giá hoặc khuyến mãi. • 4b. Hệ thống yêu cầu người dùng điều chỉnh trước khi lưu.
------------------------	--

Use Case ID	UC-ADM-02
Use Case Name	Quản lý vai trò và phân quyền
Primary Actor	Quản trị viên, Quản lý
Description	Use case này mô tả việc gán vai trò và phân quyền cho người dùng trong hệ thống nhằm kiểm soát các thao tác theo cơ chế RBAC.
Trigger	Quản trị viên mở màn hình quản lý người dùng hoặc phân quyền hệ thống.
Preconditions	• Tài khoản người dùng đã tồn tại hoặc đang được tạo mới. • Chính sách RBAC và danh mục quyền đã được định nghĩa.
Postconditions	• Người dùng được gán vai trò và quyền truy cập phù hợp. • Hệ thống cập nhật AuthorizationPolicy cho các thao tác liên quan. • Thay đổi quyền được ghi nhận trong audit log.
Main Flow	1. Quản trị viên tìm kiếm người dùng cần cập nhật. 2. Hệ thống hiển thị vai trò và quyền hiện tại. 3. Quản trị viên gán hoặc thay đổi vai trò/quyền theo chính sách. 4. Hệ thống lưu thay đổi phân quyền. 5. Hệ thống yêu cầu người dùng đăng nhập lại. 6. Hệ thống ghi nhận thay đổi vào audit log.

Alternative Flows	A1. Gán vai trò tạm thời • 3a. Quản trị viên chọn vai trò có thời hạn. • 3b. Hệ thống gán vai trò theo khoảng thời gian hiệu lực. A2. Giới hạn theo khu vực • 3a. Quản trị viên giới hạn quyền theo khu vực (bếp, sảnh, quầy). • 3b. Hệ thống áp dụng phạm vi quyền tương ứng.
Exception Flows	E1. Gỡ quyền quản trị cuối cùng • 3a. Hệ thống phát hiện đây là quyền quản trị cuối cùng. • 3b. Hệ thống chặn thao tác để tránh mất quyền hệ thống. E2. Xung đột chính sách quyền • 3a. Hệ thống phát hiện vai trò xung đột chính sách RBAC. • 3b. Yêu cầu quản trị viên điều chỉnh trước khi lưu.

Use Case ID	UC-ADM-03
Use Case Name	Xem nhật ký kiểm toán và truy vết thao tác nhạy cảm
Primary Actor	Quản lý, Quản trị viên
Description	Use case này mô tả việc tra cứu và truy vết các hành động nhạy cảm như hoàn tiền, hủy món, ghi đề giá, thay đổi quyền hoặc mở lại hóa đơn.
Trigger	Người dùng có thẩm quyền mở màn hình kiểm toán để điều tra, rà soát hoặc kiểm tra nội bộ.
Preconditions	• Hệ thống đã ghi nhận AuditLog cho các thao tác nhạy cảm. • Người dùng có quyền truy cập nhật ký kiểm toán. • Dữ liệu giao dịch và correlation id có thể truy xuất.
Postconditions	• Hệ thống hiển thị chuỗi hành động liên quan đến truy vấn. • Có thể xuất báo cáo hoặc đánh dấu bản ghi cần theo dõi. • Hành vi truy cập audit log được ghi lại.

Main Flow	1. Người dùng mở màn hình nhật ký kiểm toán. 2. Hệ thống kiểm tra quyền truy cập. 3. Người dùng nhập bộ lọc tìm kiếm (thời gian, người thao tác, loại hành động, mã liên kết). 4. Hệ thống truy vấn và hiển thị danh sách audit log. 5. Người dùng chọn một bản ghi để xem chi tiết (giá trị trước, sau, lý do, ngữ cảnh). 6. Người dùng xuất báo cáo hoặc gắn cờ theo dõi nếu cần.
Alternative Flows	A1. Truy vết từ giao dịch • 3a. Người dùng truy xuất audit log từ hóa đơn/ payment/order. • 3b. Hệ thống hiển thị toàn bộ chuỗi liên quan. A2. Che dữ liệu theo quyền • 4a. Hệ thống ẩn thông tin nhạy cảm nếu không đủ quyền. • 4b. Chỉ hiển thị dữ liệu rút gọn.
Exception Flows	E1. Không đủ quyền truy cập • 2a. Hệ thống từ chối truy cập hoặc hiển thị bản rút gọn. E2. Không có dữ liệu phù hợp • 4a. Hệ thống thông báo không tìm thấy kết quả. • 4b. Người dùng điều chỉnh bộ lọc tìm kiếm.

Use Case ID	UC-ADM-04
Use Case Name	Tạo món mới
Primary Actor	Quản lý
Description	Use case này mô tả việc quản lý tạo mới một món ăn trong thực đơn và khai báo thông tin nguyên liệu để phục vụ bán hàng và quản lý tồn kho.
Trigger	Quản lý chọn chức năng tạo món mới trong màn hình quản lý menu.
Preconditions	• Người dùng có quyền quản trị menu. • Hệ thống đã có danh mục nguyên liệu trong kho.

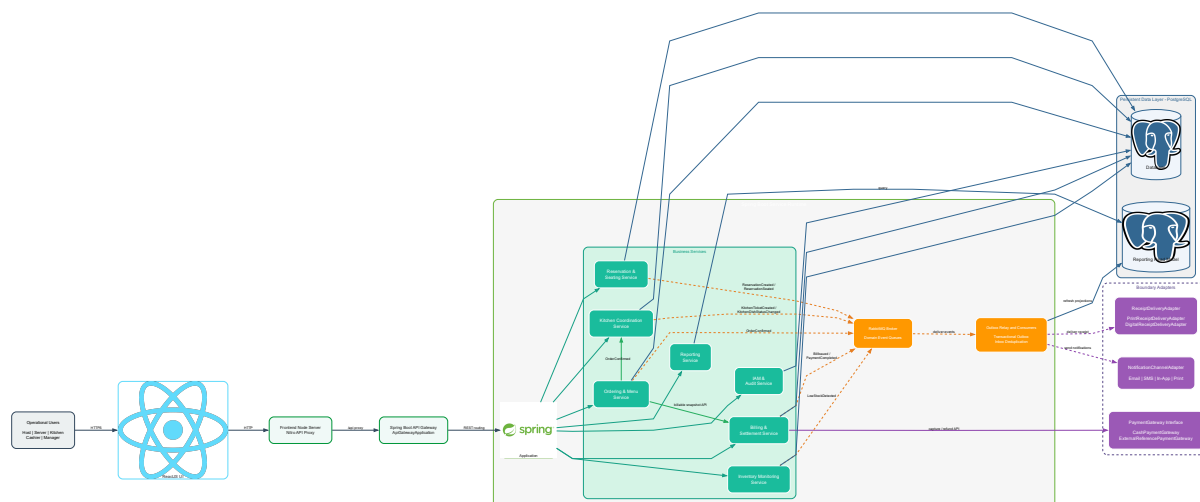
Postconditions	- Món mới được tạo và lưu trong hệ thống menu. - Công thức nguyên liệu (recipe) được liên kết với món. - Món có thể được sử dụng trong gọi món; tồn kho được trừ theo công thức khi món bắt đầu được chế biến.
Main Flow	- Quản lý chọn chức năng tạo món mới. - Hệ thống hiển thị form nhập thông tin món. - Quản lý nhập thông tin món (tên, giá, mô tả, danh mục, trạng thái bán). - Quản lý khai báo danh sách nguyên liệu và định lượng. - Quản lý lưu thông tin món. - Hệ thống kiểm tra dữ liệu hợp lệ. - Hệ thống lưu món và công thức nguyên liệu vào hệ thống.
Alternative Flows	A1. Thiếu thông tin bắt buộc 3a. Hệ thống phát hiện thiếu tên món hoặc giá. 3b. Hệ thống hiển thị thông báo lỗi. 3c. Quay lại bước 3. A2. Thiếu nguyên liệu 4a. Hệ thống phát hiện chưa khai báo nguyên liệu. 4b. Hệ thống yêu cầu bổ sung nguyên liệu trước khi lưu.
Exception Flows	E1. Lỗi hệ thống khi lưu 5a. Hệ thống không thể lưu món mới. 5b. Hệ thống hiển thị thông báo lỗi và yêu cầu thử lại.

Use Case ID	UC-OPS-01
Use Case Name	Quản lý ca làm nhân viên
Primary Actor	Quản lý
Description	Use case này mô tả việc phân công nhân sự cho từng khu vực và ca làm việc trong nhà hàng.
Trigger	Quản lý mở lịch làm việc và tạo hoặc cập nhật ca làm.
Preconditions	- Tài khoản nhân viên đã tồn tại. - Mẫu ca làm và cấu hình khu vực đã được thiết lập.

Postconditions	• Ca làm (ShiftAssignment) được tạo hoặc cập nhật. • Nhân viên nhận được lịch làm việc mới.
Main Flow	1. Quản lý chọn ngày, khu vực và loại ca. 2. Hệ thống hiển thị danh sách nhân viên và các ca đã có. 3. Quản lý gán nhân viên vào ca và khu vực. 4. Hệ thống kiểm tra thời điểm kết thúc và trùng ca làm việc. 5. Quản lý lưu lịch làm việc. 6. Hệ thống ghi nhận thông báo nội bộ cho lịch ca vừa thay đổi.
Alternative Flows	A1. Cập nhật ca đã có • 1a. Quản lý chọn một ca làm việc đã tồn tại. • 1b. Hệ thống hiển thị thông tin ca để quản lý chỉnh sửa. A2. Ca bị trùng thời gian • 4a. Hệ thống phát hiện nhân viên đã có ca trùng khoảng thời gian. • 4b. Hệ thống chặn lưu và yêu cầu điều chỉnh ca làm.
Exception Flows	E1. Trùng ca hoặc thời gian không hợp lệ • 4a. Hệ thống phát hiện xung đột lịch làm. • 4b. Chặn lưu và yêu cầu điều chỉnh. E2. Không tìm thấy nhân viên hoặc ca cần cập nhật • 3a. Hệ thống không tìm thấy nhân viên hoặc ca làm việc cần cập nhật. • 3b. Hệ thống thông báo lỗi và không lưu thay đổi.

3 Kiến trúc phần mềm

3.1 Tổng quan kiến trúc



Hình 8: Sơ đồ tổng quan kiến trúc của IRMS

IRMS được thiết kế theo hướng service-based architecture kết hợp event-driven processing. Kiến trúc này tổ chức hệ thống thành ba lớp chính: lớp giao diện người dùng, lớp dịch vụ nghiệp vụ và lớp dữ liệu cùng các luồng xử lý nền. Cách tổ chức này cho phép hệ thống phân tách rõ ràng các năng lực nghiệp vụ của nhà hàng, đồng thời sử dụng cơ chế sự kiện để xử lý các hoạt động phụ mà không làm ảnh hưởng đến tuyến giao dịch chính.

Ở lớp ngoài cùng, ReactJS đóng vai trò là giao diện người dùng thống nhất cho các vai trò vận hành trong nhà hàng như lễ tân, phục vụ, bếp, thu ngân và quản lý. Việc sử dụng một nền giao diện chung giúp các vai trò khác nhau vẫn chia sẻ cùng cơ chế xác thực, điều hướng và đồng bộ trạng thái hệ thống. Những trạng thái quan trọng như tình trạng bàn, đơn gọi món, phiếu bếp hay hóa đơn cần được cập nhật nhất quán giữa các bộ phận để đảm bảo quá trình phục vụ diễn ra liên tục.

Phía sau lớp giao diện là Frontend Node Server của TanStack Start/Nitro, đóng vai trò phục vụ giao diện React và chuyển tiếp các yêu cầu /api đến ApiGatewayApplication. Thành phần công vào thật trong backend là GatewayProxyController, nơi định tuyến yêu cầu đến các service Spring Boot theo cấu hình trong docker-compose.yml. Nhờ có lớp cổng vào này, frontend không cần biết chi tiết cách backend được tổ chức, từ đó giúp hệ thống có thể thay đổi cấu trúc bên trong mà không ảnh hưởng đến ứng dụng phía người dùng.

Lớp backend được xây dựng bằng Spring Boot và được tổ chức thành các service theo miền nghiệp vụ. Mỗi service chịu trách nhiệm cho một phần rõ ràng trong quy trình vận hành của nhà hàng. Ví dụ, Reservation & Seating Service quản lý đặt bàn và trạng thái bàn; Ordering Service xử lý đơn gọi món; Kitchen Coordination Service điều phối việc chuẩn bị món tại bếp; Billing & Settlement Service chịu trách nhiệm lập hóa đơn và thanh toán; Inventory Monitoring Service theo dõi tồn kho; trong khi Reporting Service phục vụ các truy vấn đọc cho mục đích quản lý. Bên cạnh đó, IAM / Audit Service đảm nhiệm việc kiểm soát truy cập và ghi nhận các hành động quan trọng trong hệ thống.

Việc phân chia các service theo miền nghiệp vụ giúp kiến trúc phản ánh cách nhà hàng vận hành trong thực tế. Khi ranh giới trách nhiệm được xác định rõ, các thay đổi ở một miền nghiệp vụ sẽ ít ảnh hưởng đến các phần còn lại của hệ thống.

Lớp dữ liệu của IRMS sử dụng PostgreSQL để lưu trữ dữ liệu nghiệp vụ cốt lõi và đảm bảo tính nhất quán

của các giao dịch quan trọng như xác nhận đơn gọi món, cập nhật trạng thái bàn hay hoàn tất thanh toán. Bên cạnh dữ liệu giao dịch, hệ thống cũng duy trì một mô hình đọc phục vụ báo cáo, được cập nhật thông qua các luồng xử lý nền thay vì truy vấn trực tiếp lên dữ liệu giao dịch nóng.

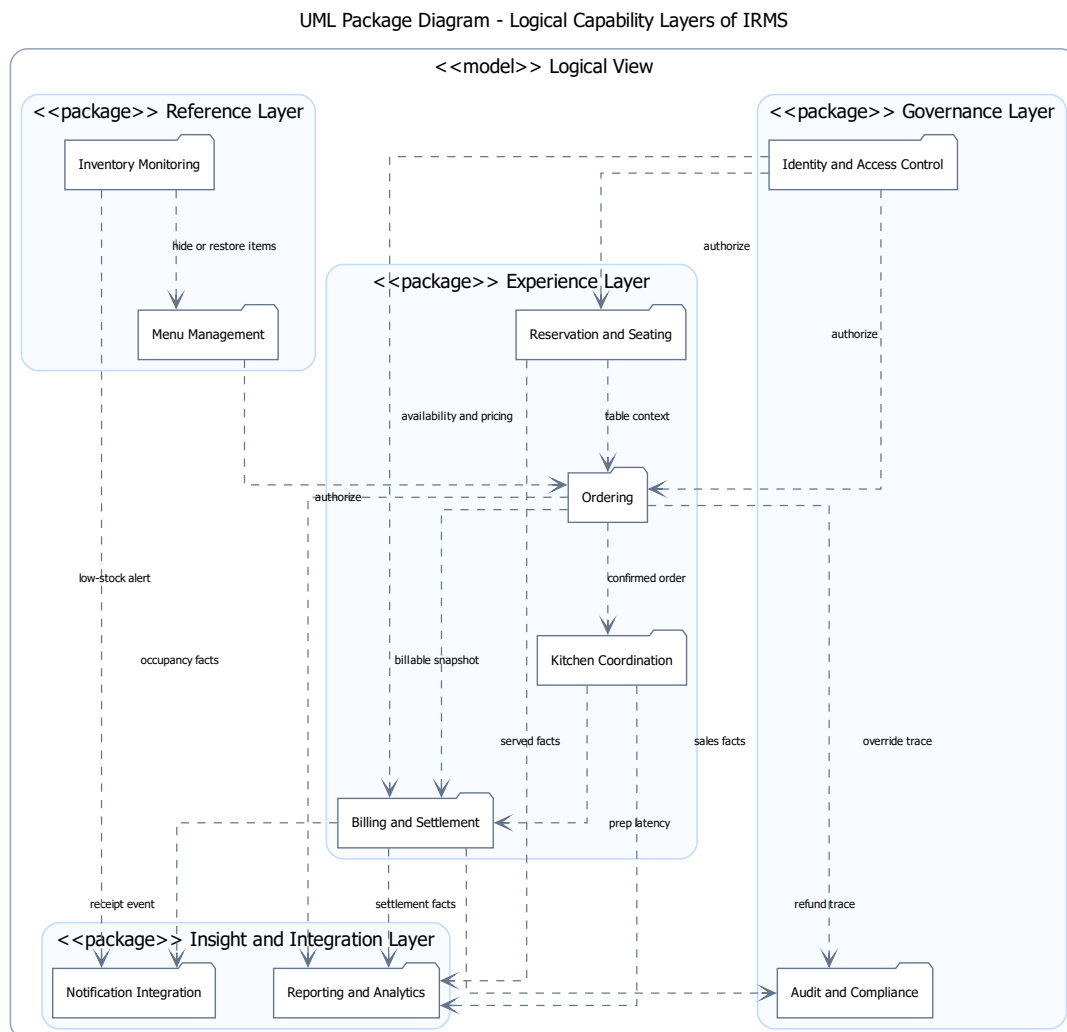
Ngoài tuyến xử lý chính, kiến trúc còn sử dụng `outbox_events`, `processed_events`, RabbitMQ và các consumer nền để xử lý các tác vụ phụ như gửi thông báo, phát hành biên lai hoặc cập nhật mô hình đọc. Các sự kiện phát sinh từ các service nghiệp vụ được ghi nhận và xử lý bất đồng bộ, giúp giảm tải cho các thao tác trực tiếp như gọi món hoặc thanh toán.

Cuối cùng, các điểm tích hợp được đặt ở rìa kiến trúc thông qua các interface và adapter như `PaymentGateway`, `NotificationChannelAdapter` và `ReceiptDeliveryAdapter`. Trong code hiện tại chưa thấy bằng chứng về một cổng thanh toán hay máy in vật lý bên ngoài; các adapter hiện thực hành vi nội bộ hoặc tham chiếu giao dịch để cô lập lỗi nghiệp vụ của IRMS.

Nhìn tổng thể, kiến trúc IRMS hướng đến một cấu trúc rõ ràng và thực dụng: giao diện thống nhất ở tuyến đầu, các service nghiệp vụ giữ ranh giới trách nhiệm rõ ràng ở lõi hệ thống, và các luồng sự kiện được sử dụng để xử lý các hoạt động phụ một cách linh hoạt. Cách tổ chức này giúp hệ thống vừa đảm bảo tính nhất quán cho các giao dịch quan trọng, vừa duy trì khả năng mở rộng và thích nghi khi hệ thống phát triển.

3.2 Tổng quan cho các góc nhìn chính

3.2.1 Tổng quan cho góc nhìn logic



Hình 9: Sơ đồ tổng quan cho góc nhìn logic của IRMS

Hình 9 trình bày góc nhìn logic của hệ thống IRMS dưới dạng UML Package, trong đó hệ thống được tổ chức dựa trên các năng lực nghiệp vụ (capabilities) thay vì theo giao diện người dùng hoặc cấu trúc kỹ thuật. Mục tiêu của góc nhìn này là xác định các miền nghiệp vụ độc lập, đồng thời thể hiện mối quan hệ tương tác giữa chúng trước khi được ánh xạ sang các service trong kiến trúc triển khai.

Trong kiến trúc IRMS, góc nhìn logic được xây dựng theo hướng service-based architecture kết hợp với cơ chế event-driven interaction ở mức khái niệm. Mỗi package tương ứng với một miền năng lực có ranh giới rõ ràng. Sự tương tác giữa các miền được thể hiện thông qua hai cơ chế chính, bao gồm giao tiếp đồng bộ theo mô hình request/response trong luồng nghiệp vụ chính và giao tiếp bất đồng bộ thông qua domain events nhằm truyền tải thông tin vận hành mà không làm tăng mức độ phụ thuộc giữa các miền.

Từ cách tổ chức này, các năng lực nghiệp vụ trong hệ thống được phân nhóm theo vai trò trong kiến trúc tổng

thể.

Thứ nhất, nhóm năng lực tham chiếu và điều kiện vận hành đóng vai trò cung cấp dữ liệu nền và đảm bảo điều kiện cho các luồng nghiệp vụ chính. *Menu Management* chịu trách nhiệm quản lý thông tin món ăn, giá bán và trạng thái hiển thị, trong khi *Inventory Monitoring* theo dõi tình trạng tồn kho và phát hiện các điều kiện thiếu hụt nguyên liệu. Nhóm này không trực tiếp tham gia xử lý giao dịch nhưng ảnh hưởng đến tính hợp lệ của dữ liệu trong toàn bộ hệ thống.

Thứ hai, chuỗi năng lực cốt lõi phản ánh toàn bộ vòng đời phục vụ trong nhà hàng. *Reservation & Seating* đảm nhiệm việc quản lý đặt bàn và phân bổ chỗ ngồi, tạo ngữ cảnh cho *Ordering* trong quá trình tiếp nhận đơn hàng. Sau đó, *Kitchen Coordination* tiếp nhận thông tin từ đơn hàng để điều phối quá trình chế biến món ăn, trong khi *Billing & Settlement* thực hiện thanh toán và kết thúc phiên phục vụ. Các năng lực trong chuỗi này có quan hệ phụ thuộc logic rõ ràng nhằm đảm bảo luồng nghiệp vụ được xử lý nhất quán.

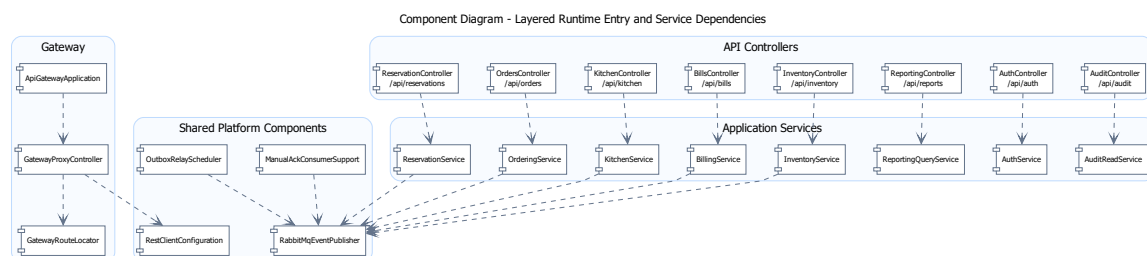
Thứ ba, nhóm năng lực hỗ trợ và phân tích chịu trách nhiệm xử lý thông tin vận hành theo hướng bất đồng bộ. *Notification Integration* tiếp nhận các sự kiện phát sinh để thực hiện thông báo, trong khi *Reporting & Analytics* tổng hợp dữ liệu từ nhiều miền nghiệp vụ nhằm phục vụ thống kê và phân tích. Việc tách nhóm này giúp giảm tải cho luồng xử lý chính và tăng khả năng mở rộng hệ thống.

Thứ tư, nhóm năng lực kiểm soát hệ thống đảm bảo các yêu cầu về bảo mật, phân quyền và truy vết. *Identity & Access Control* quản lý xác thực và phân quyền người dùng, trong khi *Audit & Compliance* ghi nhận lịch sử hoạt động để phục vụ kiểm toán và tuân thủ. Đây là các năng lực xuyên suốt toàn hệ thống nhưng không tham gia trực tiếp vào luồng nghiệp vụ chính.

Tổng thể, góc nhìn logic đóng vai trò như bản đồ năng lực nghiệp vụ của hệ thống IRMS, giúp xác định rõ ranh giới giữa các miền chức năng và cách chúng tương tác với nhau. Đây cũng là nền tảng quan trọng để ánh xạ sang các service trong kiến trúc service-based và hỗ trợ triển khai mô hình event-driven trong các góc nhìn tiếp theo.

3.2.2 Tổng quan cho góc nhìn mô-đun

Trong báo cáo này, góc nhìn phân rã dịch vụ được trình bày theo tinh thần **decomposition view** kết hợp với **uses/layered constraints**. Nói cách khác, phần service không chỉ liệt kê hệ thống có những khối nào, mà còn chỉ ra khối nào được phép phụ thuộc vào khối nào, thông tin nào là phụ thuộc tĩnh, và đâu là những liên hệ chỉ nên xuất hiện dưới dạng hệ quả sự kiện sau giao dịch.



Hình 10: Component Diagram cho góc nhìn phân rã dịch vụ của IRMS

Hình 10 làm rõ quy tắc phụ thuộc theo lớp bằng Component Diagram, bám vào các thành phần runtime thật trong backend. Hình này không phải sơ đồ lớp miền lõi; nó trả lời yêu cầu đi qua thành phần nào, thành phần

nào điều phối use case, thành phần nào phát sự kiện, và thành phần nào là hạ tầng dùng chung chứ không phải service nghiệp vụ. Vì vậy hình này cần được đọc theo các nhóm gateway, api controllers, application services và shared platform components.

Các ý chính cần đọc từ hình.

- GatewayProxyController đi qua GatewayRouteLocator và RestClientConfiguration để định tuyến REST, không gọi trực tiếp repository.
- Các controller như OrdersController, KitchenController và BillsController phụ thuộc vào service ứng dụng tương ứng, đúng với cấu trúc package trong backend.
- Các thành phần hạ tầng dùng chung như RabbitMqEventPublisher, OutboxRelayScheduler và ManualAckConsumerSupport được đặt riêng để xử lý sự kiện và consumer nền.

Vì sao sơ đồ tổng quan này quan trọng.

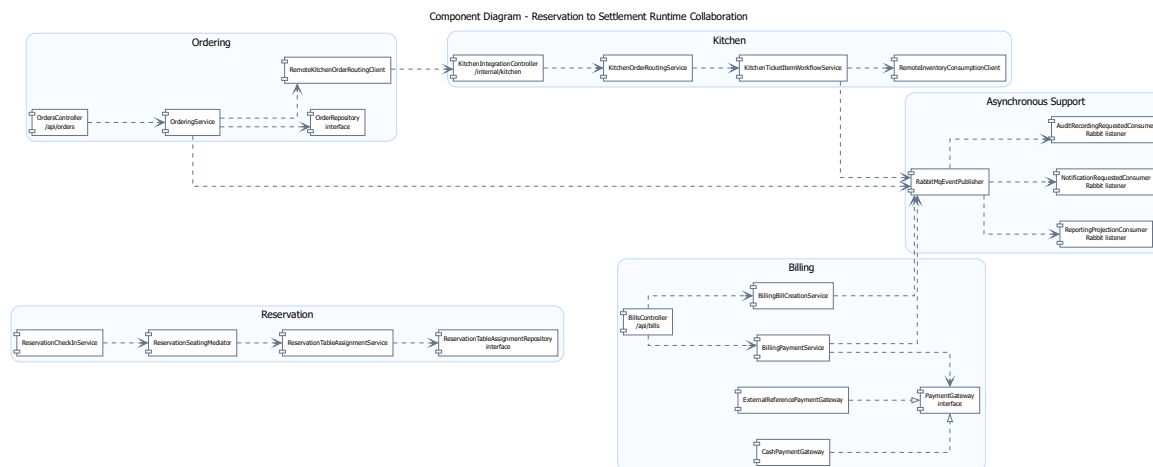
- Nó giúp phần góc nhìn mô-đun không bị dừng ở mức “danh sách service”, mà nâng lên thành **quy tắc tổ chức thành phần và phụ thuộc runtime**.
- Nó tạo cầu nối trực tiếp sang góc nhìn thành phần: từ phụ thuộc runtime, người đọc có thể suy ra vì sao tồn tại bộ điều khiển, dịch vụ ứng dụng, bộ phát sự kiện và consumer nền như những loại thành phần khác nhau.
- Nó cũng hỗ trợ phần SOLID vì từ sơ đồ có thể thấy rõ đường phụ thuộc đi qua thành phần trung gian hoặc interface thay vì gọi chéo tùy tiện.

Catalog mô-đun và ánh xạ mã nguồn. Để tránh nhầm Module View với sơ đồ triển khai runtime, Bảng 31 ghi rõ ranh giới mô-đun theo package thật trong backend. Các hình Component Diagram ở phần này giúp đọc luồng cộng tác giữa thành phần, còn catalog dưới đây neo lại cấu trúc tĩnh của mã nguồn.

Bảng 31: Catalog mô-đun chính của IRMS theo code hiện tại

Module	Trách nhiệm	Source path/package	Class/interface và phụ thuộc chính
Gateway	Điểm vào REST và định tuyến tới các service nghiệp vụ.	irms_project/backend/GatewayProxyController	SA/irms/gateway/GatewayRouteLocator, cấu hình URL service trong application-api-gateway.properties
Reservation	Đặt bàn, danh sách chờ, sơ đồ bàn và phiên phục vụ.	SA/irms/reservation	Controller/service/repository của đặt bàn; phụ thuộc Identity để kiểm soát người thao tác và phát event sau thay đổi trạng thái.
Ordering	Thực đơn, tạo đơn gọi món, tùy biến món, combo và chuyển đơn sang bếp.	SA/irms/ordering	OrderingService, OrderMenuSelectionService, ComboPricingPolicy, OrderStatusPolicy; gọi Kitchen qua client tích hợp khi gửi bếp.
Kitchen	Điều phối phiếu bếp, trạng thái món, deadline, ưu tiên và trừ tồn kho sau phục vụ.	SA/irms/kitchen	KitchenWorkflowMediator, KitchenTicketTransitionPolicy, KitchenDeadlinePolicy, KitchenTicketItemRepositoryPort; tiêu thụ event bằng ManualAckConsumerSupport.
Billing	Lập hóa đơn, tách hóa đơn, thanh toán, hoàn tiền và phát hành biên nhận.	SA/irms/billing	BillingPaymentService, BillingReceiptService, PaymentGateway, ReceiptDeliveryAdapter, BillSplitStrategy; ghi event qua DomainEventPublisher.
Inventory	Theo dõi tồn kho, tiêu hao, cảnh báo mức tồn thấp và đề xuất nhập hàng.	SA/irms/inventory	LowStockSeverityPolicy, ReorderRecommendationStrategy, UsageBasedReorderRecommendationStrategy, repository JDBC và consumer sự kiện tồn kho.
Notification	Ghi nhận yêu cầu thông báo và kiểm tra kênh gửi nội bộ.	SA/irms/notification	NotificationChannelAdapter, EmailNotificationChannelAdapter, SmsNotificationChannelAdapter, InAppNotificationChannelAdapter, PrintNotificationChannelAdapter; chưa có cấu hình nhà cung cấp ngoài.
Reporting	Dashboard, báo cáo bán hàng, vận hành và mô hình đọc.	SA/irms/reporting	Các query/report service đọc dữ liệu giao dịch và projection; nhận hệ quả qua tuyến sự kiện thay vì nằm trên đường ghi nóng.
Identity/Audit	Xác thực, vai trò, quyền, ca làm và audit log.	SA/irms/identity	Policy, session, role/permission và audit service; cung cấp SharedIdentityPolicyPort cho

3.2.3 Góc nhìn phân rã dịch vụ cho tuyến order-to-cash



Hình 11: Component Diagram cho tuyến order-to-cash của IRMS

Hình 11 làm rõ ở mức chi tiết hơn tuyến từ nhận bàn, gọi món, điều phối bếp, quyết toán cho tới phát sinh dư kiện báo cáo và thông báo. Đây là Component Diagram cho cộng tác runtime, không phải sơ đồ lớp miền lõi. Ở mức này, hệ thống được nhìn qua các thành phần runtime thật như ReservationCheckInService, OrderingService, KitchenOrderRoutingService, BillingPaymentService và RabbitMqEventPublisher. Với IRMS, tuyến order-to-cash là nơi mọi quyết định về ranh giới service, API đồng bộ và event bất đồng bộ lộ ra rõ nhất.

Điểm cốt lõi của sơ đồ là **mỗi service sở hữu class điều phối và cổng hạ tầng tương ứng**. Trong hình, ReservationSeatingMediator phối hợp nhận khách và xếp bàn; OrderingService tạo và xác nhận đơn; RemoteKitchenOrderRoutingClient gọi KitchenIntegrationController; KitchenTicketItemWorkflowService cập nhật tiến độ bếp; BillingPaymentService làm việc qua interface PaymentGateway. Các phụ thuộc giữa chúng đi qua API, event hoặc interface, chứ không qua việc truy cập thẳng vào cấu trúc bên trong nhau.

Hình này còn nhấn mạnh sự khác nhau giữa **đường thực thi đồng bộ** và **đường hệ quả sau giao dịch**. Đường thực thi đồng bộ xuất hiện ở các thao tác cần phản hồi ngay như nhận bàn, xác nhận đơn gọi món, lập hóa đơn hoặc ghi nhận thanh toán. Đường hệ quả sau giao dịch đi qua RabbitMqEventPublisher và các consumer như ReportingProjectionConsumer, NotificationRequestedConsumer và AuditRecordingRequestedConsumer. Việc tách hai đường này bảo vệ tuyến đầu khỏi các tác vụ phụ như gửi thông báo, làm giàu audit hoặc làm mới báo cáo.

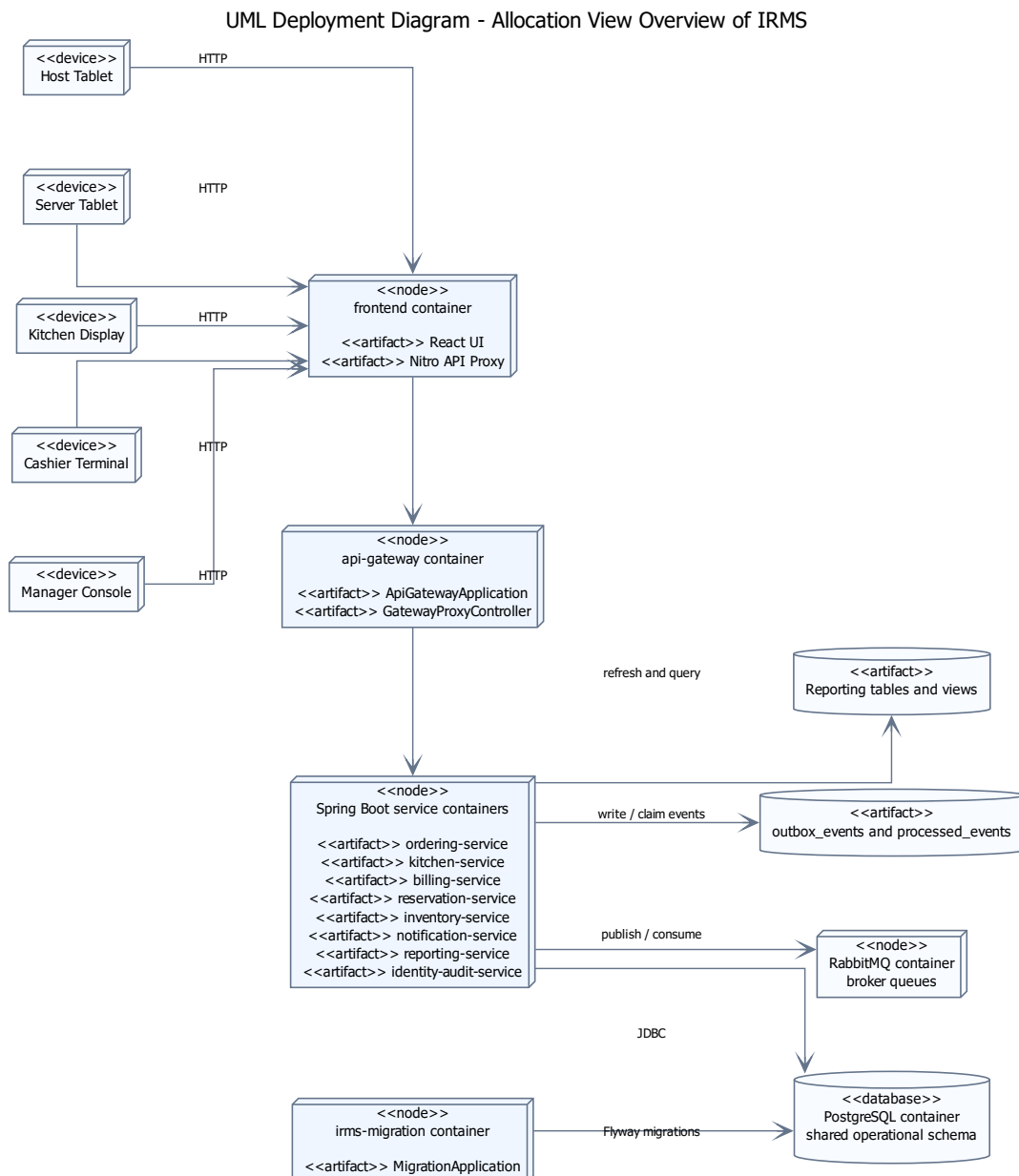
Ngoài ra, sơ đồ còn cho thấy vai trò của các controller và gateway như một biên truy cập nằm trước các service nghiệp vụ. Đây là nơi phù hợp để gom tiếp nhận yêu cầu, xác thực và chuẩn hóa lời gọi. Nhờ có tầng biên này, các service phía sau không phải lặp lại quá nhiều logic kỹ thuật, trong khi phần điều phối ứng dụng vẫn giữ được biên giao dịch riêng của từng service.

Những điểm chính cần đọc từ hình.

- **Lát cắt service** được thể hiện bằng thành phần runtime thật: controller, service ứng dụng, interface và client tích hợp.
- **Tuyến phụ thuộc tĩnh** chỉ đi qua giao diện công khai và client REST cần thiết, nhờ đó tránh phụ thuộc chéo vào chi tiết nội bộ.

- **Tuyến hệ quả sự kiện** được tách sang RabbitMqEventPublisher và các consumer nền, bảo vệ đường nóng khỏi tác vụ phụ.
- **PaymentGateway** là interface thật với hai hiện thực CashPaymentGateway và ExternalReferencePaymentGateway.
- **Quy tắc phụ thuộc runtime** ở hình này là cơ sở để kiểm chứng phần hiện thực mã nguồn: nếu service truy cập xuyên qua chi tiết nội bộ của nhau, kiến trúc sẽ bị phá vỡ ngay ở mức code.

3.2.4 Tổng quan cho góc nhìn phân bổ



Hình 12: Sơ đồ tổng quan cho góc nhìn phân bổ của IRMS

Hình 12 đưa ra bức tranh tổng quan cho góc nhìn phân bổ. Nếu góc nhìn logic trả lời “hệ thống gồm những năng lực nào” và góc nhìn phân rã dịch vụ trả lời “hệ thống phân rã thành những service nào”, thì góc nhìn phân bổ phải trả lời “những service và năng lực đó cuối cùng được đặt ở đâu trong vùng thực thi, dữ liệu và hạ tầng hỗ trợ”. Hình này vì vậy ánh xạ ba chiều: **logic/service** → **phân bổ thực thi** → **phân bổ dữ liệu**.

Các lớp phân bổ chính thể hiện trong hình.

- **Lớp biên và thiết bị:** là nơi phát sinh thao tác nghiệp vụ thực tế như máy của lễ tân, máy của phục vụ, màn hình bếp, máy thu ngân và màn hình quản lý.
- **Lớp phân bổ thực thi:** gồm frontend container, api-gateway container, nhóm container service Spring Boot và irms-migration container; đây là nơi các năng lực được hiện thực thành tiến trình hoặc vùng thực thi cụ thể.
- **Lớp phân bổ dữ liệu và sự kiện:** gồm PostgreSQL container, RabbitMQ container, outbox_events, processed_events và các bảng hoặc view báo cáo; chúng tách dữ liệu giao dịch, dữ liệu đọc tổng hợp và hàng đợi sự kiện.
- **Lớp hệ thống ngoài:** hình này không vẽ hệ thống ngoài vì chưa tìm thấy cấu hình endpoint thật cho cổng thanh toán, nhà cung cấp thông báo hoặc máy in vật lý trong docker-compose.yml và code hiện tại.

Lý do thiết kế của sơ đồ này.

- Sơ đồ giúp người đọc nhìn ra ngay **container nào phục vụ giao diện, container nào làm cổng API, và nhóm service Spring Boot nào chia sẻ PostgreSQL và RabbitMQ**.
- Nó cho thấy rõ vì sao Notification Service, Reporting Service và các consumer không nên bị buộc vào tuyến giao dịch nóng.
- Nó là phần nối đúng giữa góc nhìn thành phần và góc nhìn triển khai: sơ đồ thành phần cho thấy thành phần hợp tác ra sao; góc nhìn phân bổ cho thấy các thành phần đó được đặt trên vùng thực thi và nơi lưu trữ nào.

3.3 Đặc trưng kiến trúc

Dựa trên các yêu cầu phi chức năng đã được lượng hóa ở phần tổng quan, những *đặc trưng kiến trúc* quan trọng nhất của IRMS là:

1. **Khả năng đáp ứng:** màn hình gọi món, màn hình bếp và màn hình thu ngân phải phản hồi nhanh để không làm chậm tuyến phục vụ trực tiếp với khách.
2. **Độ tin cậy:** đơn gọi món, thanh toán, hoàn tiền và cập nhật trạng thái bàn là các luồng không được mất dữ liệu hoặc xử lý lặp sai lệch.
3. **Khả năng mở rộng:** điểm nóng thường dồn vào giờ cao điểm và tập trung ở các miền Ordering, Kitchen Coordination và Billing thay vì toàn hệ thống.
4. **Bảo mật và khả năng kiểm toán:** mọi thao tác nhạy cảm phải được kiểm soát bằng quyền và truy vết lại được về sau.
5. **Khả năng quan sát:** hệ thống phải đủ nhật ký, vết thực thi và số đo để thấy độ trễ của phiếu bếp, thanh toán và các điểm nghẽn vận hành.

6. **Khả năng sửa đổi:** thực đơn, giá, khuyến mãi, ngưỡng cảnh báo tồn kho, kênh gửi thông báo và bộ kết nối thanh toán thay đổi thường xuyên nên cần biên thay đổi hẹp.

Sáu đặc trưng trên không độc lập tuyệt đối mà thường kéo nhau theo hướng căng thẳng. Ví dụ, để tăng **khả năng đáp ứng**, hệ thống có xu hướng rút ngắn chuỗi gọi đồng bộ; nhưng để giữ **độ tin cậy**, một số bước như thanh toán vẫn phải nằm trong giao dịch hoặc giao thức chặt hơn. Tương tự, **khả năng sửa đổi** khuyến khích tách ranh giới nghiệp vụ, nhưng nếu tách quá sớm hoặc quá mạnh thì có thể làm tăng độ phức tạp tích hợp, ảnh hưởng ngược lại **khả năng đáp ứng** và **khả năng bảo trì**. Vì vậy, các quyết định kiến trúc ở phần sau đều được đọc dưới góc nhìn cân bằng giữa các đặc trưng này, không phải tối ưu một đặc tính duy nhất.

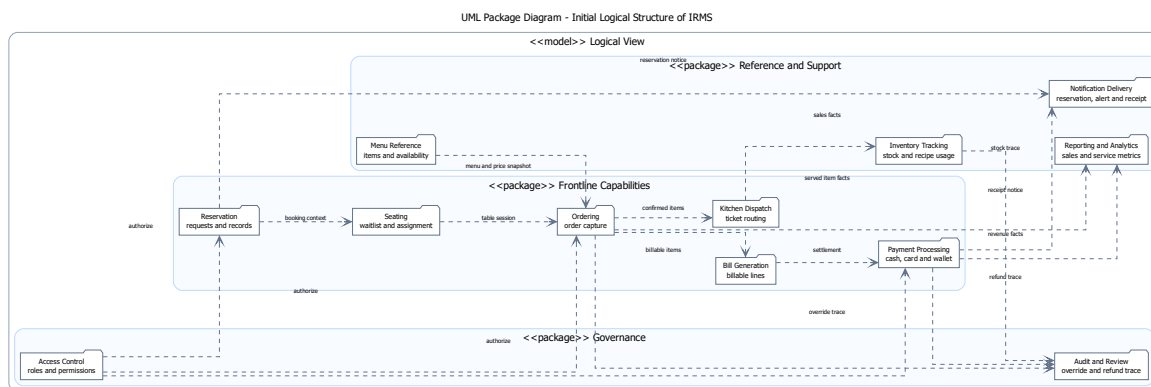
3.4 Góc nhìn logic

Góc nhìn logic mô tả các năng lực nghiệp vụ cốt lõi của hệ thống IRMS và cách các năng lực này được tổ chức thành các miền trách nhiệm có ranh giới rõ ràng. Nếu góc nhìn ngữ cảnh tập trung vào việc hệ thống tương tác với các tác nhân và hệ thống bên ngoài nào, thì góc nhìn logic trả lời câu hỏi: hệ thống cần cung cấp những năng lực nghiệp vụ nào để hiện thực hóa các tương tác đó.

Trong kiến trúc IRMS, góc nhìn logic đóng vai trò như một bản đồ năng lực nghiệp vụ (capability map), làm cơ sở cho việc tổ chức hệ thống theo hướng service-based architecture ở lớp triển khai. Tuy nhiên, ở mức này, các năng lực chưa được hiện thực hóa thành service cụ thể mà chỉ được xem như các miền nghiệp vụ logic độc lập, có ranh giới trách nhiệm rõ ràng.

Các năng lực được xác định dựa trên vòng đời vận hành của nhà hàng, bao gồm tiếp nhận khách, quản lý bàn, gọi món, chế biến, thanh toán và các hoạt động quản trị – giám sát. Về mặt tương tác, các năng lực có thể trao đổi thông tin theo hai cơ chế: đồng bộ trong luồng xử lý nghiệp vụ chính hoặc bất đồng bộ thông qua các sự kiện nghiệp vụ (domain events).

3.4.1 Sơ đồ phân rã năng lực logic ban đầu



Hình 13: Sơ đồ phân rã năng lực logic ban đầu của IRMS

Hình 13 thể hiện bước phân rã ban đầu các năng lực nghiệp vụ trong hệ thống IRMS. Ở giai đoạn này, hệ thống được nhìn dưới góc độ chức năng trực tiếp, phản ánh gần với thao tác người dùng và các quy trình vận hành thực tế trong nhà hàng.

Trong sơ đồ này, các năng lực được xác định theo hướng chức năng bề mặt như quản lý bàn, tiếp nhận đơn

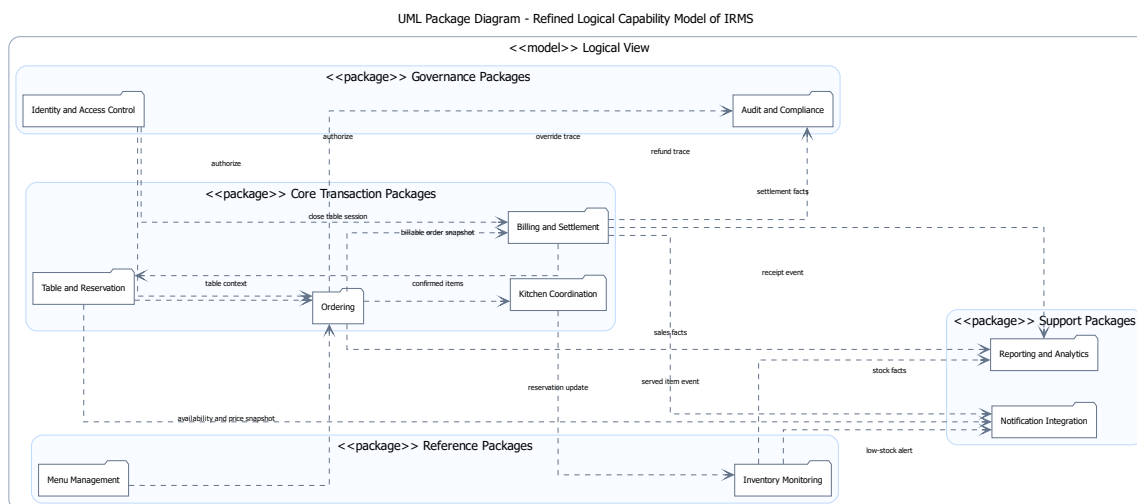
gọi món, điều phối bếp, thanh toán, quản lý tồn kho, báo cáo, kiểm soát truy cập và giám sát hệ thống. Cách phân rã này phù hợp trong giai đoạn đầu của quá trình phân tích, vì giúp đảm bảo bao phủ đầy đủ các yêu cầu nghiệp vụ và phản ánh trực tiếp các hoạt động chính của hệ thống.

Mục tiêu của bước phân rã này không phải là tối ưu kiến trúc, mà là nhận diện đầy đủ phạm vi năng lực của hệ thống IRMS. Do đó, các năng lực được mô tả theo cách gần với quy trình vận hành thực tế để dễ kiểm tra và tránh bỏ sót chức năng quan trọng.

Tuy nhiên, ở mức phân rã này, các năng lực vẫn mang tính rời rạc và chưa hình thành các miền nghiệp vụ ổn định. Một số năng lực có quan hệ chặt chẽ về mặt nghiệp vụ nhưng lại bị tách theo góc nhìn chức năng, dẫn đến phân mảnh trong mô hình logic. Ví dụ, thanh toán và lập hóa đơn thuộc cùng một chuỗi quyết toán nghiệp vụ, hoặc kiểm soát truy cập và kiểm toán có mối quan hệ chặt chẽ trong việc đảm bảo an toàn và tuân thủ hệ thống.

Vì vậy, sơ đồ này được xem là bước khởi tạo trong quá trình phân tích logic, đóng vai trò nền tảng để tiếp tục tái cấu trúc và gom nhóm các năng lực thành các miền nghiệp vụ ổn định hơn ở bước tiếp theo.

3.4.2 Sơ đồ phân rã năng lực logic hoàn chỉnh



Hình 14: Sơ đồ phân rã năng lực logic của IRMS

Dựa trên quá trình tinh chỉnh và gom nhóm các năng lực nghiệp vụ, hệ thống IRMS được tái cấu trúc thành các miền năng lực ổn định hơn như thể hiện trong Hình 14.

Trong cấu trúc này, các năng lực được tổ chức lại theo các miền nghiệp vụ có mức độ kết dính cao hơn và phản ánh đầy đủ vòng đời vận hành của nhà hàng, bao gồm:

- Table & Reservation
- Menu Management
- Ordering
- Kitchen Coordination
- Billing & Settlement

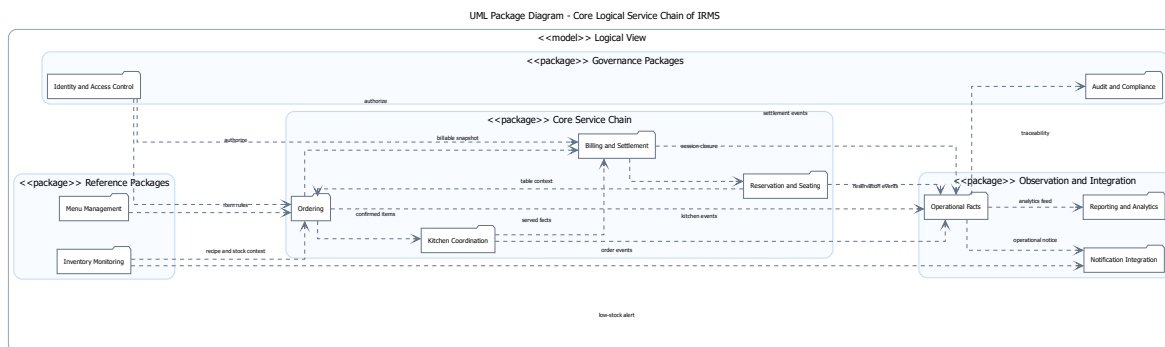
- Inventory Monitoring
- Reporting & Analytics
- Identity & Access Control
- Audit & Compliance
- Notification Integration

Việc tái cấu trúc này giúp chuyển từ cách nhìn theo chức năng thao tác người dùng sang cách nhìn theo miền trách nhiệm nghiệp vụ. Thay vì xem hệ thống như tập hợp các chức năng rời rạc, mỗi năng lực được định nghĩa như một miền logic độc lập, có ranh giới rõ ràng và chịu trách nhiệm cho một phần xác định trong vòng đời vận hành của nhà hàng.

Từ góc độ kiến trúc, đây là bước chuyển từ functional decomposition sang domain-oriented decomposition. Các năng lực lúc này không chỉ mang ý nghĩa chức năng, mà trở thành các đơn vị nghiệp vụ có tính độc lập tương đối, có thể được ánh xạ trực tiếp hoặc gián tiếp sang các service trong kiến trúc service-based.

Bên cạnh đó, cấu trúc này cũng tạo nền tảng cho mô hình event-driven processing trong các góc nhìn kiến trúc tiếp theo. Các miền năng lực có thể trao đổi thông tin thông qua sự kiện nghiệp vụ thay vì phụ thuộc trực tiếp lẫn nhau, từ đó giảm coupling giữa các thành phần và tăng khả năng mở rộng của hệ thống trong dài hạn.

3.4.3 Sơ đồ chuỗi phục vụ lỗi



Hình 15: Góc nhìn logic cho chuỗi phục vụ lỗi của IRMS

Hình 15 mô tả cách các năng lực logic lỗi tương tác với nhau trong chuỗi phục vụ chính của hệ thống IRMS.

Chuỗi này bao gồm bốn miền năng lực chính:

- Reservation & Seating: thiết lập và duy trì ghế cảnh phục vụ
- Ordering: ghi nhận và quản lý đơn gọi món
- Kitchen Coordination: điều phối quá trình chế biến món ăn
- Billing & Settlement: xử lý thanh toán và kết thúc phiên phục vụ

Sơ đồ thể hiện rằng hệ thống không vận hành như tập hợp các chức năng độc lập, mà được tổ chức theo một chuỗi năng lực có thứ tự rõ ràng, phản ánh trực tiếp vòng đời phục vụ của nhà hàng. Mỗi năng lực tương ứng với một giai đoạn trong quy trình nghiệp vụ tổng thể và chỉ chịu trách nhiệm trong phạm vi của giai đoạn đó.

Trong chuỗi này, các miền nghiệp vụ chỉ trao đổi lượng thông tin tối thiểu cần thiết để đảm bảo tính nhất quán của luồng xử lý. Reservation & Seating cung cấp ngữ cảnh phiên bàn hợp lệ; Ordering tạo và xác nhận thông tin đơn gọi món; Kitchen Coordination tiếp nhận và xử lý trạng thái chế biến; trong khi Billing & Settlement sử dụng dữ liệu đã được chốt để thực hiện quyết toán và kết thúc phiên phục vụ.

Cách tổ chức này giúp đảm bảo luồng nghiệp vụ chính có tính tuyến tính và dễ kiểm soát, đồng thời giảm thiểu sự phụ thuộc trực tiếp giữa các miền nghiệp vụ. Về mặt kiến trúc, đây là cơ sở để hiện thực hóa các tương tác giữa các năng lực thông qua cơ chế gọi dịch vụ đồng bộ trong luồng chính và cơ chế sự kiện nghiệp vụ (domain events) cho các tương tác mở rộng trong các góc nhìn triển khai phía sau.

3.4.4 Ý nghĩa kiến trúc của góc nhìn logic

Góc nhìn logic đóng vai trò là lớp nền quan trọng trong kiến trúc service-based của IRMS. Thay vì mô tả hệ thống theo giao diện người dùng hoặc các thành phần kỹ thuật, góc nhìn này xác định các miền nghiệp vụ cốt lõi dưới dạng các capability độc lập, từ đó làm cơ sở trực tiếp cho việc thiết kế các service ở lớp backend.

Trong kiến trúc tổng thể, các miền năng lực logic này được ánh xạ trực tiếp hoặc gián tiếp sang các service tương ứng trong hệ thống (ví dụ: Ordering Service, Billing & Settlement Service, Kitchen Coordination Service). Tùy theo mức độ mở rộng và tối ưu, một miền nghiệp vụ có thể được triển khai thành một service hoặc được tách nhỏ hơn thành nhiều service con. Tuy nhiên, ranh giới nghiệp vụ được xác định ở mức logic cần được giữ ổn định, ngay cả khi cách hiện thực thay đổi.

Bên cạnh đó, góc nhìn logic cũng là cơ sở để tổ chức tương tác giữa các miền nghiệp vụ theo hướng event-driven. Các năng lực nằm ngoài tuyến giao dịch chính như Reporting & Analytics, Notification Integration và Audit & Compliance được xử lý bất đồng bộ thông qua domain events. Cách tiếp cận này giúp giảm phụ thuộc trực tiếp giữa các miền nghiệp vụ, đồng thời hạn chế tải lên luồng xử lý đồng bộ trong các tác vụ quan trọng như gọi món hoặc thanh toán.

Như vậy, góc nhìn logic không chỉ đóng vai trò mô tả trung gian, mà còn là nền tảng định hình cấu trúc service, thiết kế luồng dữ liệu và xác định ranh giới xử lý trong toàn bộ kiến trúc IRMS.

3.5 Góc nhìn phân rã

Ở góc nhìn phân rã, hệ thống được chia thành hai lớp lớn: **các ứng dụng phía người dùng** và **các service nghiệp vụ phía sau**. Việc phân rã bám theo các miền nghiệp vụ của nhà hàng.

3.5.1 Các ứng dụng phía người dùng

- **Host/Reservation UI:** phục vụ đặt bàn, danh sách chờ, sơ đồ bàn và check-in.
- **Server POS UI:** phục vụ gọi món tại bàn, thay đổi món, theo dõi phiếu bếp và thực hiện tính tiền.
- **Kitchen Display UI:** hiển thị phiếu bếp theo trạm, mức ưu tiên, thời gian dự kiến hoàn thành và tiến độ chế biến.
- **Cashier UI:** tạo hóa đơn, tách hóa đơn, thanh toán, biên lai và hoàn tiền.
- **Manager Portal:** quản lý thực đơn, chương trình khuyến mãi, cảnh báo tồn kho, lịch làm việc và báo cáo.

3.5.2 Các dịch vụ nghiệp vụ phía sau

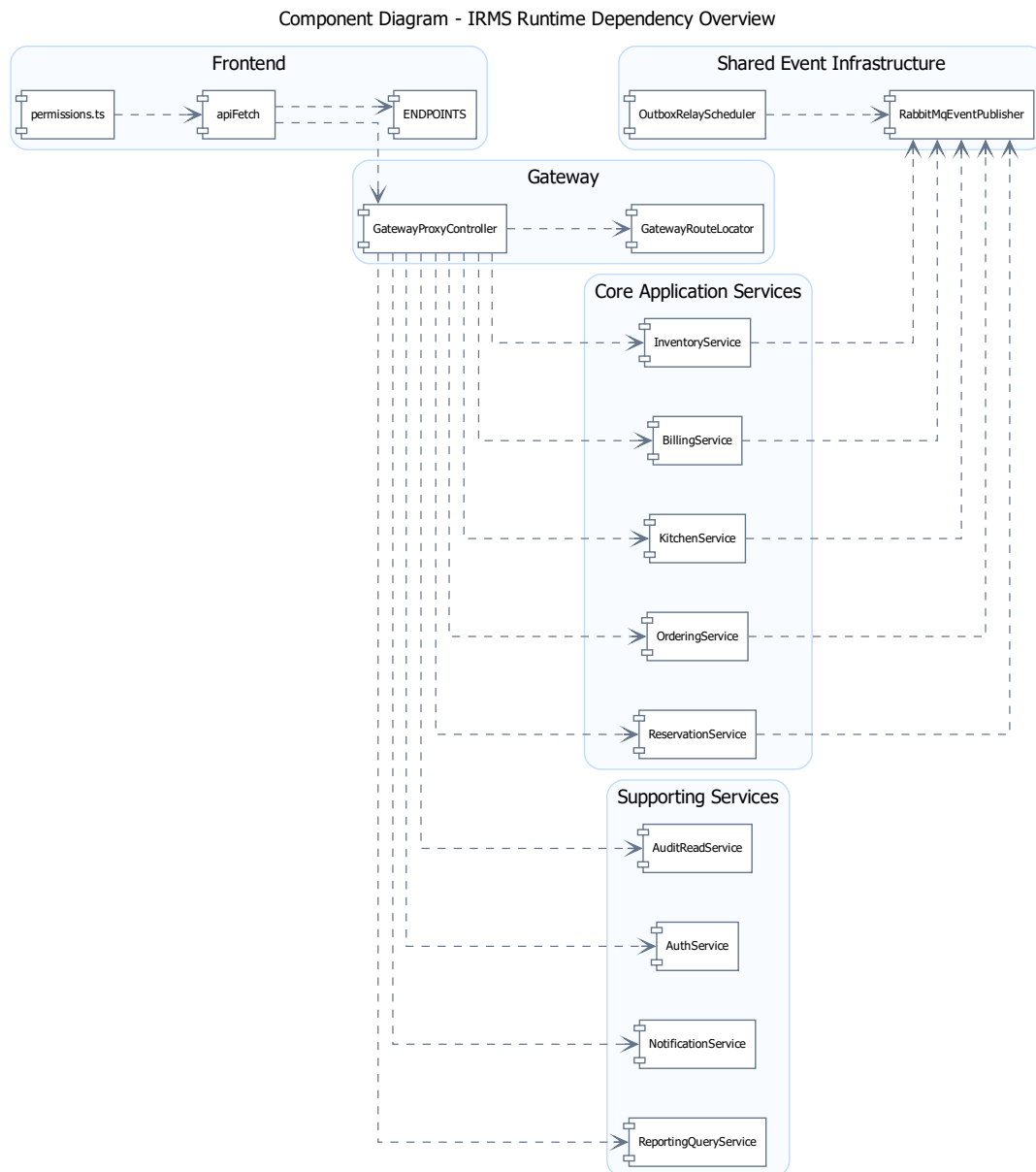
- **Reservation & Seating Service:** quản lý đặt bàn, danh sách chờ, bàn ăn và phiên phục vụ tại bàn.

- **Ordering Service:** quản lý thực đơn, tùy chọn món, đơn gọi món, dòng món và trạng thái của đơn từ đầu đến cuối.
- **Kitchen Coordination Service:** điều phối phiếu bếp, trạm bếp và cam kết thời gian ra món.
- **Billing Service:** quản lý hóa đơn, tách hóa đơn, thanh toán, biên lai, hoàn tiền và áp dụng khuyến mãi.
- **Inventory Service:** quản lý công thức, mặt hàng tồn kho, giao dịch kho và quy tắc cảnh báo mức tồn thấp.
- **Reporting Service:** quản lý mô hình đọc, ảnh chụp dữ liệu báo cáo và xuất báo cáo.
- **IAM/Access Service:** định danh, vai trò, quyền và thực thi chính sách truy cập.

Lưu ý về thuật ngữ “dịch vụ”. Trong góc nhìn phân rã này, từ “service” được dùng để chỉ **ranh giới năng lực nghiệp vụ có hợp đồng giao tiếp rõ ràng**. Mỗi service có trách nhiệm, dữ liệu và luật nghiệp vụ riêng; đồng thời công bố API đồng bộ cho thao tác cần phản hồi trực tiếp và event bất đồng bộ cho hệ quả sau giao dịch. Điểm này giúp tránh nhầm lẫn giữa service nghiệp vụ với các lớp kỹ thuật bên trong chương trình.

Giải thích chi tiết cho góc nhìn phân rã. Phân rã như trên giúp ranh giới trách nhiệm rõ ràng: thay đổi ở bếp không kéo trực tiếp vào thanh toán; thay đổi ở thực đơn không làm nứt mô hình đặt bàn; báo cáo không đung vào luồng ghi thời gian thực. Đây là cách phân rã phù hợp với IRMS vì ranh giới miền nghiệp vụ của nhà hàng đã rất rõ theo thực tế vận hành.

3.5.3 Sơ đồ tổng quan



Hình 16: Component Diagram tổng quan cho phân rã dịch vụ của IRMS

Hình 16 thể hiện góc nhìn mô-đun bằng Component Diagram ở mức đủ rộng để nhìn ra các thành phần chính đang giữ trách nhiệm runtime trong hệ thống. Đây không phải sơ đồ lớp miền lõi; nó trả lời câu hỏi mà bất kỳ người phản biện nào cũng sẽ quan tâm: yêu cầu từ frontend đi qua thành phần nào, service ứng dụng nào chịu trách nhiệm chính, và ranh giới thành phần nào là ranh giới kiến trúc đáng bảo vệ.

Điều cần đọc đầu tiên ở hình này không phải là số lượng service, mà là **cách các thành phần runtime được chia theo nghĩa nghiệp vụ**. ReservationService đại diện cho vùng đặt bàn; OrderingService xử lý tạo và

xác nhận đơn; KitchenService xử lý trạng thái phiếu bếp; BillingService xử lý lập hóa đơn; còn InventoryService, ReportingQueryService, NotificationService, AuthService và AuditReadService giữ các vùng hỗ trợ và kiểm soát.

Một điểm quan trọng khác của sơ đồ là **các phụ thuộc giữa thành phần**. apiFetch gọi các endpoint đã khai báo trong ENDPOINTS, sau đó GatewayProxyController định tuyến đến service phù hợp. Các service nghiệp vụ cùng phát sự kiện qua RabbitMqEventPublisher, còn OutboxRelayScheduler xử lý phân chuyển tiếp nền. Cách đặt đó giúp người đọc thấy ngay nơi nào phải ưu tiên tính đúng đắn tức thời, nơi nào có thể chấp nhận độ trễ nhỏ và nơi nào phải ưu tiên khả năng truy vết.

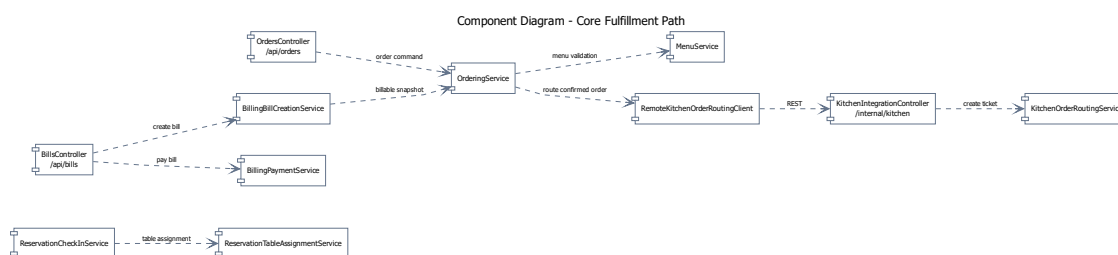
Sơ đồ còn quan trọng ở chỗ nó làm rõ **điểm dễ hỏng nếu thiết kế kém kỷ luật**. Trong một hệ thống nhà hàng, rất nhiều khái niệm có liên hệ chặt với nhau: đơn gọi món liên quan đến hóa đơn; phiếu bếp liên quan đến dòng món; tồn kho lại suy diễn từ các món đã phục vụ hoặc các quy tắc nghiệp vụ khác. Nếu không có ranh giới class đủ chặt, hệ thống sẽ rất dễ rơi vào tình trạng bất kỳ thay đổi nhỏ nào ở khuyến mãi, thực đơn hay thanh toán cũng kéo theo một chuỗi sửa chéo trên nhiều vùng mã. Hình này vì vậy không chỉ mô tả cấu trúc hiện tại, mà còn là công cụ để tự kiểm tra chất lượng kiến trúc trong suốt vòng đời phát triển.

Các thành phần chính thể hiện trong hình.

- **Các thành phần frontend và gateway:** apiFetch, ENDPOINTS, permissions.ts, GatewayProxyController và GatewayRouteLocator.
- **Các service ứng dụng:** ReservationService, OrderingService, KitchenService, BillingService, InventoryService, ReportingQueryService, NotificationService, AuthService và AuditReadService.
- **Các thành phần sự kiện dùng chung:** RabbitMqEventPublisher và OutboxRelayScheduler.

Lý do thiết kế của sơ đồ này.

- Đây là sơ đồ tổng quan của góc nhìn mô-đun vì nó mô tả toàn bộ cấu trúc thành phần runtime trước khi tách riêng tuyến lỗi và tuyến hỗ trợ.
- Việc nhìn dependencies ở mức thành phần giúp người đọc đánh giá nhanh mức kết tụ và mức phụ thuộc chéo của kiến trúc.



Hình 17: Component Diagram cho luồng thực thi nghiệp vụ lõi

Hình 17 đi sâu vào tuyến thực thi nghiệp vụ lõi của IRMS bằng các thành phần runtime thật trong code, tức tuyến mà mỗi giây chậm đi hoặc mỗi quyết định sai lệch đều tác động trực tiếp tới khách hàng đang ngồi tại bàn. Đây là vùng kiến trúc phải được đọc cẩn thận nhất vì nó quyết định hệ thống có thực sự dùng được trong giờ cao điểm hay không.

Trung tâm của hình là OrderingService. Cách đặt class này ở tâm không nhằm làm vùng gọi món trở nên “quyền lực”, mà để phản ánh đúng bản chất dữ liệu của bài toán. Tại thời điểm gọi món, hệ thống phải biết phiên

bàn, món còn bán hay không, cấu hình tùy chọn có hợp lệ hay không, và sau khi xác nhận thì thông tin nào phải được lan truyền sang bếp qua RemoteKitchenOrderRoutingClient. Vì vậy, nếu đặt trung tâm vào nơi khác, mô hình sẽ méo theo kỹ thuật chứ không còn bám đúng nghiệp vụ.

Tuy nhiên, chính vì ở trung tâm nên OrderingService cũng là class dễ bị phình to nhất. Nếu cả khuyến mãi phức tạp, tính toán kho, gửi thông báo, phân tích và kiểm toán chi tiết đều bị dồn vào cùng một chỗ, class này sẽ nhanh chóng trở thành “điểm dồn mọi thứ”. Hình này vì vậy không chỉ cho thấy vai trò trung tâm của OrderingService, mà còn nhắc rất rõ biên của nó: nó phải tập trung vào dữ liệu gọi món và chuyển tiếp nghiệp vụ trực tiếp; còn các hệ quả hỗ trợ cần được chuyển sang service khác hoặc luồng xử lý khác.

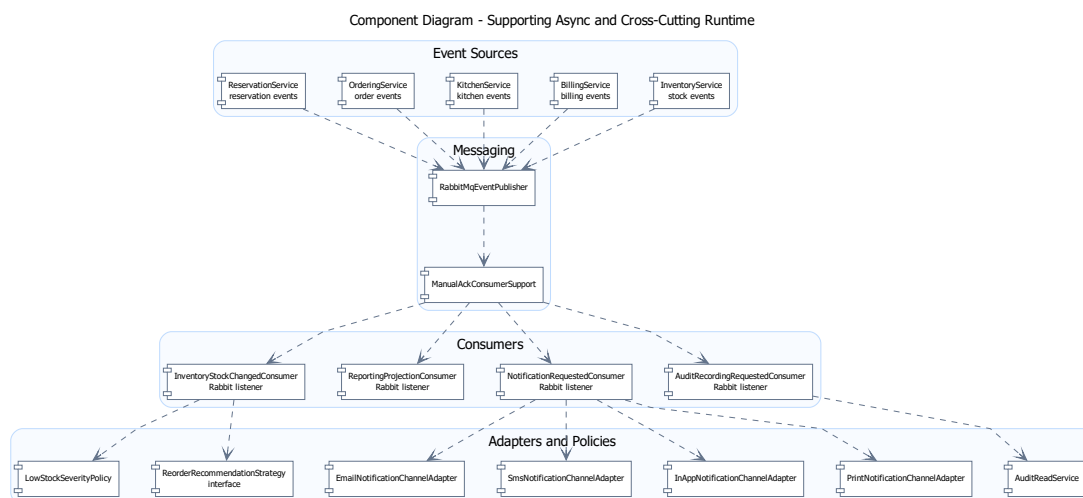
Mối quan hệ giữa OrderingService, KitchenOrderRoutingService và BillingBillCreationService trong hình cũng rất quan trọng. Sau khi đơn gọi món được xác nhận, tuyến bếp nhận thông tin qua endpoint nội bộ để tổ chức chế biến nhưng không quyết định ngược trở lại những bất biến gốc của đơn gọi món. Tương tự, vùng quyết toán cần ảnh chụp đáng tin cậy của món đã gọi và món đã phục vụ; nhưng không nên trở thành nơi quay lại chỉnh sửa cấu trúc gọi món gốc. Cách sắp xếp này giúp chuỗi nghiệp vụ có hướng đi rõ ràng: từ ngữ cảnh bàn, sang xác nhận đơn gọi món, sang điều phối bếp, rồi tới quyết toán.

Các thành phần chính thể hiện trong hình.

- **ReservationCheckInService và ReservationTableAssignmentService:** cung cấp ngữ cảnh bàn đang phục vụ.
- **OrderingService:** giữ vai trò trục trung tâm của tuyến nghiệp vụ nóng.
- **KitchenOrderRoutingService và BillingBillCreationService:** là hai hướng lan truyền quan trọng sau khi đơn được xác nhận.

Lý do thiết kế của sơ đồ này.

- Sơ đồ được tách riêng để người đọc nhìn rõ đường nghiệp vụ nóng thay vì bị chìm trong toàn bộ hệ thống.
- Việc nhấn mạnh tâm của OrderingService giúp giải thích vì sao class này cần được bảo vệ khỏi việc ôm thêm quá nhiều trách nhiệm.



Hình 18: Component Diagram cho các service hỗ trợ và kiểm soát

Hình 18 giải thích phần còn lại của kiến trúc bằng các thành phần hỗ trợ và xuyên suốt, tức những thành phần không trực tiếp đứng trên đường giao dịch nóng nhưng vẫn quyết định chất lượng vận hành của hệ thống trong dài hạn. Đây là Component Diagram cho messaging và runtime support, không phải sơ đồ lớp miền lõi. Với IRMS, InventoryStockChangedConsumer, ReportingProjectionConsumer, NotificationRequestedConsumer và AuditRecordingRequestedConsumer đều là các consumer quan trọng, chỉ khác ở chỗ chúng phục vụ mục tiêu khác với gọi món và thanh toán.

InventoryStockChangedConsumer suy diễn từ kết quả vận hành sang biến động kho; ReportingProjectionConsumer chuyển dữ liệu nghiệp vụ thành góc nhìn quản trị; NotificationRequestedConsumer chịu trách nhiệm tạo thông báo; còn AuditRecordingRequestedConsumer bảo vệ tính kiểm soát của toàn hệ thống. Các thành phần hỗ trợ không tồn tại lơ lửng, mà được nuôi bởi event phát ra từ ReservationService, OrderingService, KitchenService, BillingService và InventoryService thông qua RabbitMqEventPublisher.

Tách ra riêng không có nghĩa là xem nhẹ. Chính các service hỗ trợ này mới giúp IRMS trở thành một hệ thống vận hành hoàn chỉnh thay vì chỉ là công cụ nhập món. Nhà quản lý cần số liệu; bộ phận vận hành cần cảnh báo tồn kho; tổ chức cần khả năng truy vết thao tác; hệ thống cần phân quyền đủ chặt để tránh lạm dụng quyền hạn. Tuyến lỗi phản ánh trạng thái phục vụ đang diễn ra, còn nhóm service hỗ trợ phản ánh cách toàn bộ nhà hàng được điều hành, quan sát và kiểm soát.

Đánh đổi đi cùng cách tổ chức này là hệ thống phải chấp nhận một mức **độ trễ quan sát có kiểm soát**. Báo cáo có thể chậm hơn vài giây hoặc vài phút; cảnh báo mức tồn thấp có thể đến sau giao dịch gốc một khoảng nhỏ; dữ liệu kiểm toán có thể được làm giàu thêm chi tiết sau khi hành động chính đã hoàn tất. Đây là đánh đổi phù hợp trong bối cảnh nhà hàng, vì thao tác cần phản hồi ngay là đặt bàn, gọi món, bếp và thanh toán; còn tác vụ cần đúng, đủ và truy vết được ở tầng quản trị có thể đi sau một nhịp nhỏ.

Các thành phần chính thể hiện trong hình.

- **ReservationService, OrderingService, KitchenService, BillingService và InventoryService:** là các thành phần nguồn phát sinh dữ kiện vận hành để nhóm hỗ trợ tiêu thụ.
- **RabbitMqEventPublisher và ManualAckConsumerSupport:** là lớp trung gian chuẩn hóa và xử lý sự kiện trước khi sang nhóm hỗ trợ.
- **InventoryStockChangedConsumer, ReportingProjectionConsumer và NotificationRequestedConsumer:** đại diện cho nhóm suy diễn, tổng hợp và tích hợp có thể chạy lệch nhịp với đường nóng.
- **AuditRecordingRequestedConsumer và AuditReadService:** đại diện cho nhóm kiểm soát xuyên suốt.
- **Các adapter kênh thông báo:** gồm EmailNotificationChannelAdapter, SmsNotificationChannelAdapter, InAppNotificationChannelAdapter và PrintNotificationChannelAdapter.

Lý do thiết kế của sơ đồ này.

- Tách sơ đồ này ra khỏi sơ đồ tuyến lỗi giúp người đọc thấy rõ phần nào không nên chen trực tiếp vào tuyến giao dịch.
- Hình này cũng làm rõ tại sao một số phần có thể chấp nhận độ trễ nhỏ nhưng vẫn phải giữ được tính truy vết và khả năng kiểm soát.

3.6 Góc nhìn thành phần và kết nối

Ở góc nhìn thành phần và kết nối, hệ thống được mô tả bằng các thành phần thực thi chính và các kiểu liên kết giữa chúng. Góc nhìn này đặc biệt quan trọng vì nó cho biết một yêu cầu đi qua những khối nào, khối nào giữ quyết định nghiệp vụ và khối nào chỉ đóng vai trò tích hợp hoặc hỗ trợ kỹ thuật.

3.6.1 Các thành phần chính

Góc nhìn này bóc tách hệ thống thành các khối thực thi cụ thể, đảm bảo sự phân tách rạch ròi giữa trải nghiệm giao diện, điều phối luồng, luật nghiệp vụ và tích hợp hạ tầng phần cứng lẫn phần mềm:

- **Các ứng dụng giao diện (Client Applications):** Gồm giao diện cho lễ tân, phục vụ, bếp, thu ngân và quản lý. Lớp này tiếp nhận thao tác người dùng, gọi các endpoint trong ENDPOINTS của frontend và hiển thị lại dữ liệu do backend trả về:
 - Dùng `apiFetch` và ENDPOINTS để gọi REST API qua Nitro API Proxy.
 - Dùng `permissions.ts` và dữ liệu vai trò từ `/api/auth/me` để kiểm soát điều hướng theo quyền trên giao diện.
- **Cổng vào hệ thống (API Gateway & Frontend Proxy):** Frontend Node Server phục vụ React và chuyển tiếp `/api`; `GatewayProxyController` trong `api-gateway` định tuyến yêu cầu REST đến các service phía sau. Code hiện tại chưa cho thấy WebSocket/SSE, rate limiting hay cấu hình Nginx riêng.
- **Các dịch vụ ứng dụng (Application Services):** Chịu trách nhiệm điều phối các ca sử dụng (Use Cases). Đặc trưng kiến trúc của lớp này là *tuyệt đối không chứa bất kỳ luật nghiệp vụ nào*. Nó chỉ làm nhiệm vụ quản lý ranh giới giao dịch (Transaction Boundaries), gọi các đối tượng miền tương ứng và ủy quyền việc phát sự kiện.
- **Các service miền nghiệp vụ (Domain Services):** Là trái tim của kiến trúc, nơi cư trú của các **Thực thể gốc (Aggregate Roots)** và các chính sách bảo vệ **Bất biến nghiệp vụ (Business Invariants)**. Quyết định nghiệp vụ (ví dụ: công thức tính tiền, trạng thái khả dụng của bàn) chỉ được chốt tại đây.
- **Lớp lưu trữ dữ liệu (Data Persistence):** Thực hiện lưu trữ thông qua mẫu Repository nhằm cô lập nghiệp vụ khỏi nền tảng SQL. Lớp này phân tách rõ giữa **Mô hình ghi (Transactional Schema)** phục vụ cho tốc độ giao dịch nóng và **Mô hình đọc (Reporting Projections)** phục vụ truy vấn phân tích để không gây nghẽn cổ chai cơ sở dữ liệu.
- **Bộ truyền sự kiện và xử lý nền (Event Bus & Background Workers):** Đảm bảo tính nhất quán cuối cùng (Eventual Consistency) của hệ thống. Hệ thống áp dụng **Outbox Pattern** với **Outbox Dispatcher** để quét và đẩy sự kiện một cách đáng tin cậy. Các hệ quả bất đồng bộ (như cập nhật kho, gửi thông báo) được đưa vào hàng đợi nền để không làm chậm luồng thao tác trực tiếp của nhân viên.
- **Các bộ kết nối thích nghi (Adapters):** Được thiết kế theo nguyên lý Đảo ngược phụ thuộc (Dependency Inversion), chia làm hai nhóm đã thấy trong code nhằm cách ly lỗi nghiệp vụ:
 - *Bộ kết nối thanh toán:* `CashPaymentGateway` và `ExternalReferencePaymentGateway` hiện thực interface `PaymentGateway`.
 - *Bộ kết nối thông báo và biên lai:* `EmailNotificationChannelAdapter`, `SmsNotificationChannelAdapter`, `InAppNotificationChannelAdapter`, `PrintNotificationChannelAdapter`, `PrintReceiptDeliveryAdapter` và `DigitalReceiptDeliveryAdapter`.

3.6.2 Các kiểu kết nối chính

Để đáp ứng yêu cầu về hiệu năng cao và tính nhất quán của dữ liệu trong môi trường vận hành nhà hàng, IRMS kết hợp các kiểu kết nối đã có trong code và cấu hình. Các tương tác giữa các thành phần được định nghĩa qua bảng sau:

Bảng 32: Các kiểu kết nối chính trong IRMS

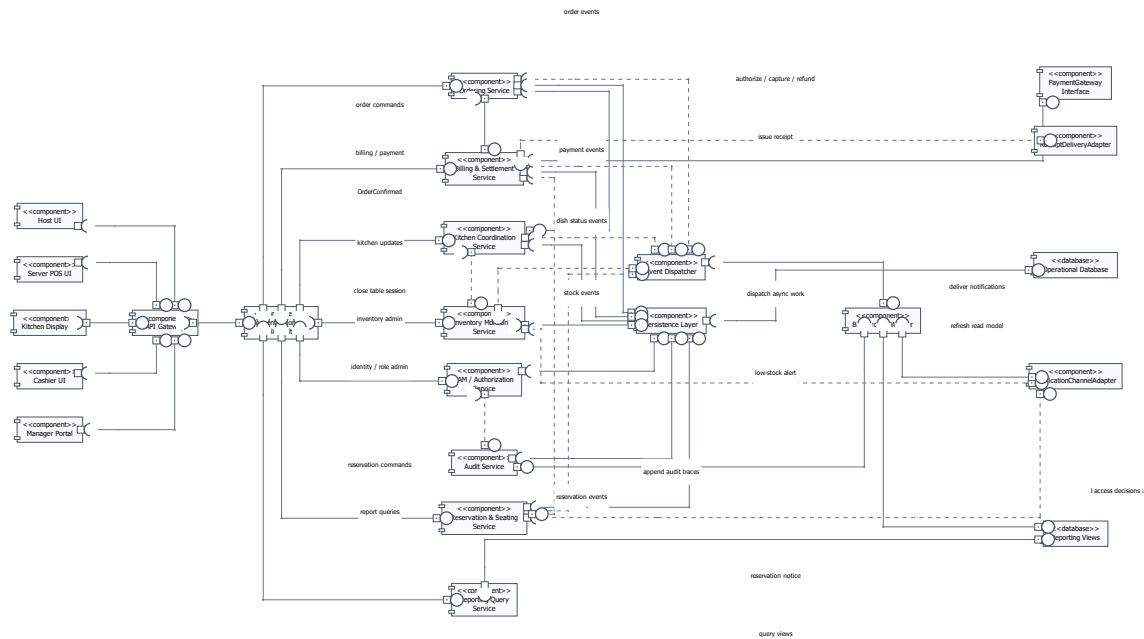
Kiểu kết nối	Vai trò và Đặc tính kỹ thuật	Ví dụ thực tế trong IRMS
Giao thức HTTP (REST API)	Kết nối đồng bộ (Synchronous), ưu tiên độ trễ thấp và phản hồi trạng thái giao dịch tức thì.	POS App gửi yêu cầu xác nhận đơn gọi món hoặc lập hóa đơn.
REST qua apiFetch	Frontend gọi định kỳ hoặc gọi lại sau thao tác người dùng để tải trạng thái mới.	Màn hình bếp gọi /api/kitchen/overview; màn hình báo cáo gọi các endpoint trong /api/reports.
Thông điệp Sự kiện (Pub/Sub)	Kết nối bất đồng bộ (Asynchronous) qua Message Broker, giúp giảm sự phụ thuộc trực tiếp (Decoupling) giữa các module.	Lan truyền sự kiện đơn hàng hoàn tất để trừ tồn kho và làm giàu dữ liệu báo cáo.
Adapter nội bộ	Interface và adapter trong code để cô lập biến thể nghiệp vụ hoặc kênh phát hành.	PaymentGateway, NotificationChannelAdapter và ReceiptDeliveryAdapter.
Kết nối Database (JDBC / Driver)	Truy xuất dữ liệu có cấu trúc và quản lý ranh giới giao dịch (Atomic Transactions).	Business Services lưu trữ trạng thái bàn và phiên phục vụ vào PostgreSQL.

3.6.3 Biện minh cho mô hình kết nối hỗn hợp

Kiến trúc IRMS không sử dụng duy nhất một kiểu kết nối mà kết hợp linh hoạt dựa trên hai nguyên tắc cốt lõi:

- Tuyến đầu ưu tiên đồng bộ (Sync for UX):** Các thao tác trực tiếp trước mặt khách hàng (đặt bàn, gọi món, thanh toán) được thực hiện qua API đồng bộ. Điều này giúp nhân viên nhận được phản hồi "Thành công" hoặc "Thất bại" ngay lập tức, đảm bảo luồng phục vụ không bị gián đoạn.
- Hậu phương ưu tiên bất đồng bộ (Async for Resilience):** Các hệ quả phát sinh sau giao dịch như thông báo, cập nhật kho, audit và báo cáo được đẩy sang tuyến sự kiện qua RabbitMQ, outbox_events và consumer nền. Việc này giúp bảo vệ hệ thống: nếu tác vụ phụ bị chậm, nó không làm nghẽn luồng gọi món chính.

Giải thích chi tiết cho góc nhìn thành phần và kết nối. Điểm then chốt của IRMS là phân biệt *kết nối phục vụ trải nghiệm người dùng* với *kết nối phục vụ phối hợp nội bộ*. Chọn đúng kiểu kết nối cho từng nhóm thành phần sẽ quyết định trực tiếp đến độ trễ phản hồi, khả năng giải thích lỗi và khả năng mở rộng của toàn hệ thống.



Hình 19: Sơ đồ thành phần và kết nối tổng quan của IRMS

Hình 19 cho thấy cấu trúc thực thi tổng quan của hệ thống theo cách nhìn thành phần. Thiết bị người dùng chỉ giao tiếp với lớp cổng vào; từ đó yêu cầu đi qua lớp điều phối ca sử dụng, xuống miền nghiệp vụ, lớp lưu trữ và các adapter biên. Ý nghĩa học thuật của hình này là chứng minh hệ thống không được tổ chức như một khối gọi chéo tùy ý, mà có hướng phụ thuộc rõ ràng từ ngoài vào trong.

Các thành phần chính thể hiện trong hình.

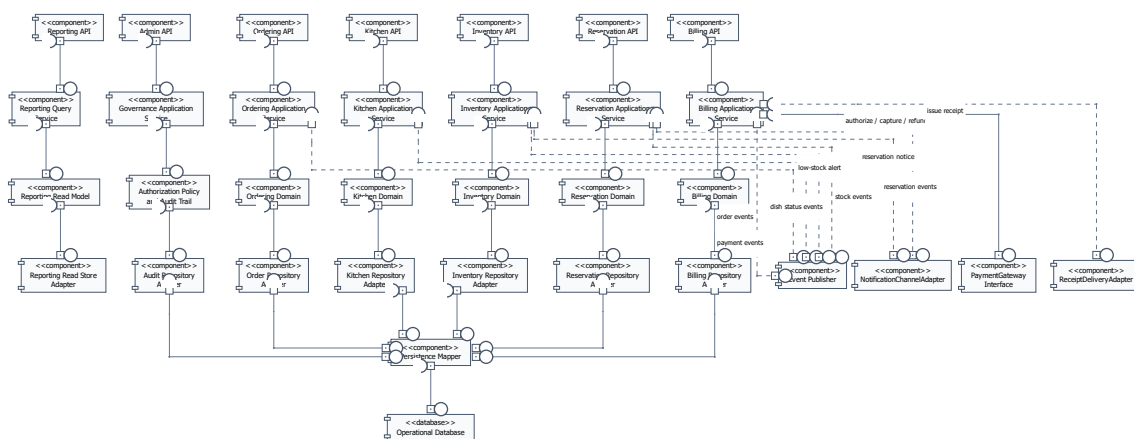
- **Các ứng dụng giao diện:** là nơi phát sinh yêu cầu từ lễ tân, phục vụ, bếp, thu ngân và quản lý.
- **Cổng vào giao diện lập trình ứng dụng:** tiếp nhận yêu cầu, xác thực, kiểm tra đầu vào và chuyển tiếp tới đúng ca sử dụng.
- **Các dịch vụ ứng dụng:** điều phối từng ca sử dụng cụ thể như tạo đặt bàn, xác nhận đơn gọi món, cập nhật phiếu bếp hoặc chốt hóa đơn.
- **Các service miền nghiệp vụ:** giữ luật nghiệp vụ cốt lõi, bắt biến dữ liệu và quyết định nghiệp vụ của nhà hàng.
- **Lớp lưu trữ dữ liệu:** chịu trách nhiệm lưu trữ trạng thái giao dịch và bảo vệ ranh giới giao dịch.
- **Bộ phát sự kiện và bộ xử lý nền:** xử lý sự kiện và tác vụ nền không nên nằm trên tuyến giao dịch nóng.
- **Các port/adaptor biên:** bao bọc cách xử lý thanh toán, gửi thông báo và phát hành biên lai.

Lý do thiết kế của sơ đồ này.

- Việc đặt **Cổng vào giao diện lập trình ứng dụng** ở phía trước giúp thống nhất xác thực, kiểm tra yêu cầu và ghi nhận ký ở biên thay vì lặp lại ở mọi điểm vào.
- Việc tách **Các dịch vụ ứng dụng** khỏi **Các service miền nghiệp vụ** giúp phân điều phối ca sử dụng không lẫn với luật nghiệp vụ cốt lõi.

- Việc đẩy **Các port/adapter biên** ra rìa giúp miền nghiệp vụ không phụ thuộc trực tiếp vào cách xử lý thanh toán, kênh gửi thông báo hay cách phát hành biên lai.
- Việc tách **Bộ phát sự kiện và bộ xử lý nền** thành thành phần riêng giúp các tác vụ phụ không làm chậm các thao tác đang diễn ra trực tiếp trước khách hàng.

Ưu điểm của sơ đồ này là chiều phụ thuộc được kiểm soát rõ: giao diện không phụ thuộc trực tiếp vào miền nghiệp vụ, miền nghiệp vụ không gọi ngược bộ điều khiển, còn các port/adapter biên chỉ xuất hiện ở rìa hệ thống. Nhược điểm là việc hiện thực phải giữ kỷ luật rõ ràng; nếu đưa luật nghiệp vụ vào bộ điều khiển hoặc để miền nghiệp vụ gọi thẳng adapter cụ thể thì giá trị của kiến trúc sẽ bị phá vỡ.



Hình 20: Sơ đồ thành phần chi tiết của lớp service nghiệp vụ trong IRMS

Hình 20 là sơ đồ quan trọng nhất của góc nhìn thành phần vì nó mở lớp service nghiệp vụ phía sau và cho thấy hệ thống được tổ chức theo trật tự trách nhiệm như thế nào. Rất nhiều tài liệu chỉ dừng ở việc nói rằng hệ thống có giao diện, dịch vụ và cơ sở dữ liệu. Hình này đi xa hơn: nó chỉ ra thành phần nào được quyền giữ luật nghiệp vụ, thành phần nào chỉ nên điều phối tiến trình, và thành phần nào tuyệt đối không được chen ngược vào lỗi.

Lớp tiếp nhận chỉ làm việc của lớp tiếp nhận: nhận yêu cầu, kiểm tra hình dạng dữ liệu, chuyển đổi dữ liệu vào ra và trả kết quả. Điều quan trọng ở đây là nó không giữ luật nghiệp vụ. Khi lớp này phình to, mọi quy tắc quan trọng sẽ bị rải khắp các bộ điều khiển và hệ thống rất khó kiểm thử. **Lớp điều phối ứng dụng** là nơi ghép các bước của từng ca sử dụng: mở giao dịch, gọi đúng đối tượng miền nghiệp vụ, điều phối kho lưu trữ và quyết định điểm nào cần phát sự kiện. Lớp này không nên chứa bản thân các bất biến nghiệp vụ, nhưng phải đủ gần nghiệp vụ để điều phối đúng trình tự. **Lớp miền nghiệp vụ** mới là nơi cư trú của các đối tượng gốc, chính sách, quy tắc và bất biến cốt lõi. Đây là trái tim của thiết kế. Cuối cùng, **lớp hạ tầng kỹ thuật** giữ các chi tiết thay đổi nhanh như cơ sở dữ liệu, PaymentGateway, NotificationChannelAdapter và cơ chế phát sự kiện.

Sự phân lớp này đặc biệt quan trọng với IRMS vì hệ thống có nhiều loại thay đổi với tốc độ khác nhau. Quy trình thu ngân có thể thay đổi nhẹ theo chính sách nhà hàng; cách tính phí và khuyến mãi có thể đổi theo mùa; cách hiện thực thanh toán hoặc phát hành biên lai có thể thay. Nếu không có ranh giới giữa miền nghiệp vụ và chi tiết công nghệ, mọi thay đổi dạng đó sẽ lan thẳng vào lõi. Hình này cho thấy kiến trúc đã chủ động chặn điều đó: port/adapter biên bị giữ ở rìa, kho lưu trữ bị ẩn sau giao diện trừu tượng, và phần quyết định nghiệp vụ được bảo vệ ở trung tâm.

Một giá trị khác của hình là nó chứng minh rằng các nguyên lý thiết kế được nói ở phần sau không chỉ mang

tính khẩu hiệu. Khi quan sát sơ đồ này, người đọc có thể thấy trực tiếp nơi áp dụng trách nhiệm đơn nhất, nơi áp dụng phân tách giao diện và nơi áp dụng đảo ngược phụ thuộc. Nói cách khác, đây không chỉ là sơ đồ đẹp; đây là sơ đồ dùng để kiểm tra xem những lời biện minh về thiết kế có thực sự được phản ánh vào cấu trúc hay không.

Các thành phần chính thể hiện trong hình.

- **Lớp tiếp nhận:** tiếp nhận yêu cầu, chuyển đổi dữ liệu vào và dữ liệu ra, nhưng không giữ luật nghiệp vụ.
- **Lớp điều phối ứng dụng:** điều phối từng ca sử dụng, kiểm soát ranh giới giao dịch và gọi đúng đối tượng miền nghiệp vụ.
- **Lớp miền nghiệp vụ:** chứa thực thể gốc, chính sách nghiệp vụ, quy tắc kiểm tra và các bất biến cốt lõi.
- **Lớp hạ tầng kỹ thuật:** cung cấp lưu trữ, công bố sự kiện, kết nối cơ sở dữ liệu và các adapter biên.
- **Kho lưu trữ, bộ kết nối, trực sự kiện nội bộ:** là các thành phần kỹ thuật cụ thể, được đặt ở lớp hạ tầng thay vì chen vào lõi.

Lý do thiết kế của sơ đồ này.

- Sơ đồ chứng minh khối xử lý phía sau không phải một khối mờ, mà có cấu trúc nội bộ chặt chẽ và có chiều phụ thuộc rõ ràng.
- Việc tách lớp tiếp nhận, lớp điều phối ứng dụng, lớp miền nghiệp vụ và lớp hạ tầng giúp truy ra chính xác nơi nên đặt từng loại thay đổi.
- Đây là sơ đồ then chốt để nối góc nhìn thành phần với phần thiết kế chi tiết và phần cài đặt.

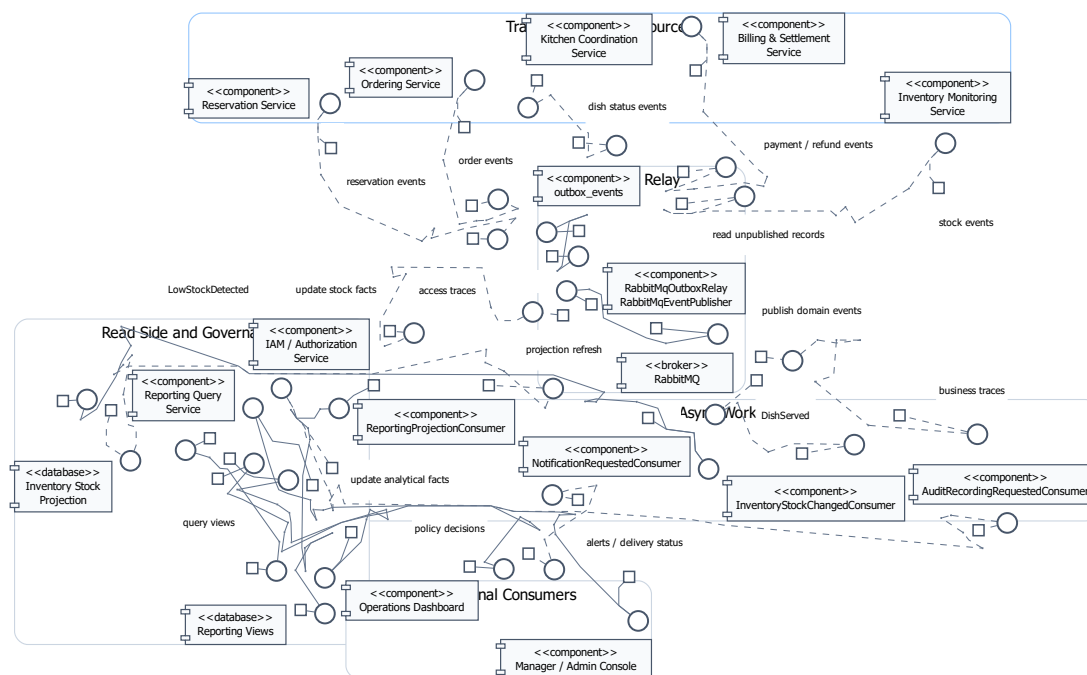
Điểm mạnh lớn nhất của cấu trúc này là khả năng giữ cho hệ thống phát triển lâu dài mà không bị rối. Đổi lại, nó đòi hỏi kỷ luật tổ chức mã nguồn ngay từ đầu: phải chấp nhận viết nhiều giao diện hơn, nhiều lớp điều phối hơn và nhiều lớp chuyển đổi dữ liệu hơn. Trong bối cảnh của IRMS, đó là chi phí hợp lý để đổi lấy một lõi nghiệp vụ không bị bào mòn bởi thay đổi công nghệ.

- **Cổng vào giao diện lập trình ứng dụng và lớp bộ điều khiển** cùng **bộ lọc xác thực / kiểm tra chính sách**: tiếp nhận yêu cầu, chuẩn hóa dữ liệu vào, xác thực và chặn hành vi vượt quyền trước khi đi vào lõi.
- **Các dịch vụ ứng dụng** của từng vùng Reservation, Ordering, Kitchen và Billing: điều phối ca sử dụng, gọi đúng thành phần miền nghiệp vụ và kiểm soát biên giao dịch.
- **Các thành phần miền nghiệp vụ và kho lưu trữ**: hiện thực quy tắc miền và nơi lưu trạng thái bền vững.
- **Hộp thư ra / bộ phát sự kiện** cùng các port/adaptor như PaymentGateway, ReceiptDeliveryAdapter và NotificationChannelAdapter: tách bước chốt giao dịch khỏi bước xử lý biên hoặc xử lý nền.

Lý do cần thêm sơ đồ này.

- Nó mô tả đường đi của một lệnh nghiệp vụ cụ thể, thay vì chỉ liệt kê danh mục thành phần.
- Nó bảo vệ luận điểm kiến trúc rằng những gì cần xác nhận ngay phải được chốt ở lõi trước, còn các hệ quả phụ đi sau qua cơ chế có kiểm soát.
- Nó làm rõ nơi đặt kiểm tra chính sách, giúp bảo mật và kiểm toán trở thành một phần hữu cơ của kiến trúc, không phải đoạn xử lý thêm về sau.

3.6.5 Sơ đồ thành phần cho xử lý nền, quản trị và mô hình đọc



Hình 22: Sơ đồ thành phần cho xử lý nền, quản trị và mô hình đọc

Hình 22 mô tả kiến trúc event-driven sau khi các giao dịch lõi đã được xác lập. Đây là Component Diagram của C&C View, không phải Class Diagram: các phần tử trong hình là service, outbox, RabbitMQ và consumer nên đang tham gia vào tương tác runtime. Tại đây, IRMS tập trung vào việc hiện thực hóa nguyên tắc **Nhất quán cuối**

cùng (Eventual Consistency) và Tách biệt trách nhiệm truy vấn (CQRS - Command Query Responsibility Segregation) ở mức độ thực dụng.

Cấu trúc phối hợp giữa các thành phần hỗ trợ được tổ chức như sau:

1. **Cơ chế phát và chuyển tiếp sự kiện:** Các service giao dịch ghi sự kiện vào outbox_events. Sau đó RabbitMqOutboxRelay đọc bản ghi chưa phát, dùng RabbitMqEventPublisher đẩy sự kiện sang RabbitMQ. Cấu trúc này khớp với TransactionalOutboxPublisher, RabbitMqOutboxRelay và cấu hình spring.rabbitmq.* trong backend.
2. **Cơ chế diễn dịch sự kiện (Event Projection):** Các sự kiện sau khi đi qua RabbitMQ được tiêu thụ bởi các consumer như ReportingProjectionConsumer và InventoryStockChangedConsumer. Các thành phần này chuyển dữ liệu từ dạng giao dịch sang mô hình đọc hoặc projection phục vụ báo cáo và tồn kho, giúp yêu cầu từ màn hình quản lý không tranh chấp trực tiếp với luồng gọi món.
3. **Quản trị và Tuân thủ (Governance & Compliance):**
 - AuditRecordingRequestedConsumer: Không tham gia vào luồng quyết định nghiệp vụ nhưng tiêu thụ sự kiện để ghi dữ liệu truy vết, làm rõ "ai, làm gì, khi nào, bối cảnh nào" một cách độc lập.
 - IAM / Authorization Service: Cung cấp các chính sách bảo vệ tài nguyên (Policy-based Access Control), đảm bảo các thao tác quản trị nhạy cảm như hoàn tiền hoặc điều chỉnh tồn kho luôn được kiểm soát chặt chẽ.
4. **Tích hợp thông báo và liên lạc (Notification Hub):** NotificationRequestedConsumer tiếp nhận sự kiện từ RabbitMQ, sau đó NotificationService ghi nhận thông báo qua các adapter kênh như EmailNotificationChannelAdapter, SmsNotificationChannelAdapter, InAppNotificationChannelAdapter và PrintNotificationChannelAdapter. Code hiện tại chưa thể hiện nhà cung cấp SMS/Email bên ngoài hay máy in vật lý, nên phần này chỉ được hiểu là biên adapter nội bộ.

Biện minh cho thiết kế xử lý nền:

- **Tối ưu hóa tài nguyên ghi (Write Optimization):** Bằng cách đẩy các tác vụ nặng (tổng hợp báo cáo, gửi thông báo) sang luồng nền, hệ thống đảm bảo cơ sở dữ liệu giao dịch chính luôn ở trạng thái nhẹ nhất, ưu tiên tuyệt đối cho tốc độ xác nhận món và thanh toán.
- **Khả năng quan sát (Observability):** Việc tách biệt các consumer hỗ trợ giúp đội ngũ vận hành dễ theo dõi độ trễ của từng tác vụ nền như cập nhật projection, ghi audit hoặc xử lý thông báo thông qua log và trạng thái xử lý trong outbox_events, processed_events.

3.7 Góc nhìn triển khai

3.7.1 Tổng quan phân bổ hệ thống

Ở góc nhìn triển khai, hệ thống được ánh xạ lên hạ tầng vật lý và hạ tầng thực thi nhằm làm rõ cách các thành phần phần mềm được phân bổ trên các nút triển khai (deployment nodes), cách các nút này giao tiếp với nhau và cách hệ thống đáp ứng các yêu cầu phi chức năng như độ trễ, khả năng mở rộng và độ tin cậy.

Hệ thống được tổ chức thành bốn tầng chính như sau:

1. Tầng thiết bị đầu cuối (Client Layer)

- Máy tính bảng dành cho nhân viên phục vụ, hỗ trợ thao tác đặt bàn và gọi món theo thời gian thực.

- Thiết bị lễ tân hoặc ki-ốt tiếp nhận khách, phục vụ việc đăng ký và phân bổ chỗ ngồi.
- Màn hình bếp tại từng trạm chế biến, hiển thị danh sách món cần chuẩn bị với yêu cầu cập nhật gần thời gian thực.
- Máy thu ngân, thực hiện các giao dịch thanh toán với yêu cầu độ trễ thấp và tính nhất quán cao.
- Trình duyệt của quản lý, phục vụ truy vấn báo cáo và giám sát hoạt động hệ thống.

Các thiết bị này hoạt động như các client gửi yêu cầu đồng bộ (synchronous request) tới hệ thống thông qua giao thức HTTP/HTTPS. Đặc điểm chung của tầng này là phụ thuộc vào mạng và yêu cầu độ trễ thấp đối với các thao tác nghiệp vụ chính.

2. Tầng cổng vào và phân phối (Gateway & Delivery Layer)

- Bộ đảo ngược lưu lượng hoặc **API Gateway**, đóng vai trò điểm truy cập duy nhất của hệ thống.
- Máy chủ phân phối giao diện tính hoặc giao diện phía máy chủ, chịu trách nhiệm phục vụ nội dung frontend.

Tầng này chịu trách nhiệm:

- Điều phối yêu cầu từ client tới các service tương ứng.
- Thực hiện các kiểm tra kỹ thuật chung như xác thực, phân quyền và ghi nhận nhật ký truy cập.
- Giảm tải cho các service phía sau thông qua việc gom và chuẩn hóa các điểm truy cập.

Việc sử dụng API Gateway giúp cô lập tầng client với tầng ứng dụng, đồng thời tạo điều kiện thuận lợi cho việc mở rộng và thay đổi cấu trúc backend trong tương lai.

3. Tầng ứng dụng (Application Layer)

- Khối xử lý phía sau bao gồm các service nghiệp vụ chính:
 - Service đặt bàn và quản lý chỗ ngồi.
 - Service gọi món và thực đơn.
 - Service điều phối bếp.
 - Service thanh toán.
 - Service quản lý tồn kho.
 - Service báo cáo và quản trị.
 - Service quản lý truy cập và kiểm toán.
- Bộ xử lý nền (Background Worker) thực hiện các tác vụ bất đồng bộ:
 - Xử lý sự kiện phát sinh từ hệ thống (event-driven processing).
 - Thực hiện các tác vụ retry khi xảy ra lỗi tạm thời.
 - Làm mới các ảnh chụp dữ liệu phục vụ báo cáo (reporting projections).

Các service trong tầng này có thể được triển khai trên cùng một cụm máy chủ hoặc container, nhưng được tách biệt về mặt logic theo miền nghiệp vụ. Các giao tiếp giữa service có thể bao gồm:

- Giao tiếp đồng bộ (REST API) cho các thao tác nghiệp vụ chính.
- Giao tiếp bất đồng bộ (message queue) cho các tác vụ nền.

4. Tầng dữ liệu (Data Layer)

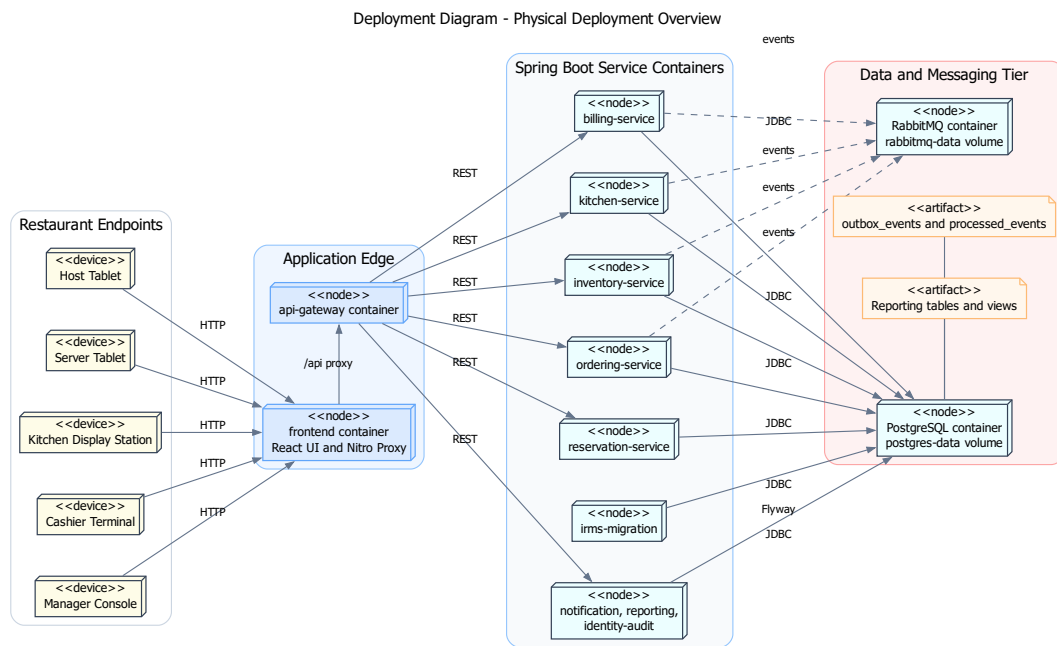
- Cơ sở dữ liệu giao dịch, lưu trữ dữ liệu vận hành chính như đơn hàng, thanh toán, tồn kho và người dùng.
- Mô hình đọc và khung nhìn báo cáo, được tối ưu cho truy vấn và phân tích dữ liệu.
- Các bảng `outbox_events`, `processed_events` và mô hình đọc báo cáo phục vụ xử lý nền và truy vấn quản trị.
- Hạ tầng RabbitMQ phục vụ hàng đợi sự kiện giữa các service.

Tầng dữ liệu được thiết kế tách biệt với tầng ứng dụng nhằm đảm bảo tính toàn vẹn dữ liệu, đồng thời hỗ trợ mở rộng theo nhu cầu đọc/ghi. Các thành phần như mô hình đọc và hệ thống logging thường được cập nhật thông qua luồng xử lý bất đồng bộ để giảm tải cho hệ thống giao dịch chính.

3.7.2 Giải thích thiết kế triển khai

- Thiết bị gọi món và thiết bị bếp phải đặt gần môi trường mạng nội bộ của nhà hàng và có cơ chế kết nối lại tốt vì đây là hai điểm bị ảnh hưởng trực tiếp nhất khi mạng dao động. Việc bố trí này giúp giảm độ trễ thao tác và đảm bảo tính liên tục của hoạt động vận hành ngay cả khi kết nối không ổn định.
- Khối thanh toán và khối gọi món có yêu cầu sẵn sàng cao hơn các khối còn lại, nên cần được ưu tiên về giám sát, sao lưu và khả năng mở rộng. Đây là các điểm chạm trực tiếp với khách hàng, nơi mọi gián đoạn đều ảnh hưởng rõ rệt đến trải nghiệm và doanh thu.
- Khối báo cáo cần tách khỏi tuyến giao dịch để truy vấn phân tích không cản trở đường xử lý gọi món và thanh toán. Việc tách này cho phép hệ thống duy trì thời gian phản hồi thấp cho các thao tác thời gian thực, đồng thời vẫn hỗ trợ các nhu cầu phân tích dữ liệu với khối lượng lớn.
- Các nút dữ liệu phải có sao lưu và chính sách phân quyền riêng; dữ liệu thanh toán, hoàn tiền và nhật ký kiểm toán là nhóm cần ưu tiên bảo vệ cao. Điều này phản ánh yêu cầu về bảo mật và tuân thủ trong môi trường vận hành thực tế.
- Việc tách bộ xử lý nền khỏi khối xử lý yêu cầu trực tuyến giúp những tác vụ như gửi thông báo, in biên lai điện tử hoặc cập nhật báo cáo không tranh chấp tài nguyên với thao tác tại quầy.

Giải thích chi tiết cho góc nhìn triển khai. Góc nhìn triển khai giúp chứng minh rằng kiến trúc được chọn không chỉ hợp lý trên sơ đồ logic mà còn có thể triển khai thực tế. Với IRMS, bản chất *nhiều điểm chạm vật lý* — quầy lễ tân, bàn phục vụ, trạm bếp, quầy thu ngân — làm cho góc nhìn triển khai quan trọng hơn nhiều so với các hệ thống chỉ có một giao diện web thông thường.



Hình 23: Sơ đồ triển khai tổng quan về bố trí vật lý của IRMS

Hình 23 mô tả cách hệ thống được triển khai trong môi trường thực tế của nhà hàng. Ở mức này, mục tiêu không còn là mô tả logic nghiệp vụ, mà là trả lời câu hỏi hệ thống hiện diện như thế nào trong không gian vật lý và các thành phần phần mềm được gắn vào những thiết bị cụ thể nào.

Điểm quan trọng nhất cần rút ra là IRMS được tổ chức theo mô hình **nhiều điểm chạm vật lý nhưng một lõi điều phối tập trung** . Các thiết bị tại hiện trường — như máy tính bảng phục vụ, thiết bị lễ tân, màn hình bếp và quầy thu ngân — được đặt gần người dùng để đảm bảo thao tác nhanh và thuận tiện. Tuy nhiên, toàn bộ trạng thái nghiệp vụ cốt lõi vẫn được duy trì tại khối xử lý trung tâm và hạ tầng dữ liệu, thay vì phân tán xuống các thiết bị.

Cách bố trí này giúp tránh tình trạng mất nhất quán khi mạng dao động hoặc khi nhiều thao tác xảy ra đồng thời tại các điểm khác nhau. Nó cũng phản ánh một nguyên tắc quan trọng: các thiết bị đầu cuối chỉ nên đóng vai trò tương tác, còn quyết định nghiệp vụ bền vững phải được neo tại backend.

Một đặc điểm thiết kế quan trọng khác là sự tách biệt giữa hai tuyến xử lý:

- **Tuyến xử lý trực tuyến (synchronous path)** phục vụ các thao tác yêu cầu phản hồi ngay như gọi món, cập nhật trạng thái bếp và thanh toán.
- **Tuyến xử lý bất đồng bộ (asynchronous path)** xử lý các hệ quả phụ thông qua **Outbox / hàng đợi** và bộ xử lý nền, bao gồm gửi thông báo, phát hành biên lai và cập nhật dữ liệu báo cáo.

Việc phân tách này cho phép hệ thống giữ được thời gian phản hồi ổn định trong giờ cao điểm, khi các thao tác trực tuyến cần được ưu tiên tuyệt đối.

Ngoài ra, sơ đồ cũng làm rõ vai trò của PostgreSQL, RabbitMQ, outbox_events và các bảng hoặc view báo cáo. Các tác vụ không yêu cầu phản hồi ngay được xử lý tách biệt, trong khi dữ liệu giao dịch chính vẫn được neo trong PostgreSQL để giữ tính nhất quán.

Từ góc độ vận hành, cách bố trí này còn mang lại khả năng **cô lập lỗi theo thành phần**. Khi consumer nền,

hàng đợi hoặc mô hình đọc báo cáo gặp sự cố, tuyến xử lý trực tuyến vẫn có thể tiếp tục hoạt động với mức suy giảm có kiểm soát.

Các thành phần chính thể hiện trong hình.

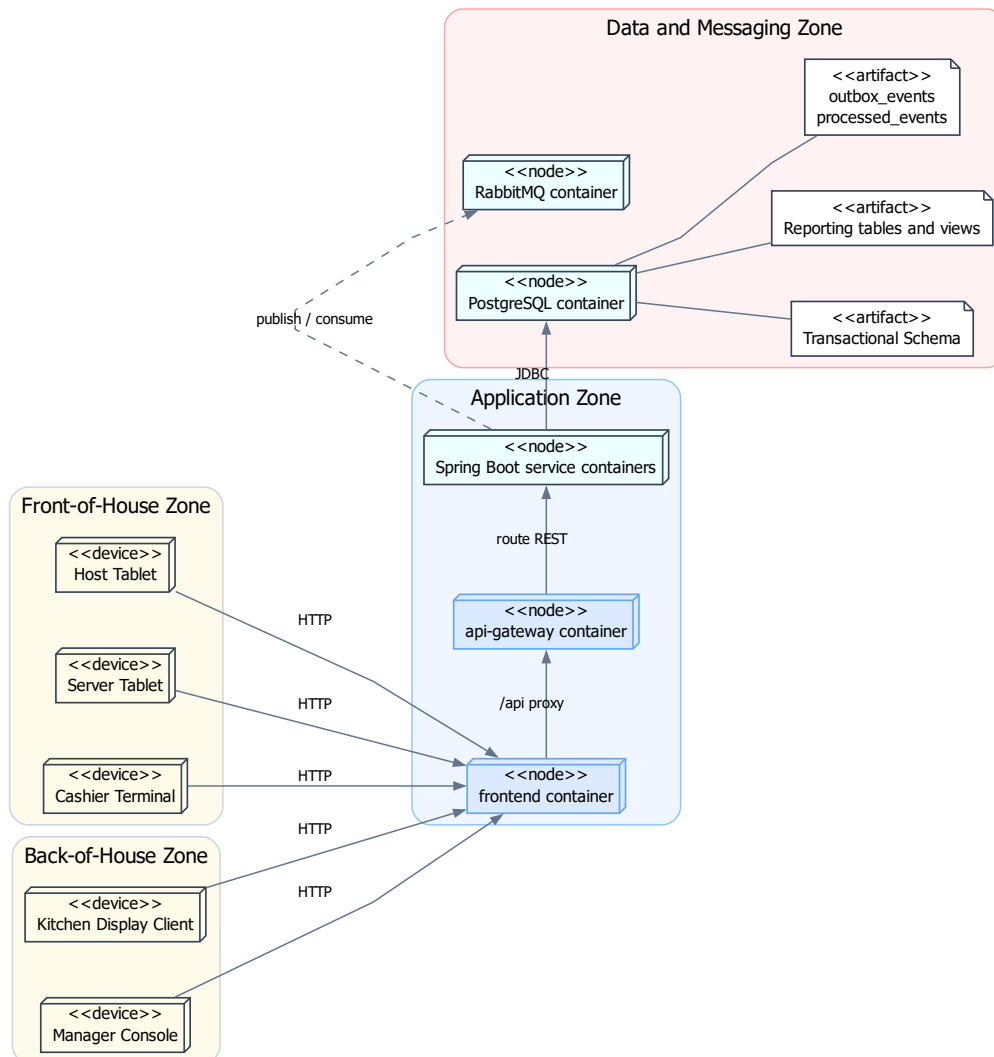
- **Thiết bị đầu cuối:** máy tính bảng phục vụ, thiết bị lễ tân, màn hình bếp, máy thu ngân và trình duyệt quản lý.
- **Khối cổng vào:** frontend container và api-gateway container tiếp nhận, proxy và định tuyến yêu cầu từ thiết bị.
- **Khối xử lý trung tâm:** các container service Spring Boot thực hiện logic nghiệp vụ chính.
- **Hạ tầng sự kiện:** RabbitMQ, outbox_events và processed_events xử lý các tác vụ bất đồng bộ.
- **Hạ tầng dữ liệu:** PostgreSQL lưu dữ liệu giao dịch và mô hình đọc báo cáo.

Lý do thiết kế.

- Giữ một lõi xử lý trung tâm giúp đảm bảo tính nhất quán và đơn giản hóa triển khai.
- Tách tuyến bất đồng bộ giúp giảm tải cho tuyến trực tuyến.
- Phân lớp lưu trữ giúp hệ thống linh hoạt và dễ mở rộng hơn.

Phương án này đơn giản và thực dụng, phù hợp với quy mô hệ thống, nhưng cũng đặt áp lực lên khối xử lý trung tâm, đòi hỏi giám sát và mở rộng phù hợp.

Deployment Diagram - Network Zones and Trust Boundaries



Hình 24: Sơ đồ triển khai theo vùng mạng và ranh giới tin cậy

Hình 24 mở rộng góc nhìn triển khai bằng cách làm rõ **ranh giới tin cậy** và cách hệ thống được phân chia thành các vùng mạng khác nhau.

Kiến trúc được chia thành các vùng: vùng thiết bị hiện trường, vùng ứng dụng, vùng dữ liệu và vùng hàng đợi sự kiện. Mỗi vùng đại diện cho một mức độ tin cậy và quyền truy cập khác nhau.

Nguyên tắc quan trọng nhất là **không có thành phần nào được phép vượt qua ranh giới mà không qua kiểm soát**. Các thiết bị đầu cuối chỉ giao tiếp với hệ thống thông qua frontend container và api-gateway. Các service phía sau mới được truy cập PostgreSQL và RabbitMQ.

Việc tách riêng vùng dữ liệu giúp bảo vệ dữ liệu nhạy cảm và giảm nguy cơ truy cập trái phép. Đồng thời, việc đặt RabbitMQ ở vùng riêng giúp cô lập tuyến sự kiện khỏi tuyến yêu cầu trực tuyến.

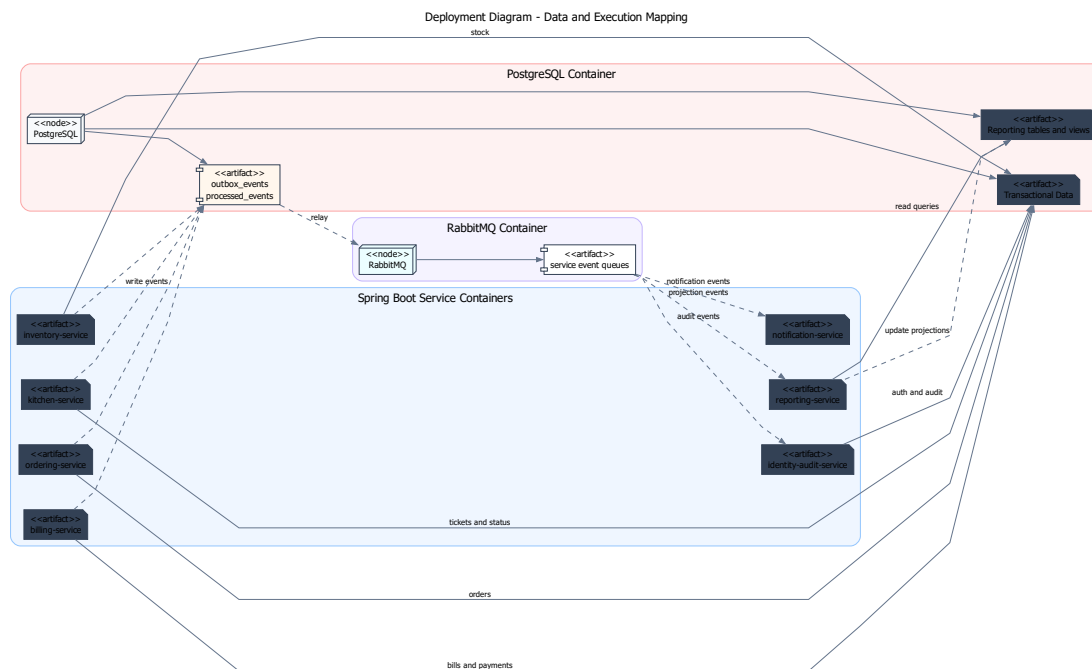
Sơ đồ này cũng hỗ trợ vận hành thực tế: khi xảy ra sự cố, việc phân vùng giúp nhanh chóng xác định lỗi thuộc lớp nào.

Các vùng chính.

- Vùng thiết bị hiện trường
- Vùng ứng dụng
- Vùng dữ liệu
- Vùng dữ liệu và hàng đợi sự kiện

Lý do thiết kế.

- Tăng cường bảo mật thông qua phân tách vùng
- Kiểm soát luồng truy cập giữa các thành phần
- Hỗ trợ vận hành và xử lý sự cố



Hình 25: Sơ đồ triển khai về ánh xạ dữ liệu và thực thi

Hình 25 làm rõ cách các service, dữ liệu và luồng thực thi được tổ chức trong hệ thống.

Sơ đồ này thể hiện rõ rằng IRMS theo **kiến trúc hướng dịch vụ kết hợp điều phối bằng sự kiện**. Các service nghiệp vụ xử lý giao dịch chính thông qua API đồng bộ, trong khi các hệ quả phụ được xử lý thông qua cơ chế bất đồng bộ dựa trên **Outbox và hàng đợi sự kiện**.

Luồng xử lý có thể chia thành hai phần:

- **Luồng giao dịch chính:** các service ghi dữ liệu vào cơ sở dữ liệu giao dịch.
- **Luồng bất đồng bộ:** các sự kiện được ghi vào Outbox, sau đó được bộ xử lý nền tiêu thụ để cập nhật mô hình đọc, gửi thông báo hoặc xử lý các tác vụ phụ.

Cách tổ chức này giúp đảm bảo rằng các giao dịch cốt lõi vẫn rõ ràng và nhất quán, trong khi hệ thống vẫn linh hoạt trong việc xử lý các nhu cầu mở rộng như báo cáo và tích hợp.

Ngoài ra, việc tách mô hình đọc khỏi dữ liệu giao dịch cho phép tối ưu hóa truy vấn mà không ảnh hưởng đến hiệu năng của hệ thống chính.

Các thành phần chính.

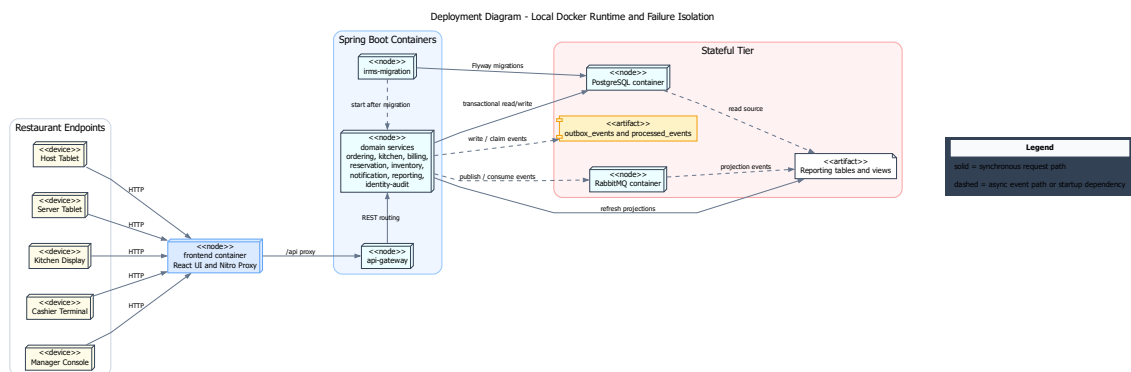
- Service nghiệp vụ
- Cơ sở dữ liệu giao dịch
- Outbox / hàng đợi sự kiện
- Bộ xử lý nền
- Mô hình đọc
- RabbitMQ và các consumer sự kiện

Lý do thiết kế.

- Giữ ranh giới service rõ ràng
- Tách xử lý bất đồng bộ để giảm phụ thuộc
- Tối ưu hóa truy vấn thông qua mô hình đọc

Ưu điểm của kiến trúc này là vừa đảm bảo tính rõ ràng của giao dịch, vừa hỗ trợ mở rộng linh hoạt. Đổi lại, hệ thống cần được thiết kế cẩn thận về hợp đồng API, sự kiện và quyền sở hữu dữ liệu để tránh phụ thuộc chéo giữa các service.

3.7.3 Sơ đồ triển khai cho runtime Docker local và cô lập lỗi



Hình 26: Sơ đồ triển khai cho runtime Docker local và cô lập lỗi

Hình 26 đi sâu vào câu hỏi triển khai thực tế hơn với repo hiện tại: khi chạy bằng docker-compose.yml, các container, dữ liệu và tuyến sự kiện được đặt ở đâu, và lỗi của một phần hỗ trợ được cô lập khỏi tuyến yêu cầu trực tuyến như thế nào. Giá trị chính của sơ đồ này là bám vào runtime có bằng chứng trong repo, thay vì mô tả một cấu hình sẵn sàng cao chưa được cấu hình.

Điểm đầu tiên cần rút ra là **tuyến yêu cầu trực tuyến** đi qua frontend container, api-gateway rồi đến các service Spring Boot phía sau. Frontend Node Server phục vụ React và proxy /api; GatewayProxyController định tuyến REST

đến từng service. Cấu hình hiện tại chưa có load balancer hoặc hai nút ứng dụng song song, nên sơ đồ không biểu diễn failover giữa các app node.

Điểm thứ hai là sơ đồ làm rõ cách **dữ liệu giao dịch lõi** được đặt trong PostgreSQL container với volume postgres-data. irms-migration chạy Flyway trước khi các service nghiệp vụ khởi động. Repo hiện tại chưa có bảng chứng về Read Replica / Warm Standby, nên phần sao chép dữ liệu không còn được mô tả như một khả năng đã triển khai.

Điểm thứ ba là sơ đồ thể hiện rõ việc **cô lập lỗi giữa giao dịch chính và các hệ quả phụ**. Các service ghi dữ liệu giao dịch theo tuyến đồng bộ, trong khi các tác vụ như làm mới mô hình đọc, gửi thông báo và ghi audit được tách qua outbox_events, processed_events, RabbitMQ và các consumer. Nhờ vậy, sự cố ở tuyến thông báo hoặc tiến trình làm mới báo cáo không cần làm sập ngay tuyến giao dịch. Đây là một dạng cô lập lỗi rất quan trọng trong bối cảnh vận hành nhà hàng: nếu thông báo bị chậm, khách vẫn phải thanh toán được; nếu báo cáo chưa được làm mới kịp thời, bàn vẫn phải mở và món vẫn phải ra đúng.

Một cách đọc thực tế hơn của sơ đồ là thông qua các kịch bản lỗi cụ thể:

- Nếu api-gateway hoặc một service container lỗi, endpoint tương ứng sẽ suy giảm theo healthcheck của Docker Compose; chưa có cấu hình chuyển lưu lượng sang nút dự phòng.
- Nếu consumer nền bị lỗi tạm thời, trạng thái xử lý còn được giữ trong outbox_events, processed_events hoặc hàng đợi RabbitMQ để xử lý lại theo cơ chế đã cấu hình.
- Nếu tuyến thông báo hoặc báo cáo phản hồi chậm, giao dịch lõi vẫn được ghi nhận trước; phần hệ quả phụ được xử lý sau qua sự kiện.

Điểm thứ tư là sơ đồ thể hiện rõ **đường suy giảm dịch vụ chấp nhận được**. Không phải mọi lỗi đều dẫn tới dừng toàn hệ thống. Một số lỗi chỉ nên làm chậm phần đọc, làm chậm thông báo hoặc đưa một số tác vụ sang hàng chờ. Khả năng chấp nhận suy giảm có kiểm soát như vậy là dấu hiệu của một kiến trúc vận hành trưởng thành hơn so với mô hình mà mọi thành phần đều phụ thuộc chặt vào nhau.

Các ý chính cần rút ra từ sơ đồ.

- **frontend container** tách giao diện React và proxy /api khỏi các service backend.
- **api-gateway** định tuyến REST đến các container service Spring Boot theo cấu hình thật.
- **PostgreSQL container** lưu dữ liệu giao dịch và các bảng hỗ trợ báo cáo; chưa có cấu hình replica trong repo.
- **RabbitMQ container** cùng outbox_events và processed_events ngăn các tác vụ nền làm nghẽn trực tiếp tuyến giao dịch.
- **Reporting tables and views** cho phép tách nhu cầu đọc báo cáo khỏi phần xử lý lệnh nghiệp vụ.

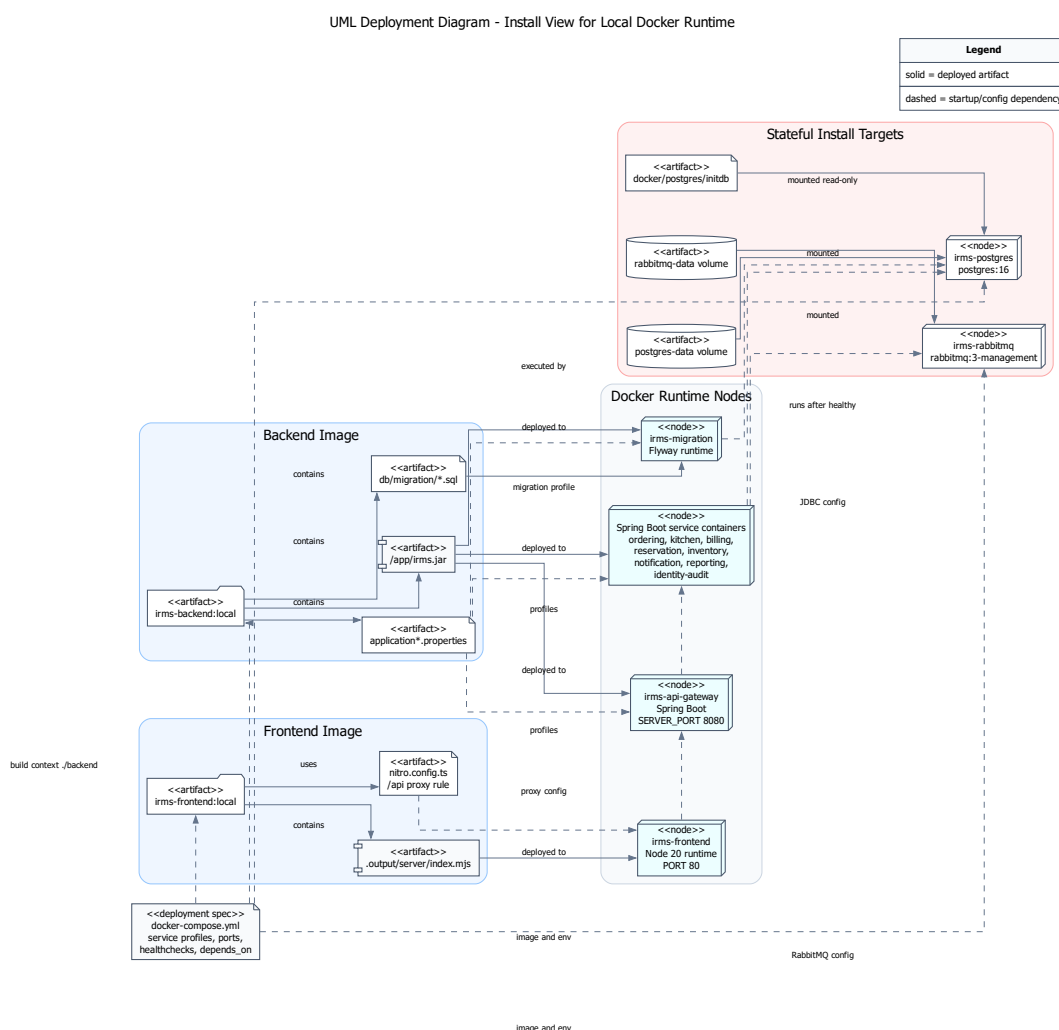
Vì sao sơ đồ này cần có trong góc nhìn triển khai.

- Góc nhìn triển khai không chỉ cần cho thấy hệ thống được đặt ở đâu, mà còn phải cho thấy ranh giới lỗi giữa tuyến REST, dữ liệu và sự kiện.
- Với IRMS, các phần hỗ trợ như thông báo, audit và báo cáo có thể chậm hoặc lỗi từng phần; sơ đồ này cho thấy cách hệ thống cô lập các lỗi đó khỏi lỗi giao dịch.
- Sơ đồ cũng làm rõ hướng mở rộng về sau: có thể bổ sung load balancer, replica hoặc nhiều consumer hơn, nhưng đó là hướng mở rộng chứ chưa phải cấu hình đã có trong repo.

Ngoài khả năng cô lập lỗi, sơ đồ này cũng thể hiện rõ hướng mở rộng của hệ thống. Việc bổ sung thêm nút ứng dụng hoặc bộ xử lý nền có thể được thực hiện về sau, nhưng trong tài liệu hiện tại cần phân biệt rõ giữa cấu hình đã có trong docker-compose.yml và phương án mở rộng tương lai.

3.7.4 Install View cho gói triển khai Docker local

Deployment View ở trên trả lời các container và node runtime được đặt ở đâu. Install View dưới đây bổ sung góc nhìn còn lại: artifact, cấu hình và migration nào được đóng gói vào từng container khi chạy theo docker-compose.yml. Phần này chỉ dựa trên file thật trong repo, không thêm Nginx, Kubernetes, cloud service, load balancer, read replica hoặc external provider nếu chưa có cấu hình.



Hình 27: UML Deployment Diagram cho Install View của gói Docker local

Hình 27 dùng UML Deployment Diagram để thể hiện các node runtime và artifact cài đặt thật trong repo. irms-backend:local chứa /app/irms.jar, các file application*.properties và migration SQL; cùng artifact JAR này được triển khai vào api-gateway, các service Spring Boot và irms-migration, nhưng vai trò runtime khác nhau nhờ profile và biến môi trường trong docker-compose.yml. irms-frontend:local chứa bundle Nitro ở .output/server/index.mjs; nitro.config.ts điều khiển proxy /api/** sang api-gateway. Các node trạng thái chỉ gồm PostgreSQL với postgres-

data và RabbitMQ với rabbitmq-data, đúng với cấu hình hiện tại.

Bảng 33: Install View cho gói triển khai Docker local của IRMS

Artifact/config item	Được cài/dóng gói ở đâu	Nguồn trong repo	Vai trò runtime
Backend Spring Boot artifact irms.jar	Image irms-backend:local; chạy lại cho api-gateway, ordering-service, kitchen-service, billing-service, reservation-service, inventory-service, notification-service, reporting-service, identity-audit-service.	irms_project/backend/Dockerfile, irms_project/docker-compose-backend/pom.xml.	Mã nguồn, build tạo bằng Maven, container chạy bằng java -jar /app/irms.jar; vai trò service được chọn bằng SPRING_PROFILES_ACTIVE, IRMS_RUNTIME_SERVICE và SERVER_PORT.
Migration runtime và SQL migrations	Container irms-migration, dùng cùng image backend nhưng profile docker,migration.	application-migration.properties, application.properties, src/main/resources/db/migration/*.sql; docker-compose.yml.	Chạy trước các service nghiệp vụ; IRMS_FLYWAY_ENABLED=true và depends_on.condition=service_completed_successfully làm rõ thứ tự khởi động.
Frontend bundle / Nitro server	Image irms-frontend:local, container irms-frontend.	irms_project/frontend/Dockerfile, frontend/package.json, frontend/nitro.config.ts.	npm install, build tạo thư mục .output; container production chạy node .output/server/index.mjs phục vụ giao diện React/TanStack và proxy /api/** tới api-gateway.
Runtime application properties	Được nạp trong các container backend theo profile Docker và profile service.	application.properties, application-docker.properties, application-*-service.properties.	Điều khiển data source PostgreSQL, RabbitMQ, Flyway, security token, CORS, timeout phiên, outbox và cấu hình service URL.
PostgreSQL data store	Container irms-postgres, volume postgres-data.	docker-compose.yml, docker/postgres/initdb/01-init-devoutbox.sh.	Lưu dữ liệu giao dịch, outbox processed events và mô hình đọc; healthcheck pg_isready là điều kiện cho migration/service.
RabbitMQ broker	Container irms-rabbitmq, volume rabbitmq-data.	docker-compose.yml, application.properties,	Cung cấp broker cho event-

Từ bảng này có thể thấy thứ tự cài đặt/khởi động của repo hiện tại khá rõ: postgres và rabbitmq phải healthy trước, irms-migration chạy thành công để tạo/cập nhật schema, sau đó các service Spring Boot và cuối cùng là frontend phụ thuộc vào api-gateway. Đây là Install View của runtime Docker local hiện có, không phải mô tả triển khai cloud hay cụm sẵn sàng cao.

3.8 Nguyên tắc thiết kế

Kiến trúc của IRMS bám theo các nguyên tắc thiết kế sau:

- **Kết tụ cao, phụ thuộc thấp:** mỗi service chỉ công khai API, event và hợp đồng dữ liệu thật sự cần thiết, đồng thời tránh làm lộ chi tiết cài đặt.
- **Phân tách mối quan tâm:** bộ điều khiển, dịch vụ ứng dụng, miền nghiệp vụ và hạ tầng kỹ thuật không bị trộn vai trò.
- **Phụ thuộc thông qua lớp trừu tượng:** các adapter biên đi qua giao diện trừu tượng thay vì để lỗi nghiệp vụ phụ thuộc trực tiếp vào nhà cung cấp hoặc bộ thư viện cụ thể.
- **API đồng bộ cho thao tác cần phản hồi trực tiếp:** các giao dịch lõi như xác nhận đặt bàn, tạo đơn gọi món, lập hóa đơn và thanh toán ưu tiên xử lý đồng bộ để trạng thái phản hồi rõ ràng.
- **Event bất đồng bộ cho hệ quả sau giao dịch:** các hệ quả phụ như cảnh báo tồn kho, làm mới báo cáo, gửi thông báo, ghi audit và in biên lai đi qua Outbox, Event Bus hoặc Background Worker.
- **Mô hình hóa bắt đầu từ miền nghiệp vụ:** sơ đồ lớp phải phản ánh hành vi thật của nhà hàng, không bị gián lược thành sơ đồ quan hệ dữ liệu thuần túy.

Các nguyên tắc này là tiêu chí xuyên suốt: giúp xác định ranh giới nghiệp vụ rõ ràng, tách biệt các thành phần, phân luồng xử lý hợp lý và ưu tiên mô hình hóa theo nghiệp vụ thay vì chỉ theo dữ liệu.

3.9 Áp dụng nguyên lý SOLID vào thiết kế

Thay vì trình bày SOLID như một danh sách lý thuyết, phần này đối chiếu trực tiếp các nguyên tắc với những điểm thiết kế đang tồn tại trong mã nguồn. Các ví dụ dưới đây chỉ dùng class, interface hoặc cấu hình có trong repository; vì vậy chúng cho thấy nơi project áp dụng SOLID rõ nhất, không khẳng định mọi class đều tuân thủ hoàn hảo.

Bảng 34: Đối chiếu nguyên tắc SOLID với hiện thực mã nguồn

Nguyên tắc	Biểu hiện trong project	Bằng chứng code	Ý nghĩa thiết kế
SRP	Trách nhiệm được tách theo vai trò: service xử lý một use case chính, policy giữ quy tắc, adapter xử lý biên tích hợp, consumer xử lý sự kiện nền.	BillingPaymentService, BillingReceiptService, KitchenTicketTransitionPolicy, ManualAckConsumerSupport, RabbitMqOutboxRelay.	Khi thay đổi thanh toán, trạng thái bếp hoặc retry RabbitMQ, phạm vi sửa được khoanh vào nhóm class đúng trách nhiệm.
OCP	Các điểm dễ thay đổi được mở qua interface/strategy/policy thay vì sửa trực tiếp service lõi.	PaymentGateway với CashPaymentGateway, ExternalReferencePaymentGateway, BillSplitStrategy với AmountSplitStrategy, EqualSplitStrategy, ItemSplitStrategy, SeatSplitStrategy; ReorderRecommendationStrategy.	Có thể thêm biến thể mới bằng class hiện thực mới và cấu hình bean, giảm sửa chéo ở lõi nghiệp vụ.
LSP	Các implementation được dùng thông qua cùng contract và được chọn theo khả năng hỗ trợ nghiệp vụ.	BillingPaymentService nhận List<PaymentGateway> và gọi supports(...); BillingSplitService nhận List<BillSplitStrategy>.	Service sử dụng contract ổn định; implementation có thể thay thế trong phạm vi hợp đồng đã định nghĩa.
ISP	Interface được giữ nhỏ, gắn với một vai trò cụ thể thay vì gom thành interface đa năng.	PaymentGateway, ReceiptDeliveryAdapter, NotificationChannelAdapter, KitchenTicketItemRepositoryPort, DomainEventPublisher.	Client chỉ phụ thuộc vào hành vi nó thật sự cần, để kiểm thử và ít bị kéo theo thay đổi ngoài phạm vi.
DIP	Service cấp cao phụ thuộc abstraction/port, còn hạ tầng hiện thực chi tiết JDBC, RabbitMQ hoặc adapter kênh.	BillingPaymentService phụ thuộc BillPaymentRepository, PaymentGateway, DomainEventPublisher; BillingReceiptService phụ thuộc ReceiptDeliveryAdapter; JdbcKitchenTicketItemRepository implements KitchenTicketItemRepositoryPort.	Chiều phụ thuộc đi từ hạ tầng vào hợp đồng nghiệp vụ, giúp lõi ít bị khóa bởi công nghệ cụ thể.

3.9.1 Các ví dụ áp dụng rõ nhất trong code

Thanh toán và biên nhận. BillingPaymentService không biết chi tiết xử lý từng phương thức thanh toán, mà làm việc qua PaymentGateway. Code hiện tại có CashPaymentGateway và ExternalReferencePaymentGateway; tên “external reference” ở đây chỉ là adapter nội bộ theo code, chưa chứng minh có nhà cung cấp thanh toán thật. Tương tự, BillingReceiptService phát hành biên nhận qua ReceiptDeliveryAdapter với DigitalReceiptDeliveryAdapter và PrintReceiptDeliveryAdapter. Đây là ví dụ mạnh cho OCP, ISP và DIP: service lõi giữ luồng thanh toán, còn biến thể phương thức/kênh nằm ở lớp hiện thực.

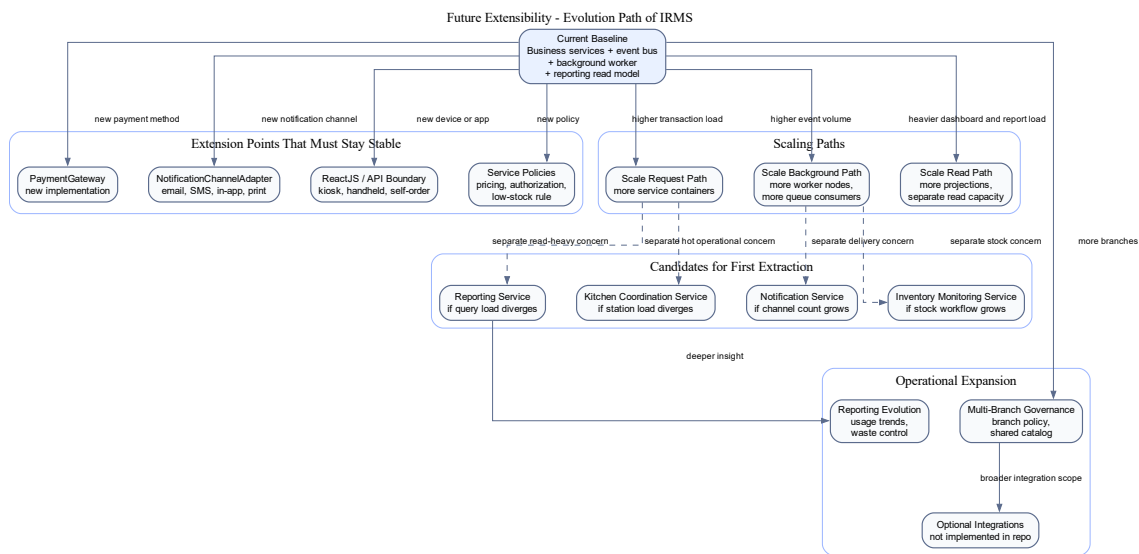
Thông báo. Nhóm NotificationChannelAdapter được tách thành các kênh nhỏ như EmailNotificationChannelAdapter, SmsNotificationChannelAdapter, InAppNotificationChannelAdapter, PrintNotificationChannelAdapter và PhoneNotificationChannelAdapter. Những class này giúp service thông báo kiểm tra và ghi nhận kênh gửi mà không biến phần notification thành một interface quá lớn. Tuy nhiên, tài liệu chỉ mô tả đây là adapter trong code; chưa diễn giải thành nhà cung cấp SMS/email/push hoặc máy in vật lý vì chưa có cấu hình provider tương ứng.

Policy và strategy nghiệp vụ. DomainPolicyConfiguration đăng ký các policy/strategy như OrderStatusPolicy, ComboPricingPolicy, KitchenTicketTransitionPolicy, KitchenDeadlinePolicy, LowStockSeverityPolicy, ReorderRecommendationStrategy và các BillSplitStrategy. Cách tách này làm rõ SRP: quy tắc chuyển trạng thái, tính deadline, đề xuất nhập hàng hoặc tách hóa đơn không bị trộn vào controller. Đồng thời, các strategy như BillSplitStrategy và ReorderRecommendationStrategy là điểm mở rộng OCP có bằng chứng trực tiếp trong code.

Outbox, RabbitMQ và consumer nền. Ở tuyến event-driven, TransactionalOutboxPublisher hiện thực DomainEventPublisher, RabbitMqOutboxRelay phụ trách chuyển outbox sang RabbitMQ, còn ManualAckConsumerSupport gom logic ack, retry, deduplication và dead-letter cho consumer. Phần này là ví dụ SRP và DIP ở tầng hạ tầng: service nghiệp vụ chỉ publish event qua abstraction, còn cách phát, retry hoặc dead-letter nằm trong component kỹ thuật riêng.

Giới hạn cần tránh diễn giải quá mức. Các ví dụ trên cho thấy IRMS có áp dụng SOLID tại những điểm thiết kế chính, đặc biệt ở service, adapter, policy, strategy, repository port và event infrastructure. Điều đó không có nghĩa mọi class đều hoàn hảo theo SOLID. Ngoài ra, các interface như PaymentGateway, ReceiptDeliveryAdapter và NotificationChannelAdapter chỉ chứng minh có biên trừu tượng trong code; chúng không tự động chứng minh hệ thống đã tích hợp cổng thanh toán ngoài, nhà cung cấp SMS/email thật hoặc máy in vật lý.

3.10 Khả năng mở rộng trong tương lai



Hình 28: Lộ trình mở rộng kiến trúc của IRMS trong tương lai

Hình 28 tập trung riêng vào câu hỏi mà hầu như mọi đồ án kiến trúc đều phải trả lời tương minh: nếu phạm vi vận hành lớn hơn, số thiết bị tăng lên, số chi nhánh tăng lên hoặc yêu cầu tích hợp đa dạng hơn, kiến trúc hiện tại sẽ tiến hóa theo đường nào mà không phải viết lại phần lõi. Hình này không mô tả một trạng thái giả định mơ hồ trong tương lai. Nó mô tả **đường tiến hóa hợp lý** từ kiến trúc hiện tại sang những khả năng mở rộng có xác suất xảy ra cao nhất đối với IRMS.

Ở trạng thái hiện tại, IRMS vận hành như một **kiến trúc hướng dịch vụ kết hợp điều phối bằng sự kiện**, trong đó các service nghiệp vụ có ranh giới rõ, tuyến giao dịch nóng dùng API đồng bộ và các hệ quả sau giao dịch được xử lý qua event. Đây là nền tảng tốt vì giao dịch lõi vẫn gọn, ranh giới service vẫn rõ và các service hỗ trợ như Reporting Service, Notification Service, Inventory Monitoring Service hoặc một phần Kitchen Coordination Service có thể mở rộng theo nhịp tải riêng trước khi cần thay đổi sâu tuyến giao dịch lõi.

Điểm quan trọng của hình là nó nêu rõ **điều kiện để mở rộng mà không phá kiến trúc**. Nếu muốn thêm phương thức thanh toán mới, hệ thống chỉ nên chạm vào PaymentGateway và chính sách liên quan. Nếu muốn thêm kênh thông báo, thay đổi phải chủ yếu nằm ở NotificationChannelAdapter. Nếu muốn mở thêm chi nhánh, hệ thống cần tăng sức chứa ở lớp cấu hình chi nhánh, vùng dữ liệu và tăng triển khai, nhưng không được buộc viết lại mô hình đơn gọi món hay quyết toán. Nếu muốn có kiosk tự phục vụ hoặc thiết bị cầm tay mới, thay đổi chủ yếu nằm ở lớp client và giao diện lập trình ứng dụng, còn chuỗi miền nghiệp vụ chính vẫn giữ nguyên. Những điều kiện đó chính là thước đo thật sự của extensibility, chứ không chỉ là khả năng thêm lớp mã mới.

Sơ đồ cũng chỉ ra các **điểm mở rộng chiến lược** cần được giữ ổn định từ sớm: PaymentGateway, NotificationChannelAdapter, chính sách phân quyền, quy tắc định giá, quy tắc cảnh báo tồn kho, mô hình đọc báo cáo và các giao diện ứng dụng khách. Khi những điểm này được giữ ở đúng ranh giới, hệ thống có thể tiến hóa dần từng năng lực mà không cần đánh đổi sự ổn định của tuyến lõi. Đây là lý do mục “Future Extensibility” không nên chỉ được nói bằng một đoạn văn chung chung; nó cần một hình kiến trúc riêng để cho thấy đường phát triển của hệ thống là hữu hình và có kiểm soát.

Các hướng tiến hóa chính thể hiện trong hình.

- **Mở rộng tích hợp:** thêm phương thức thanh toán, kênh thông báo, thiết bị in hoặc thiết bị đầu cuối mới thông qua các cổng và bộ kết nối hiện có.
- **Mở rộng theo tải:** tăng số nút ứng dụng, nút bộ xử lý nền, năng lực đọc báo cáo hoặc mở rộng riêng một số service có tải đặc thù.
- **Mở rộng theo phạm vi vận hành:** thêm nhiều chi nhánh, thêm chính sách theo chi nhánh và tăng năng lực quản trị tập trung mà không phải làm lại mô hình miền lõi.
- **Mở rộng theo dịch vụ:** chỉ tách thành dịch vụ mạng độc lập với những vùng có nhịp thay đổi hoặc mức tải thực sự khác biệt, thay vì tách đồng loạt vì lý do hình thức.

4 Thiết kế chi tiết

4.1 Góc nhìn lớp

Góc nhìn lớp là phần chi tiết hóa của Module View, tập trung vào class, interface, enum và các quan hệ tĩnh trong miền lõi hoặc tại những điểm mở rộng kỹ thuật có bằng chứng trong code. Vì vậy, các sơ đồ ở mục này không dùng để mô tả broker, container, consumer runtime hay luồng publish/subscribe; các nội dung đó đã được đặt ở góc nhìn thành phần và kết nối.

Code backend hiện tại dùng Spring Boot với JDBC repository và schema migration thay vì các lớp @Entity. Vì vậy, các sơ đồ lớp trong phần này được đọc như mô hình lớp nghiệp vụ hiện trạng: phần lớn lớp tương ứng với bảng/record trong migration, còn các interface và policy được giữ khi có class/interface thật trong code. Các thao tác nghiệp vụ như xác nhận, thanh toán, cập nhật trạng thái hoặc phát sự kiện được hiện thực chủ yếu trong application service, domain policy và adapter, nên diagram không trình bày chúng như method của entity.

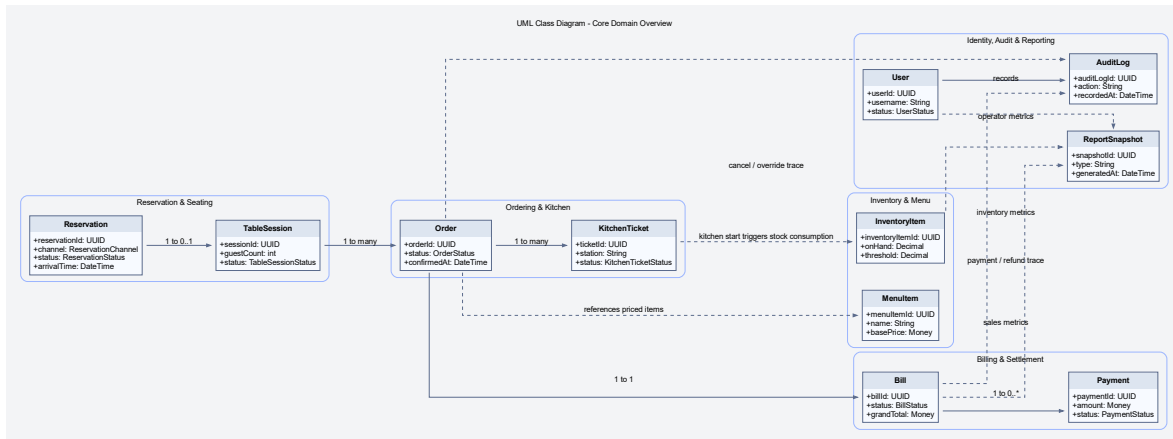
4.1.1 Phạm vi bao phủ của sơ đồ lớp

Sơ đồ lớp được tổ chức theo năm hướng chính nhằm bao phủ đầy đủ miền nghiệp vụ của IRMS:

1. **Ngữ cảnh khách hàng và chi nhánh:** sơ đồ thể hiện CustomerProfile và Branch để việc đặt bàn, lập hóa đơn, quản lý tồn kho và phân công nhân sự luôn được xác định rõ theo khách hàng và theo địa điểm vận hành.
2. **Bao phủ đầy đủ hơn phần tùy biến món:** sơ đồ thể hiện ModifierOption và OrderItemModifier để lưu được lựa chọn chi tiết ở mức tùy chọn, thay vì chỉ dừng ở mức nhóm modifier.
3. **Bao phủ khuyến mãi và giảm giá:** sơ đồ thể hiện PromotionCampaign và AppliedDiscount để hóa đơn phản ánh đúng các chương trình ưu đãi được áp dụng trong từng giao dịch.
4. **Bao phủ quản trị nhân sự theo ca:** sơ đồ thể hiện ShiftAssignment vì IRMS có nêu rõ nhu cầu quản trị và sắp lịch nhân viên.
5. **Làm rõ các giao diện mở rộng:** các giao diện cho thanh toán, thông báo và phân quyền được thể hiện riêng để chỉ ra rõ các điểm mở rộng và các chính sách nhạy cảm của hệ thống.

Sơ đồ lớp trong báo cáo này không được hiểu như một ERD thuần dữ liệu. Thay vào đó, nó được dùng để biểu diễn **trạng thái nghiệp vụ, ranh giới record/aggregate và các điểm mở rộng có bằng chứng trong code**. Cách mô hình hóa này giúp làm rõ hệ thống lưu giữ và liên kết các khái niệm nghiệp vụ nào, đồng thời tránh mô

tả nhằm các thao tác của service thành method nằm trên entity; phần biện minh cho góc nhìn lớp, mức chi tiết của từng cụm lớp và mối liên hệ của chúng với kiến trúc tổng thể được dẫn ngay tới Bảng 36, Bảng 37 và Bảng 38.



Hình 29: Sơ đồ lớp miền lõi tổng thể của IRMS

Hình 29 là sơ đồ lớp ở mức nhìn toàn hệ thống, có nhiệm vụ thể hiện các cụm nghiệp vụ quan trọng và các liên kết chính giữa chúng. Thay vì chỉ tập trung vào một số lớp giao dịch quen thuộc như Order hay Bill, sơ đồ này giữ lại các điểm neo chính của đặt bàn, phiên bàn, gọi món, bếp, thanh toán, tồn kho, quản trị, kiểm toán và báo cáo. Các lớp chi tiết hơn như WaitlistEntry, DiningTable, ModifierGroup, BillSplit hoặc ShiftAssignment được trình bày ở các sơ đồ cụm phía sau. Nhờ đó, người đọc có thể nắm được **bức tranh miền nghiệp vụ tổng thể** trước khi đi sâu vào từng cụm lớp chi tiết hơn.

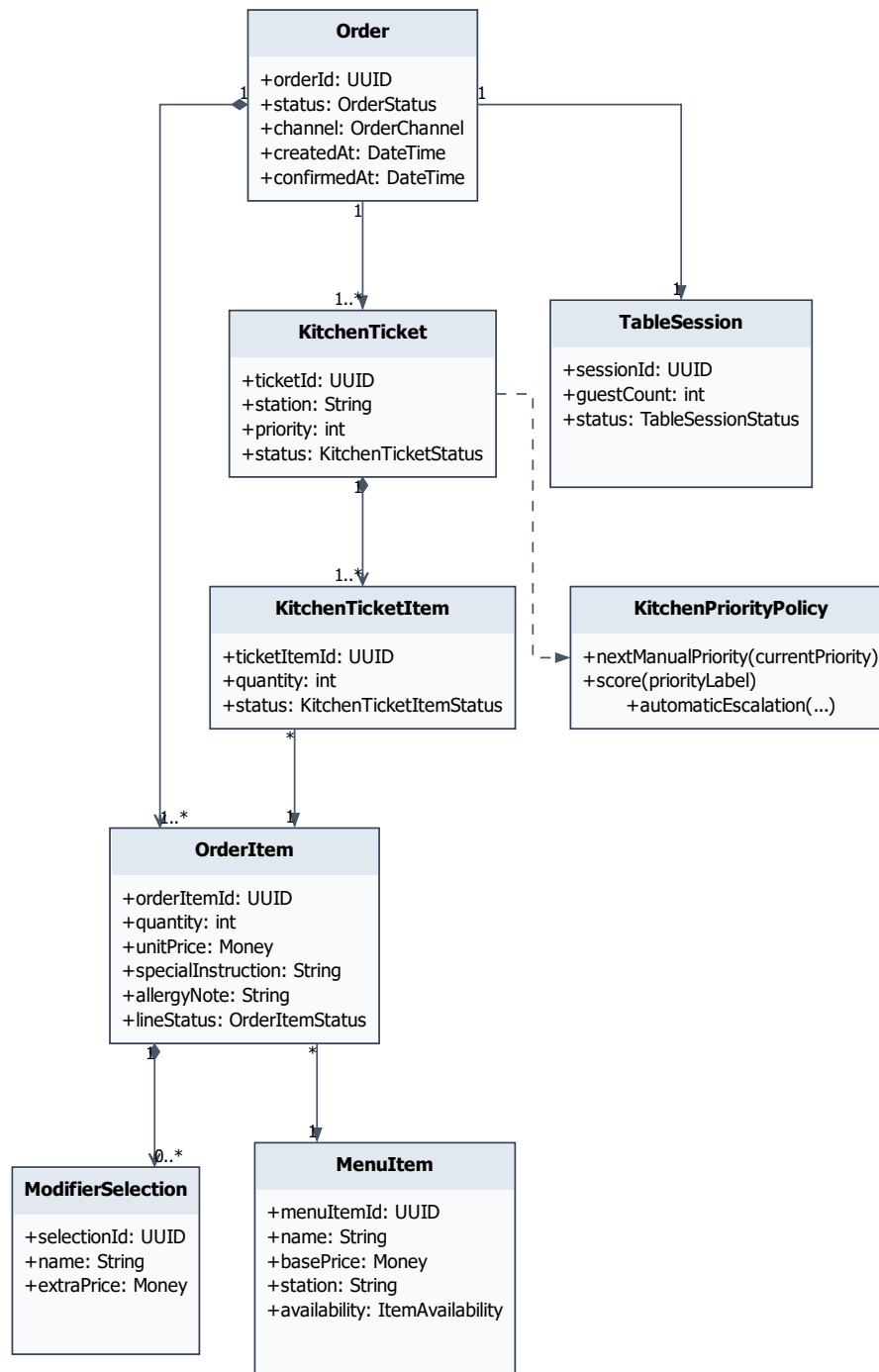
Các nhóm lớp chính thể hiện trong hình.

- **Đặt bàn và phục vụ tại bàn:** Reservation và TableSession biểu diễn điểm nối giữa lịch đặt bàn với phiên phục vụ tại bàn.
- **Thực đơn, gọi món và bếp:** MenuItem, Order và KitchenTicket thể hiện đường đi chính từ món trong thực đơn tới đơn gọi món và phiếu bếp.
- **Quyết toán và thanh toán:** Bill và Payment mô tả liên kết giữa hóa đơn và các lần thu tiền.
- **Tồn kho:** InventoryItem là điểm neo để các luồng chế biến làm phát sinh biến động tồn kho.
- **Quản trị, kiểm toán và báo cáo:** User, AuditLog và ReportSnapshot đảm nhiệm phân truy vết hành động và tổng hợp dữ liệu vận hành.

Lý do thiết kế của sơ đồ này.

- Sơ đồ tổng thể được đặt ở mức đủ rộng để người đọc nhận ra ranh giới giữa các miền nghiệp vụ, nhưng chưa đi quá sâu vào chi tiết cài đặt bên trong từng service.
- Các quan hệ được lựa chọn theo tiêu chí nghiệp vụ, nghĩa là chỉ thể hiện những liên kết có ý nghĩa vận hành rõ ràng như đặt bàn gắn với bàn ăn, phiên bàn sinh đơn gọi món, đơn gọi món sinh phiếu bếp, và hóa đơn quyết toán phiên bàn.
- Các điểm mở rộng như PaymentGateway, NotificationChannelAdapter và AuthorizationPolicy được đưa vào ngay trong sơ đồ tổng thể để nhấn mạnh rằng mô hình lớp không chỉ mô tả dữ liệu, mà còn chỉ ra **ranh giới mở rộng và kiểm soát**.

Điểm mạnh của sơ đồ tổng thể là hỗ trợ kiểm tra nhanh các liên kết miền lỗi của IRMS trước khi đi vào từng sơ đồ chi tiết. Nếu cần kiểm tra phạm vi đầy đủ của một cụm nghiệp vụ, người đọc tiếp tục đối chiếu các hình cụm phía sau vì những hình đó mới trình bày đầy đủ các lớp chi tiết trong từng miền.



Hình 30: Sơ đồ lớp miền lỗi của cụm gọi món, thực đơn và bếp

Hình 30 đi vào cụm lớp có mật độ tương tác cao nhất trong vận hành hằng ngày của nhà hàng. Đây là nơi

luồng nghiệp vụ bắt đầu từ dữ liệu thực đơn, đi qua quá trình chọn món và tùy biến, sau đó chuyển thành các đơn vị điều phối ở bếp. Vì vậy, sơ đồ này có vai trò quan trọng trong việc chứng minh rằng mô hình gọi món của IRMS không bị giản lược thành một bảng dữ liệu phẳng.

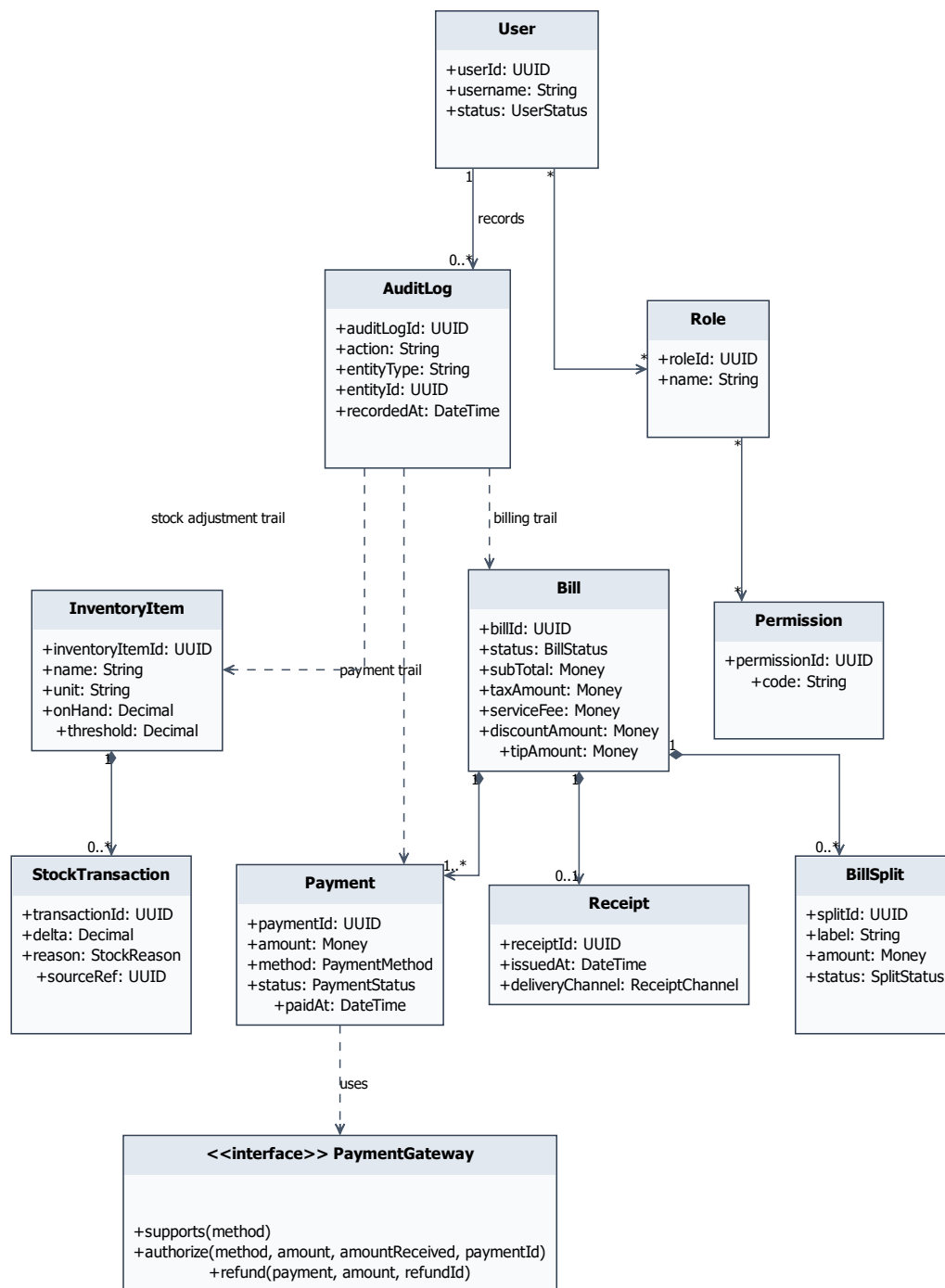
Các lớp trọng tâm thể hiện trong hình.

- **MenuCategory** và **MenuItem**: biểu diễn cấu trúc thực đơn và là nguồn gốc của giá cơ bản, trạng thái còn bán và thông tin điều phối.
- **ModifierGroup** và **ModifierOption**: chịu trách nhiệm ràng buộc việc tùy biến món, ví dụ giới hạn số lựa chọn tối thiểu và tối đa hoặc thay đổi giá do tùy chọn bổ sung.
- **Order** và **OrderItem**: là lõi của giao dịch gọi món; Order giữ trạng thái đơn tổng thể, còn OrderItem là đơn vị bán hàng cụ thể.
- **OrderItemModifier**: lưu ảnh chụp của tùy biến đã chọn tại thời điểm gọi món, giúp hệ thống không bị lệ thuộc ngược vào cấu hình thực đơn hiện tại.
- **KitchenStation**, **KitchenTicket** và **KitchenTicketItem**: chuyển dữ liệu từ miền bán hàng sang miền chế biến, đồng thời giữ được trạng thái của từng trạm bếp thay vì dồn toàn bộ tiến độ vào Order.

Lý do thiết kế của sơ đồ này.

- Tách OrderItem khỏi KitchenTicketItem để phân biệt rõ **đơn vị bán hàng** với **đơn vị điều phối chế biến**; đây là quyết định cần thiết nếu một món phải được định tuyến sang trạm bếp cụ thể.
- Dùng cơ chế ảnh chụp ở OrderItem và OrderItemModifier để bảo vệ tính đúng đắn của hóa đơn và phiếu bếp ngay cả khi thực đơn thay đổi sau thời điểm gọi món.
- Giữ KitchenStation như một lớp riêng để quy tắc định tuyến, mức ưu tiên và trạng thái bếp không làm phình to aggregate Order.

Ưu điểm của cụm lớp này là phản ánh được đồng thời **cấu hình thực đơn**, **dữ liệu giao dịch** và **trạng thái chế biến**. Nếu thiếu một trong ba lớp nghĩa đó, hệ thống rất dễ rơi vào tình trạng hoặc khó mở rộng nghiệp vụ, hoặc không theo dõi được đường đi thực sự của món từ màn hình gọi món xuống bếp.



Hình 31: Sơ đồ lớp miền lõi của cụm thanh toán, tồn kho và quản trị

Hình 31 tập trung vào vùng nhạy cảm nhất của kiến trúc, nơi các quyết định của hệ thống tác động trực tiếp đến tiền, tồn kho và quyền thao tác của người dùng. Vì đây là khu vực dễ phát sinh sai sót có hậu quả vận hành lớn, sơ đồ được trình bày chi tiết hơn để người đọc thấy rõ lớp nào chịu trách nhiệm giao dịch, lớp nào chịu trách nhiệm truy vết, và lớp nào đóng vai trò cổng mở rộng.

Các lớp trọng tâm thể hiện trong hình.

- **Cụm quyết toán:** Bill, BillSplit, AppliedDiscount và PromotionCampaign cùng nhau biểu diễn logic tổng hợp tiền, tách hóa đơn và áp dụng khuyến mãi.
- **Cụm thanh toán:** Payment, Refund, Receipt và PaymentGateway mô tả việc thu tiền, hoàn tiền, phát hành biên lai và giao tiếp qua adapter biên trừ tương.
- **Cụm tồn kho:** InventoryItem, RecipeLine, StockTransaction, LowStockRule và NotificationChannelAdapter mô tả đường suy diễn từ món bán ra tới biến động tồn kho và cảnh báo mức tồn thấp.
- **Cụm quản trị và kiểm soát:** User, Role, Permission, AuditLog, AuthorizationPolicy, ShiftAssignment và ReportSnapshot tạo nền cho phân quyền, kiểm toán, lịch ca và báo cáo.

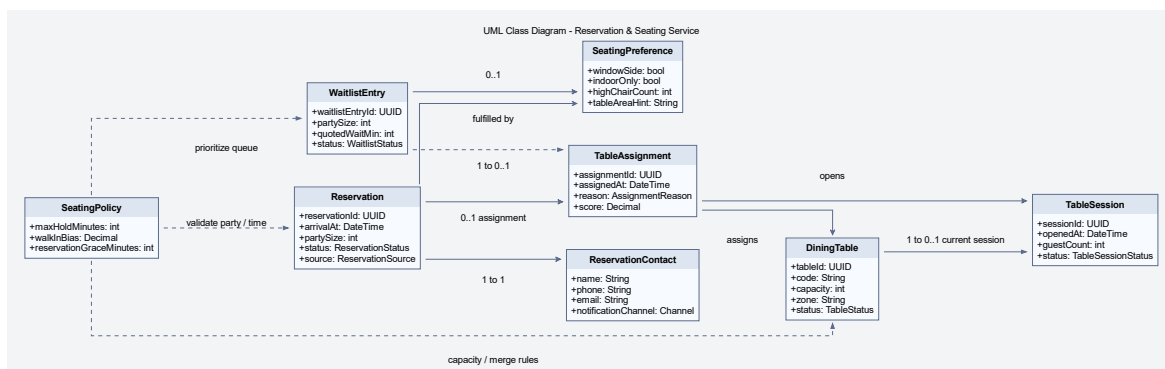
Lý do thiết kế của sơ đồ này.

- Đặt PaymentGateway và NotificationChannelAdapter dưới dạng giao diện để miền nghiệp vụ không phụ thuộc trực tiếp vào một lớp hiện thực cụ thể.
- Tách Refund ra khỏi Payment để phản ánh đúng việc hoàn tiền là một hành động nghiệp vụ riêng, có lý do, ngưỡng phê duyệt và trạng thái theo dõi riêng.
- Đưa AuditLog và AuthorizationPolicy vào cùng sơ đồ với giao dịch để nhấn mạnh rằng kiểm soát quyền và truy vết không phải phần phụ, mà là một phần của thiết kế lõi.
- Giữ ReportSnapshot ở cụm này để thể hiện rõ dữ liệu báo cáo được suy diễn từ dữ liệu vận hành, thay vì can thiệp trực tiếp vào tuyến giao dịch nóng.

Ưu điểm lớn nhất của sơ đồ này là cho thấy kiến trúc lớp của IRMS không chỉ giải quyết việc “thực hiện đúng chức năng”, mà còn giải quyết việc “kiểm soát được hậu quả vận hành” khi xảy ra hoàn tiền, điều chỉnh kho, ghi đề giá hay tranh chấp thanh toán. Đây là điểm làm cho phần thiết kế chặt chẽ hơn dưới góc nhìn kiến trúc phần mềm trong môi trường vận hành thực tế.

4.2 Giải thích theo từng cụm lớp

Phần này được trình bày theo trình tự *quan sát sơ đồ* → *xác định cụm trách nhiệm* → *làm rõ lý do mô hình hóa*. Cách trình bày này giúp phần sơ đồ lớp không dừng lại ở mức liệt kê lớp, mà làm rõ vai trò trong miền, ranh giới trách nhiệm và quyết định thiết kế của từng service.



Hình 32: Sơ đồ lớp miền lõi chi tiết cho Reservation & Seating Service

Hình 32 đi vào Reservation & Seating Service ở mức chi tiết và đúng tinh thần mô hình miền. Trọng tâm của service này không chỉ là “quản lý danh sách bàn”, mà là quản lý **năng lực phục vụ theo thời gian**: ai đến trước, ai có cam kết trước, bàn nào phù hợp, khi nào được mở phiên, và trên cơ sở nào quyết định gán bàn được đưa ra. Vì vậy, sơ đồ được tổ chức quanh ba trục: **nguồn nhu cầu** (Reservation, WaitlistEntry), **tài nguyên phục vụ** (DiningTable, TableSession) và **quy tắc điều phối** (TableAssignment, SeatingPolicy, SeatingPreference).

Các cụm lớp và trách nhiệm thể hiện trong hình.

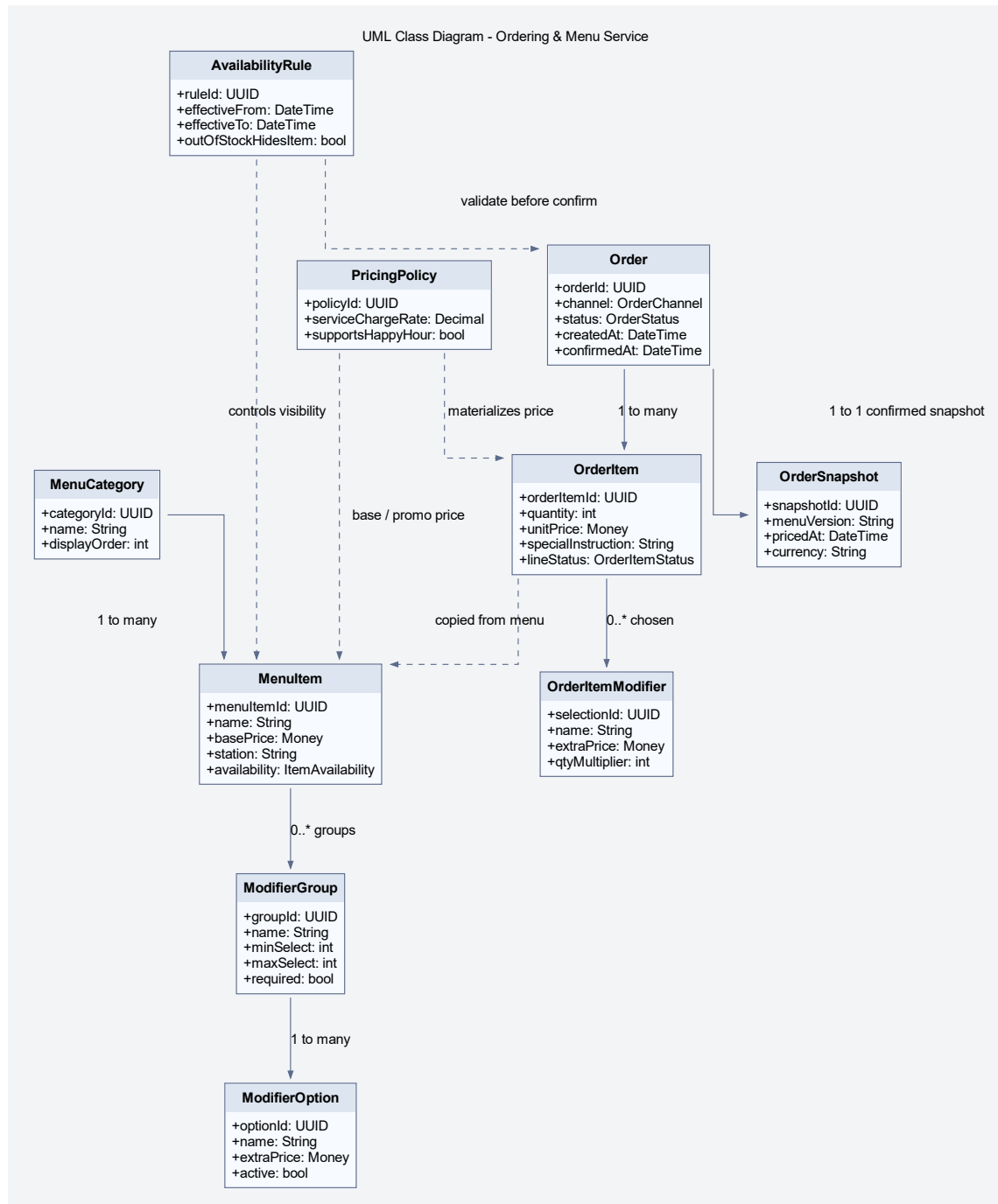
- **Nguồn nhu cầu trước khi phục vụ:** Reservation giữ thông tin đặt trước, trạng thái và các hành vi như xác nhận, check-in, no-show và hủy; ReservationContact tách riêng thông tin liên hệ để tránh dồn dữ liệu truyền thông vào thực thể đặt bàn.
- **Nguồn nhu cầu chưa được đáp ứng ngay:** WaitlistEntry mô hình hóa khách vắng lai hoặc trường hợp chưa có bàn phù hợp tại thời điểm hiện tại; điều này quan trọng vì waitlist không nên bị ép vào cùng vòng đời với reservation.
- **Tài nguyên vật lý và phiên vận hành:** DiningTable biểu diễn bàn vật lý với sức chứa và vùng; TableSession biểu diễn phiên phục vụ thực tế, độc lập với việc bàn đó có được đặt trước hay không.
- **Lớp điều phối:** TableAssignment lưu quyết định gán bàn cụ thể, còn SeatingPolicy và SeatingPreference làm rõ phần “cơ sở chọn bàn” thay vì để quy tắc nằm rải rác trong service.

Giải thích chi tiết theo từng cụm lớp.

- **Reservation không đồng nhất với TableSession.** Một reservation có thể bị hủy, no-show hoặc dời giờ mà không sinh phiên phục vụ. Ngược lại, một table session có thể được mở cho khách walk-in mà không có reservation. Việc tách hai lớp này là quyết định mô hình hóa quan trọng nhất của Reservation & Seating Service.
- **WaitlistEntry là đối tượng hạng nhất của miền.** Nếu chỉ xem waitlist là một trạng thái phụ của reservation, hệ thống sẽ khó diễn tả đúng khách vắng lai, khách chưa chắc ngồi, khách đổi ý hoặc khách chấp nhận vùng bàn khác sau một khoảng chờ.
- **TableAssignment giữ lịch sử quyết định gán bàn.** Điều này hữu ích cho giải thích vận hành, tái gán bàn, và về sau có thể nối với audit hoặc analytics để đo hiệu quả xếp bàn theo giờ cao điểm.
- **SeatingPolicy là nơi đặt rule thay vì hard-code trong controller.** Các rule như thời gian giữ chỗ, ưu tiên walk-in hay reservation, giới hạn mở session và năng lực gộp bàn nên được gom vào chính sách có tên gọi rõ ràng.

Lý do thiết kế của sơ đồ này.

- Tách ReservationContact và SeatingPreference giúp thực thể Reservation gọn hơn, chỉ giữ phần nhận diện và trạng thái lỗi.
- Dùng TableAssignment như đối tượng trung gian để lưu lý do và điểm chấm bàn, nhờ đó quyết định xếp bàn có thể giải thích được.
- Giữ TableSession như bản ghi riêng nhằm bảo vệ toàn bộ tuyến gọi món, hóa đơn và thanh toán khỏi việc phụ thuộc trực tiếp vào đặt bàn.



Hình 33: Sơ đồ lớp miền lõi chi tiết cho Ordering & Menu Service

Hình 33 mô tả Ordering & Menu Service, service có cường độ vận hành cao nhất của toàn hệ thống. Về mặt miền, service này phải giải quyết mâu thuẫn giữa **thực đơn biến đổi liên tục** và **giao dịch cần được cố định**. Thực đơn có thể thay đổi thường xuyên: ăn món, đổi giá, thêm tùy chọn món hoặc đổi giờ phục vụ. Trong khi đó, đơn gọi món đã xác nhận phải giữ nguyên đúng tên món, giá, tùy chọn và điều kiện áp dụng tại thời điểm xác nhận. Vì vậy, sơ đồ tách rõ hai nhóm đối tượng: Menu* là phần cấu hình có thể thay đổi, còn Order* là phần giao dịch đã được chốt.

Các cụm lớp và trách nhiệm thể hiện trong hình.

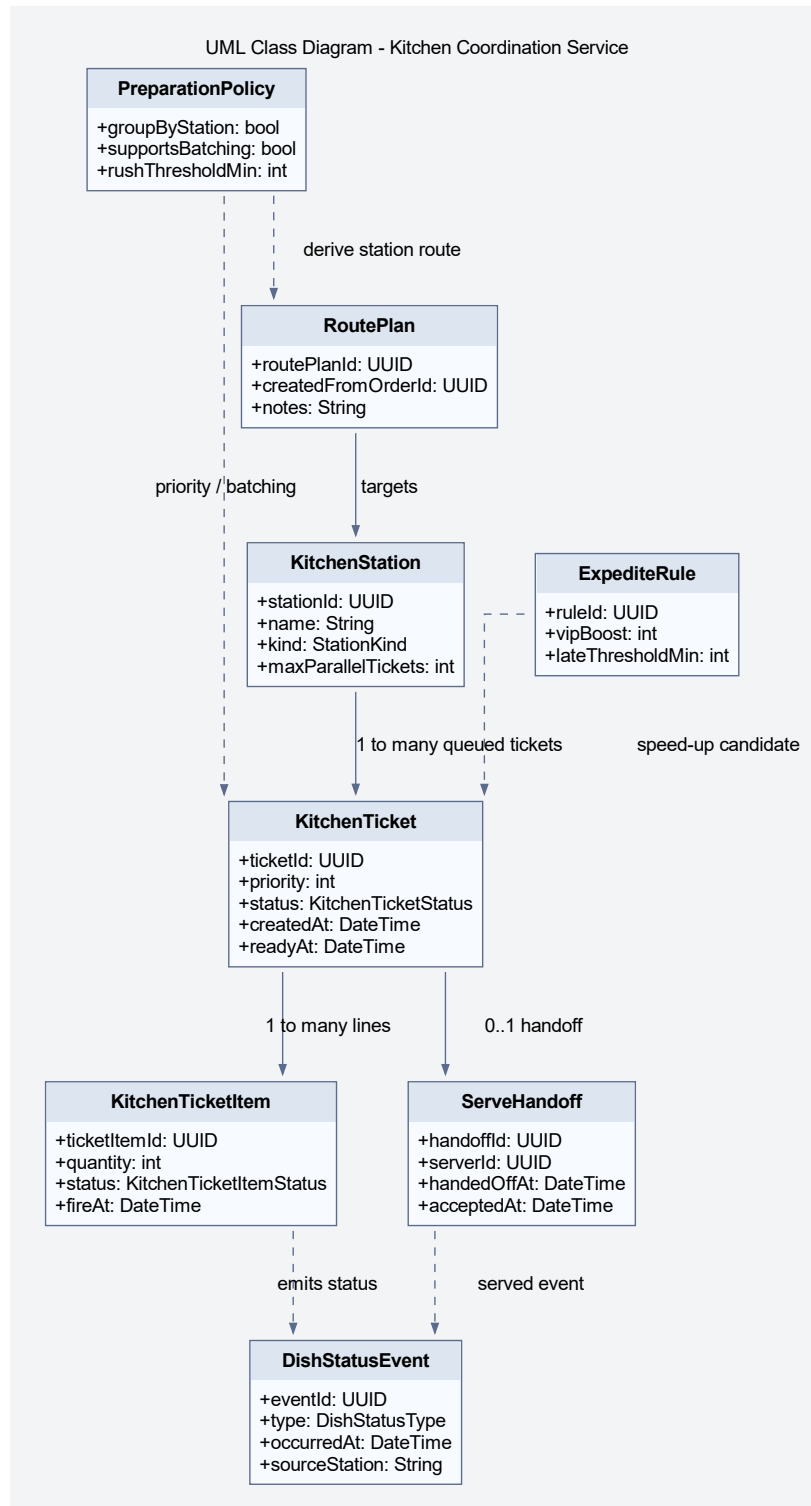
- **Cấu hình thực đơn:** MenuCategory, MenuItem, ModifierGroup và ModifierOption tạo nên cấu trúc menu mà quản lý có thể thay đổi.
- **Quy tắc khả dụng và định giá:** AvailabilityRule và PricingPolicy gom logic giờ bán, trạng thái hết hàng, giá cơ sở và các quy tắc điều chỉnh giá.
- **Giao dịch đơn gọi món:** Order, OrderItem và OrderItemModifier lưu lại món đã chọn, tùy chọn đã chọn, số lượng, ghi chú và trạng thái của từng dòng món.
- **Ảnh chụp giao dịch:** OrderSnapshot cho thấy tại thời điểm xác nhận, hệ thống có điểm neo để lần vết lại phiên bản menu và cách tính giá đã được áp dụng.

Giải thích chi tiết theo từng cụm lớp.

- **MenuItem không nên được dùng trực tiếp như OrderItem.** Nếu dùng trực tiếp, chỉ cần đổi tên hoặc giá món sau đó là lịch sử giao dịch và hóa đơn sẽ bị biến dạng.
- **ModifierGroup và ModifierOption là cấu trúc mang quy tắc.** Chúng không chỉ là dữ liệu hiển thị; chúng còn quyết định số lựa chọn bắt buộc, số lượng tối đa và phần cộng giá thêm.
- **PricingPolicy nên tách khỏi MenuItem.** Giá của một món không chỉ là basePrice; nó còn phụ thuộc khung giờ, phí phục vụ, chương trình áp dụng hoặc logic định giá theo bối cảnh. Tách chính sách này giúp Ordering & Menu Service tuân thủ trách nhiệm đơn nhất và dễ mở rộng hơn.
- **OrderSnapshot khóa ngữ nghĩa tại thời điểm xác nhận.** Đây là điểm rất quan trọng khi cần kiểm toán, đối soát hóa đơn hoặc tái hiện đơn hàng trong trường hợp tranh chấp.

Lý do thiết kế của sơ đồ này.

- Việc tách rule và data giúp Ordering & Menu Service vừa linh hoạt cho quản trị thực đơn, vừa chắc chắn cho tuyến giao dịch.
- Đặt AvailabilityRule và PricingPolicy trong cùng sơ đồ để nhấn mạnh rằng validate và price phải là một phần của domain model, chứ không chỉ là service logic.
- Duy trì OrderItemModifier như thực thể riêng để ghi nhận modifier thực tế đã chọn thay vì chỉ giữ quan hệ tham chiếu tới modifier cấu hình.



Hình 34: Sơ đồ lớp miền lõi chi tiết cho Kitchen Coordination Service

Hình 34 làm rõ rằng Kitchen Coordination Service là một service nghiệp vụ riêng, chứ không chỉ là một trạng thái phụ của đơn gọi món. Khi đơn gọi món đã được xác nhận, hệ thống bắt đầu giải quyết một bài toán khác: chia việc theo trạm, xếp hàng đợi, điều chỉnh mức ưu tiên, theo dõi tiến độ và bàn giao món đã hoàn tất cho tuyến phục vụ. Nếu dồn toàn bộ logic này vào Order hoặc OrderItem, mô hình sẽ nhanh chóng lẫn lộn giữa góc nhìn bán

hàng và góc nhìn chế biến.

Các cụm lớp và trách nhiệm thể hiện trong hình.

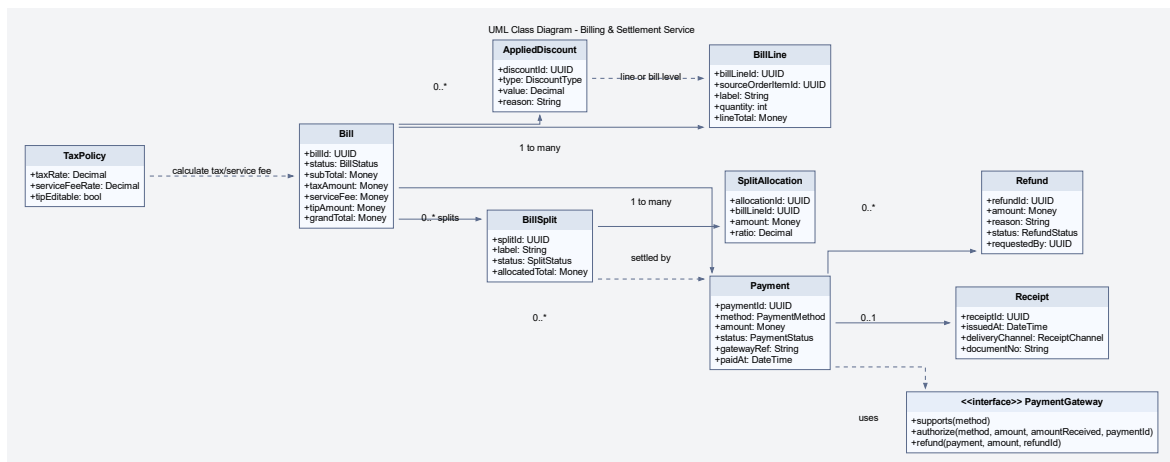
- **Tài nguyên bếp:** KitchenStation biểu diễn từng trạm với năng lực xử lý, loại trạm và hàng chờ tương ứng.
- **Đơn vị điều phối:** KitchenTicket là phiếu công việc ở mức phiếu bếp; KitchenTicketItem là đơn vị thực thi cụ thể theo món hoặc nhóm món.
- **Chính sách điều phối:** PreparationPolicy, RoutePlan và ExpediteRule gom logic tách phiếu, nhóm món, định tuyến và tăng ưu tiên.
- **Bàn giao trạng thái:** ServeHandoff và DishStatusEvent neo đường truyền trạng thái món ra khỏi bếp.

Giải thích chi tiết theo từng cụm lớp.

- **KitchenTicket là aggregate riêng.** Nó có vòng đời chờ, đang làm, sẵn sàng và đã bàn giao; vòng đời này không nên bị gộp vào trạng thái đơn gọi món.
- **KitchenTicketItem cho phép xử lý ở mức hạt mịn.** Trong thực tế, không phải toàn bộ phiếu bếp luôn đi cùng một tốc độ; từng dòng có thể bị giữ, sẵn sàng hay bị chậm riêng.
- **PreparationPolicy và ExpediteRule là nơi đặt thuật toán điều phối.** Nhờ đó, logic ưu tiên khách VIP, món bị trễ hoặc gom nhóm theo trạm bếp có vị trí rõ ràng trong mô hình.
- **ServeHandoff tạo điểm chốt giữa bếp và phục vụ.** Một món ở trạng thái ready chưa đồng nghĩa với trạng thái served; thêm lớp bàn giao giúp phân biệt rõ hai giai đoạn này.

Lý do thiết kế của sơ đồ này.

- Tách miền bếp thành Kitchen Coordination Service giúp hệ thống mở rộng thuận lợi hơn khi cần nhiều station hoặc nhiều màn hình bếp khác nhau.
- Dùng lớp sự kiện DishStatusEvent để cho phép mô hình đọc, thông báo hoặc phân tích tiến độ mà không chạm trực tiếp vào aggregate bếp.
- Mô hình này cũng làm rõ vì sao góc nhìn thành phần cần KitchenTicketFactory và KitchenApplicationService riêng.



Hình 35: Sơ đồ miền lõi chi tiết cho Billing & Settlement Service

Hình 35 trình bày vùng quyết toán với độ sâu phù hợp cho báo cáo. Ở IRMS, bài toán quyết toán không chỉ dừng ở việc “tính tổng tiền”: hệ thống còn phải biểu diễn dòng hóa đơn, tách hóa đơn, phân bổ dòng vào từng phần thanh toán, áp khuyến mãi, tính thuế và phí, thu tiền theo một hay nhiều giao dịch thanh toán, phát hành biên lai và theo dõi hoàn tiền. Mỗi quyết định trong vùng này đều có hậu quả tài chính, vì vậy mô hình lớp phải được phân tách đủ rõ để tránh một lớp Bill quá lớn ôm toàn bộ trách nhiệm.

Các cụm lớp và trách nhiệm thể hiện trong hình.

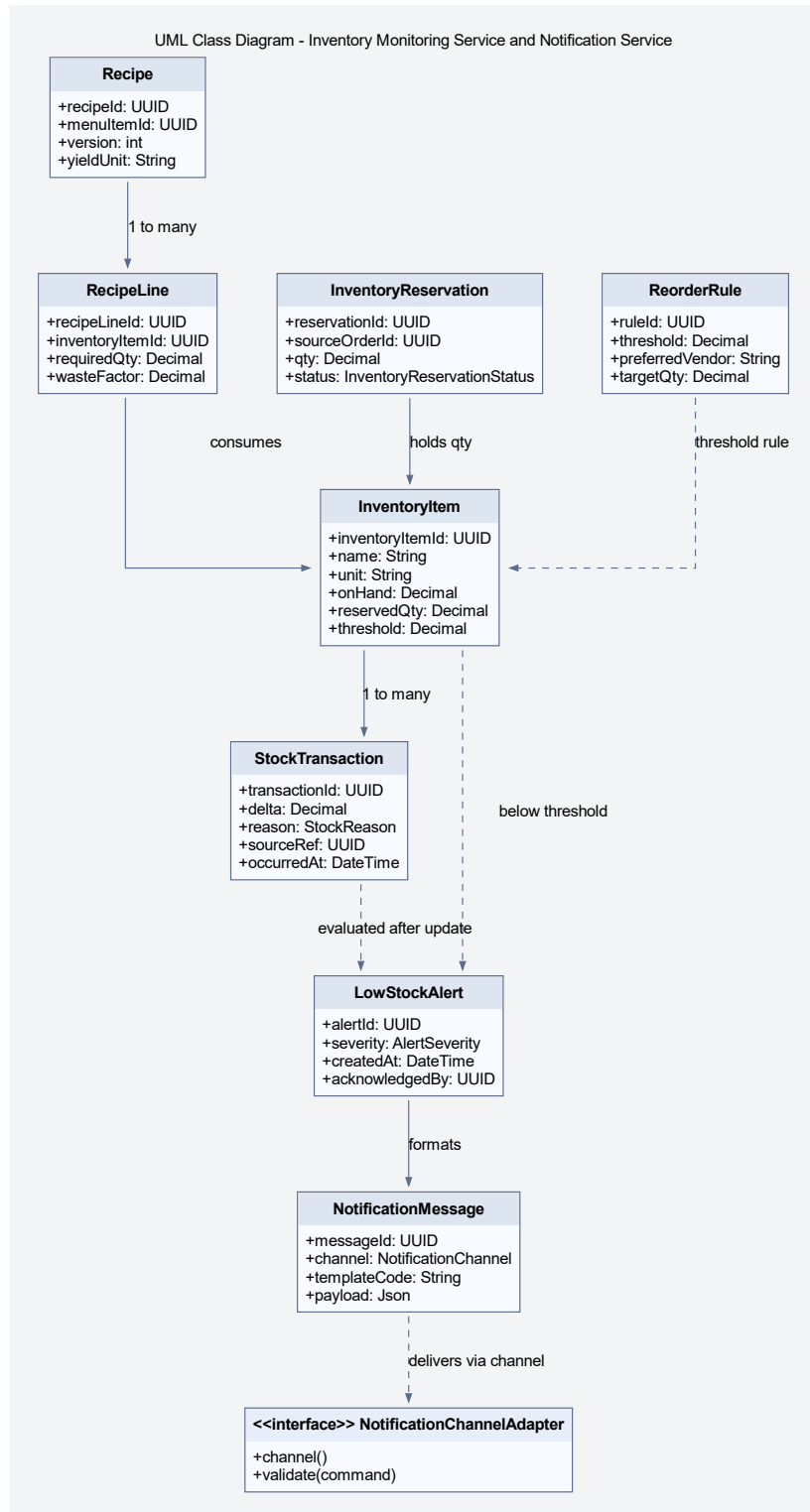
- **Hóa đơn và dòng hóa đơn:** Bill và BillLine là trung tâm quyết toán, nơi dữ liệu đơn gọi món đã xác nhận được gom lại thành đối tượng tài chính.
- **Tách hóa đơn:** BillSplit và SplitAllocation biểu diễn đúng nghiệp vụ chia hóa đơn, thay vì coi split chỉ là một vài cờ trạng thái hoặc danh sách tạm.
- **Thanh toán và hoàn tiền:** Payment, Refund và Receipt tạo thành vòng đời thu tiền, hoàn tiền và chứng từ.
- **Quy tắc quyết toán:** AppliedDiscount, TaxPolicy và giao diện PaymentGateway gom logic giảm giá, làm tròn, thuế và adapter thanh toán.

Giải thích chi tiết theo từng cụm lớp.

- **BillSplit là đối tượng hạng nhất.** Trong nhà hàng, việc tách hóa đơn không phải thao tác hiếm; nó có nhãn, phạm vi, tổng phân bổ và trạng thái riêng. Mô hình hóa tường minh giúp hỗ trợ chia theo người, theo dòng món hoặc theo tỷ lệ.
- **Payment tách khỏi Bill.** Một hóa đơn có thể có nhiều giao dịch thanh toán; một giao dịch có thể thất bại, cần thử lại, hoặc bị hoàn tiền một phần. Dồn logic này vào hóa đơn sẽ làm Bill có quá nhiều lý do thay đổi.
- **Refund là thực thể riêng, không chỉ là số âm của payment.** Nó cần lý do, người yêu cầu, trạng thái và về sau có thể cần cơ chế phê duyệt.
- **PaymentGateway phải là lớp trừu tượng.** Lỗi quyết toán cần thực hiện các thao tác nghiệp vụ như ủy quyền, ghi nhận giao dịch và hoàn tiền, nhưng không nên phụ thuộc vào SDK hoặc giao thức đặc thù của từng nhà cung cấp.

Lý do thiết kế của sơ đồ này.

- Mô hình này bám sát tuyến giao dịch nhưng vẫn chừa đúng điểm mở cho bộ kết nối.
- Việc tách TaxPolicy và AppliedDiscount làm rõ đâu là dữ liệu nghiệp vụ, đâu là quy tắc tính toán.
- Sơ đồ cũng chuẩn bị tốt cho kiểm toán: hoàn tiền, tách hóa đơn và giao dịch thanh toán đều đã có chỗ neo riêng để nối với nhật ký kiểm toán hoặc luồng phê duyệt.



Hình 36: Sơ đồ lớp kỹ thuật cho Inventory Monitoring Service và Notification Service

Hình 36 mở rộng phần tồn kho theo hướng **truy vết được nguồn gốc biến động** và **phân biệt rõ dữ liệu kho với kênh cảnh báo**. Hình này được gọi là sơ đồ lớp kỹ thuật vì ngoài các record nghiệp vụ trong migration, nó còn giữ NotificationChannelAdapter như một interface mở rộng thật trong code. Một hệ thống nhà hàng thực tế không thể chỉ có trường onHand; nó phải biết món nào dùng nguyên liệu nào, thời điểm nào thì giữ tạm nguyên

liệu, khi nào thì trừ thật, và khi nào cần phát ra cảnh báo mức tồn thấp. Vì vậy, sơ đồ bổ sung các lớp Recipe, RecipeLine, InventoryReservation, ReorderRule, LowStockAlert và NotificationMessage.

Các cụm lớp và trách nhiệm thể hiện trong hình.

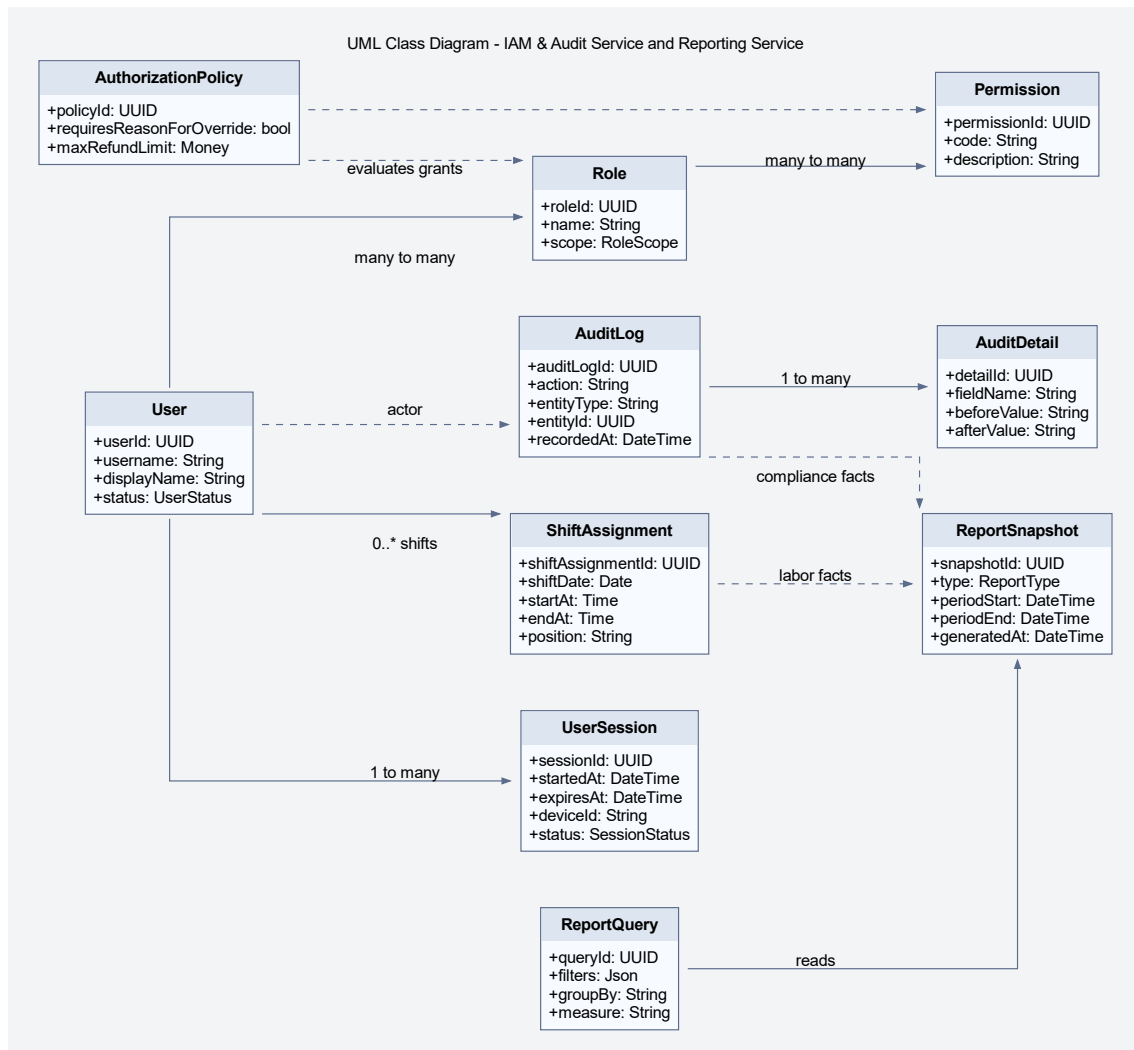
- **Số dư và giao dịch kho:** InventoryItem và StockTransaction giữ trạng thái kho hiện tại và lịch sử biến động.
- **Suy diễn mức tiêu hao:** Recipe và RecipeLine nối món bán ra với nguyên liệu cần dùng.
- **Giữ chỗ và chốt trừ:** InventoryReservation cho phép mô tả giai đoạn giữ tạm nguyên liệu trước khi thanh toán hoàn tất hoặc đơn gọi món được chốt.
- **Cảnh báo và kênh gửi:** ReorderRule, LowStockAlert, NotificationMessage và giao diện NotificationChannelAdapter tách rõ điều kiện phát cảnh báo khỏi hạ tầng chuyển thông báo.

Giải thích chi tiết theo từng cụm lớp.

- **Recipe là cầu nối giữa thực đơn và kho.** Nếu thiếu lớp này, phần tiêu hao nguyên liệu sẽ bị mã hóa cứng trong logic gọi món hoặc quyết toán.
- **InventoryReservation giúp mô hình linh hoạt hơn.** Một số phương án triển khai muốn trừ kho ngay khi xác nhận đơn gọi món, số khác chỉ chốt hẳn khi thanh toán hoàn tất; có lớp giữ chỗ giúp mở được cả hai chiến lược.
- **StockTransaction là bằng chứng, không chỉ là lịch sử phụ.** Nó cho phép giải thích vì sao tồn kho thay đổi, hỗ trợ reconciliation và audit sau ca.
- **Notification cần đi qua lớp trừu tượng.** Cảnh báo tồn thấp là nghiệp vụ; xử lý kênh email, sms, in-app hay print lại là bài toán của NotificationChannelAdapter.

Lý do thiết kế của sơ đồ này.

- Mô hình tách được tồn kho khỏi kênh thông báo nhưng vẫn giữ liên hệ chặt thông qua LowStockAlert.
- Nó hỗ trợ tốt cho sơ đồ thành phần bất đồng bộ, nơi bộ xử lý nền có thể gửi cảnh báo sau khi service tồn kho phát hiện vượt ngưỡng.
- Nó cũng tạo điểm nối dữ liệu sạch hơn cho phần báo cáo doanh thu–tồn kho ở giai đoạn sau.



Hình 37: Sơ đồ lớp kỹ thuật cho IAM & Audit Service và Reporting Service

Hình 37 trình bày phần governance ở mức phù hợp với kiến trúc của một hệ thống vận hành thực tế. Hình này được gọi là sơ đồ lớp kỹ thuật vì nó kết hợp các record lưu trữ, policy và lớp truy vấn báo cáo để làm rõ cơ chế kiểm soát thay vì chỉ mô tả thực thể nghiệp vụ thuần. Trong IRMS, user, role và permission mới chỉ là phần đầu. Hệ thống còn cần mô tả phiên đăng nhập (UserSession), chính sách kiểm tra quyền theo bối cảnh (AuthorizationPolicy), nhật ký kiểm toán đa thuộc tính (AuditLog, AuditDetail), phân công ca (ShiftAssignment) và ảnh chụp báo cáo cùng truy vấn báo cáo (ReportSnapshot, ReportQuery). Chỉ khi các lớp này được biểu diễn tường minh, phần “quản trị, kiểm toán, báo cáo” mới vượt khỏi mức mô tả khái quát.

Các cụm lớp và trách nhiệm thể hiện trong hình.

- **Danh tính và quyền:** User, Role, Permission và UserSession giữ nền tảng xác thực–phân quyền.
- **Chính sách truy cập:** AuthorizationPolicy gom logic bối cảnh như giới hạn hoàn tiền, yêu cầu nêu lý do ghi đề hoặc yêu cầu phê duyệt.
- **Dấu vết thay đổi:** AuditLog và AuditDetail lưu hành động, thực thể bị tác động và giá trị trước–sau.
- **Nhân sự và phân tích:** ShiftAssignment, ReportSnapshot và ReportQuery nổi dữ liệu vận hành với góc nhìn

quản trị.

Giải thích chi tiết theo từng cụm lớp.

- **AuthorizationPolicy cần là lớp riêng.** Nếu mọi quy tắc quyền đều nằm ở bộ điều khiển hoặc chú giải, hệ thống sẽ khó diễn tả các quyết định phụ thuộc ngữ cảnh nghiệp vụ như số tiền hoàn tiền hay lý do ghi đề.
- **AuditDetail giúp nhật ký kiểm toán có giá trị thực tế.** Chỉ lưu “ai làm gì” là chưa đủ; với thao tác nhạy cảm, cần biết trường nào đã đổi từ gì sang gì.
- **UserSession là đối tượng đáng được mô hình hóa.** Nó hỗ trợ khóa phiên, truy vết thiết bị và quản lý timeout, vốn là nhu cầu thực tế trong nhà hàng khi đổi ca hoặc bàn giao máy.
- **ReportSnapshot tách khỏi ReportQuery.** Snapshot là dữ liệu tổng hợp bền vững cho một kỳ; query là cách người dùng truy cập và lọc dữ liệu đó.

Lý do thiết kế của sơ đồ này.

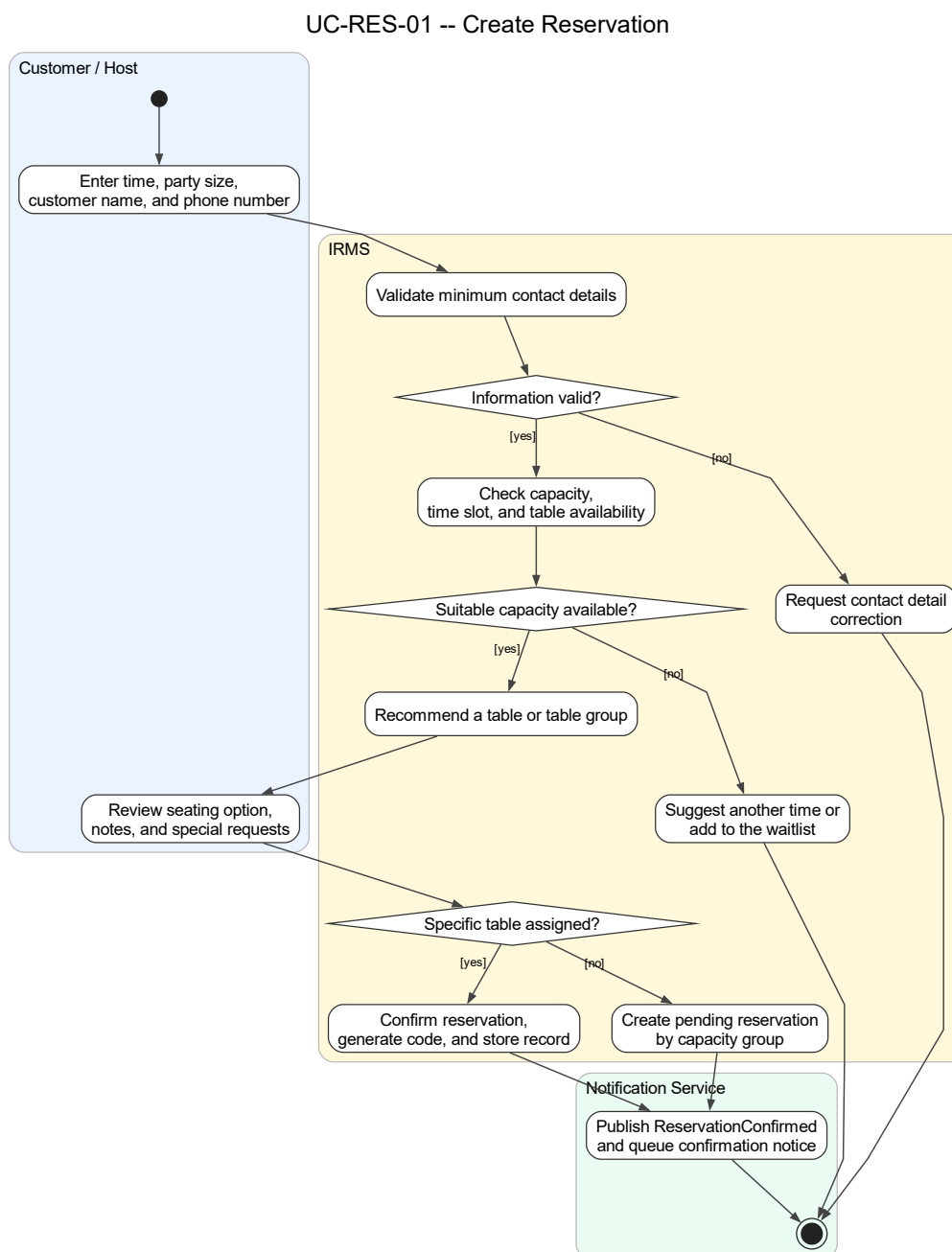
- Vùng governance được biểu diễn chi tiết giúp phân bảo mật, kiểm toán và báo cáo giữ đúng vai trò kiến trúc, thay vì bị xem như lớp chức năng phụ.
- Nó tạo điểm nối rõ ràng giữa sơ đồ lớp với góc nhìn thành phần hỗ trợ và quản trị đã trình bày ở phần trước.
- Nó cũng hỗ trợ tốt cho phân biện minh nguyên lý SOLID, đặc biệt là SRP, OCP và DIP ở vùng policy, audit và reporting.

4.3 Thiết kế hành vi

Phần này mở rộng trực tiếp từ các ca sử dụng đã đặc tả ở trên. Phần đặc tả ca sử dụng xác định tác nhân, mục tiêu, điều kiện và kết quả nghiệp vụ; còn sơ đồ hoạt động diễn đạt thứ tự xử lý, điểm rẽ nhánh và nơi phát sinh ngoại lệ trong từng luồng vận hành. Vì vậy, các sơ đồ dưới đây được xây dựng theo đúng ranh giới vận hành hiện tại của IRMS: lễ tân, phục vụ, bếp, thu ngân, quản lý và hệ thống IRMS.

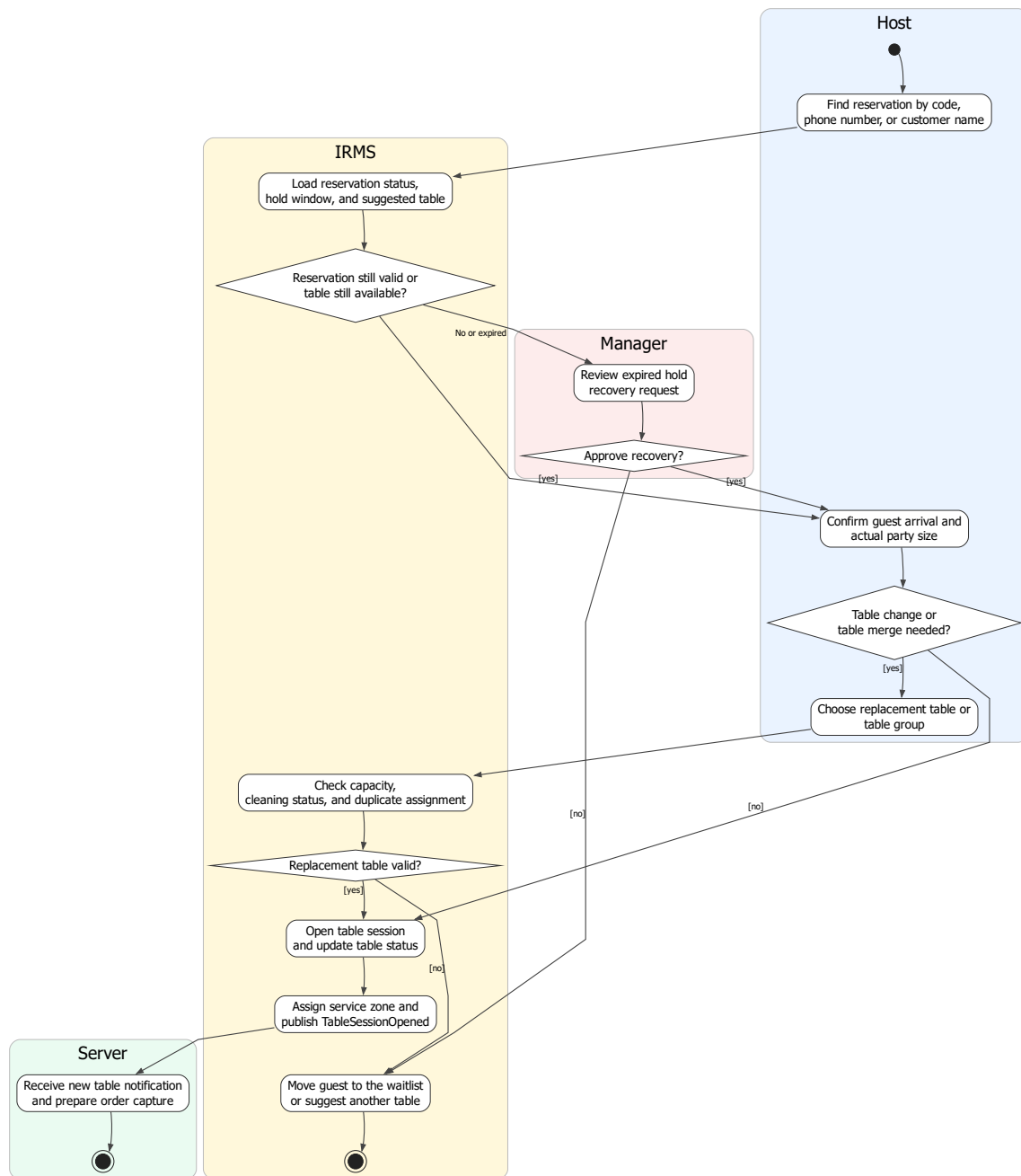
Một điểm quan trọng của nhóm sơ đồ này là mức độ chi tiết không hoàn toàn giống nhau. Những ca sử dụng nằm trên tuyến nghiệp vụ nóng như nhận khách, gọi món, gửi bếp, cập nhật chế biến, tạo hóa đơn, thanh toán, hoàn tiền và cập nhật tồn kho được mở rộng sâu hơn vì đây là các vị trí quyết định trực tiếp đến độ trễ phản hồi, độ tin cậy giao dịch và khả năng truy vết của toàn hệ thống.

4.3.1 Nhóm đặt bàn và bàn ăn



Hình 38: Sơ đồ hoạt động cho UC-RES-01 – Tạo đặt bàn

UC-RES-02 -- Check In Guest and Seat Table

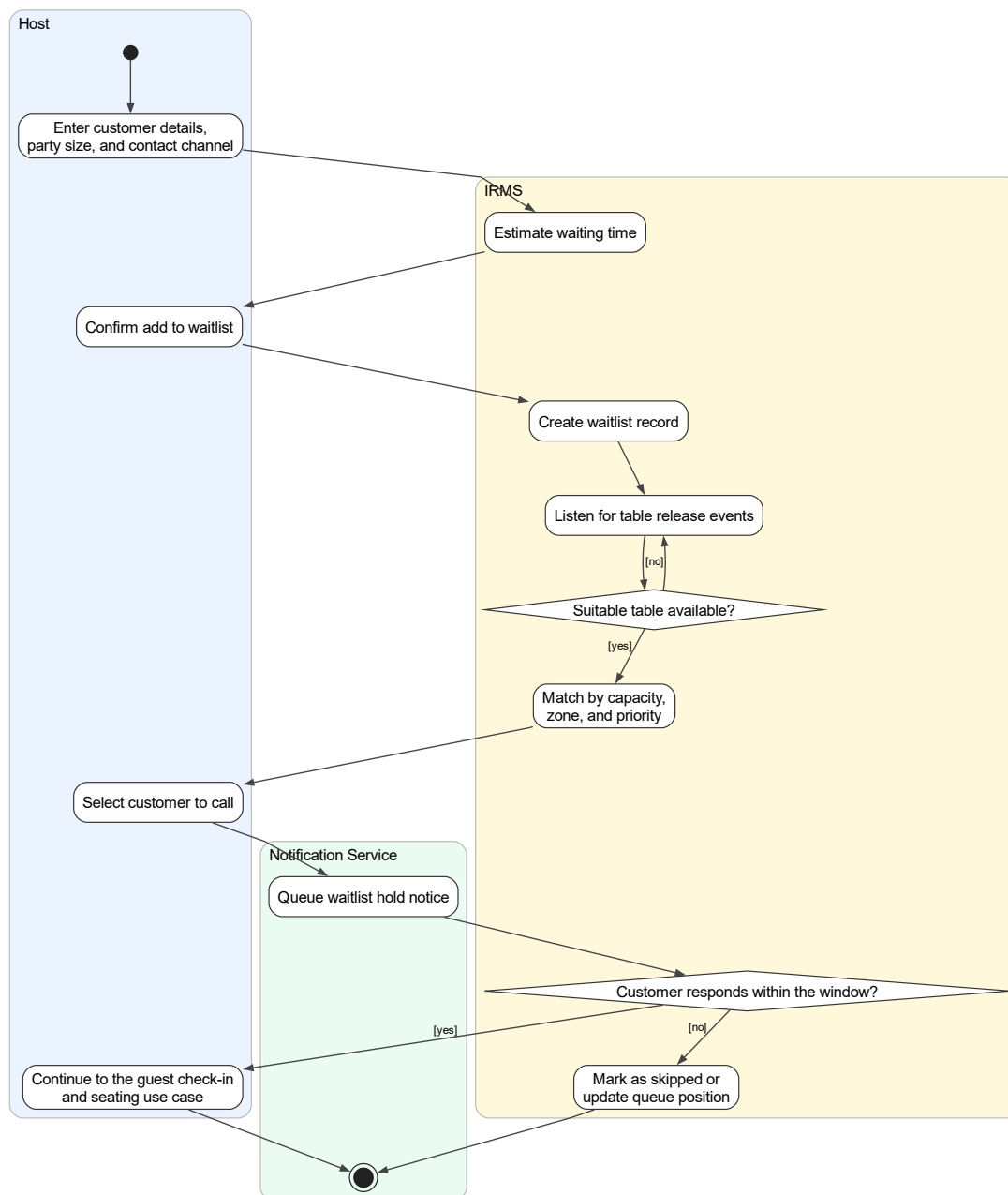


Hình 39: Sơ đồ hoạt động cho UC-RES-02 – Nhận khách và sắp bàn

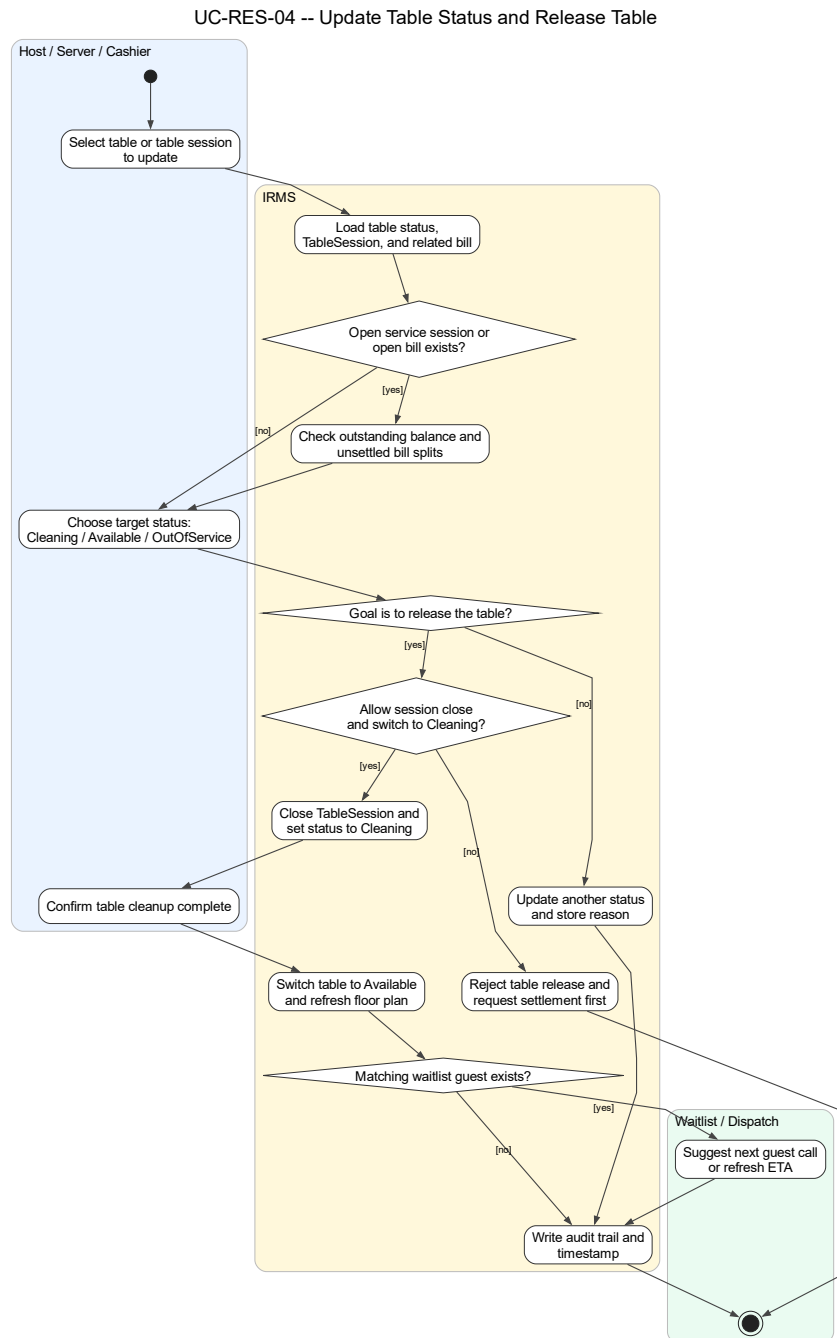
Phân tích sơ đồ UC-RES-02. Luồng hoạt động trong trường hợp này bám sát thực tế tại quầy lễ tân: tìm đặt bàn, kiểm tra hiệu lực, đối chiếu số khách thực tế, xác định có cần đổi bàn hoặc ghép bàn hay không, sau đó mới mở TableSession. Điểm mấu chốt là việc **mở phiên phục vụ bàn** luôn diễn ra sau bước kiểm tra bàn khả dụng, nhờ đó báo cáo giữ được bất biến quan trọng rằng một bàn không có hai phiên phục vụ hoạt động cùng lúc.

Ngoài ra, sơ đồ cũng làm rõ nhánh ngoại lệ khi đặt bàn đã quá giờ giữ chỗ. Trong tình huống này, hệ thống không tự động bỏ qua mà chuyển sang bước phê duyệt của quản lý. Cách mô tả này bám sát phần đặc tả ca sử dụng phía trên và phản ánh đúng yêu cầu kiểm soát nghiệp vụ của IRMS.

UC-RES-03 -- Manage Waitlist



Hình 40: Sơ đồ hoạt động cho UC-RES-03 – Quản lý danh sách chờ



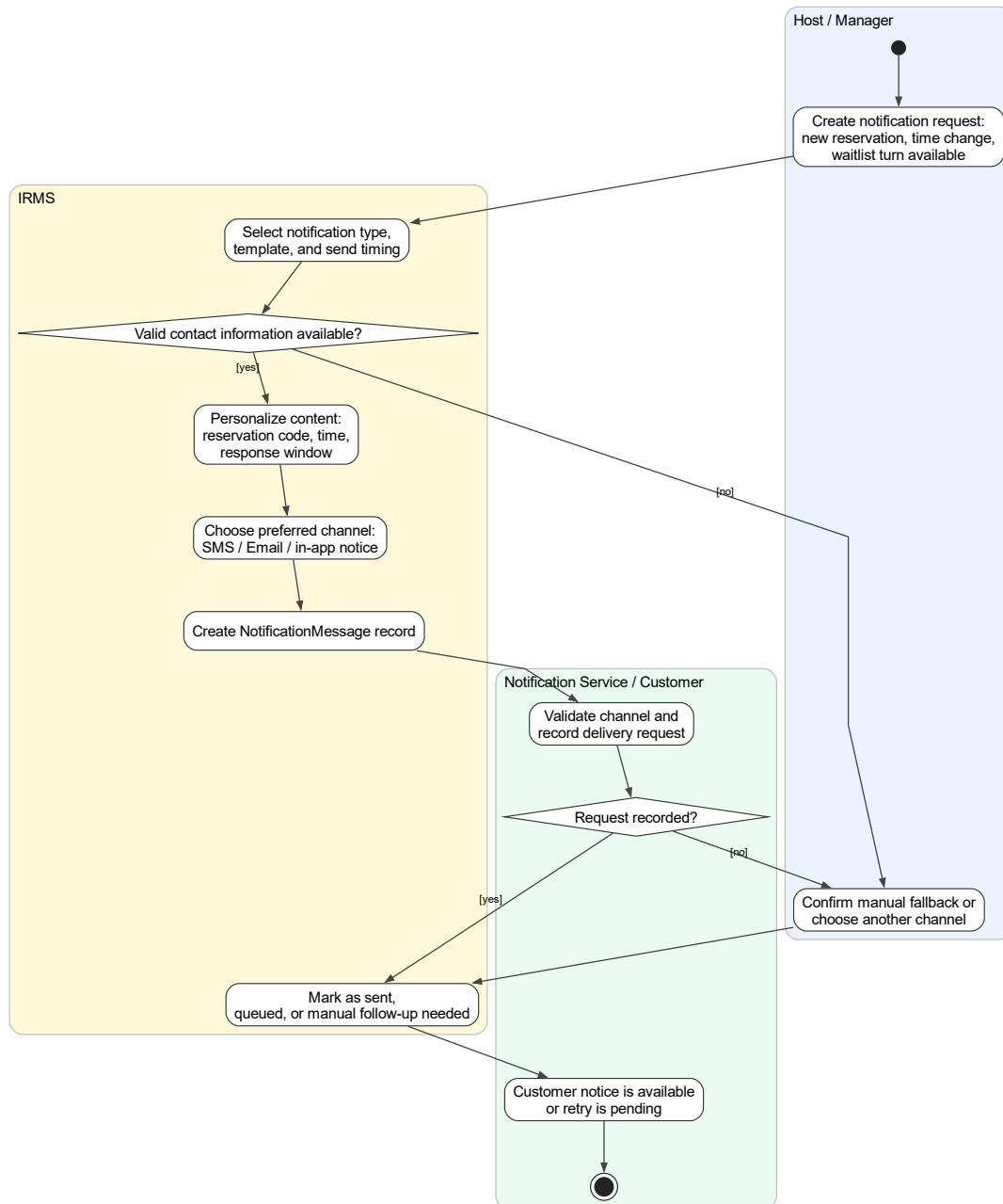
Hình 41: Sơ đồ hoạt động cho UC-RES-04 – Cập nhật trạng thái bàn và giải phóng bàn

Phân tích sơ đồ UC-RES-04. Luồng này làm rõ phần nằm giữa **kết thúc phục vụ** và **bắt đầu vòng quay bàn mới**. Trong vận hành thực tế, việc một bàn trở thành Available không thể chỉ là hệ quả ngầm sau thanh toán. Nó phải đi qua chuỗi kiểm tra rõ ràng: còn phiên phục vụ nào đang mở không, còn split hoặc công nợ nào chưa tất toán không, bàn đã chuyển sang Cleaning chưa, và nhân viên đã xác nhận dọn xong chưa. Nhờ đó, trạng thái bàn trên sơ đồ vận hành phản ánh đúng thực tế thay vì chỉ là ước đoán theo cảm tính của từng bộ phận.

Giá trị nổi bật của sơ đồ là khả năng nối được ba miền vốn thường bị mô tả rời nhau: TableSession, Billing và Waitlist. Khi bàn được giải phóng thành công, hệ thống không chỉ đổi một nhãn trạng thái; nó còn cập nhật nền

cho việc gọi khách tiếp theo, tính vòng quay bàn và tránh lỗi gán trùng trong giờ cao điểm. Đây là lý do ca sử dụng này cần được tách riêng thay vì ẩn trong một mô tả chung về “đóng bàn”.

UC-RES-05 -- Notify Customer

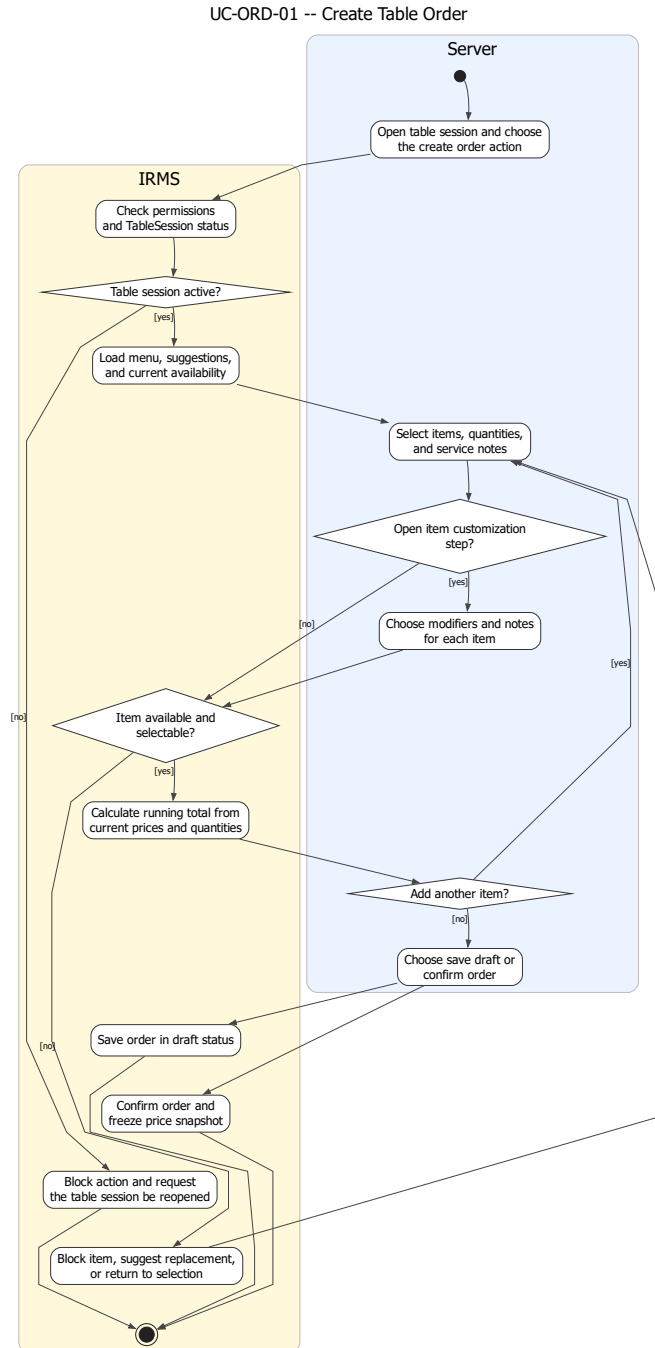


Hình 42: Sơ đồ hoạt động cho UC-RES-05 – Gửi thông báo cho khách hàng

Phân tích sơ đồ UC-RES-05. Sơ đồ thông báo khách hàng cho thấy đây không phải một thao tác phụ có thể bỏ qua, mà là một luồng nghiệp vụ có đầu vào, điều kiện và trạng thái riêng. Trong IRMS, việc thông báo đặt bàn thành công, nhắc giờ đến, gọi khách từ danh sách chờ hay xác nhận thay đổi không nên được mô tả như vài dòng tích hợp kỹ thuật ở rìa hệ thống. Trái lại, nó cần được hiểu là **hệ quả được kiểm soát của sự kiện nghiệp**

vụ. Cách mô tả này giúp tách rõ phần nào thuộc trách nhiệm của lỗi nghiệp vụ, phần nào thuộc bộ kết nối gửi thông báo và phần nào cần cơ chế retry hoặc fallback.

4.3.2 Nhóm gọi món và điều phối bếp

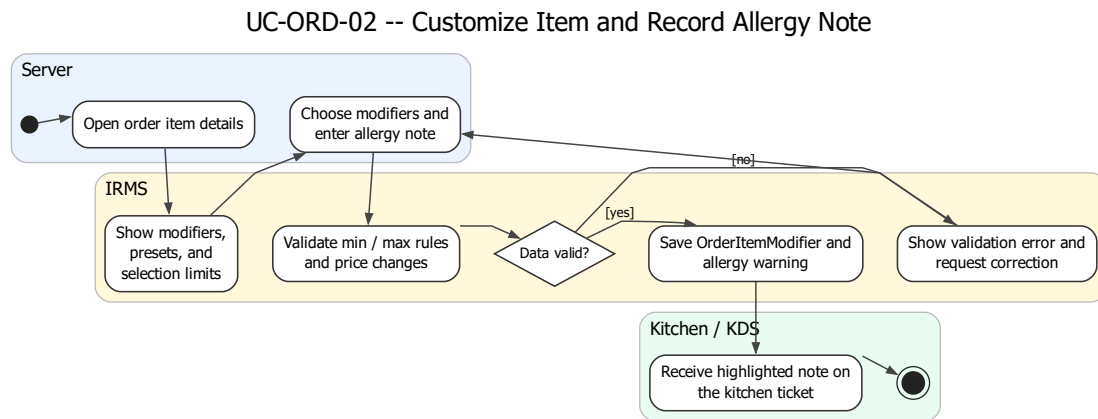


Hình 43: Sơ đồ hoạt động cho UC-ORD-01 – Tạo đơn gọi món tại bàn

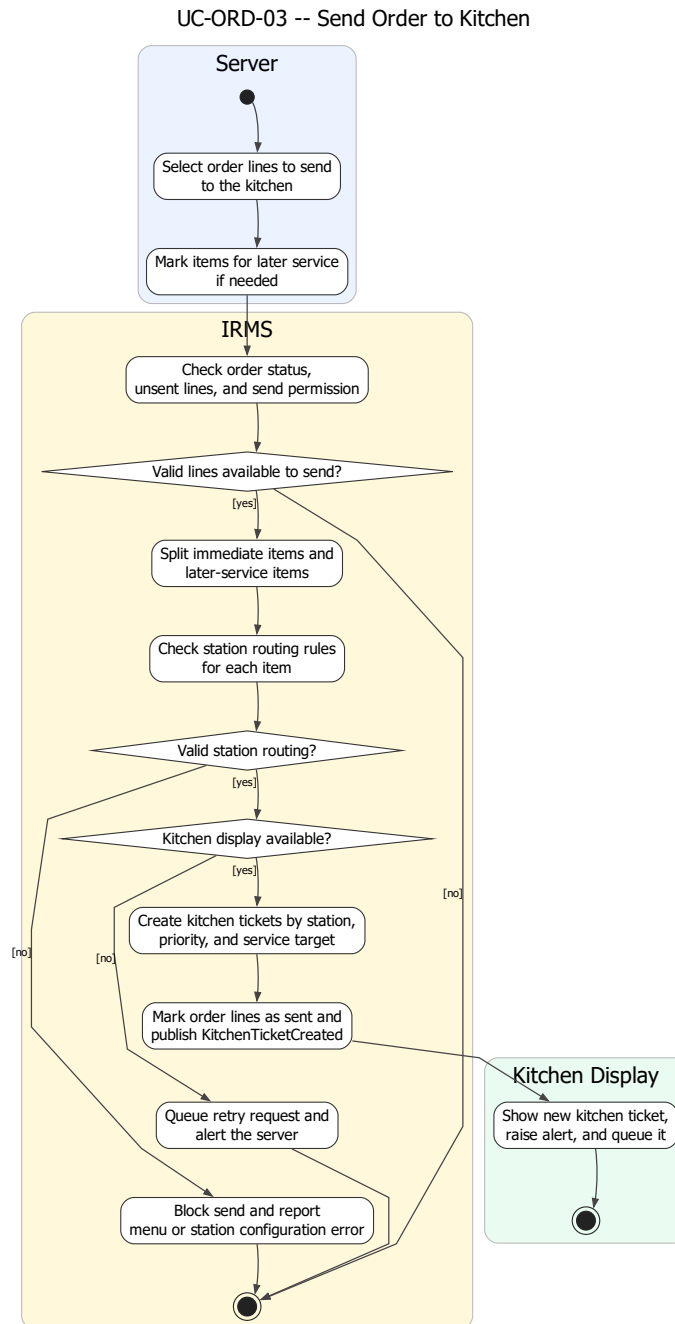
Phân tích sơ đồ UC-ORD-01. Phần hoạt động này làm rõ các bước tải thực đơn, kiểm tra quyền thao tác trên phiên bản, chọn món, kiểm tra trạng thái còn bán, quay lại vòng chọn món nếu món vừa hết hàng, tính tạm tính và tách rõ hai nhánh “lưu nháp” và “xác nhận đơn”. Điều này quan trọng vì ca sử dụng gọi món không chỉ là thao

tác nhập dữ liệu; nó còn là điểm chụp trạng thái giá và tính hợp lệ của món tại thời điểm nhân viên phục vụ thao tác.

Sơ đồ cũng giữ đúng ranh giới với ca sử dụng gửi bếp. Ở đây, đơn gọi món chỉ được tạo và xác nhận ở mức nghiệp vụ gọi món; việc tạo phiếu bếp và điều phối sang trạm bếp được tách sang sơ đồ riêng, nhờ đó ranh giới trách nhiệm giữa Ordering và Kitchen Coordination rõ ràng hơn.



Hình 44: Sơ đồ hoạt động cho UC-ORD-02 – Tùy biến món và ghi chú dị ứng

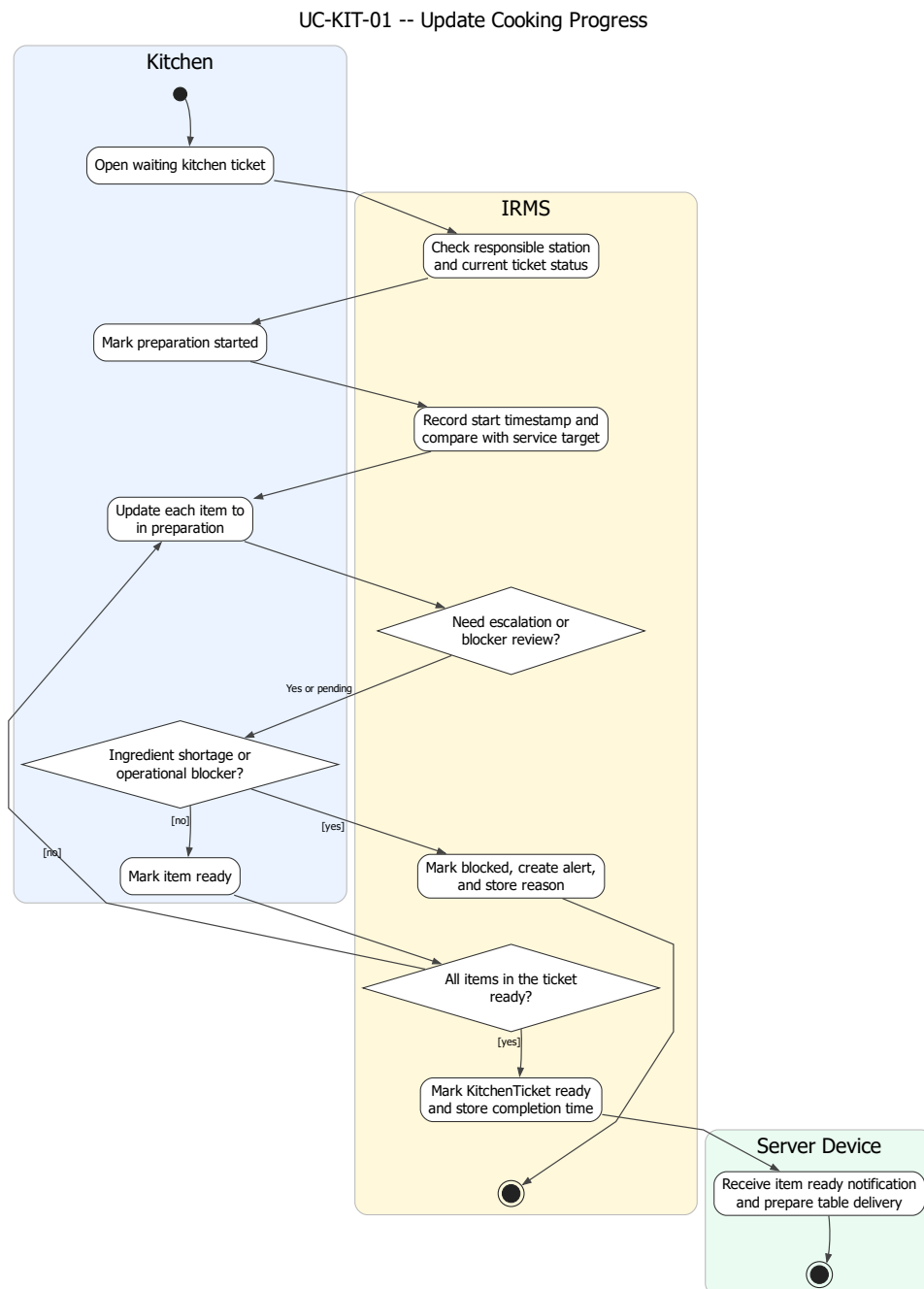


Hình 45: Sơ đồ hoạt động cho UC-ORD-03 – Gửi đơn gọi món sang bếp

Phân tích sơ đồ UC-ORD-03. Đây là một trong những sơ đồ hoạt động quan trọng nhất của báo cáo vì nó nằm đúng trên ranh giới giữa tuyến gọi món của phục vụ và tuyến điều phối của bếp. Luồng hoạt động này làm rõ các bước kiểm tra điều kiện gửi, tách riêng các dòng món gửi ngay và các dòng món phục vụ sau, kiểm tra quy tắc định tuyến theo trạm, tạo KitchenTicket, xếp hàng theo độ ưu tiên và cập nhật trạng thái đơn gọi món sau khi tạo phiếu.

Các nhánh ngoại lệ cũng được mô tả rõ ràng: nếu không xác định được trạm bếp hoặc màn hình bếp không khả dụng, hệ thống không được phép làm mất dữ liệu đã xác nhận mà phải chuyển sang hàng chờ gửi lại và phát

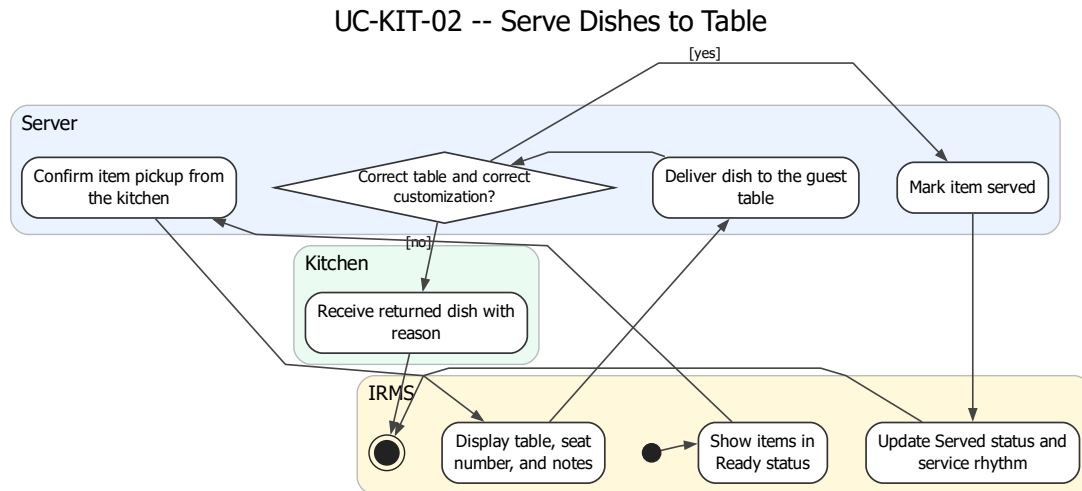
cảnh báo cho phục vụ. Cách mô tả này phù hợp với yêu cầu phi chức năng về độ tin cậy và bảo toàn đơn đã xác nhận.



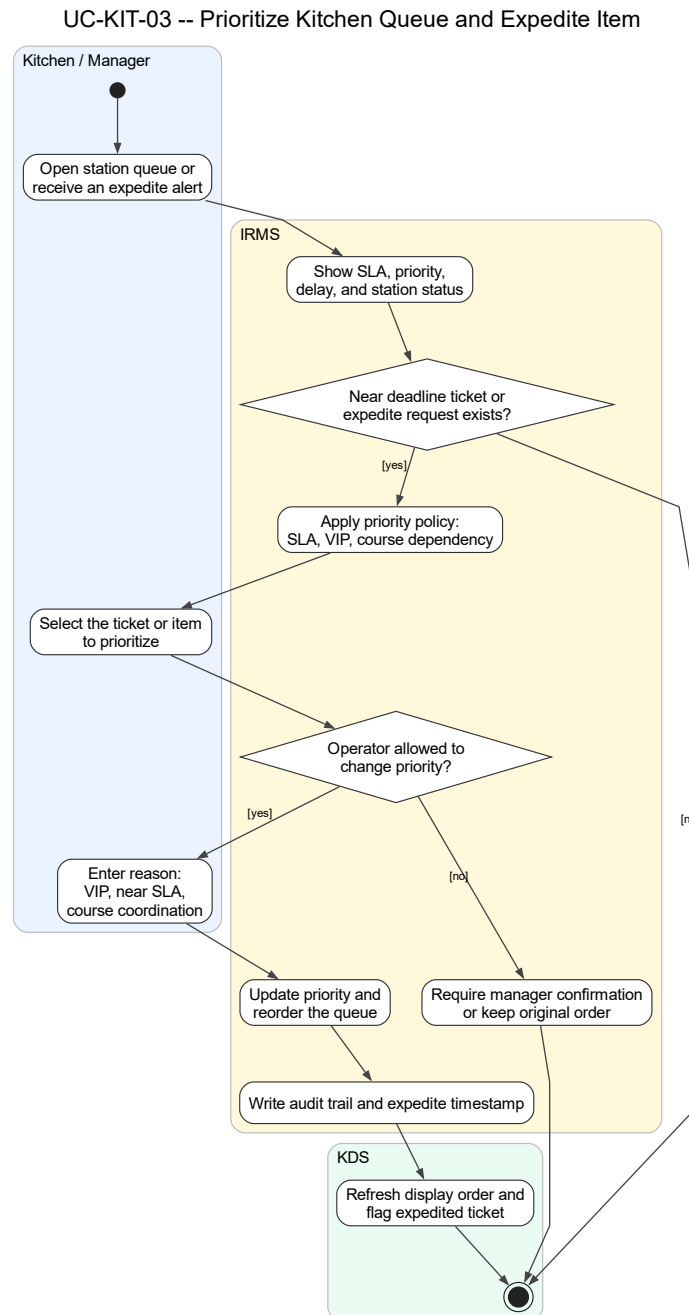
Hình 46: Sơ đồ hoạt động cho UC-KIT-01 – Cập nhật tiến độ chế biến

Phân tích sơ đồ UC-KIT-01. Luồng hoạt động ở đây bám đúng nhịp vận hành của bếp: nhận phiếu, kiểm tra trạm chịu trách nhiệm, đánh dấu bắt đầu, chuyển sang chế biến, ghi dấu thời gian cho từng lần đổi trạng thái, đánh giá nguy cơ chậm hạn và cuối cùng xác nhận món sẵn sàng. Mức chi tiết này giúp phần báo cáo không chỉ dừng ở mô tả “bếp cập nhật trạng thái”, mà còn cho thấy trạng thái nào cần được ghi nhận để phục vụ phân tích vận hành về sau.

Phần nhánh “thiếu nguyên liệu hoặc bị chặn” cũng rất quan trọng. Trong trường hợp đó, hệ thống không được kết thúc im lặng mà phải đánh dấu chặn, phát cảnh báo và trả thông tin ngược về phía phục vụ. Điều này giữ cho luồng giao tiếp giữa bếp và khu vực phục vụ thống nhất với phần kiến trúc đã mô tả ở các góc nhìn trước.



Hình 47: Sơ đồ hoạt động cho UC-KIT-02 – Ra món cho bàn

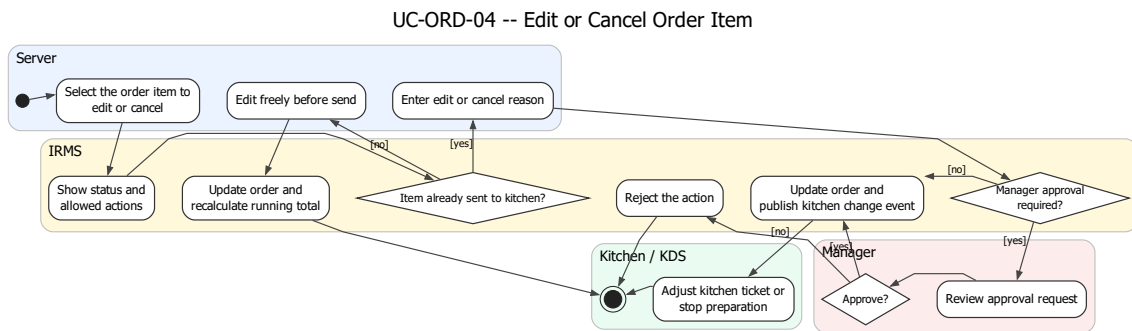


Hình 48: Sơ đồ hoạt động cho UC-KIT-03 – Ưu tiên hàng chờ bếp và xử lý món gấp

Phân tích sơ đồ UC-KIT-03. Luồng này làm rõ một điểm thường được nhắc đến trong tài liệu nhà hàng nhưng ít được mô hình hóa đầy đủ: **bếp không chỉ nhận phiếu bếp, mà còn phải điều phối thứ tự thực thi**. Trong giờ cao điểm, nếu mọi ticket đều bị xử lý đúng theo thứ tự đến mà không xét SLA, cấu trúc course, khách VIP hay món sắp trễ hạn, hệ thống KDS sẽ chỉ là một bảng hiển thị thụ động. Vì vậy, ca sử dụng ưu tiên hàng chờ bếp được trình bày như một luồng độc lập để phản ánh đúng yêu cầu “prioritize queues” của đề bài.

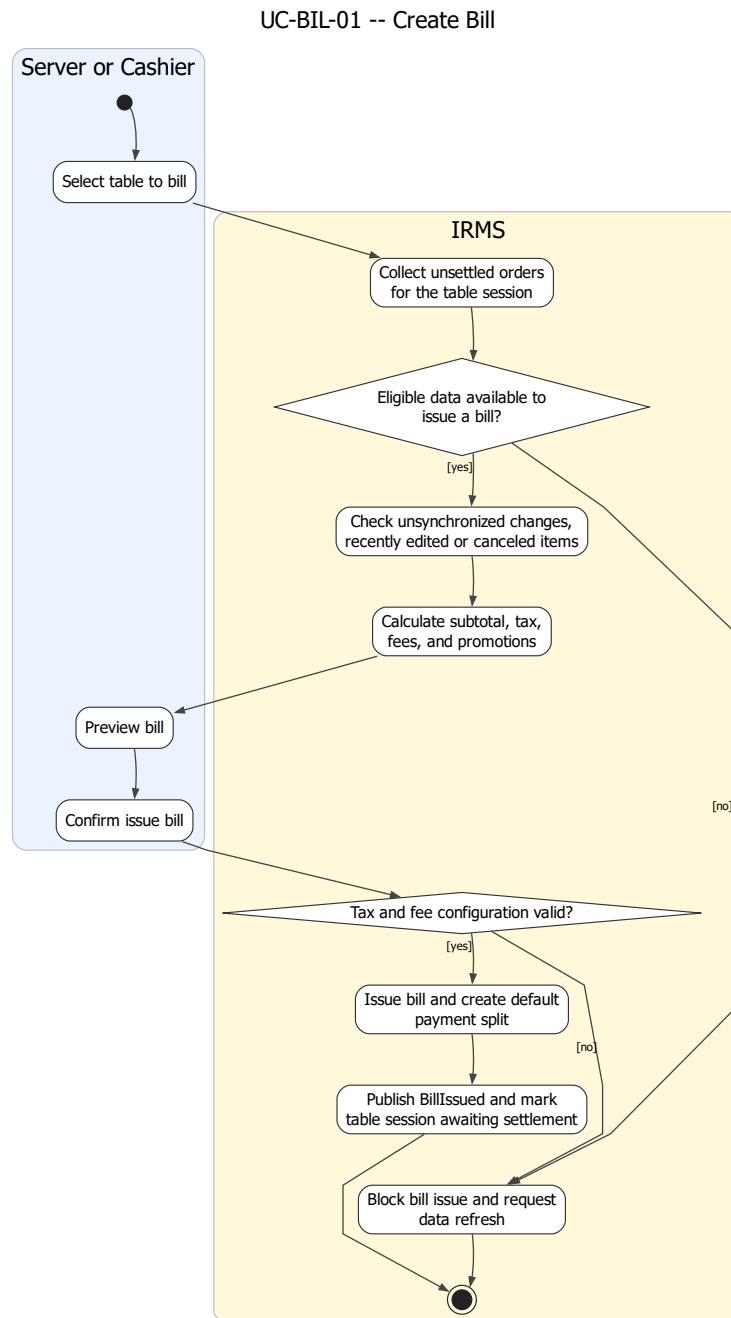
Về mặt kiến trúc, sơ đồ cũng cho thấy quyết định ưu tiên phải đi qua chính sách có kiểm soát, có ghi lý do và có giới hạn quyền, chứ không phải bất kỳ nhân viên nào cũng có thể tùy ý đổi thứ tự chế biến. Điều này giúp

IRMS vừa hỗ trợ vận hành linh hoạt, vừa giữ được khả năng truy vết khi có khiếu nại về món ra chậm hoặc món bị đẩy lùi không hợp lý.



Hình 49: Sơ đồ hoạt động cho UC-ORD-04 – Sửa hoặc hủy dòng món

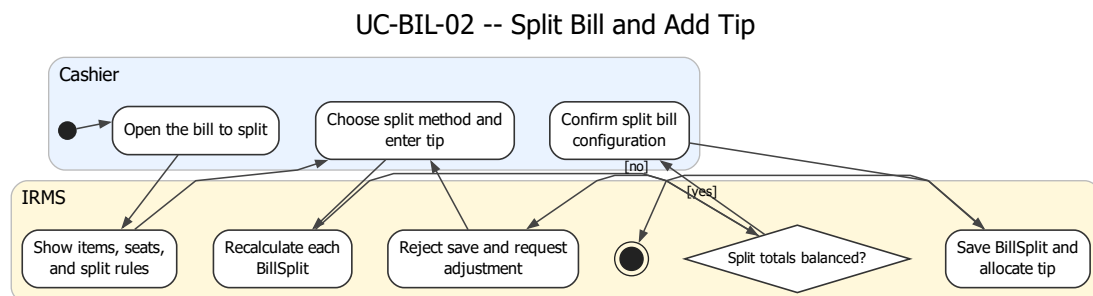
4.3.3 Nhóm hóa đơn, thanh toán và biên lai



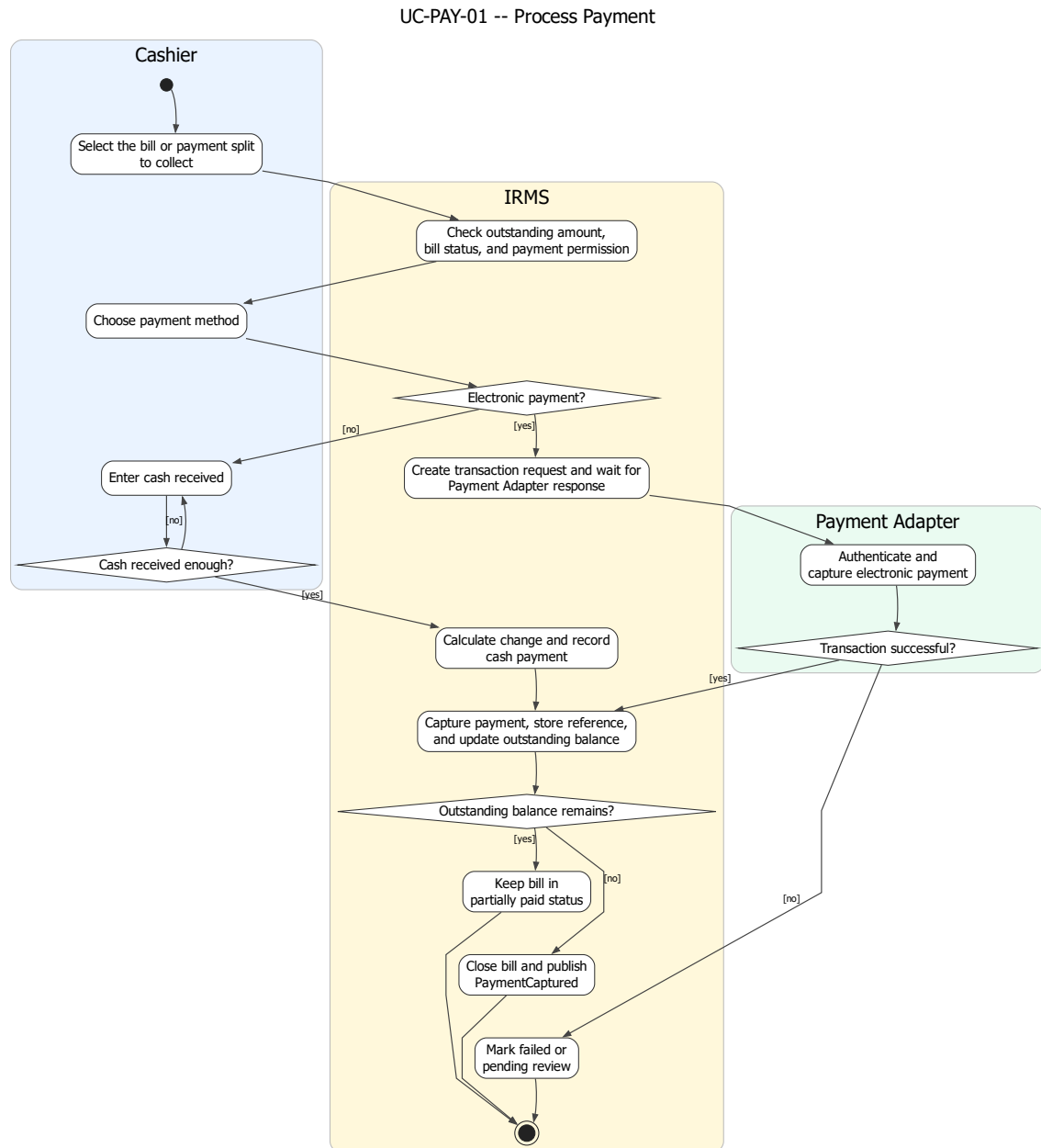
Hình 50: Sơ đồ hoạt động cho UC-BIL-01 – Tạo hóa đơn cho bàn

Phân tích sơ đồ UC-BIL-01. Sơ đồ tạo hóa đơn phản ánh đúng thứ tự xử lý của phần quyết toán: gom các đơn gọi món chưa quyết toán, kiểm tra điều kiện xuất hóa đơn, tính tiền hàng, thuế, phí và khuyến mãi, sau đó mới cho phép phục vụ hoặc thu ngân xem trước và xác nhận phát hành. Việc đặt bước “xem trước hóa đơn” trước “phát hành” là cần thiết vì đây là điểm cuối cùng để phát hiện sai lệch nếu một dòng món vừa bị hủy hoặc vừa có thay đổi cấu hình giá.

Sơ đồ cũng thể hiện rõ nhánh chặn phát hành khi dữ liệu chưa hợp lệ. Điều này phù hợp với đặc tả ca sử dụng ở trên và làm nổi bật rằng Bill không chỉ là phép cộng cuối cùng, mà là một thực thể nghiệp vụ có điều kiện phát hành chặt chẽ.



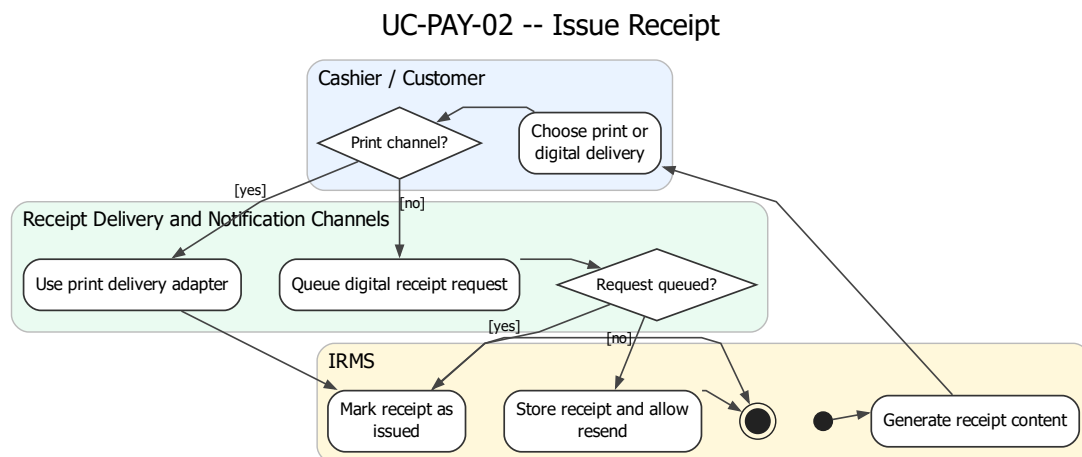
Hình 51: Sơ đồ hoạt động cho UC-BIL-02 – Tách hóa đơn và thêm tiền boa



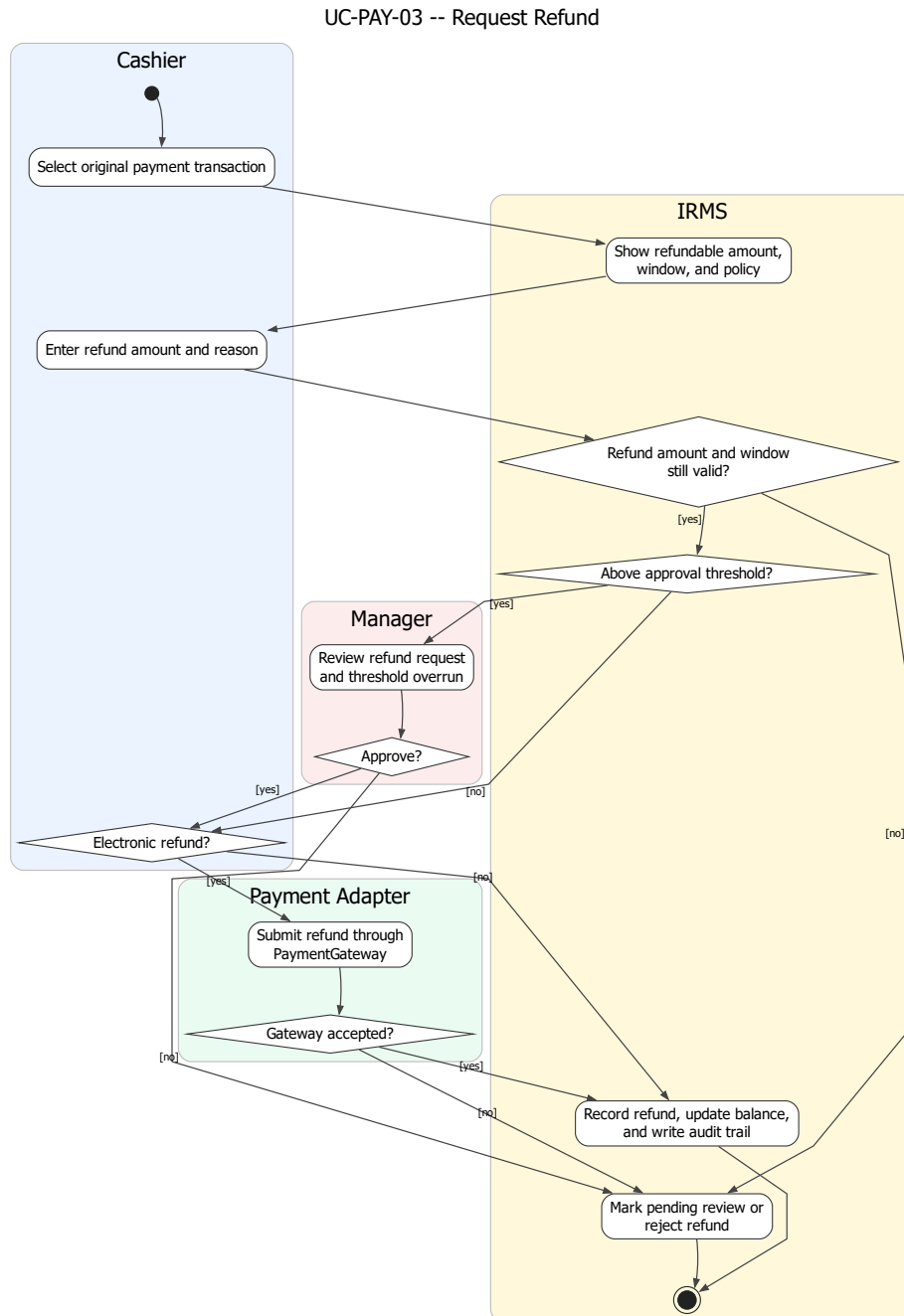
Hình 52: Sơ đồ hoạt động cho UC-PAY-01 – Xử lý thanh toán

Phân tích sơ đồ UC-PAY-01. Đây là sơ đồ hoạt động có ý nghĩa kiểm soát cao nhất trong toàn bộ phần hành vi. Luồng này làm rõ hai tuyến thanh toán khác nhau: thanh toán qua PaymentGateway và thanh toán tiền mặt tại quầy. Với nhánh PaymentGateway, hệ thống phải tạo yêu cầu giao dịch, nhận kết quả, lưu mã tham chiếu và xử lý riêng các trạng thái thành công, thất bại hoặc chờ rà soát. Với thanh toán tiền mặt, hệ thống tính số tiền thối rồi mới cập nhật số dư còn lại.

Sau bước ghi nhận thanh toán, hệ thống còn phải đánh giá hóa đơn đã được thanh toán đủ hay chưa. Nếu chưa đủ, hóa đơn vẫn mở; nếu đủ, hóa đơn được đóng và luồng phát hành biên lai mới được mở ra. Cách mô tả này bám đúng kiến trúc hiện tại của IRMS và tránh nhập nhằng giữa “đã có một giao dịch thanh toán” và “đã tắt toán hóa đơn”.



Hình 53: Sơ đồ hoạt động cho UC-PAY-02 – Phát hành biên lai

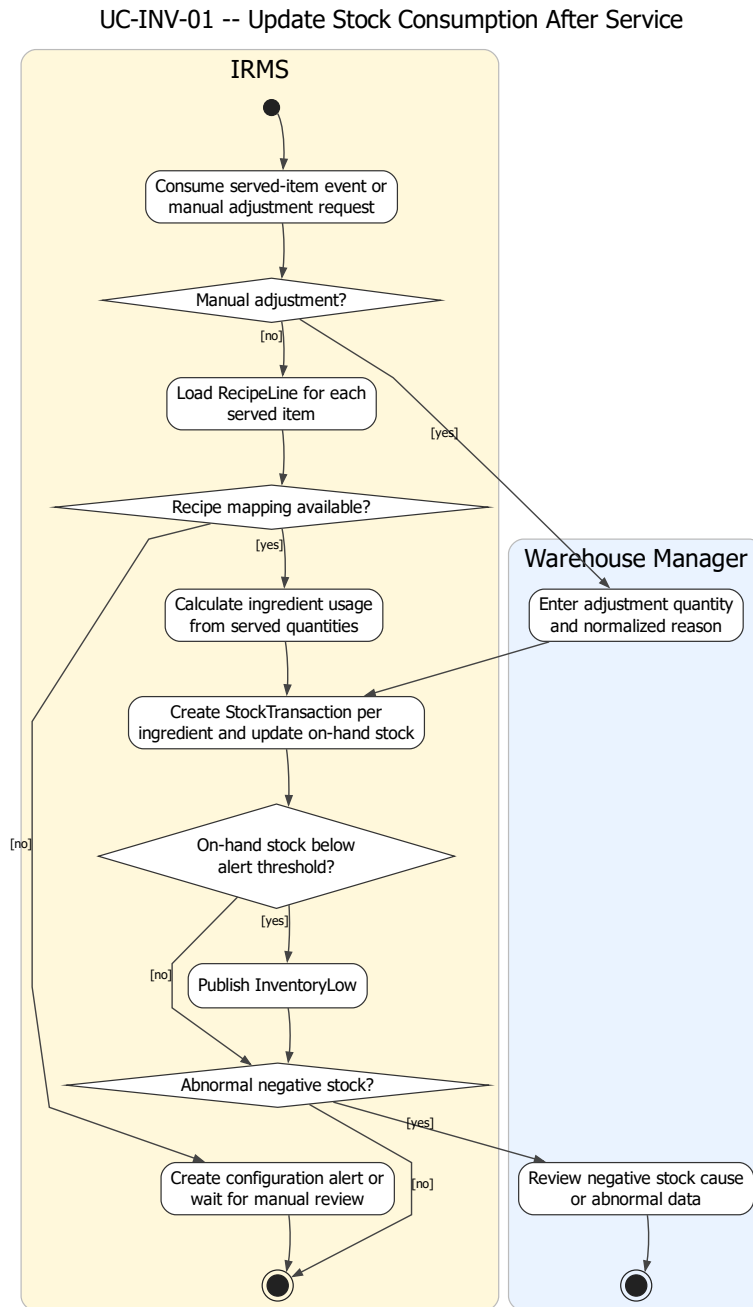


Hình 54: Sơ đồ hoạt động cho UC-PAY-03 – Hoàn tiền

Phân tích sơ đồ UC-PAY-03. Sơ đồ hoàn tiền hiện đã thể hiện đầy đủ hơn các điểm kiểm soát bắt buộc: chọn giao dịch gốc, kiểm tra số tiền còn được hoàn, nhập lý do, so ngưỡng phê duyệt, xin xác nhận của quản lý nếu cần, xử lý hoàn tiền qua PaymentGateway hoặc ghi nhận hoàn tiền thủ công, rồi mới cập nhật sổ kiểm toán. Điều này rất quan trọng vì hoàn tiền là thao tác nhạy cảm nhất trong vùng thanh toán.

Một chi tiết đáng chú ý là sơ đồ đã tách rõ nhánh “chờ rà soát” khi PaymentGateway không chấp nhận hoặc phản hồi không rõ ràng. Nhờ đó báo cáo có thể giải thích trạng thái nghiệp vụ chặt chẽ hơn, thay vì gom mọi trường hợp không thành công vào một nhãn lỗi duy nhất.

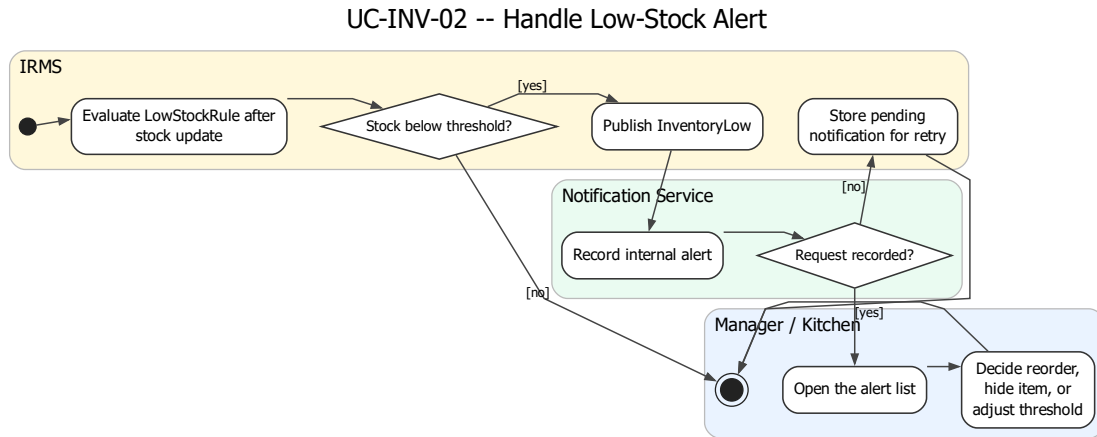
4.3.4 Nhóm tồn kho, cảnh báo và phân tích



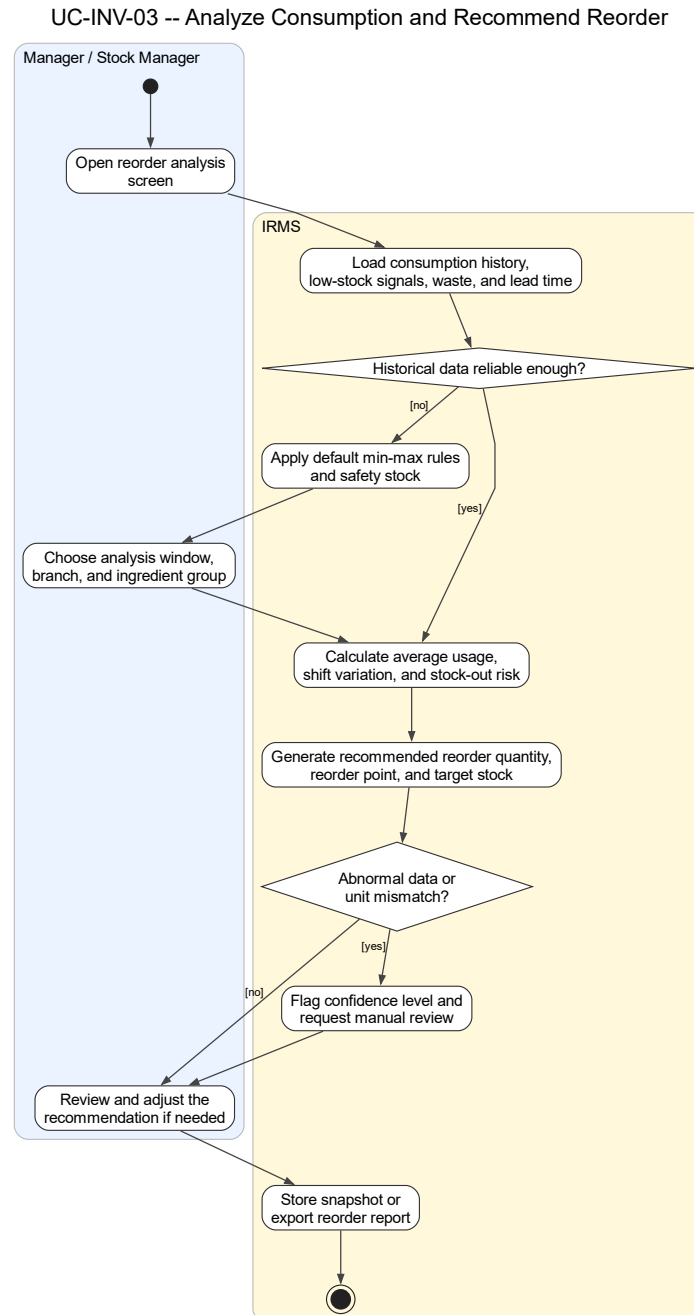
Hình 55: Sơ đồ hoạt động cho UC-INV-01 – Cập nhật tiêu hao tồn kho sau phục vụ

Phân tích sơ đồ UC-INV-01. Sơ đồ cập nhật tiêu hao tồn kho phản ánh đúng bản chất suy diễn của vùng tồn kho: nhận sự kiện từ món đã phục vụ hoặc điều chỉnh thủ công, tra công thức nguyên liệu, tính lượng tiêu hao theo số lượng món, sinh StockTransaction theo từng nguyên liệu, cập nhật số lượng tồn hiện có, rồi tiếp tục đánh giá các điều kiện bất thường. Đây là mô tả phù hợp với kiến trúc của IRMS, nơi vùng tồn kho không tự suy đoán món bán ra từ giao diện mà dựa vào tín hiệu nghiệp vụ từ tuyến phục vụ.

Sơ đồ cũng làm rõ hai nhánh ngoại lệ quan trọng: thiếu ánh xạ công thức và âm kho bất thường. Nhờ đó phần thiết kế hành vi bám sát hơn với ca sử dụng đã đặc tả, đồng thời nổi được với sơ đồ cảnh báo sắp hết hàng ngay bên dưới.



Hình 56: Sơ đồ hoạt động cho UC-INV-02 – Nhận cảnh báo sắp hết hàng

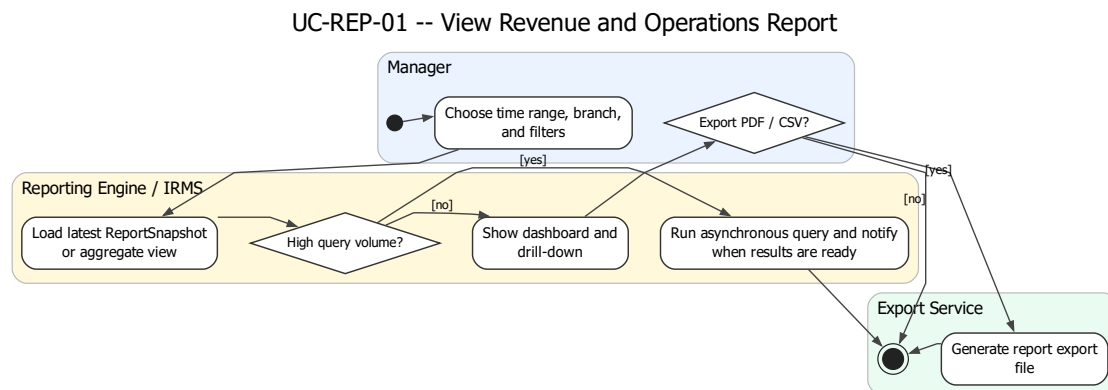


Hình 57: Sơ đồ hoạt động cho UC-INV-03 – Phân tích tiêu hao và đề xuất nhập hàng

Phân tích sơ đồ UC-INV-03. Ca sử dụng này làm cho phần tồn kho bám sát hơn với yêu cầu “historical usage helps predict reorder quantities and manage waste”. Nếu chỉ dừng ở cập nhật tiêu hao và cảnh báo sắp hết hàng, IRMS mới hỗ trợ tốt cho phản ứng vận hành ngắn hạn; nó chưa hỗ trợ đầy đủ cho quyết định chuẩn bị hàng hóa ở chu kỳ ngày, tuần hoặc mùa cao điểm. Vì vậy, sơ đồ chuyển trọng tâm của vùng tồn kho từ **theo dõi hiện trạng** sang **hỗ trợ ra quyết định**.

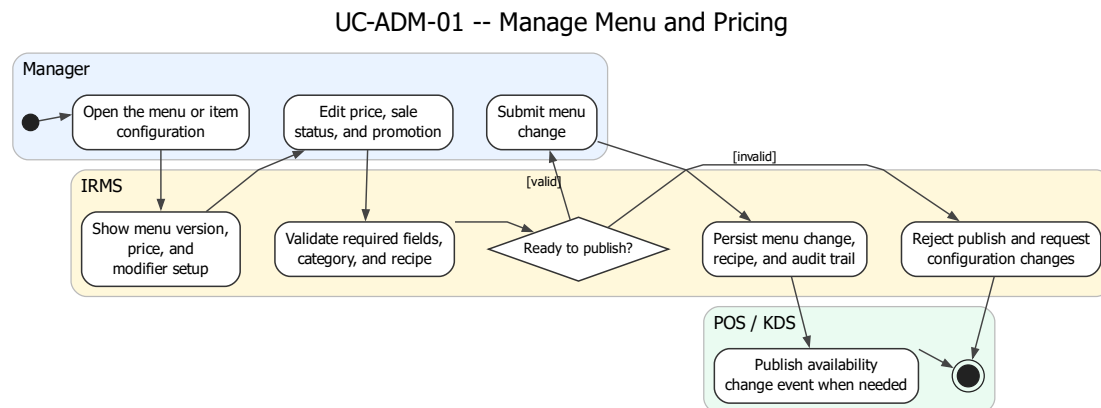
Điểm quan trọng của sơ đồ là không mô tả việc dự báo như một thuật toán mơ hồ, mà làm rõ chuỗi dữ liệu đầu vào: tiêu hao lịch sử, low-stock alert, hao hụt, lead time và mức tồn an toàn. Điều này có giá trị trong báo

cáo kiến trúc vì nó cho thấy đề xuất nhập hàng là kết quả của mô hình dữ liệu và luồng xử lý đã có, không phải một chức năng được bổ sung rời rạc ở phần quản trị mà không có nền dữ liệu đủ tin cậy.



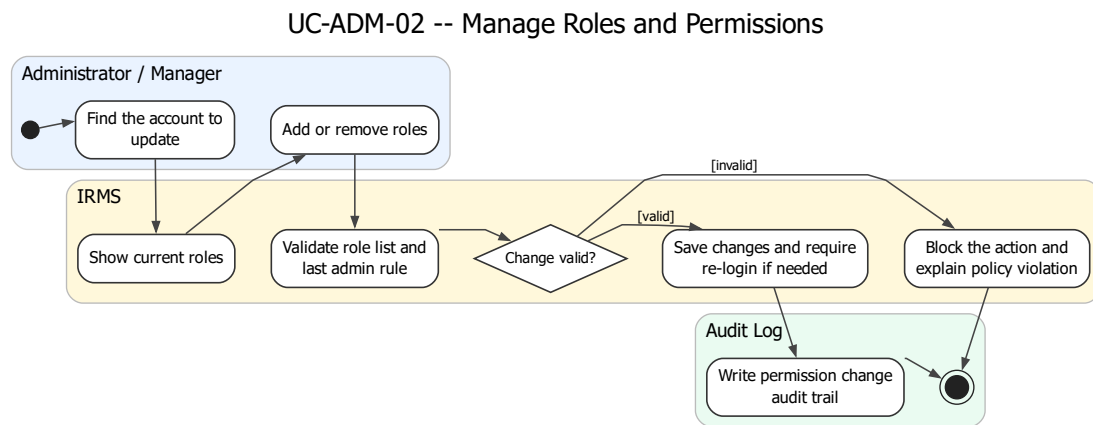
Hình 58: Sơ đồ hoạt động cho UC-REP-01 – Xem báo cáo doanh thu và vận hành

4.3.5 Nhóm quản trị cấu hình và vận hành



Hình 59: Sơ đồ hoạt động cho UC-ADM-01 – Quản lý thực đơn và giá bán

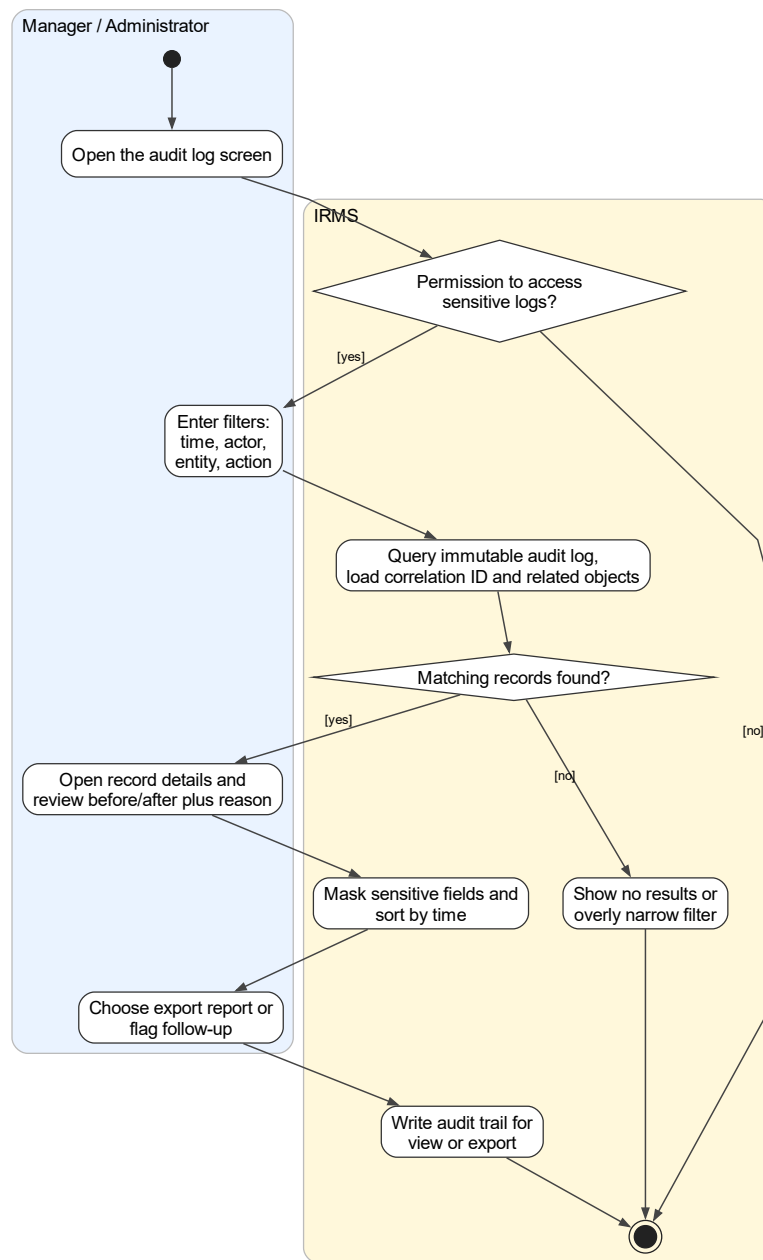
- Quản lý mở màn hình cấu hình thực đơn và chọn món hoặc danh mục cần chỉnh sửa.
- Hệ thống hiển thị thông tin chi tiết của món, bao gồm giá, trạng thái và các chương trình khuyến mãi hiện tại.
- Quản lý thực hiện chỉnh sửa thông tin món và có thể áp dụng khuyến mãi nếu cần.
- Khi lưu thay đổi, quản lý gửi cấu hình món, giá, trạng thái bán, khuyến mãi hoặc công thức nguyên liệu theo form hiện tại.
- Hệ thống kiểm tra dữ liệu cấu hình:
 - Nếu thiếu thông tin bắt buộc, hệ thống yêu cầu bổ sung trước khi tiếp tục.
 - Nếu danh mục không tồn tại hoặc công thức nguyên liệu sai định dạng, hệ thống yêu cầu điều chỉnh trước khi lưu.
- Khi dữ liệu hợp lệ, hệ thống lưu thay đổi thực đơn, công thức liên quan và ghi nhận audit trail; nếu trạng thái bán thay đổi, hệ thống phát event cập nhật khả dụng.
- Các đơn hàng đang mở vẫn giữ nguyên giá đã ghi nhận trước đó để đảm bảo tính nhất quán.



Hình 60: Sơ đồ hoạt động cho UC-ADM-02 – Quản lý vai trò và phân quyền

- Quản trị viên mở màn hình quản lý người dùng và tìm kiếm người dùng cần cập nhật quyền.
- Hệ thống hiển thị vai trò và danh sách quyền hiện tại của người dùng.
- Quản trị viên thực hiện gán hoặc điều chỉnh vai trò và quyền theo chính sách RBAC.
- Hệ thống xử lý các trường hợp mở rộng:
 - Nếu không còn vai trò nào được chọn, hệ thống chặn thao tác vì mỗi nhân viên phải giữ ít nhất một vai trò.
 - Nếu vai trò được chọn không tồn tại trong danh sách hiện hành, hệ thống chặn thao tác và yêu cầu chọn lại.
- Hệ thống kiểm tra tính hợp lệ của chính sách phân quyền:
 - Nếu phát hiện xung đột chính sách RBAC, hệ thống yêu cầu quản trị viên điều chỉnh lại quyền.
 - Nếu thao tác dẫn đến việc gỡ bỏ quyền quản trị cuối cùng, hệ thống chặn thao tác để đảm bảo luôn tồn tại ít nhất một tài khoản quản trị.
- Khi dữ liệu hợp lệ, hệ thống thay thế danh sách vai trò của người dùng bằng các vai trò đã chọn.
- Hệ thống yêu cầu người dùng đăng nhập lại để áp dụng quyền mới.
- Cuối cùng, hệ thống ghi nhận toàn bộ thay đổi vào audit log nhằm phục vụ truy vết và kiểm soát.

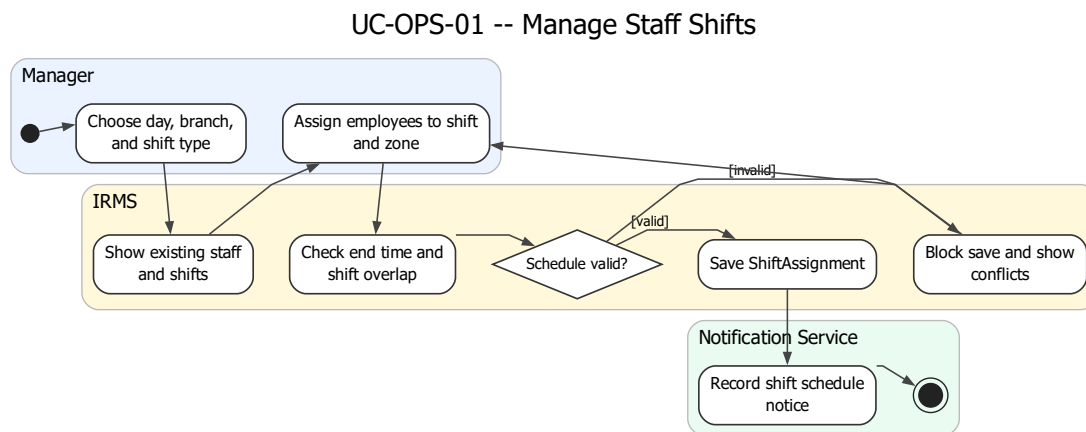
UC-ADM-03 -- Review Audit Log and Trace Sensitive Actions



Hình 61: Sơ đồ hoạt động cho UC-ADM-03 – Xem nhật ký kiểm toán và truy vết thao tác nhạy cảm

- Người dùng có thẩm quyền mở màn hình nhật ký kiểm toán để thực hiện tra cứu.
- Hệ thống kiểm tra quyền truy cập:
 - Nếu không đủ quyền, hệ thống từ chối truy cập hoặc chỉ hiển thị dữ liệu rút gọn.
 - Nếu hợp lệ, hệ thống cho phép tiếp tục truy vấn.
- Người dùng nhập bộ lọc tìm kiếm (thời gian, người thao tác, loại hành động, mã liên kết).
- Hệ thống truy vấn và hiển thị danh sách audit log:

- Nếu không có dữ liệu phù hợp, hệ thống thông báo và yêu cầu điều chỉnh bộ lọc.
- Nếu người dùng không đủ quyền, hệ thống làm mờ hoặc ẩn thông tin nhạy cảm.
- Người dùng chọn một bản ghi để xem chi tiết, bao gồm giá trị trước và sau thay đổi, lý do và ngữ cảnh thực hiện.
- Người dùng có thể xuất báo cáo hoặc đánh dấu bản ghi cần theo dõi.
- Hệ thống ghi nhận hành vi truy cập và thao tác trên audit log nhằm phục vụ kiểm soát và truy vết sau này.



Hình 62: Sơ đồ hoạt động cho UC-OPS-01 – Quản lý ca làm nhân viên

- Quản lý mở màn hình lịch làm việc và chọn ngày, khu vực cùng loại ca cần phân công.
- Hệ thống hiển thị danh sách nhân viên và các ca làm việc đã có.
- Quản lý thực hiện gán nhân viên vào ca và khu vực làm việc.
- Hệ thống hỗ trợ các trường hợp mở rộng:
 - Nếu thời điểm kết thúc không sau thời điểm bắt đầu, hệ thống chặn lưu ca.
 - Nếu nhân viên đã có ca trùng khoảng thời gian, hệ thống chặn lưu ca và yêu cầu chọn lại.
- Hệ thống kiểm tra các ràng buộc lập lịch:
 - Nếu phát hiện trùng ca, hệ thống chặn lưu và yêu cầu điều chỉnh.
 - Nếu dữ liệu hợp lệ, hệ thống lưu ShiftAssignment, ghi audit log và xếp hàng thông báo nội bộ cho nhân viên.
- Khi dữ liệu hợp lệ, quản lý lưu lịch làm việc.
- Hệ thống cập nhật ShiftAssignment, ghi audit log và xếp hàng thông báo nội bộ cho nhân viên liên quan.

4.4 Phản tư về toàn bộ báo cáo và định hướng hiện thực

Điểm mạnh lớn nhất của báo cáo nằm ở việc duy trì một mạch lập luận nhất quán xuyên suốt từ bối cảnh nghiệp vụ tới yêu cầu phi chức năng, ca sử dụng, quyết định kiến trúc và thiết kế chi tiết. Các sơ đồ không xuất hiện như các minh họa rời rạc. Mỗi góc nhìn được sử dụng để làm rõ một quyết định cụ thể của hệ thống. Nhờ đó, người đọc có thể theo dõi một logic liên tục: áp lực vận hành của nhà hàng dẫn tới việc tách miền nghiệp vụ, từ đó hình thành cách tổ chức service, dữ liệu và luồng xử lý.

Một điểm mạnh quan trọng khác là việc đặt ranh giới năng lực nghiệp vụ làm trung tâm. Các miền như Reservation & Seating, Ordering, Kitchen Coordination, Billing & Settlement được xác định rõ trách nhiệm. Các ranh giới này được giữ nhất quán khi ánh xạ sang các service và luồng xử lý. Mỗi service sở hữu dữ liệu và logic riêng. Tương tác giữa các service đi qua giao diện công khai, không truy cập chéo vào chi tiết nội bộ. Cách tổ chức này giúp giảm phụ thuộc, đồng thời cho phép thay đổi từng phần mà không ảnh hưởng toàn hệ thống.

Bên cạnh đó, việc tách biệt luồng giao dịch chính và luồng xử lý bất đồng bộ là một quyết định mang tính nền tảng. Các thao tác tuyến đầu như gọi món, điều phối bếp, thanh toán được giữ trong chuỗi đồng bộ ngắn nhằm đảm bảo phản hồi nhanh và trạng thái nhất quán. Các hệ quả sau giao dịch như thông báo, cập nhật báo cáo, ghi nhận audit được chuyển sang xử lý qua cơ chế sự kiện và xử lý nền. Cách tiếp cận này phản ánh trực tiếp yêu cầu vận hành: tuyến phục vụ cần ổn định, các tác vụ phụ cần linh hoạt.

Các nguyên lý SOLID được thể hiện thông qua cấu trúc hệ thống thay vì tách thành phần lý thuyết. Trách nhiệm được phân tách rõ giữa lớp tiếp nhận yêu cầu, lớp điều phối và mô hình nghiệp vụ. Các tích hợp bên ngoài như thanh toán, thông báo được đặt sau các giao diện trừu tượng. Chiều phụ thuộc được kiểm soát để tránh ràng buộc trực tiếp vào công nghệ cụ thể. Nhờ đó, lõi nghiệp vụ giữ được tính ổn định trong khi các thành phần bên ngoài vẫn có thể thay đổi.

Tuy nhiên, kiến trúc hiện tại phản ánh một bối cảnh vận hành tương đối tập trung. Khi mở rộng theo hướng nhiều chi nhánh hoặc nhiều tích hợp, các vấn đề về triển khai, quan sát hệ thống và quản lý dữ liệu giữa các service sẽ trở nên rõ rệt hơn. Một số năng lực như dự báo tồn kho, tối ưu điều phối bếp, xử lý lỗi tích hợp mới dừng ở mức kiến trúc, chưa đi sâu vào chiến lược vận hành chi tiết.

Về mặt kinh nghiệm, giá trị quan trọng nhất của báo cáo nằm ở cách chuyển trọng tâm từ việc liệt kê chức năng sang việc làm rõ cách hệ thống vận hành. Khi lập luận xoay quanh giao dịch cốt lõi, ranh giới service, luồng đồng bộ và bất đồng bộ, tài liệu phản ánh đúng bản chất của thiết kế kiến trúc. Từ đó có thể rút ra một nguyên tắc: giá trị của báo cáo không nằm ở số lượng sơ đồ, mà nằm ở khả năng mỗi sơ đồ làm rõ một quyết định thiết kế cụ thể.

Trong giai đoạn hiện thực, các quyết định kiến trúc có thể được sử dụng trực tiếp làm khung triển khai. Các service được hiện thực theo đúng ranh giới nghiệp vụ. Lớp API và cổng vào đảm nhiệm kiểm soát truy cập và điều phối yêu cầu. Cơ chế Outbox và xử lý nền được sử dụng để xử lý các sự kiện bất đồng bộ. Dữ liệu được tách giữa giao dịch và mô hình đọc nhằm cân bằng tính nhất quán và hiệu năng.

Nếu các ranh giới service và nguyên tắc tách luồng xử lý được giữ vững, kiến trúc có thể đóng vai trò như một tiêu chuẩn kiểm soát chất lượng mã nguồn. Khi đó, báo cáo không chỉ là tài liệu mô tả, mà trở thành cơ sở dẫn hướng cho quá trình phát triển hệ thống.

A Phụ lục

A.1 Lý do lựa chọn kiến trúc và hồ sơ quyết định kiến trúc

Phụ lục này tổng hợp phần biện minh cho lựa chọn kiến trúc của IRMS dưới dạng **so sánh phương án** và **hồ sơ quyết định kiến trúc** (*Architecture Decision Records – ADR*). Mục tiêu của phần này không chỉ dừng ở việc nêu kiến trúc được chọn, mà còn làm rõ **bối cảnh ra quyết định, các phương án đã cân nhắc, lập luận loại trừ** và **hệ quả thiết kế** của phương án cuối cùng. Cách trình bày này giúp phụ lục đóng vai trò như một hồ sơ giải thích quyết định, thay vì chỉ là phần bổ sung hình thức cho báo cáo chính.

Hồ sơ quyết định dạng file được lưu tại `docs/adr/`. Các hộp ADR trong phụ lục giữ vai trò trình bày ngắn gọn trong báo cáo, còn file ADR tương ứng ghi rõ bằng chứng code/config và các hệ quả đánh đổi. Cách tách này giúp phần chính vẫn dễ đọc, đồng thời tránh việc các quyết định kiến trúc chỉ xuất hiện như mô tả không có nguồn kiểm chứng.

Các mục trong phụ lục này không đứng riêng với thân báo cáo mà được dùng để giải thích ngược cho phần mô hình hóa ở Mục 2.1, phần tổng quan và các góc nhìn kiến trúc ở Mục 3.1 và Mục 3.2, cùng phần thiết kế chi tiết ở Mục 4.1 và Mục 4.2.

A.1.1 So sánh các kiểu kiến trúc ứng viên

Bảng 35: So sánh các kiểu kiến trúc ứng viên cho IRMS

Kiểu kiến trúc	Ưu điểm chính	Hạn chế trong bối cảnh IRMS
Kiến trúc phân lớp tập trung thuần	Dễ khởi tạo ban đầu, ít thành phần triển khai, đơn giản cho giao dịch nội bộ.	Dễ tổ chức theo tầng kỹ thuật thay vì theo năng lực nghiệp vụ. Trong IRMS, các vùng như Ordering, Kitchen, Billing, Inventory và Reporting có nguy cơ chồng lấn trách nhiệm nếu chỉ bám vào cấu trúc <code>controller-service-repository</code> , làm suy giảm khả năng kiểm soát ranh giới nghiệp vụ.
Service-Based phân tán	Cho phép tổ chức hệ thống theo các service tương ứng với từng năng lực nghiệp vụ; thuận lợi cho mở rộng theo miền và tích hợp qua adapter biên.	Làm tăng độ phức tạp trong giao tiếp qua mạng, truy vết liên service, quản lý hợp đồng dữ liệu và phối hợp triển khai. Nếu áp dụng quá sớm hoặc quá mạnh sẽ tạo chi phí vận hành chưa cần thiết so với quy mô của IRMS.
Microservices	Cô lập mạnh về triển khai, hỗ trợ mở rộng độc lập và vòng đời phát hành riêng.	Chi phí vận hành cao, kéo theo các vấn đề như giao dịch phân tán, quan sát hệ thống và quản trị hợp đồng dịch vụ. Không phù hợp với phạm vi và mục tiêu của hệ thống trong giai đoạn hiện tại.
Event-Driven thuần	Phù hợp cho xử lý nền, phát tán sự kiện, retry và xây dựng mô hình đọc.	Không phù hợp cho toàn bộ hệ thống. Các tương tác như tạo order, lập bill và thanh toán yêu cầu phản hồi tức thời và trạng thái rõ ràng, nên không thể chuyển hoàn toàn sang bất đồng bộ.
Service-Based kết hợp Event-Driven	Giữ được ranh giới năng lực nghiệp vụ rõ ràng thông qua các service; đảm bảo luồng xử lý chính đơn giản và phản hồi nhanh; đồng thời hỗ trợ tốt các tác vụ nền thông qua sự kiện.	Đòi hỏi thiết kế cẩn thận ranh giới service và quản lý luồng sự kiện để tránh phụ thuộc ngược hoặc lan truyền lỗi giữa các miền.

Từ các góc nhìn logic, thành phần và triển khai đã trình bày trong báo cáo chính, phương án phù hợp cho IRMS là một kiến trúc **service-based theo miền nghiệp vụ** kết hợp với **cơ chế xử lý hướng sự kiện**.

Ở mức logic, hệ thống được phân rã theo các năng lực như Reservation & Seating, Ordering, Kitchen Coordination, Billing & Settlement, Inventory, Reporting và IAM / Audit. Ở mức thực thi, các service này được tổ chức trong một backend thống nhất để giữ cho các giao dịch lõi đơn giản, giảm độ trễ và đảm bảo tính nhất quán.

Song song với đó, các tác vụ không yêu cầu phản hồi ngay được xử lý thông qua **cơ chế bất đồng bộ dựa trên sự kiện**, cho phép tách biệt tuyến giao dịch chính với các hệ quả phụ như gửi thông báo, cập nhật báo cáo và ghi nhận kiểm toán.

ADR-0001: Lựa chọn kiến trúc service-based kết hợp event-driven cho IRMS

1. **Bối cảnh:** IRMS cần hỗ trợ nhiều luồng nghiệp vụ có tính chất khác nhau: gọi món, điều phối bếp, thanh toán yêu cầu phản hồi nhanh và nhất quán, trong khi báo cáo, thông báo và kiểm toán lại phù hợp với xử lý nền.
2. **Quyết định:** Chọn kiến trúc gồm:
 - **service-based theo miền nghiệp vụ** cho tổ chức hệ thống;
 - **giao tiếp đồng bộ** cho các luồng tuyến đầu;
 - **giao tiếp bất đồng bộ dựa trên sự kiện** cho các tác vụ nền.
3. **Trạng thái:** Đã phê duyệt.
4. **Lý do lựa chọn:** Kiến trúc này giữ được sự rõ ràng về ranh giới nghiệp vụ, đồng thời đảm bảo hiệu năng cho các thao tác trực tiếp với người dùng. Việc bổ sung event-driven giúp giảm tải cho tuyến giao dịch chính và tăng khả năng mở rộng cho các xử lý nền.
5. **Hệ quả kiến trúc:** Hệ thống cần quản lý hai loại luồng xử lý: luồng đồng bộ cho giao dịch chính và luồng bất đồng bộ cho hệ quả phụ. Điều này kéo theo nhu cầu kiểm soát idempotency, retry và theo dõi trạng thái xử lý nền.
6. **Các phương án không được chọn:** Layered monolith bị loại do không phản ánh rõ ranh giới nghiệp vụ; microservices bị loại do chi phí vận hành cao; event-driven thuần bị loại do không phù hợp với giao dịch cần phản hồi tức thời.
7. **Bản ghi ADR:** docs/adr/ADR-0001-service-based-event-driven-architecture.md.

ADR-0002: Sử dụng Frontend Node Server và API Gateway làm điểm vào thống nhất

1. **Bối cảnh:** Hệ thống phục vụ nhiều loại client như POS, KDS và ứng dụng quản lý.
2. **Quyết định:** Sử dụng **Frontend Node Server** của TanStack Start/Nitro để phục vụ React và proxy /api, sau đó dùng **API Gateway** trong backend làm điểm vào chung cho các service.
3. **Lý do lựa chọn:** Tập trung hóa định tuyến ở biên, giữ frontend không phụ thuộc trực tiếp vào từng service backend.
4. **Hệ quả:** Giảm phụ thuộc giữa client và backend, đồng thời chuẩn hóa giao tiếp ở biên hệ thống.
5. **Bản ghi ADR:** docs/adr/ADR-0002-use-nitro-proxy-and-api-gateway.md.

ADR-0003: Kết hợp REST đồng bộ với RabbitMQ, Outbox và manual ack

1. **Bối cảnh:** IRMS bao gồm hai loại tương tác chính: (1) các thao tác cần phản hồi tức thời như tạo order, lập bill, thanh toán; (2) các tác vụ nền như gửi thông báo, cập nhật báo cáo và ghi nhận kiểm toán.
2. **Quyết định:** Sử dụng **giao tiếp đồng bộ** (REST) cho các thao tác tuyến đầu, đồng thời sử dụng **sự kiện miền bất đồng bộ** qua RabbitMQ, outbox_events, processed_events và consumer manual ack cho các tác vụ không yêu cầu phản hồi ngay.
3. **Trạng thái:** Đã phê duyệt.

4. **Lý do lựa chọn:** Cách kết hợp này đảm bảo trải nghiệm người dùng nhanh và rõ ràng về trạng thái lỗi, đồng thời cho phép các tác vụ nền được xử lý độc lập, có thể retry và mở rộng mà không ảnh hưởng đến luồng chính. RabbitMQ là broker có bằng chứng trong code/config hiện tại; không có bằng chứng về Kafka hay Redis Streams.
5. **Hệ quả kiến trúc:** Hệ thống cần hỗ trợ cơ chế **idempotency**, **retry** và theo dõi trạng thái xử lý nền. Ngoài ra, cần có cơ chế quan sát luồng sự kiện để phát hiện lỗi và tắc nghẽn.
6. **Ví dụ áp dụng trong IRMS:**
 - Tuyển đồng bộ: `CreateOrder`, `CreateBill`, `ProcessPayment`.
 - Tuyển bất đồng bộ: `OrderConfirmed`, `DishReady`, `LowStockDetected`, `ReportRefreshRequested`.
7. **Bản ghi ADR:** `docs/adr/ADR-0003-use-rabbitmq-outbox-and-manual-ack.md`.

ADR-0004: Sử dụng PostgreSQL dùng chung theo miền dữ liệu và read model cho báo cáo

1. **Bối cảnh:** Các miền như Ordering, Billing, Inventory và Reporting có đặc điểm truy cập dữ liệu khác nhau. Nếu dùng chung một nhóm bảng cho mọi mục đích, truy vấn báo cáo có thể ảnh hưởng trực tiếp đến hiệu năng giao dịch.
2. **Quyết định:** Các service sở hữu phần dữ liệu theo miền trong PostgreSQL dùng chung của runtime hiện tại. Phần báo cáo sử dụng **read model** riêng, được cập nhật thông qua sự kiện hoặc các tiến trình nền.
3. **Trạng thái:** Đã phê duyệt.
4. **Lý do lựa chọn:** Tách dữ liệu giúp giảm phụ thuộc giữa các miền, đồng thời cho phép tối ưu truy vấn theo từng mục đích sử dụng. Read model giúp tránh việc truy vấn trực tiếp vào dữ liệu giao dịch nóng.
5. **Hệ quả kiến trúc:** Hệ thống chấp nhận một mức **trễ dữ liệu có kiểm soát** giữa dữ liệu giao dịch và dữ liệu báo cáo. Đồng thời cần cơ chế cập nhật và đồng bộ read model một cách đáng tin cậy.
6. **Phạm vi áp dụng:** Áp dụng cho các báo cáo như doanh thu, hiệu suất bếp, vòng quay bàn và cảnh báo tồn kho.
7. **Bản ghi ADR:** `docs/adr/ADR-0004-use-postgresql-shared-database-and-reporting-read-model`.

ADR-0005: Cô lập thanh toán, thông báo và biên lai qua adapter nội bộ

1. **Bối cảnh:** Thanh toán, thông báo và phát hành biên lai là các điểm dễ thay đổi theo chính sách vận hành. Code hiện tại có các interface `PaymentGateway`, `NotificationChannelAdapter` và `ReceiptDeliveryAdapter`, nhưng chưa có cấu hình provider thanh toán, SMS/Email hay máy in vật lý bên ngoài.
2. **Quyết định:** Giữ các biến thể này sau adapter nội bộ, để lỗi nghiệp vụ chỉ phụ thuộc vào năng lực cần dùng thay vì phụ thuộc trực tiếp vào provider cụ thể.
3. **Trạng thái:** Đã phê duyệt.
4. **Hệ quả kiến trúc:** Hệ thống dễ thay đổi kênh xử lý hơn, nhưng tài liệu phải phân biệt rõ adapter nội bộ với hệ thống ngoài đã triển khai thật.

5. **Bản ghi ADR:** docs/adr/ADR-0005-use-boundary-adapters-for-payment-notification-a

ADR-0006: Triển khai runtime hiện tại bằng Docker Compose

1. **Bối cảnh:** Repo hiện tại mô tả runtime bằng docker-compose.yml, gồm frontend, api-gateway, các service Spring Boot, PostgreSQL, RabbitMQ và container migration.
2. **Quyết định:** Sử dụng Docker Compose làm mô hình triển khai hiện trạng trong báo cáo. Các cơ chế như load balancer, Kubernetes, read replica hoặc warm standby chỉ được xem là hướng mở rộng nếu chưa có cấu hình trong repo.
3. **Trạng thái:** Đã phê duyệt.
4. **Hệ quả kiến trúc:** Góc nhìn triển khai phải bám vào các container thật và không mô tả khả năng sẵn sàng cao như đã hiện thực.

5. **Bản ghi ADR:** docs/adr/ADR-0006-use-docker-compose-for-current-runtime-deploymen

ADR-0007: Kiểm soát truy cập và ghi nhận kiểm toán

1. **Bối cảnh:** IRMS chứa nhiều thao tác nhạy cảm như hoàn tiền, thay đổi giá, điều chỉnh tồn kho và phân quyền người dùng. Các thao tác này cần được kiểm soát chặt chẽ và có khả năng truy vết.
2. **Quyết định:** Áp dụng **RBAC** tại lớp IAM và lớp biên, kết hợp với **AuthorizationPolicy** trong miền nghiệp vụ. Đồng thời ghi **AuditLog** cho các hành động quan trọng.
3. **Trạng thái:** Đã phê duyệt.
4. **Lý do lựa chọn:** RBAC cho phép kiểm soát truy cập theo vai trò, trong khi policy giúp xử lý các điều kiện nghiệp vụ chi tiết hơn. Audit log đảm bảo khả năng truy vết và hỗ trợ kiểm toán.
5. **Hệ quả kiến trúc:** Kiểm soát truy cập trở thành một phần của thiết kế miền, không chỉ là lớp kỹ thuật ở biên. Hệ thống cần đảm bảo tính toàn vẹn và không thể sửa đổi của log kiểm toán.
6. **Ví dụ áp dụng trong IRMS:**
 - hoàn tiền vượt ngưỡng cần kiểm tra policy;
 - thay đổi giá phải được ghi log;
 - truy cập audit log yêu cầu quyền riêng.

7. **Bản ghi ADR:** docs/adr/ADR-0007-use-rbac-authorization-policy-and-audit-log.md.

A.2 Liên kết giữa phụ lục và các phần chính

Phần này cho thấy mỗi phụ lục kỹ thuật được dùng để đọc ngược và biện minh cho đúng phần tương ứng trong báo cáo chính. Nhờ vậy, khi một quyết định đã xuất hiện trong thân báo cáo, người đọc có thể lần ngay sang phụ lục phù hợp để xem lý do chọn kiến trúc, thành phần hoặc loại hình sơ đồ.

Bảng 36: Liên kết giữa các phần chính và các phụ lục biện minh

Phần chính	Điểm cần được giải thích	Phụ lục dùng để biện minh
Mục 2.1	Vì sao phần mô hình hóa mở đầu bằng sơ đồ ngữ cảnh, sơ đồ tổng quan ca sử dụng và sơ đồ hoạt động đầu-cuối thay vì đi thẳng vào đặc tả chi tiết.	Phụ lục A.3 giải thích giá trị của từng loại sơ đồ; Phụ lục A.1 cho thấy các sơ đồ này là nền để đọc kiến trúc, không phải hình minh họa rời rạc.
Mục 3.1	Vì sao IRMS được mô tả như một kiến trúc service-based theo năng lực nghiệp vụ, kết hợp lớp biên web, tuyến xử lý nền và mô hình đọc tách riêng.	Phụ lục A.1 chứa các ADR-0001, ADR-0002, ADR-0003 và ADR-0004 về kiểu kiến trúc, điểm vào, kiểu kết nối và dữ liệu đọc; Phụ lục A.4 giải thích vì sao dùng các thành phần nền tương ứng.
Mục 3.2 và Mục 3.8	Vì sao báo cáo đồng thời dùng góc nhìn logic, mô-đun, thành phần, triển khai và nguyên tắc thiết kế.	Phụ lục A.3 giải thích vai trò của từng góc nhìn, đồng thời phần này giúp nối mỗi góc nhìn với đúng phần giải thích tương ứng trong báo cáo.
Mục 4.1 và Mục 4.2	Vì sao phần thiết kế chi tiết dùng sơ đồ lớp ở mức aggregate, điểm mở rộng và hành vi thay vì chỉ mô tả dữ liệu; vì sao các sơ đồ hoạt động chi tiết đi theo từng cụm nghiệp vụ.	Phụ lục A.3 giải thích lý do chọn góc nhìn lớp và hoạt động; Phụ lục A.4 giải thích vì sao các điểm mở rộng như thanh toán, thông báo, phân quyền được đưa vào mô hình chi tiết.

A.3 Lý do chọn các góc nhìn và loại sơ đồ

Mỗi loại sơ đồ trong báo cáo được lựa chọn nhằm trả lời một câu hỏi kiến trúc cụ thể, từ việc xác định ranh giới hệ thống, phân rã chức năng, cho đến mô tả hành vi chi tiết và cách hệ thống được triển khai trong thực tế. Việc tổ chức các góc nhìn theo nhiều lớp giúp đảm bảo tính nhất quán giữa yêu cầu, kiến trúc và thiết kế chi tiết của IRMS.

Bảng 37: Lý do chọn từng góc nhìn và loại sơ đồ trong báo cáo

Loại sơ đồ / góc nhìn	Dùng ở phần nào	Lý do chọn
Sơ đồ ngữ cảnh	Mục 2.1	Dùng để xác định ranh giới hệ thống IRMS, các tác nhân bên ngoài và các hệ thống liên quan. Đây là bước nền để tránh việc các thành phần kiến trúc bị hiểu sai như các module nội bộ thuần kỹ thuật.
Sơ đồ use case tổng quan	Mục 2.1	Dùng để gom các ca sử dụng thành các nhóm nghiệp vụ lớn, giúp hình dung phạm vi chức năng của hệ thống trước khi đi vào kịch bản chi tiết và đặc tả. Đây là cầu nối giữa yêu cầu và kiến trúc.
Sơ đồ hoạt động (kịch bản end-to-end)	Mục 2.2	Dùng để mô tả luồng nghiệp vụ xuyên suốt như đặt bàn, gọi món, thanh toán. Qua đó xác định rõ các điểm đồng bộ, bất đồng bộ và các bước có tính phụ thuộc cao trong hệ thống.
Góc nhìn logic	Mục 3.4	Dùng để biểu diễn hệ thống dưới dạng các năng lực nghiệp vụ (capability view), giúp kiểm tra mức độ phù hợp giữa kiến trúc và nghiệp vụ nhà hàng, đặc biệt là các ranh giới trách nhiệm giữa các phần.
Góc nhìn phân rã (module view)	Mục 3.5	Dùng để chuyển từ năng lực nghiệp vụ sang cấu trúc mô-đun thực thi trong hệ thống. Góc nhìn này giúp xác định rõ trách nhiệm từng module và cách hệ thống được chia theo miền chức năng.
Góc nhìn thành phần và kết nối	Mục 3.6	Dùng để mô tả các thành phần runtime và cách chúng giao tiếp với nhau (HTTP, event, async worker). Đây là góc nhìn làm rõ kiến trúc thực thi và luồng dữ liệu giữa các phần.
Góc nhìn triển khai	Mục 3.7	Dùng để mô tả cách hệ thống được triển khai trên hạ tầng thực tế, bao gồm phân tách service, database, worker và các cơ chế đảm bảo sẵn sàng cao và cô lập lỗi.
Sơ đồ lớp	Mục 4.1	Dùng để mô tả cấu trúc miền nghiệp vụ ở mức chi tiết (aggregate, entity, service), đặc biệt các điểm mở rộng như payment, notification, authorization. Phù hợp hơn ERD vì tập trung vào hành vi thay vì chỉ dữ liệu.
Sơ đồ hoạt động chi tiết	Mục 4.3	Dùng để mô tả luồng xử lý chi tiết của từng nhóm chức năng quan trọng, nhằm kiểm chứng tính đúng đắn của thiết kế lớp và các tương tác giữa các thành phần.

A.4 Lý do chọn các thành phần và loại hình kiến trúc nền

Phần này tổng hợp cơ sở giải thích cho các thành phần và kiểu tổ chức xuất hiện trong sơ đồ tổng quan, mô-đun, thành phần và triển khai; đồng thời nối từng lựa chọn với yêu cầu vận hành của IRMS. Các thuật ngữ được sử dụng nhất quán với quyết định kiến trúc đã chọn, trong đó hệ thống được tổ chức theo hướng **service-based architecture** kết hợp **event-driven communication**, giúp giảm phụ thuộc đồng bộ giữa các thành phần và tăng khả năng mở rộng theo từng miền nghiệp vụ.

Bảng 38: Lý do chọn các thành phần và loại hình kiến trúc nền của IRMS

Thành phần / loại hình	Lý do dùng trong IRMS	Liên hệ trực tiếp trong báo cáo
ReactJS cho lớp giao diện	Phù hợp với các màn hình thao tác theo vai trò (lễ tân, phục vụ, bếp, thu ngân, quản lý); hỗ trợ cập nhật trạng thái giao diện nhanh và điều hướng nhất quán giữa các luồng nghiệp vụ.	Xuất hiện trong Mục 3.1 như lớp biên người dùng và là điểm vào các kịch bản nghiệp vụ ở Mục 2.1.
Frontend Node Server / api-gateway edge boundary	Đóng vai trò lớp biên hệ thống: Frontend Node Server phục vụ giao diện React và chuyển tiếp /api, còn api-gateway định tuyến request đến các service nội bộ.	Biện minh cho ADR-0002 trong Phụ lục A.1 và xuất hiện trong sơ đồ kiến trúc tổng quan đã cập nhật.
Service-based architecture + event- driven communication	Hệ thống được phân rã thành các service theo miền nghiệp vụ, giao tiếp chủ yếu qua event để giảm coupling và hỗ trợ mở rộng độc lập từng chức năng như order, kitchen, payment. Cách tiếp cận này phù hợp với các luồng nghiệp vụ nhiều bước và có tính bất đồng bộ cao.	Là trục chính của ADR-0001; thể hiện xuyên suốt trong Mục 3.1 và Mục 3.2.
RabbitMQ / Outbox pattern / background workers	Đảm bảo tính nhất quán giữa giao dịch chính và việc phát event, đồng thời xử lý các tác vụ nền như gửi thông báo, cập nhật read model và xử lý retry một cách bất đồng bộ.	Biện minh cho ADR-0003 và xuất hiện trong các sơ đồ kiến trúc, thành phần và triển khai.
PostgreSQL giao dịch	Đảm bảo tính nhất quán dữ liệu mạnh cho các thao tác quan trọng như xác nhận đơn, thanh toán, hoàn tiền và ghi nhận audit.	Gắn với ADR-0004 và các sơ đồ dữ liệu và triển khai trong Mục 3.2.
Read model cho báo cáo	Tách biệt luồng đọc và luồng ghi nhằm tối ưu hiệu năng truy vấn báo cáo, giảm tải hệ thống giao dịch chính và chấp nhận độ trễ nhất định trong đồng bộ dữ liệu.	Biện minh bởi ADR-0004; liên kết với các sơ đồ phân tích và triển khai.
PaymentGateway	Cô lập cách xử lý thanh toán sau một interface nội bộ, giúp hệ thống linh hoạt thay đổi cơ chế xử lý mà không ảnh hưởng đến domain logic.	Xuất hiện trong Mục 3.1 và Mục 4.1 như một adapter boundary; repo hiện tại chưa có provider thanh toán ngoài được cấu hình; gắn với ADR-0005.
NotificationChannel / ReceiptDeliveryAdapter	Tách biệt thông báo và phát hành biên lai khỏi luồng nghiệp vụ chính, cho phép xử lý bất đồng bộ.	Liên kết giữa kiến trúc tổng quan và thiết kế chi tiết ở Mục 4.2; code hiện tại chưa có printer vật lý hay provider SMS/Email ngoài; gắn với ADR-

A.5 Phân công công việc

Phân phân công dưới đây được trình bày theo hướng mỗi thành viên phụ trách một mảng chính, đồng thời đều tham gia phối hợp rà soát và hiệu chỉnh để bảo đảm tính liên kết giữa các phần của báo cáo. Cách phân công này phản ánh vai trò chính của từng thành viên nhưng vẫn nhấn mạnh tính cộng tác chung của toàn nhóm trong quá trình hoàn thiện tài liệu.

Bảng 39: Phân công công việc của Nhóm 8

MSSV	Thành viên	Phạm vi đóng góp chính
2311896	Đỗ Quang Long	Phụ trách tổng hợp bối cảnh nghiệp vụ, hệ thống hóa yêu cầu chức năng và phi chức năng, đồng thời phối hợp rà soát để bảo đảm các yêu cầu được phản ánh nhất quán trong các phần phân tích và thiết kế.
2311982	Lê Hoàng Luân	Phụ trách tổ chức cấu trúc tài liệu LaTeX, chuẩn hóa hình thức trình bày và tích hợp các thành phần nội dung, đồng thời phối hợp với các thành viên khác để bảo đảm bố cục báo cáo và hệ thống sơ đồ thống nhất.
2310990	Lê Thúy Hiền	Phụ trách xây dựng phần đặc tả use case và mô tả các luồng hoạt động nghiệp vụ chính, đồng thời phối hợp đối chiếu với phần yêu cầu và phần kiến trúc để giữ sự liên thông giữa mô tả nghiệp vụ và giải pháp thiết kế.
2213698	Nguyễn Hải Trung	Phụ trách các góc nhìn phân rã dịch vụ, thành phần và kết nối của hệ thống, đồng thời phối hợp với các phần khác để bảo đảm các quyết định kiến trúc bám sát yêu cầu nghiệp vụ và thống nhất với cấu trúc tổng thể của báo cáo.
2211384	Tô Thế Hưng	Phụ trách xây dựng các sơ đồ lớp và rà soát mô hình miền nghiệp vụ, đồng thời phối hợp đối chiếu với phần kiến trúc và phần hành vi để bảo đảm cấu trúc lớp phản ánh đúng các trách nhiệm và điểm mở rộng của hệ thống.
2310737	Trần Nhật Đăng	Phụ trách phần triển khai, phân bổ và phân tư kiến trúc, đồng thời phối hợp rà soát cách diễn đạt và mối liên kết giữa các chương để bảo đảm báo cáo có tính nhất quán và hoàn chỉnh ở mức toàn cục.